MASSIMO PAPA

# PYTHON

## PRIMI PASSI

VOLUME 1

# LE BASI DEL LINGUAGGIO



Per corsi Scuola Secondaria di 2° grado, Post-Diploma e appassionati!

CON CENNI GNU/LINUX



PUBLISHED COPY

# PYTHON

## P R I M I  P A S S I

## Volume 1
## LE BASI DEL LINGUAGGIO

# WORD CLOUD


python
print
programma
metodo
funzione
lista
codice
output
stringa
file
classe
parametro
database
comando
import
lanciamo
oggetto
server
nome
def
valore
righe
directory
riga
dati
tale
tupla
dizionario
operatori
oggetti
sistema
interprete
tuple
uscita
istruzione
caso
visto
analisi
input
volta
ciclo
esempio
numero
elemento
parametri
funzioni
elementi
testo
stato
variabile
carattere
chiave
path
sequenza
inserire
utente
restituisce
installazione
open
sequenze
lettura
cursor
comandi
html
tipo
video
files
metodi
float
int
str
bool
pip
http
interno
numpy
table
console
client
dataframe
home
massimo
figura
prova
versione
stampato
eseguire
stringhe
errore
return
linux
valori
modulo
linuxmachine
contenuto
stdin
num
caso
ssh
math
set
attributi
novas
verr
suo
ip
np
ftp
loop
nostro
stat
xls
flusso
numeri
record
alcun
ambiente
api
sintassi
apre
liste
caratteri
nulla
host
bit
ax
nelle
tutto
exec
axes
codifica
creazione
centro
gestire
fare
plt
ele
web
br
ssl
test
read
line
write
cwd
key
pandas
lib
db
init
base
area
id
gnu
zip
default
studente
env
data
ls
sar
script
utf
csv
sql
array
plot
sys
cfr
fun
row
cat
sort
locale
altezza
usr
recent
users
attributo
parentesi
cerchio
classi
terzo
pop
generale
aver
dizionari
tab
andiamo
rappresenta
porta
indice
operazioni
permessi
mysqldb
count
val
min
istanza
append
full
module
flow
errori
altre
shell
pem
serie
cosa
lavoro
operatore
argomenti
grafico
install
acct
unix
librerie
step
elemento
map
pd
vi
nei
user
list
serve
zip
gnu
id
area
base
init
db
lib
pandas
false
diz
tupla
txt
riga
directory
dati
tale
stringa
def
valore
righe

# SOMMARIO


1. Premessa 15
2. Introduzione 17
3. L'interprete Python 18
3.1 Introduzione 18
3.1.1 Lanciare l'interprete 18
3.1.1.1 Interattivo 18
3.1.1.2 Linea di comando 19
3.2 Ciao Mondo! 20
3.3 Creare un eseguibile 20
4.2.1 Operatori aritmetici 22
4.2.2 Operatori di comparazione 23
4.2.3 Operatori logici 23
4.2.4 Operatori di assegnazione 24
4.2.5 Operatori di membership 24
4.2.6 Operatori di identità 25
4.2.7 Operatori bitwise 25
4.3. Commento 26
4.4 Variabili 26
4.5 Strutture di controllo flusso 27
4.5.1 Ciclo while 27
4.5.2 Ciclo do - while 29
4.5.3 Ciclo for 30
4.5.3.1 Oggetti iterabili 32
4.5.3.2 La struttura generale del ciclo for 32
4.5.4 Costrutto if 33
4.5.5 Operatori di confronto e operatori logici 36
4.5.6 Le istruzioni break e continue 39
4.5.6.1 Break 39
4.5.6.2 Continue 40
4.5.7 Approfondimento sui loop: l'istruzione else 41
4.5.8 Costrutto match-case 43
4.5.8.1 Il match-case con condizioni aggiuntive (guard) 44
4.5.8.3 Pattern matching su strutture dati 46
4.5.8.3.1 Pattern matching sulle liste 46
4.5.8.3.2 Pattern matching sulle tuple 46
4.5.8.3.3 Pattern matching sui dizionari 47
4.5.8.3.4 Pattern matching sulle stringhe 48

4.5.8.3.5 Pattern matching sugli insiemi 48
**5. Le funzioni 50**
5.1 Cosa sono le funzioni e a cosa servono 50
5.2 Definizione di funzione 50
5.2.1* Digressione sulle modalità di passaggio dei parametri 52
5.2.1.1 Modalità di passaggio di parametri 52
5.2.1.2 I tipi di variabili in Python: una puntualizzazione 53
5.2.1.3 La funzione id() 53
5.2.1.4 Tipi immutabili e mutabili 54
5.2.1.5 Passaggio di parametri in una funzione Python 55
5.3 Approfondimenti sul passaggio di parametri 57
5.3.1 Parametri con valori predefiniti 57
5.3.2 Argomenti denominati 58
5.3.3 Numero variabile di argomenti 58
5.3.4 Numero variabile di argomenti denominati 60
5.4 Regole di visibilità 62
5.4.1 Visibilità locale 63
5.4.2 Visibilità globale 63
5.5 Approfondimenti sulla funzione print 66
5.5.1 Print accetta un numero variabile di parametri 66
5.5.1.1 Sulle sequenze di escape 67
5.5.2 Parametro formale end 69
5.5.3 Parametro formale sep 69
5.5.4 Formattazione dell'output alla maniera del linguaggio C 70
5.5.5 Formattazione dell'output utilizzando le f-string 73
5.5.5.1 Cosa sono le f-string? 73
5.5.5.2 L'uso delle f-string con la funzione print 73
5.6 Sequenze e dizionari come argomento di funzioni 76
5.6.1 Stringhe e Tuple 76
5.6.2 Liste 78
5.6.3 Dizionari 79
5.6.4 Insiemi 80
5.7 Suggerimenti del tipo degli argomenti e del valore restituito 82
**6. Input da tastiera 84**
6.1 Inserimento di stringhe 84
6.2 Inserimento di valori numerici 85
6.3 Inserimento di più valori sulla stessa riga 87
**7. Strutture dati 89**
7.1 Stringhe 89
7.1.1 Creare una stringa 89

7.1.2 Accedere agli elementi di una stringa 90
7.1.3 Operazioni tra stringhe 92
7.1.4 Modificare i caratteri di una stringa 92
7.1.5 Estrazione di sottostringhe 94
7.1.6 Confrontare le stringhe 94
7.1.7 Appartenenza a una stringa 95
7.2 Liste 95
7.2.1 Creare una lista 95
7.2.2 Accedere agli elementi di una lista 96
7.2.3 Modificare gli elementi di una lista 97
7.2.4 Modificare le liste 97
7.2.5 Confrontare le liste 98
7.2.6 Alcuni metodi specifici delle liste 99
7.2.6.1 Funzione list() 99
7.2.6.2 Metodi append(), extend() e insert() 100
7.2.6.3 Metodi clear(), pop() e remove() 100
7.2.6.4 Metodi sort() e reverse() 101
7.2.6.4.1 Funzione sorted() e reversed() 102
7.3 Tuple 102
7.3.1 Creare una tupla 102
7.3.1.1 Creare una tupla con un singolo elemento 103
7.3.2 Accedere agli elementi di una tupla 104
7.3.2.1 Esempio notevole di assegnazione di tuple 105
7.3.2.2 Funzioni che restituiscono tuple 106
7.3.3 Modificare gli elementi di una tupla 107
7.3.3.1 Modificare gli elementi di un oggetto in una tupla 107
7.3.4 Modificare le tuple 108
7.3.5 Concatenare tuple e liste 109
7.4 Operazioni sulle sequenze 109
7.4.1 Ricerca di un elemento 109
7.4.2 Il numero di occorrenze di un elemento 111
7.4.3 Estrazioni di sotto-sequenze 112
7.4.4* Ricerca di sottosequenze 114
7.4.4.1 Ricerca tramite slicing 115
7.4.4.2 Confronto tra i vari metodi di ricerca delle sottosequenze 116
7.4.5* Il numero di occorrenze di una sottosequenza 116
7.5 Funzioni di utilità sulle sequenze 117
7.5.1 min() e max() 118
7.5.2 filter() 119
7.5.3 map() 121

7.5.4 zip() 122
7.6 Dizionari 123
7.6.1 Creare un dizionario 124
7.6.2 Accedere agli elementi di un dizionario 125
7.6.3 Modificare gli elementi di un dizionario 125
7.6.4 Modificare i dizionari 126
7.6.5 Confrontare i dizionari 127
7.6.5 Ricerca di elementi nel dizionario 128
7.6.6 Alcuni metodi specifici dei dizionari 129
7.6.6.1 Metodo popitem( ) 129
7.6.6.2 Metodo setdefault( ) 130
7.6.6.3 Metodo update( ) 130
7.6.6.4 Metodi keys( ) e values( ) 131
7.7 Insiemi 132
7.7.1 Creare un insieme 132
7.7.2 La funzione frozenset() 133
7.7.3 Accedere agli elementi di un insieme 134
7.7.4 Operazioni insiemistiche sugli insiemi 135
7.7.4.1 Unione 135
7.7.4.2 Intersezione 136
7.7.4.3 Differenza 136
7.7.4.4 Differenza simmetrica 136
7.7.5 Modificare un insieme 137
7.7.5.1 Il metodo add() 137
7.7.5.2 Il metodo pop() 138
7.7.5.3 Il metodo remove() 138
7.7.5.4 Il metodo discard() 139
7.7.5.5 Il metodo clear() 139
7.7.6 Confrontare gli insiemi 140
7.7.6.1 Disgiunzione 140
7.7.6.2 Sottoinsieme 140
7.7.6.2 Sovrainsieme 141
**8. Classi e Oggetti** **142**
8.1 La programmazione orientata agli oggetti 142
8.2 Creazione di una classe e la sua istanziazione 142
8.3 Creazione dei metodi di una classe 145
8.3.1 Il metodo __str__ di Object 146
8.3.2 Metodi che accettano oggetti della classe 147
8.3.3 Metodi che restituiscono oggetti della classe 148
8.3.4 Il codice della classe Punto completa 149

8.3.5 * Una piccola digressione teorica sui metodi getter e setter che ci porterà a parlare delle proprietà di un oggetto. 150
8.3.5.1 Metodi getter 150
8.3.5.2 Metodi setter 150
8.3.5.3 ** Cosa è un decoratore 151
8.3.5.4 Property decorator 151
8.3.5.5 Il codice della classe Punto con le properties 152
8.4 Ereditarietà 153
8.4.1 Classe di esempio Cerchio 153
8.4.2 La sottoclasse Cilindro 155
8.4.2.1 Piccola digressione matematica 158
8.5 Polimorfismo 158
8.5.1 Polimorfismo con operatori 160
8.5.2 Polimorfismo con collezioni eterogenee 161
8.6 I modificatori di visibilità 162
8.6.1 Modificatore di accesso pubblico 162
8.6.2 Modificatore di accesso protetto 163
8.6.3 Modificatore di accesso privato 164
8.7 Importare le classi 165
8.7.1 Utilizzare le classi definite in altri sorgenti 165
8.7.1.1 Cosa è un modulo 165
8.7.2 Il nostro primo modulo: geometria.py 166
8.7.3 Importare classi da altri moduli 167
8.7.4 Importare tutto da un modulo 168
8.7.5 Importazione con la direttiva alias 169
8.7.6 Gestire il percorso dei moduli 169
8.7.7 Consigli pratici 170
8.8 Un esempio notevole: classe Frazione 170
8.8.1 Il progetto 170
8.8.2 La classe Frazione 171
8.8.3 Il metodo di semplificazione della frazione 174
8.8.3.1 Piccolo background teorico 174
8.8.3.2 Calcolo dell'M.C.D. in Python 175
8.8.3.3 Il codice della classe con il metodo semplifica() 175
8.8.4 Introduciamo le operazioni tra oggetti frazione 176
8.8.4.1 Descrizione della funzionalità 176
8.8.4.2 Overloading dell'operatore + 177
8.8.4.3 Overloading degli operatori in Python 179
8.8.4.3 Implementazione delle quattro operazioni 180
8.8.4.3.1 Moltiplicazione 180

8.8.4.3.2 Divisione 180
8.8.4.3.3 Addizione 180
8.8.4.3.4 Sottrazione 181
8.8.5 Implementazione del confronto tra frazioni 181
8.8.5.1 Metodo alternativo 183
8.8.6 Il metodo getDecimalValue() 184
8.8.7 Il codice completo della classe Frazione 184
8.9 Classi astratte 186
8.9.1 Introduzione alle classi astratte 186
8.9.2 Definizione di una classe astratta 186
8.9.3 Precisazioni sulla definizione 187
8.9.4 Implementazione delle classi concrete 188
8.9.4.1 Tentativo di istanziare una classe astratta 188
8.9.5 A cosa servono le classi astratte? 189
8.9.6 Verifica dell'implementazione dei metodi 190
8.9.7 Metodi concreti in classi base astratte 190
8.9.8 Un esempio avanzato: gestione dei pagamenti 192
8.9.9.Vantaggi delle classi astratte 193
8.10 * Ereditarietà multipla 194
8.10.1 Ereditarietà singola: un breve ripasso 194
8.10.2 Dall'ereditarietà singola all'ereditarietà multipla 195
8.10.3 A cosa serve l'ereditarietà multipla 197
8.10.4 Le problematiche dell'ereditarietà multipla 197
8.10.4.1 Problema del conflitto tra metodi con lo stesso nome 197
8.10.4.2 Il problema del diamante 199
8.10.4.3 L'MRO: Method Resolution Order 200
8.10.5 L'uso di super() per far cooperare le classi 201
8.10.6 I Mixin: una buona pratica di design 203
8.10.6.1 Un altro esempio di Mixin senza uso di librerie esterne 204
8.10.7 Quando usare l'ereditarietà multipla 206
8.10.8 Ereditarietà e composizione 206
8.10.9 Regole d'oro per l'ereditarietà multipla 207
8.10.10 Riepilogo 209
8.11 I metodi statici 209
8.11.1 Che cos'è un metodo statico 210
8.11.2 Come si definisce un metodo statico 210
8.11.3 Un primo esempio semplice 210
8.11.4 Differenza tra metodo di istanza e metodo statico 211
8.11.5 Un esempio collegato alla classe Punto 212
8.11.6 A cosa servono i metodi statici 213

8.11.7 Quando conviene usare un metodo statico 214
8.11.8 Pro e contro dei metodi statici 215
8.11.9 Errore comune: usare self in un metodo statico 218
8.11.10 Metodi statici e metodi di classe 218
8.11.10.1 * Approfondimento: i metodi di classe 219
8.11.11 Esempio riassuntivo 221
8.11.12 Riepilogo 222
**9. Gestione degli errori 224**
9.1 Cosa sono le eccezioni 224
9.2 Struttura di base per la gestione delle eccezioni 225
9.2.1 Il blocco finally 226
9.2.2 Il blocco else 226
9.3 Eccezioni comuni 227
9.4 Utilizzare più blocchi except 227
Caso 1: Input non valido 228
Caso 2: Divisione per zero 228
Caso 3: Input valido 228
9.5 Utilizzare più eccezioni in un unico blocco except 229
Caso 1: Input non valido 229
Caso 2: Divisione per zero 229
Caso 3: Input valido 229
9.6 Sollevare eccezioni 230
9.7 Definire eccezioni personalizzate 231
9.7.1 Esempio di gestione eccezioni personalizzate 232
9.8 Utilizzare le eccezioni per il controllo del flusso 234
**10. Importazione moduli 236**
10.1 Introduzione 236
10.2 I moduli del Python 238
10.3 L'istruzione import 240
**11. I packages 242**
11.1 Packages e moduli 242
11.2 Creazione dei packages 242
11.3 Utilizzo dei packages 244
11.4 Configurazione del package con init.py 244
11.5 Importazioni assolute e relative 245
**12. Paradigmi funzionali e gestione efficiente della memoria 247**
12.1 Perché abbiamo bisogno di nuovi strumenti 247
12.2 Le espressioni lambda 247
12.2.1 Funzioni "usa e getta" 247
12.2.2 Lambda insieme a map() e filter() 248

12.3 La sintassi delle comprehension 250
12.3.1 List comprehension 250
12.3.1.1* Considerazioni sulla velocità delle list comprehension 252
12.3.2 Dictionary comprehension e set comprehension 253
12.4 Il protocollo di iterazione: iterabili e iteratori 254
12.4.1 Un'analogia per non confonderli 254
12.4.2 I metodi __iter__() e __next__() 255
12.5 Generatori e valutazione pigra 257
12.5.1 Dalle classi alla parola chiave yield 257
12.5.3 Espressioni generatrici 258
12.5.2 Lista o generatore? Un confronto riassuntivo 261
12.6 Non esagerare: la trappola della complessità 261
12.7 Ricapitolo sui concetti 262
### Esercizi 263

# 1. Premessa

Questo testo è una revisione completa degli appunti di laboratorio di matematica e statistica del corso triennale per conseguire il Diploma di Informatica presso gli Istituti Tecnici. La revisione si è resa necessaria in quanto in questa versione si vuole dare ampio spazio al linguaggio di programmazione Python versione 3.x e dare qualche cenno alle librerie più utilizzate per il data science, quali:

- numpy
- Pandas
- MatPlotLib
- Scikit-learn
- OpenCV (questa è specifica per la visione artificiale)

Si è dato anche spazio all'utilizzo del linguaggio per svolgere compiti sistemistici come:

- gestione del filesystem ( file e modulo os)
- servizi di rete ( ssh, ftp, http)
- database (mysql e sqlite3)

Lo scopo è quello di voler costruire un testo che possa essere utile a chi si vuole avvicinare al linguaggio Python avendo già una conoscenza della programmazione. Ci si rivolge agli studenti della scuola secondaria di secondo grado e  dei corsi post-diploma.

L'approccio è sostanzialmente molto pratico, si forniranno esempi di programmi che introducono i concetti peculiari del linguaggio Python, di ogni codice viene fornito l'output e un'analisi approfondita di ciò che è avvenuto nell'esecuzione delle istruzioni.
Viene fornita alla fine di ogni capitolo una serie di esercizi utili a fissare la teoria illustrate nel capitolo.

In appendice è presente una breve digressione sul sistema operativo GNU/Linux, o meglio sul suo utilizzo attraverso la console dei comandi (*shell*). Si forniscono i cenni necessari per effettuare tutte le prove e gli esercizi che mano a mano si incontreranno nella lettura. Si apprenderà come utilizzare la console da utente. Lo scopo è quello di fornire le conoscenze per

- aprire una console Linux
- utilizzare i comandi in linea forniti dalla shell
- utilizzare i comandi di ridirezione dell'output e i comandi di pipe
- editare testi dalla console

Inoltre sono presenti due capitoli che forniscono indicazioni sull'installazione del Python rispettivamente su ambienti Linux e Windows.

Nell'appendice relativa all'installazione di Python per GNU/Linux vi è un capitolo sulla gestione degli ambienti virtuali che risulta essere molto utile quando si lavora con più

librerie e si vuole evitare di creare conflitti tra le diverse versioni di librerie che si utilizzano. Oppure quando si vuole creare un ambiente di lavoro con versione di Python differente dall'installazione di default presente sul sistema.

# 2. Introduzione

Python è un linguaggio di programmazione dinamico orientato agli oggetti utilizzabile per molti tipi di sviluppo software. Offre un forte supporto all'integrazione con altri linguaggi e programmi, è fornito di una estesa libreria standard e può essere imparato in pochi giorni.[1]

Ideato da Guido van Rossum all'inizio degli anni novanta, il nome fu scelto per la passione dello stesso inventore verso i Monty Python e per la loro serie televisiva Monty Python's Flying Circus.[2]

Il Python ha una sintassi molto semplice, simile a pseudo-codice, per tale motivo è anche molto utilizzato in ambito didattico. Questa sua semplicità gli porta ad avere una curva di apprendimento molto buona.

E' un linguaggio interpretato, quindi le istruzioni vengono tradotte ed eseguite riga per riga. questa caratteristica si presta molto bene alla sperimentazione e alla prototipazione.

Un'altra importante caratteristica è l'esistenza di moltissime librerie che permettono al programmatore Python di sviluppare programmi e soluzioni in qualsiasi ambito!

Il python è portabile, il suo interprete gira in tutte le principali piattaforme hardware e software.

E' un linguaggio che supporta il paradigma della programmazione orientata agli oggetti.

Queste caratteristiche elencate (insieme ad altri fattori che non menziono perché esulerebbero troppo dallo scopo di questo testo) portano il Python a essere tra i linguaggi più diffusi e utilizzati al mondo.  A dimostrazione di quanto detto si verifica che viene impiegato pressochè in tutti i settori.

[1] https://www.python.it/
[2] https://it.wikipedia.org/wiki/Python

# 3. L'interprete Python

## 3.1 Introduzione

È un linguaggio interpretato, un classico linguaggio di scripting. Strettamente parlando l'eseguibile Python è l'interprete del linguaggio stesso.

Un interprete è un programma in grado di eseguire altri programmi a partire direttamente dal relativo codice sorgente scritto in un linguaggio di alto livello, senza la previa compilazione dello stesso (codice oggetto), eseguendo cioè le istruzioni nel linguaggio usato traducendole di volta in volta in istruzioni in linguaggio macchina del processore.[3]

### 3.1.1 Lanciare l'interprete

Ritorniamo all'eseguibile Python, è l'interprete del linguaggio ma è anche qualcosa in più. Puo' essere utilizzato in due modalità differenti:

- ➔ Interattivo
- ➔ Da linea di comando

#### 3.1.1.1 Interattivo

Una modalità per lavorare con il python è quello di digitare i singoli comandi sull'interprete e aspettare che questi fornisca l'output.

Cominciamo a lanciare l'interprete in modalità interattiva, è sufficiente lanciare il comando senza nessun argomento:

```
$ python3
```

Si otterrà il prompt dell'interprete che aspetta l'inserimento di un comando seguito da `invio` per poter essere eseguito.

```
Python 3.9.7 (default, Sep 10 2021, 14:59:43)
[GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more
information.
>>>
```

Il prompt è `>>>` appena appare l'interprete Python si mette in ascolto sullo standard input sino a quando non abbiamo digitato un'istruzione python che possa essere eseguita.

[3] https://it.wikipedia.org/wiki/Interprete_(informatica)

### 3.1.1.2 Linea di comando

Da linea di comando si passano all'interprete dei parametri, per sapere quali sono è sufficiente digitare come parametro `--help`:

```
$ python3 --help
```

Si otterrà un elenco molto lungo analogo al seguente (che è stato troncato):

```
usage: python3 [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options and arguments (and corresponding environment variables):
-b               : issue warnings about str(bytes_instance),
str(bytearray_instance)
             and comparing bytes/bytearray with str. (-bb: issue
errors)
-B               : don't write .pyc files on import; also
PYTHONDONTWRITEBYTECODE=x
-c cmd : program passed in as string (terminates option list)
-d       : turn on parser debugging output (for experts only, only
works on
         debug builds); also PYTHONDEBUG=x
-E       : ignore PYTHON* environment variables (such as PYTHONPATH)
-h       : print this help message and exit (also --help)
-i        : inspect interactively after running script; forces a
prompt even
               if stdin does not appear to be a terminal; also
PYTHONINSPECT=x
-I       : isolate Python from the user's environment (implies -E
and -s)

. . . .
. . . .
```

Noi lo utilizzeremo andandogli a passare come parametro il nome del nostro file sorgente che contiene il nostro programma python, ad esempio:

```
$ python3 nomeprogramma.py
```

L'estensione dei sorgenti python è solitamente `.py`

Si lascia al lettore la libertà di approfondire tutti gli altri parametri che possono essere passati come argomento all'interprete Python. In questa introduzione al linguaggio possiamo tralasciare questo approfondimento.

Nei prossimi capitoli utilizzeremo per la maggior parte delle volte la modalità da linea di comando.

## 3.2 Ciao Mondo!

Cominciamo subito con il primo programma.
Lanciate dalla console di Linux il seguente comando:

```
$ vi ciao.py
```

In questo modo aprite l'editor di testi per poter inserire il vostro codice sorgente python. Digitate la seguente riga:

```
print ("ciao mondo!")
```

Salvate il file ed eseguitelo con il seguente comando :

```
$ pyton3 ciao.py
```

Osservate cosa appare a video, dovreste ottenere un output simile a quello indicato di seguito:

```
ciao mondo!
```

Il vostro primo programma stampa a video la frase: "ciao mondo!".

Quindi abbiamo appreso che l'istruzione `print` permette di stampare sullo standard output e le virgolette delimitano una stringa.

## 3.3 Creare un eseguibile

Vogliamo creare un programma che si lancia direttamente digitando il nome del programma stesso.
In altre parole quello che vogliamo fare è creare uno script che si esegua senza dover richiamare esplicitamente l'interprete.

In un file di script la prima riga è destinata all'indicazione dell'interprete che si vuole utilizzare.
La sintassi è la seguente:

```
#!pathnameinterprete
```

quindi riprendendo l'esempio di prima, vogliamo lanciare il programma nella seguente modalità:

```
$ ciao.py
```

Come si può notare non stiamo chiamando esplicitamente l'interprete Python.

Prima di tutto dobbiamo individuare il pathname dell'interprete che dobbiamo utilizzare. Lanciamo il seguente comando dalla console Linux:

```
$ which python3
```

otteniamo un output simile al seguente:

```
/usr/bin/python3
```

`which` è un comando della console che ci fornisce il percorso completo del comando inserito come argomento, in questo caso il percorso completo di python3 che è il nostro interprete.

Quindi modifichiamo il nostro primo programma come segue:

```
#!/usr/bin/python3

print("Hello world!")
```

I caratteri `#!` indicano che i caratteri successivi contengono il pathname di un interprete di comandi.

Ora dobbiamo indicare al sistema operativo che il file puo' essere eseguito direttamente indicando il nome del file. Per fare questo si devono modificare i permessi del file, per un approfondimento si rimanda all'appendice in cui è riportata una digressione sui permessi di un file (paragrafo 3.2.3)

Dalla console Linux eseguiamo il seguente comando:

```
$ chmod u+x ciao.py
```

In questa maniera ciao.py diventa un file eseguibile che può essere eseguito (lanciato) nel seguente modo:

```
$ ciao.py
```

# 4. Le basi di Python (ovvero programmiamo subito in Python)

## 4.1. Il nostro secondo programma

Continuiamo con un altro programma.
Seguendo la stessa procedura del precedente esempio scriviamo il seguente programma:

```
print ("2 + 2 = ", 2+2)
print ("3 * 4 = ", 3*4)
print (100-1," = 100-1")
print ("(33+2)/5 + 11.5 = ", (33+2)/5+11.5)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
2 + 2 = 4
3 * 4 = 12
99 = 100 - 1
(33+2)/5 + 11.5 = 18.5
```

Abbiamo utilizzato il python come una calcolatrice!

Dall'analisi del codice e dell'output osserviamo che:
L'istruzione print accetta più parametri, questi sono separati dalla virgola e possono essere stringhe o valutazioni di operazioni aritmetiche
Ogni riga di codice finisce con la fine della riga
Se un operando è decimale (cioè espresso in floating point) allora il risultato sarà decimale

## 4.2 Operatori

Non esistono solamente gli operatori aritmetici, in python sono presenti 7 differenti tipi di operatori:
Operatori aritmetici;
Operatori di comparazione;
Operatori logici;
Operatori di assegnazione;

- ➔ Operatori di membership;
- ➔ Operatori bitwise;
- ➔ Operatori di identità.

### 4.2.1 Operatori aritmetici

Sono gli operatori che vengono utilizzati per eseguire operazioni aritmetiche:

| OPERATORI ARITMETICI |
|---|

| | | |
|---|---|---|
| Elevamento a potenza | `**` | `5 ** 2 = 25` |
| Moltiplicazione | `*` | `2 * 3 = 6` |
| Divisione | `/` | `14 / 3 = 4.6666666667` |
| Divisione (arrotonda per difetto) | `//` | `14 // 3 = 4`<br>`-14 // 3 = -5` |
| Resto della divisione | `%` | `14 % 3 = 2` |
| Addizione | `+` | `1 + 2 = 3` |
| Sottrazione | `-` | `4 - 3 = 1` |

### 4.2.2 Operatori di comparazione

Gli operatori di comparazione vengono utilizzati per confrontare più operandi e determinare la relazione che c'è tra essi, perciò questi operatori vengono detti anche operatori di relazione.

Supponiamo x = 10 e y = 5.

| **OPERATORI DI COMPARAZIONE** | | |
|---|---|---|
| Minore | `<` | `x < 20  True` |
| Maggiore | `>` | `x > 11  False` |
| Diverso | `!=` | `x != y  True` |
| Uguale | `==` | `x == y  False` |
| Minore o uguale | `<=` | `x <= 10 True` |
| Maggiore o uguale | `>=` | `y >= 5  True` |

### 4.2.3 Operatori logici

Gli operatori logici vengono utilizzati principalmente per combinare più condizioni, possiamo usare come esempio gli operatori di comparazione appena visti:

Supponiamo x = 10.

| **OPERATORI LOGICI** | | |
|---|---|---|
| `True` se tutte le condizioni indicate sono vere. | `and` | `x > 9 and x < 11 True` |
| `True` se almeno una delle condizioni indicate è vera. | `or` | `x > 5 or x < 9  True` |
| inverte il risultato atteso | `not` | `not x > 5  False` |

### 4.2.4 Operatori di assegnazione

Questi operatori ci permettono di assegnare dei valori alle variabili.

| OPERATORI DI ASSEGNAZIONE | | |
|---|---|---|
| assegna all'operando di sinistra il valore dell'operando di destra. | `=` | `x = 5` |
| assegna all'operando di sinistra la somma tra esso e l'operando di destra. | `+=` | `x += 5`<br>`(x = x + 5)` |
| assegna all'operando di sinistra la differenza tra esso e l'operando di destra. | `-=` | `x -= 5`<br>`(x = x - 5)` |
| assegna all'operando di sinistra la moltiplicazione tra esso e l'operando di destra. | `*=` | `x *= 5`<br>`(x = x * 5)` |
| assegna all'operando di sinistra la divisione tra esso e l'operando di destra. | `/=` | `x /= 5`<br>`(x = x / 5)` |
| assegna all'operando di sinitra la divisione arrotondata tra esso e l'operando di destra. | `//=` | `x //= 5`<br>`(x = x // 5)` |
| assegna all'operando di sinistra il resto ottenuto dividendo esso per l'operando di destra. | `%=` | `x %= 5`<br>`(x = x % 5)` |
| assegna all'operando di sinistra il numero avente esso per base e l'operando di destra come esponente. | `**=` | `x **= 5`<br>`(x = x ** 5)` |

### 4.2.5 Operatori di membership

Gli operatori di membership vengono utilizzati per verificare se un valore è presente all'interno di una sequenza come una *lista*, una *stringa*, una *tupla* o un *dizionario* (affronteremo in dettaglio queste strutture dati nel capitolo 7)

Consideriamo una stringa `x = "ciao"`.

| OPERATORI DI MEMBERSHIP |
|---|

| | | |
|---|---|---|
| `True` se l'operando di sinistra è presente nella sequenza indicata | `in` | `"a" in x  True` |
| `True` se l'operando di sinistra non è presente nella sequenza indicata. | `not in` | `"o" not in x False` |

### 4.2.6 Operatori di identità

Gli operatori di identità vengono usati per verificare se due operandi sono uguali (se si riferiscono allo stesso oggetto), ovvero se puntano alla stessa locazione di memoria.

Attenzione: il fatto che due operandi abbiano lo stesso valore non implica che essi siano uguali.

Supponiamo x = 10 e y=10.0 .

| **OPERATORI DI IDENTITÀ** | | |
|---|---|---|
| True se i due operandi sono uguali. | `is` | `x is y  False` |
| `True` se i due operandi non sono uguali (non si riferiscono allo stesso oggetto). | `is not` | `x is not y True` |

### 4.2.7 Operatori bitwise

Gli operatori bitwise servono per confrontare i valori binari degli operandi e operano bit a bit.

Supponiamo x = 4 (0000 0100 in binario) e y = 2 (0000 0010 in binario).

| **OPERATORI BITWISE** | | |
|---|---|---|
| confronta bit per bit dei due operandi e assegna 1 al risultato finale quando entrambi sono a 1 (**bitwise and**). | `&` | `x & y = 0`<br>`(0000 0000)` |
| confronta bit per bit dei due operandi e assegna 1 al risultato finale quando almeno uno dei due bit a confronto è a 1 (**bitwise or**). | `\|` | `x \| y = 5`<br>`(0000 0110)` |
| operatore unario che inverte i valori binari (**bitwise not**). | `~` | `~x = -5`<br>`(1111 1001)` |

| | | |
|---|---|---|
| confronta bit per bit dei due operandi e assegna 1 al risultato finale quando solo uno dei due bit a confronto è a 1 (**bitwise xor**). | ^ | x ^ y = 6<br>(0000 0110) |
| vengono spostati (shiftati) i bit verso sinistra in base al numero indicato. | << | x << 3 = 32<br>(0010 0000) |
| vengono spostati (shiftati) i bit verso destra in base al numero indicato. | >> | x >> 2 = 1<br>(0000 0001) |

## 4.3. Commento

Scriviamo il seguente programma:

```
# questo è un commento
print "prova commento" # anche questo è un commento
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
prova commento
```

Il carattere '#' indica che tutto ciò che è presente sulla destra viene considerato commento sino a fine riga.

## 4.4 Variabili

Scriviamo il seguente programma:

```
a= 123.4
b23='pippo'
b=432
c=a+b
d=1
nome='carlo'
print ("a+b= ",c)
print ("a+d= ",a+d)
print ('il nome è ', nome)
print (nome+b23)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
a+b= 555.4
a+d= 124.4
il nome è carlo
carlopippo
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Non è necessario dichiarare il tipo di variabili come altri linguaggi di programmazione. L'interprete python riconosce il tipo di variabile quando si effettua l'assegnazione. Questo non sempre è un vantaggio, non dichiarando le variabili si rischia di scrivere un sorgente poco leggibile. Per tali motivi si consiglia di usare abbondantemente il commento per descrivere il significato delle variabili in gioco e in generale la struttura del codice sorgente.
- ➔ I nomi delle variabili possono essere alfanumerici o alfabetici, nel primo caso il primo carattere deve essere alfabetico.
- ➔ Un'espressione viene valutata quando richiesto e gli operandi assumono i valori che hanno al momento della valutazione.
- ➔ I valori delle variabili stringhe possono essere delimitati da apici o doppi apici.
- ➔ '+' è l'operatore che ci permette di concatenare tra loro due o più stringhe alfanumeriche

## 4.5 Strutture di controllo flusso

Il python ha diverse strutture per controllare il flusso delle informazioni in un algoritmo. Noi ne studieremo tre ed ogni volta confronteremo il programma sorgente con il flow-chart.

### 4.5.1 Ciclo `while`

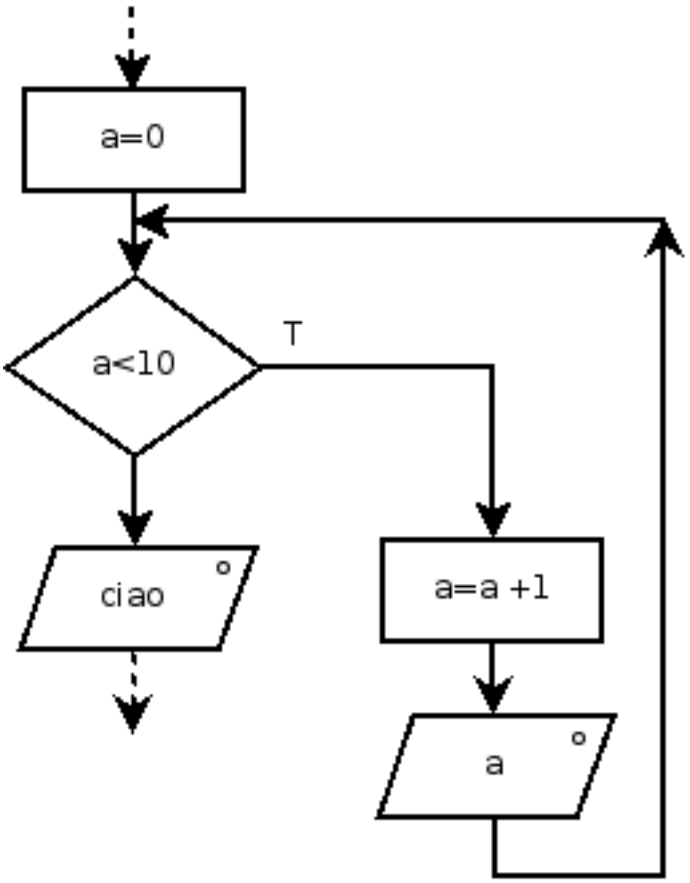
Scriviamo il seguente programma:

```
a = 0
while a < 10 :
      a = a + 1
      print(a)
print ("fine ciclo while")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
3
4
5
6
7
8
9
10
fine ciclo while
```

Il flow-chart dell'algoritmo è il seguente:




Dall'analisi del codice e dell'output osserviamo che:

- ➔ non esiste un carattere per indicare l'inizio e la fine del corpo del loop. L'interprete riconosce il corpo del loop dall'indentazione dello stesso rispetto all'istruzione while.
- ➔ Dopo la condizione di test vanno inseriti obbligatoriamente "**:**". Questi introducono l'inizio del loop.

➔ La prima istruzione eseguita dopo il ciclo while e' la prima istruzione indentata allo stesso livello dell'istruzione while.

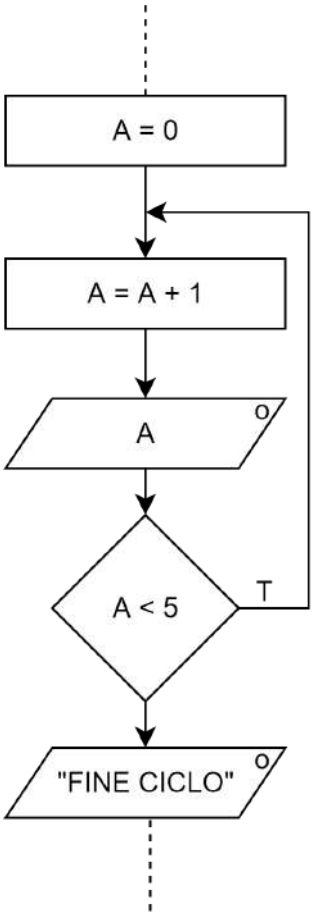
Solitamente per implementare gli algoritmi di calcolo numerico è piu' utile lavorare con i cicli while perche' piu' versatili.

### 4.5.2 Ciclo `do - while`

Il costrutto di flusso `while` è un ciclo con precondizione, cioè si controlla che la condizione sia verificata e successivamente si esegue il corpo del loop.
Caratteristica peculiare di questa tipologia di iterazione è che in alcuni casi potrebbe accadere che il corpo del loop non venga mai eseguito. Questo succede quando la condizione non viene soddisfatta subito la prima volta che si controlla la stessa.

Un altro tipo di costrutto è il `do-while`, che è un ciclo postcondizionale, cioè la condizione viene verificata dopo aver eseguito il corpo del loop.
Caratteristica peculiare di questa tipologia di iterazione è che venga sempre eseguito, almeno una volta, il corpo del loop.

Il diagramma di flusso del ciclo do-while è il seguente:




**Il costrutto `do - while` nel linguaggio Python non esiste !**

Ma non è un grosso problema in quanto può essere implementato utilizzando il solo costrutto `while`.

Scriviamo il seguente programma:

```
a = 0
a = a + 1
print(a)
while(a<5):
      a = a + 1
      print(a)
print("FINE CICLO")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
3
4
5
FINE CICLO
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Innanzitutto ci rendiamo conto che l'istruzione di azzeramento della variabile `A` non serve in quanto viene subito riassegnata
- ➔ Per riprodurre il ciclo do-while abbiamo ricopiato le istruzioni del lopp subito sopra l'istruzione `while`

Potevamo scrivere il codice in altro modo, ottimizzandolo. Infatti le prime 3 istruzioni possono essere sostituite solo dall'istruzione `print(1)`.
Quindi analizzando il diagramma di flusso possiamo di volta in volta riscrivere un loop do-while con un while comprendendo la logica. Ma se vogliamo convertire in maniera automatica un `do - while` con un `while` possiamo eseguire le istruzioni all'interno del loop subito prima dell'istruzione `while` senza preoccuparci di comprendere la logica del loop stesso.

### 4.5.3 Ciclo `for`

I cicli `for` permettono a un programma di eseguire un'operazione diverse volte come il ciclo `while`.
Ma permettono anche di iterare sugli elementi di una sequenza.

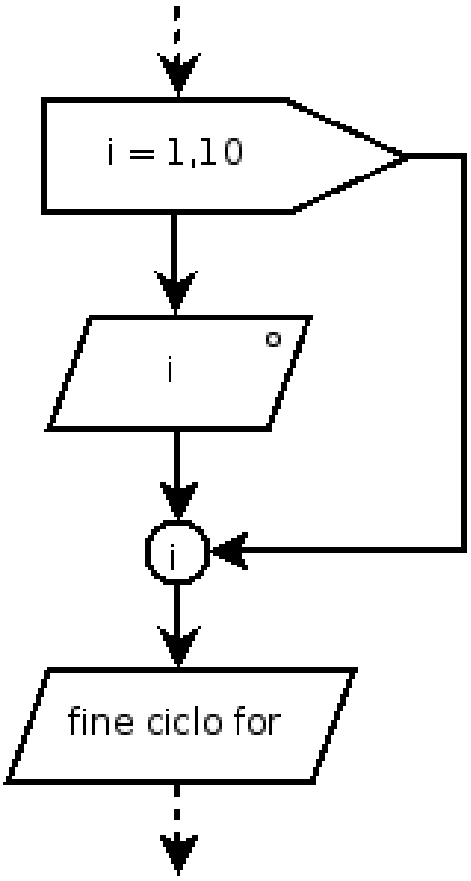
Scriviamo il seguente programma:

```
for i in range(1,11):
      print(i)
print ("fine ciclo for")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
3
4
5
6
7
8
9
10
fine ciclo for
```

Il flow-chart dell'algoritmo è il seguente:




Dall'analisi del codice e dell'output osserviamo che:

➔ non esiste un carattere per indicare l'inizio e la fine del corpo del loop. L'interprete riconosce il corpo del loop dall'indentazione dello stesso rispetto all'istruzione while.

➔ Dopo la condizione di test vanno inseriti obbligatoriamente "**:**". Questi introducono l'inizio del loop.
➔ La prima istruzione eseguita dopo il ciclo for e' la prima istruzione indentata allo stesso livello dell'istruzione for.
➔ *`range`* e' una funzione che restituisce una successione di numeri compresi tra il primo e l'ultimo escluso aventi passo unitario. La sintassi generale e' la seguente:

```
range (inizio,fine,step)
```

dove
- `inizio = numero di partenza della successione`
- `fine   = numero finale della successione escluso`
- `step   = passo della successione`

p.e. `range(1,4,2)` fornira' la seguente successione: `1,3`

Ovviamente possiamo costruire successioni decrescenti con inizio > fine e step <0 , p.e. `range(4,1,-1)` dara' origine alla seguente successione: `4,3,2`

Se omettiamo `inizio` e `step` `range` darà origine a una successione che parte da 0 e termina con `fine -1` con `step=1`
p.e. `range(4)` dara' origine alla seguente successione: `0,1,2,3`

#### 4.5.3.1 Oggetti iterabili

L'istruzione `range` in pratica ci permette di costruire una successione di valori che vengono di volta in volta assegnati alla variabile indice del for. Più o meno come avviene in altri linguaggi di programmazione come il linguaggio C.
In realtà la funzione `range` crea un *oggetto iterabile* che rappresenta una sequenza di valori interi.

Più in generale gli oggetti iterabili possono essere visti come dei contenitori di altri oggetti, che possono essere estratti uno dopo l'altro in sequenza. L'operatore che effettua l'estrazione si chiama *iteratore*, lo approfondiremo più avanti in questo testo.

Oggetti iterabili per antonomasia sono le stringhe che possono essere viste come sequenze di caratteri ( per un approfondimento vai al capitolo 7.1 Le stringhe ). Ma lo sono anche le liste, le tuple, i dizionari e gli insiemi, praticamente tutte le strutture dati affrontate nel capitolo 7 delle strutture dati.

#### 4.5.3.2 La struttura generale del ciclo `for`

La struttura generale del ciclo for è la seguente:

```
for <variabile> in <sequenza> :
     corpo del ciclo
```

dove:
➔ `<variabile>` è la variabile che conterrà l'oggetto estratto dalla sequenza

➔ `<sequenza>` è la successione di oggetti dell'iterabile a cui si sta facendo riferimento

Esaminiamo il seguente programma:

```
for carattere in "PAROLA":
     print (carattere)
print ("fine ciclo for")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
P
A
R
O
L
A
fine ciclo for
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Confrontando il codice con il precedente, notiamo che la funzione `range()` è stata sostituita con una stringa, questo ci fa comprendere come sia la funzione range() che la stringa siano oggetti simili che "esprimono" una sequenza.

➔ Nella variabile carattere ciene di volta in volta caricato l'oggetto estratto dall'oggetto iterabile che segue la keyword `in`. Tale oggetto è un carattere

➔ Nei costrutti for la variabile indice non rappresenta l'indice del `for` ma, più in generale, è l'oggetto che di volta in volta viene estratto dalla sequenza. Vale a dire che di volta in volta è un oggetto sempre differente, non è un intero che rappresenta l'indice come nel caso del ciclo for di alcuni linguaggi di programmazione. È più simile all'istruzione *foreach* che si incontra in altri linguaggi di programmazione.

Altri esempi di applicazione dei cicli `for` sugli oggetti iterabili li incontreremo sempre nel capitolo 7 delle strutture dati.

### 4.5.4 Costrutto `if`

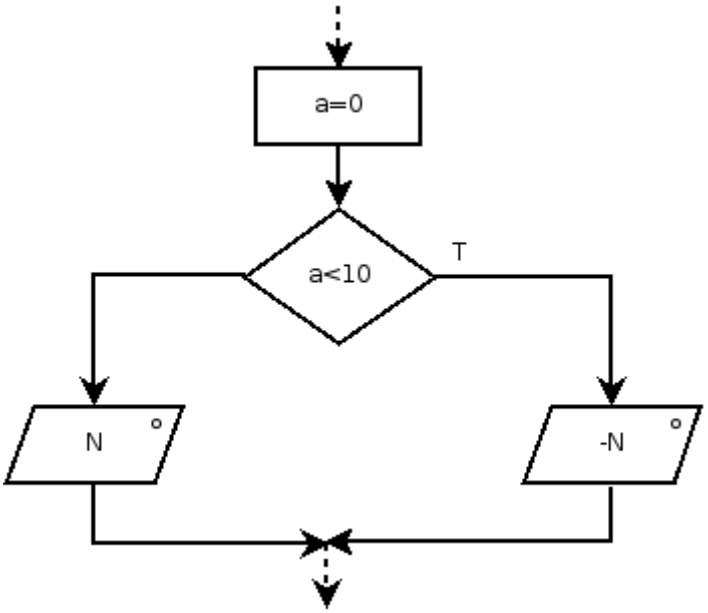
Scriviamo il seguente programma:

```
n=10
if n < 0 :
     print ("Il valore assoluto di ",n," e' ",-n)
else:
     print ("Il valore assoluto di ",n," e' ",n)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Il valore assoluto di 10 e' 10
```

Il flow-chart dell'algoritmo è il seguente:




Dall'analisi del codice e dell'output osserviamo che:

- ➔ Anche in questo caso il corpo dell'if viene indicato con l'indentazione così come quello dell'istruzione else.
- ➔ Sia la condizione dell'if che l'istruzione else terminano con ":", questi vanno inseriti obbligatoriamente.

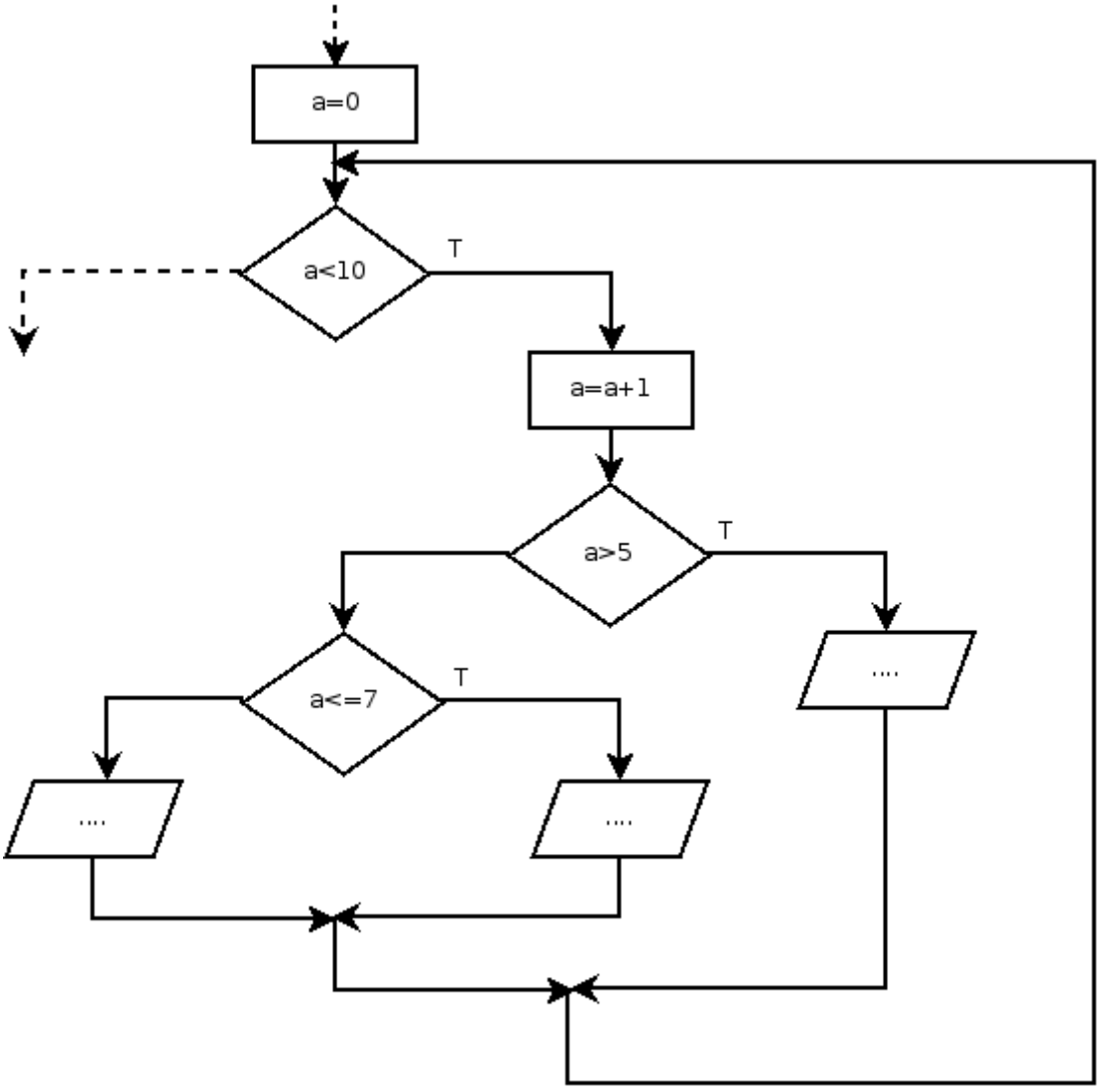
Ora esaminiamo un esempio leggermente piu' complesso.
Scriviamo il seguente programma:

```
a=0
while a < 10:
     a=a+1
     if a > 1 :
          print (a," > ",5)
     elif a <= 7:
          print (a," <= ",7)
     else:
          print ("nessuna condizione risulta verificata")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1 <= 7
2 <= 7
3 <= 7
4 <= 7
5 <= 7
6 > 5
7 > 5
8 > 5
9 > 5
10 > 5
```

Il flow-chart dell'algoritmo è il seguente:




Dall'analisi del codice e dell'output osserviamo che:

➔ L'istruzione "elif" ci permette di semplificare la scrittura del codice nel caso in cui abbiamo più test in cascata.
➔ Possiamo avere più livelli di indentazione, dipende dalla struttura e dalla complessità del nostro algoritmo.

### 4.5.5 Operatori di confronto e operatori logici

Nel capitolo 4.2 dedicato agli operatori abbiamo incontrato gli operatori di confronto e gli operatori logici.
Gli operatori di confronto sono quelli che solitamente utilizziamo per costruire le condizioni di test del ciclo while e dell'if in quanto ci consentono di mettere a confronto due oggetti.

Gli operatori logici ci permettono di costruire condizioni di test complesse mettendo in relazione espressioni logiche (cioè le condizioni di test ).

Come esempio consideriamo il problema di individuazione di un anno bisestile. Un anno è bisestile se è divisibile per 4 ma non per 100, oppure se è divisibile per 400 (ad esempio il 1900 non è stato bisestile, mentre il 2000 lo è stato). Dato in input un anno vogliamo sapere se questo sia bisestile oppure no.

L'algoritmo che risolve il problema è descritto dal seguente flow-chart:

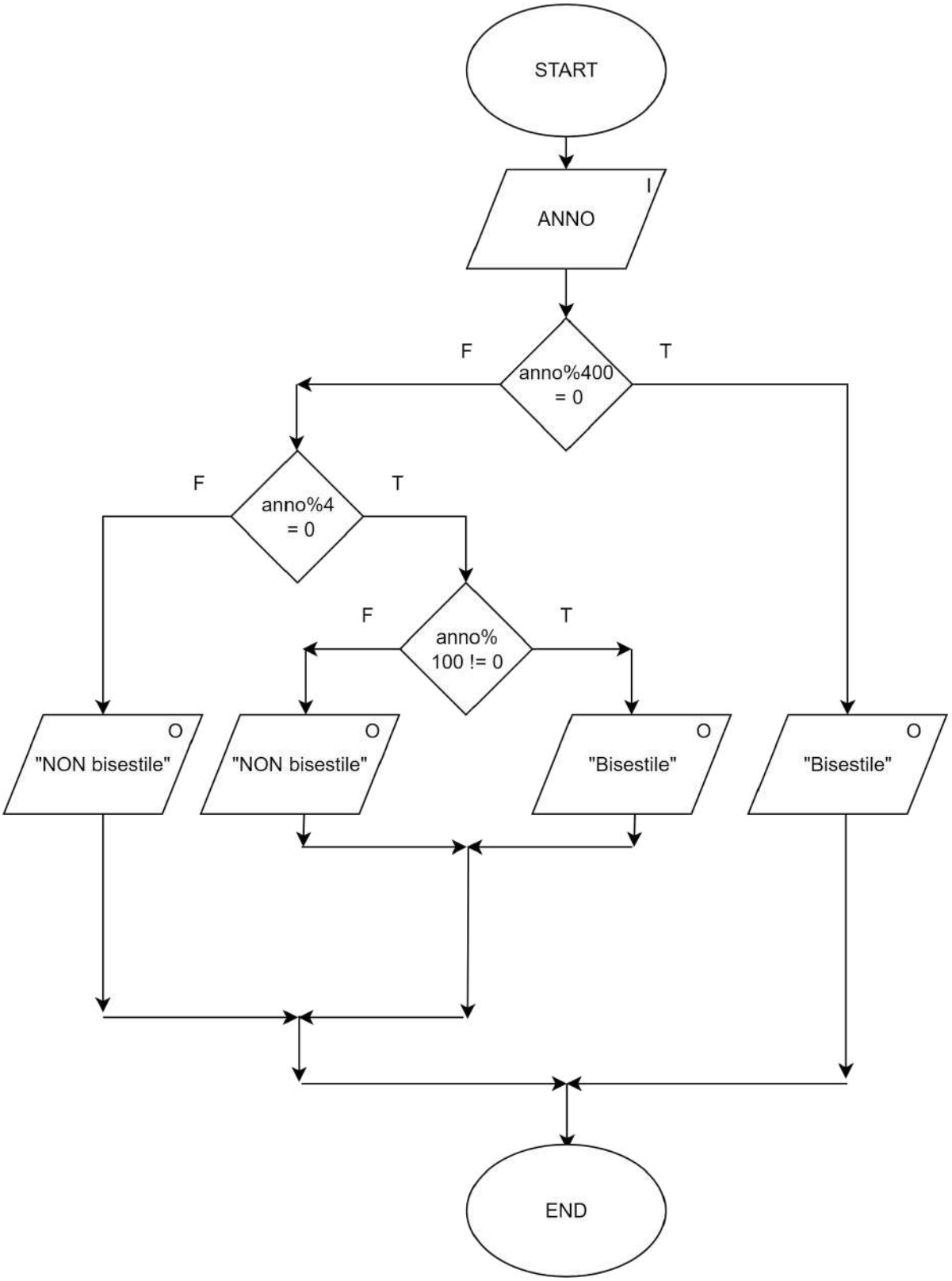



Figura 4.1

Come esercizio si esegua la codifica del precedente flow-chart.
Dopo aver appurato che riconosca correttamente gli anni bisestili, il lettore provi a riscrivere l'algoritmo utilizzando gli operatori logici (o connettivi logici).
L'algoritmo è più semplice o più complicato?

Di seguito l'algoritmo con i connettori logici:

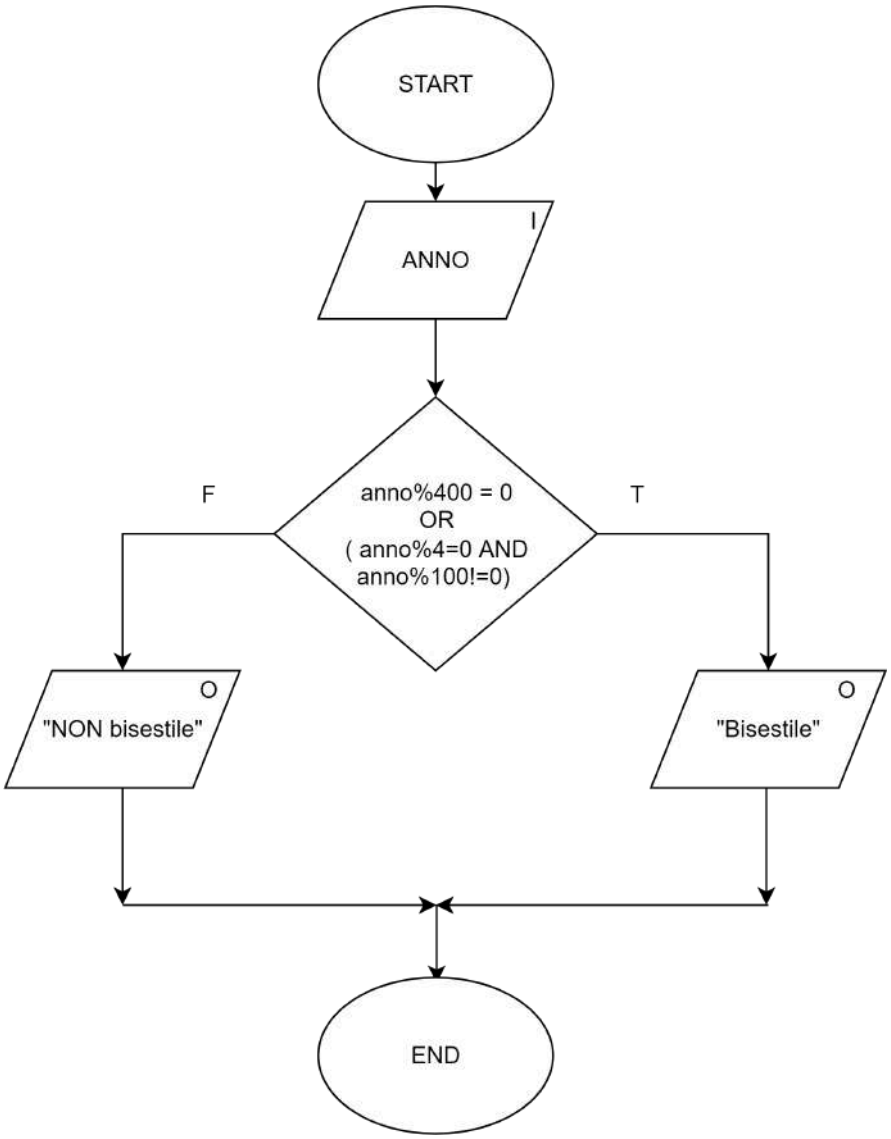



Figura 4.2

Come si può notare l'algoritmo sembra essere molto più semplice. In effetti il numero di blocchi è minore, tutta la complessità si "nasconde" nella selezione.

È comunque fuori dubbio che la codifica risulta essere molto più semplice dell'algoritmo senza i connettivi logici. Si può verificare questa osservazione confrontando i seguenti codici:

L'algoritmo di Figura 4.1 si codifica come segue:

```
anno= int(input("Inserisci anno: "))

if(anno%400==0):
    print("Anno bisestile")
elif (anno%4==0):
```

```
        if(anno%100!=0):
            print("Bisestile")
        else:
            print("NON Bisestile")
    else:
        print("NON Bisestile")
```

L'algoritmo di Figura 4.2 si codifica come segue:

```
anno= int(input("Inserisci anno: "))

if anno%400==0 or (anno%4 == 0 and anno%100!=0):
    print("Anno bisestile")
else:
    print("NON Bisestile")
```

### 4.5.6 Le istruzioni `break` e `continue`

Le istruzioni `break` e `continue` permettono di alterare il flusso di esecuzione delle istruzioni in un ciclo.
La loro introduzione snellisce la scrittura del codice aumentandone la leggibilità.
Personalmente sconsiglio l'uso intensivo di queste istruzioni in quanto l'abuso può portare all'effetto opposto di rendere poco chiaro e comprensibile il codice in cui sono state inserite.
In generale si può sempre evitare di utilizzare `break` e `continue` inserendo opportunamente le variabili flag e l'istruzione `if`.

#### 4.5.6.1 `Break`

Si utilizza all'interno di un ciclo. Il suo scopo è quello di terminare il ciclo andando a eseguire l'istruzione subito successiva al ciclo stesso in cui è presente il `break`.
L'istruzione agisce a livello di ciclo più interno. Se l'istruzione è situata all'interno di un ciclo che a sua volta è annidato all'interno di un altro ciclo, `break` permetterà di uscire solo dal ciclo più interno.

Come esempio consideriamo il seguente codice:

```
for i in range(10):
   print(i)
   if(i==5):
       break
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
0
1
2
3
4
5
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Il programma esce dal ciclo for non appena incontra l'istruzione `break`

#### 4.5.6.2 Continue

Si utilizza all'interno di un ciclo. Consente di passare all'iterazione successiva.

Consideriamo il seguente codice:

```
for i in range(5):
    if(i==3):
        continue
    print(i)
print ("fine ciclo for")

for i in range(5):
    if(i!=3):
        print(i)
print ("fine ciclo for")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
0
1
2
4
fine ciclo for
0
1
2
4
fine ciclo for
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Quando la variabile `i` assume il valore 3 allora viene eseguita l'istruzione `continue` che interrompe l'esecuzione delle istruzioni che completano il corpo del ciclo for.

➔ Il ciclo successivo produce lo stesso identico output senza utilizzare l'istruzione

`continue`.

### 4.5.7 Approfondimento sui loop: l'istruzione `else`

Un aspetto poco noto di Python, ma molto interessante, è la possibilità di associare una clausola `else` ai cicli `for` e `while`.

A differenza di quanto si potrebbe pensare in base all'uso che se ne fa con l'istruzione `if`, l'`else` applicato a un ciclo non ha il significato di: *altrimenti*, bensì quello di: *se il ciclo è terminato normalmente, senza essere stato interrotto da un break*.

In altre parole, il blocco di codice contenuto nell'`else` viene sempre eseguito al termine del ciclo, a meno che il ciclo non venga interrotto anticipatamente tramite un'istruzione `break`. Se invece il ciclo viene interrotto da un `break`, l'`else` viene saltato e non viene eseguito.

Analizziamo il seguente codice:

```
for i in range(1,4):
    print(i)
else:
    print("fine")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
3
fine
```

Consideriamo ora lo stesso comportamento applicato a un ciclo while:

```
i=1
while i<4 :
    print(i)
    i+=1
else:
    print("fine")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
3
fine
```

Dall'analisi di questi primi due esempi osserviamo che:

- ➔ In entrambi i casi il ciclo si conclude senza incontrare alcun `break`, portando a termine tutte le iterazioni previste.
- ➔ Di conseguenza, in entrambi i casi viene eseguita anche l'istruzione contenuta nel blocco `else`, che stampa "`fine`".

Vediamo ora cosa succede se all'interno dei cicli inseriamo un break.

```
for i in range(1,4):
    print(i)
    if i==2:
        break
else:
    print("fine")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
```

Ripetiamo lo stesso esperimento con il ciclo while:

```
i=1
while i<4 :
    print(i)
    if i==2:
        break
    i+=1
else:
    print("fine")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1
2
```

Dall'analisi di questi ultimi due esempi osserviamo che:

- ➔ Quando la variabile `i` assume il valore 2 viene eseguita l'istruzione `break`, che interrompe immediatamente il ciclo.
- ➔ Poiché il ciclo non si è concluso "naturalmente", ma è stato interrotto forzatamente, l'istruzione contenuta nel blocco `else` non viene eseguita e la stringa "fine" non compare in output.

In sintesi, l'`else` applicato a un ciclo risulta particolarmente utile quando si desidera eseguire un'operazione solo nel caso in cui il ciclo sia arrivato regolarmente alla fine delle sue iterazioni, senza essere stato interrotto da condizioni particolari intercettate tramite un break.
Un caso d'uso tipico è, ad esempio, la ricerca di un elemento all'interno di una sequenza: se l'elemento viene trovato si esce dal ciclo con un `break`, mentre l'`else` permette di gestire in modo elegante il caso in cui la ricerca non abbia dato esito positivo.

### 4.5.8 Costrutto match-case

A partire dalla versione 3.10, Python introduce un nuovo costrutto di controllo di flusso chiamato `match-case`, che permette di confrontare il valore di un'espressione con una serie di pattern (modelli), eseguendo il blocco di codice associato al primo pattern che risulta soddisfatto.
Chi ha già esperienza con altri linguaggi di programmazione può notare una certa somiglianza con l'istruzione `switch-case` presente, ad esempio, in *C*, *Java* o *JavaScript*. Tuttavia il `match-case` di Python è concettualmente più potente, in quanto non si limita a confrontare valori semplici ma consente il cosiddetto pattern matching, cioè il confronto strutturale di oggetti anche complessi (*liste, tuple, dizionari*), come vedremo più avanti nel testo.

Scriviamo il seguente programma:

```
giorno = 3

match giorno:
    case 1:
        print("Lunedì")
    case 2:
        print("Martedì")
    case 3:
        print("Mercoledì")
    case 4:
        print("Giovedì")
    case 5:
        print("Venerdì")
    case 6 | 7:
        print("Weekend")
    case _:
        print("Giorno non valido")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Mercoledì
```

Dall'analisi del codice e dell'output osserviamo che:

- L'istruzione `match` viene seguita dall'espressione di cui vogliamo confrontare il valore, e come nel `while` e nel `for`, la riga si chiude obbligatoriamente con "`:`".
- Ogni possibile confronto viene introdotto dalla parola chiave `case`, seguita dal pattern da confrontare e da "`:`". Il corpo di ognuno dei case viene individuato tramite l'indentazione.
- L'interprete confronta il valore di `giorno` con i vari *pattern* nell'ordine in cui sono scritti, e appena trova una corrispondenza esegue il blocco di istruzioni associato, senza controllare gli altri case successivi (non serve quindi un'istruzione equivalente al `break` di altri linguaggi).
- Il simbolo "`|`" (*pipe*) permette di associare più valori a uno stesso `case`, come si vede nella riga `case 6 | 7`, che raggruppa sabato e domenica.
- Il carattere jolly "`_`" (*underscore*) viene utilizzato come ultimo `case` per intercettare tutti i valori che non hanno trovato corrispondenza in nessuno dei case precedenti. Svolge quindi un ruolo simile a quello del `default` di altri linguaggi o dell'`else` dell'istruzione `if`.
- Se nessun case corrisponde e non è presente il wildcard "`_`", il costrutto `match` non esegue alcuna istruzione e il programma prosegue normalmente con l'istruzione successiva.

#### 4.5.8.1 Il match-case con condizioni aggiuntive (*guard*)

È possibile arricchire un pattern con una condizione aggiuntiva, chiamata *guard*, introdotta dalla parola chiave `if`. Il `case` in questione viene considerato valido solo se, oltre alla corrispondenza del pattern, risulta vera anche la condizione specificata nella *guard*.

Analizziamo il seguente programma:

```
voto = 7

match voto:
    case n if n >= 6:
        print("Promosso")
    case n if n < 6:
        print("Bocciato")
```

```
Promosso
```

Dall'analisi del codice e dell'output osserviamo che:

- Il pattern non è un valore fisso ma una variabile (`n`), viene di volta in volta associata al valore contenuto in voto. Si tratta di una forma di cattura del valore, concettualmente simile a quanto avviene con la variabile indice nel ciclo `for`.
- La *guard* `if n >= 6` viene valutata solo dopo che il pattern `n` ha trovato corrispondenza (cosa che avviene sempre, trattandosi di una semplice variabile), e determina se il case debba essere effettivamente eseguito.

Nell'esempio precedente il pattern era molto semplice in quanto si limitava a catturare un valore in una variabile (`n`).
Ora vediamo un esempio in cui il pattern verifica anche una struttura. Scriviamo, quindi, il seguente programma:

```
punto = (3, 3)

match punto:
    case (0, 0):
        print("Il punto coincide con l'origine")
    case (x, y) if x == y:
        print("Il punto si trova sulla bisettrice")
    case (x, y):
        print("Punto generico di coordinate:", x, y)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Il punto si trova sulla bisettrice
```

Dall'analisi del codice e dell'output osserviamo che:

- Il valore da confrontare, in questo caso, non è un semplice numero o una stringa, ma una struttura dati che studieremo in seguito. Si tratta della struttura dati `tupla` che in questo caso si compone di due elementi: `punto = (3, 3)`.
- Il primo `case (0, 0)`, è un pattern costante composto. La corrispondenza si verifica solo se `punto` è esattamente la `tupla (0, 0)`.
- Il secondo `case (x, y) if x == y`, è quello che ci interessa maggiormente: il pattern (x, y) verifica innanzitutto che punto sia una tupla di due elementi (in caso contrario il case verrebbe scartato a prescindere dalla guard), e contestualmente cattura il primo elemento in x e il secondo in y. Solo dopo che questa corrispondenza strutturale è avvenuta, viene valutata la guard if x == y.
- A differenza dell'esempio precedente, qui la guard non si limita a confrontare un valore già catturato per intero, ma lavora su una corrispondenza che dipende anche dalla forma dell'oggetto: se punto fosse stato, ad esempio, una tupla di tre elementi (p.e. `(3,4,8)`), il `case (x, y)` non avrebbe trovato corrispondenza, indipendentemente dal valore della guard.
- Il terzo `case (x, y)`, privo di guard, funge da caso generico per tutte le altre coppie di coordinate che non soddisfano le condizioni precedenti.

Questo esempio mostra quindi la differenza sostanziale tra una guard applicata a un pattern semplice (come la variabile n dell'esempio precedente, che corrisponde sempre) e una guard applicata a un pattern strutturato (come la tupla (x, y)), dove la guard entra in gioco solo se la struttura dell'oggetto rispetta prima il pattern indicato.

In generale la modalità di utilizzo del `match-case` con le *guard* è particolarmente utile quando i pattern semplici (valori fissi o wildcard) non sono sufficienti a esprimere la condizione di test che vogliamo verificare.

#### 4.5.8.3 Pattern matching su strutture dati

Le vere potenzialità del costrutto `match-case` si manifestano quando i pattern non si limitano a confrontare valori semplici, ma verificano la forma di strutture dati più complesse, catturandone contestualmente il contenuto in nuove variabili.

Vediamo alcuni esempi applicati alle principali strutture dati che incontreremo nel capitolo 7.

##### 4.5.8.3.1 Pattern matching sulle liste

Scriviamo il seguente programma:

```
lista = [1, 2, 3]

match lista:
    case []:
        print("Lista vuota")
    case [x]:
        print("Lista con un solo elemento:", x)
    case [x, y]:
        print("Lista con due elementi:", x, y)
    case [x, y, *resto]:
        print("Primo:", x, "Secondo:", y, "Resto:", resto)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Primo: 1 Secondo: 2 Resto: [3]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Ogni `case` verifica sia la lunghezza della lista e, contestualmente, ne cattura gli elementi in altrettante variabili: il `case []` corrisponde solo alla lista vuota, il `case [x]` solo a una lista di un elemento, e così via.
- ➔ Il simbolo `*` (già incontrato nel capitolo dedicato alle funzioni, a proposito del numero indefinito di argomenti) permette di catturare in un'unica variabile tutti gli elementi restanti della lista, qualunque sia il loro numero. Nel nostro esempio `*resto` cattura tutti gli elementi dal terzo in poi, restituendo una nuova lista.
- ➔ L'interprete confronta i `case` nell'ordine in cui sono scritti: poiché la nostra lista ha tre elementi, i primi tre case vengono scartati (non corrispondono per lunghezza) e viene eseguito il quarto.

##### 4.5.8.3.2 Pattern matching sulle tuple

Il funzionamento è analogo a quello delle liste, con la sola differenza della sintassi

(parentesi tonde anziché quadre), come già visto nell'esempio del paragrafo precedente con le coordinate di un punto.
Approfondiamo qui un caso con tuple di lunghezza variabile:

```
comando = ("sposta", 10, 20)

match comando:
    case ("saluta",):
        print("Ciao!")
    case ("sposta", x, y):
        print("Spostamento a coordinate:", x, y)
    case ("ruota", angolo):
        print("Rotazione di", angolo, "gradi")
    case _:
        print("Comando sconosciuto")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Spostamento a coordinate: 10 20
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Ogni case verifica sia il primo elemento della tupla (una sorta di "etichetta" del comando), sia il numero di elementi successivi, catturandoli in altrettante variabili.
- ➔ Questo schema è molto diffuso quando si vogliono rappresentare comandi o messaggi strutturati, in cui il primo elemento identifica il tipo di operazione e gli elementi successivi ne rappresentano i parametri.

#### 4.5.8.3.3 Pattern matching sui dizionari

Analizziamo il seguente programma:

```
utente = {"nome": "Marco", "eta": 30, "citta": "Roma"}

match utente:
    case {"nome": n, "eta": e} if e >= 18:
        print(n, "è maggiorenne")
    case {"nome": n, "eta": e}:
        print(n, "è minorenne")
    case _:
        print("Dati non validi")
```

Lanciamo il codice, l'output stampato a video è il seguente:

```
Marco è maggiorenne
```

Dall'analisi del codice e dell'output osserviamo che:

- A differenza di quanto avviene con *liste* e *tuple*, un pattern applicato a un dizionario non richiede la corrispondenza di tutti gli elementi presenti: il pattern `{"nome": n, "eta": e}` corrisponde a utente anche se quest'ultimo contiene ulteriori chiavi (come `"citta"`), purché siano presenti almeno le chiavi `"nome"` ed `"eta"`.
- Ogni chiave indicata nel pattern viene verificata singolarmente, mentre il valore associato viene catturato nella variabile indicata (`n` ed `e` nel nostro esempio).
- Anche in questo caso è possibile arricchire il pattern con una *guard*, come abbiamo fatto nella riga `if e >= 18`, per aggiungere condizioni sui valori catturati.

#### 4.5.8.3.4 Pattern matching sulle stringhe

Possiamo effettuare il confronto confronto di una stringa con dei valori costanti.
È possibile però anche verificare che una variabile sia di tipo stringa, catturandone il contenuto, tramite un pattern di classe.
Esaminiamo, a tale scopo, il seguente codice:

```
valore = "Milano"

match valore:
    case str() as s:
        print(s, "è una stringa")
    case _:
        print(valore, "Non è una stringa")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Milano è una stringa
```

Dall'analisi del codice e dell'output osserviamo che:

- Il pattern `str()` non confronta valore con una stringa specifica, ma ne verifica il tipo. La corrispondenza si verifica per qualunque stringa, indipendentemente dal suo contenuto.
  - * `str()` è la classe che definisce il tipo (oggetto) stringa. Nel `case`, quindi, possiamo inserire le classi che ci permettono di individuare se un'oggetto appartiene a quella classe oppoure no (approfondiremo il concetto di classe e di oggetto più avanti).
- La clausola as s cattura l'intero oggetto (in questo caso la stringa stessa) nella variabile s.

#### 4.5.8.3.5 Pattern matching sugli insiemi

Gli *insiemi* (che approfondiremo nel capitolo 7) rappresentano un caso particolare, a differenza di *liste* e *tuple*, non essendo strutture ordinate, il `match-case` non consente di

catturarne singolarmente gli elementi in base alla posizione.
È comunque possibile verificarne il tipo e, tramite le *guard*, applicare condizioni sul contenuto.
Esaminiamo il seguente programma:

```
gruppo = {1, 2, 3}

match gruppo:
    case set() as s if len(s) == 0:
        print("Insieme vuoto")
    case set() as s if 2 in s:
        print("L'insieme contiene il valore 2")
    case set() as s:
        print("Insieme generico:", s)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
L'insieme contiene il valore 2
```

Dall'analisi del codice e dell'output osserviamo che:

- Anche in questo caso il pattern `set()` verifica il tipo dell'oggetto, mentre la clausola `as s` ne cattura il contenuto nella variabile `s`.
- Non essendo gli insiemi strutture ordinate, la verifica sul contenuto (come la presenza del valore 2) viene delegata interamente alla *guard*, tramite l'operatore `in` già incontrato nel capitolo dedicato agli operatori.

**IN SINTESI:**

il pattern matching su *liste*, *tuple* e *dizionari* consente di verificare contemporaneamente la forma di una struttura dati e di estrarne il contenuto in nuove variabili.

il pattern matching su per *stringhe* e *insiemi*, non essendo strutture "*scomponibili*" allo stesso modo, il `match-case` viene utilizzato principalmente per verificarne il tipo, delegando alle *guard* i controlli più specifici sul contenuto.

# 5. Le funzioni

## 5.1 Cosa sono le funzioni e a cosa servono

Una funzione è una successione di istruzioni che restituisce il valore di un calcolo, in generale esegue un algoritmo e alla quale viene assegnato un nome. Per definire una funzione, si deve specificare il nome e scrivere la serie di istruzioni. Una volta definita la funzione la si potrà "chiamare" mediante il nome che gli è stato assegnato dal programmatore.
Parafrasando Bjarne Stroustrup, il creatore del linguaggio c++: "uno degli scopi primari delle funzioni è quello di rendere comprensibile il codice. Se il codice è comprensibile allora è più facilmente manutenibile ed è anche più esente da errori.
Il primo passo per arrivare alla comprensibilità è quello di spezzare i processi di calcolo in blocchi, appunto le funzioni e dargli un nome. Tali funzioni forniscono quindi il vocabolario di base del calcolo, proprio come i tipi di dati forniscono il vocabolario di base dei dati. Successivamente, possiamo comporre funzioni, rappresentare compiti comuni o specializzati in calcoli più ampi.
Il numero di errori nel codice è fortemente correlato alla quantità di codice e alla complessità del codice. Entrambi i problemi possono essere risolti utilizzando più funzioni e funzioni più brevi. L'utilizzo di una funzione per svolgere un compito specifico spesso ci evita di scrivere un pezzo di codice nel mezzo di altro codice. Rendere questo "pezzo" una funzione ci costringe a nominare l'attività e documentarne le dipendenze. Il consiglio più elementare è quello di mantenere il numero di righe del codice tale per cui di riesca ad averla visualizzata nella sua interezza sullo schermo. I bugs tendono a insinuarsi quando possiamo visualizzare solo una parte di un algoritmo alla volta. Molti programmatori si pongono un limite di circa 40 righe massimo per una funzione. Secondo Stroustrup l'ideale sarebbe ancora una taglia molto più piccola, probabilmente una media di 7 righe."

## 5.2 Definizione di funzione

Scriviamo il seguente programma:

```
a=23
b=-40
def my_abs(num):
     if num < 0 :
          num=-num
     return num
if my_abs(a) >= my_abs(b) :
     print (my_abs(a))
else:
     print (my_abs(b))
```

Lanciamo il programma, l'output stampato a video è il seguente:

40

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'istruzione def permette di definire una funzione, nel nostro caso la funzione si chiama: "my_abs"
- ➔ Non esistono i prototype per le funzioni python, si definisce la funzione e subito dopo il suo corpo.
- ➔ Dopo il def è necessario digitare ":", il corpo della funzione si dovrà scrivere indentato. N.B. Il return fa parte del corpo della funzione e quindi va scritto anch'esso indentato.
- ➔ La lista dei parametri formali viene indicata tra parentesi dopo il nome della funzione.
- ➔ La funzione può essere definita in qualsiasi posto nel codice, l'importante è che sia definita prima che venga chiamata.
- ➔ I valori di ritorno si possono assegnare a delle variabili, ma si possono anche richiamare le funzioni direttamente da altre istruzioni come, p.e., l'istruzione di selezione `if` o l'istruzione di stampa `print`.

Vediamo altri esempi di definizione di funzione:

```
def ciao():
     print ("Ciao mondo!")

ret = ciao()
print (ret)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Ciao mondo!
None
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ In questo caso la funzione non accetta argomenti, quindi sia nella definizione che nella chiamata si hanno solo le parentesi tonde aperte e chiuse
- ➔ La funzione non restituisce nessun valore dato che l'istruzione return non appare.
- ➔ Lanciando il programma l'istruzione di assegnazione esegue la funzione ed a video appare l'argomento della print, alla variabile *a* non viene assegnato alcun valore. Se si stampa a si ottiene:"None" che indica valore "vuoto".

Altro esempio in cui abbiamo una funzione che accetta più di un argomento:

```
def areaRettangolo(base,altezza):
     return base*altezza
```

In quest'altro esempio vediamo la possibilità di definire una funzione che restituisca più di

un valore:

```
def valori(x,y):
     a=x+1
     b=x*y
     c=3*y+x
     return a,b,c

k,l,m=valori(2,3)
print (k,l,m)
print (l)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
3 6 11
6
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione restituisce 3 valori. (vedremo più in dettaglio come questo avviene nel capitolo delle tuple)
- ➔ L'assegnazione a 3 variabili dei 3 valori di ritorno avviene simultaneamente.
- ➔ Le variabili assegnate possono essere utilizzate indipendentemente l'una dalle altre.

## 5.2.1* Digressione sulle modalità di passaggio dei parametri

Cerchiamo di capire in che modalità vengono passati i parametri ad una funzione Python.

E' necessario soffermarsi , prima, su due importanti questioni: come avviene generalmente, in un linguaggio di programmazione, il passaggio dei parametri e come il Python gestisce le variabili in memoria.

### 5.2.1.1 Modalità di passaggio di parametri

Nello studio dei vari linguaggi di programmazione a riguardo delle modalità di passaggio dei parametri a una funzione si incontrano principalmente due modalità:

- ➔ passaggio per valore
- ➔ passaggio per riferimento

Nel primo caso la funzione chiamata riceve una *copia* del valore dell'argomento. Ogni modificazione del valore della copia *non* si riflette sul valore *originale* della variabile passata nella chiamata.
Nel secondo caso la funzione chiamata riceve l'indirizzo di memoria della variabile passata, quindi ogni modificazione all'interno della funzione si riflette sul valore originale della variabile passata.

Naturalmente ci sono altre accezioni per questo importante topic, ma per quello che a noi serve è necessario avere in mente solo queste due.

#### 5.2.1.2 I tipi di variabili in Python: una puntualizzazione

Abbiamo compreso come il Python sia un linguaggio *debolmente tipizzato* e che abbia la *tipizzazione dinamica*, questa è una caratteristica abbastanza comune tra i linguaggi interpretati.

Inoltre il linguaggio Python non ha tipi primitivi come, p.e., il linguaggio C. Ogni tipo è definito da una classe e quindi i valori delle variabili sono tutti oggetti di tali classi (per una spiegazione sul significato di *classe* e *oggetto* si rimanda al capitolo che tratta la programmazione orientata agli oggetti.

Quindi i tipi `int`, `float`, `bool`, `str` rispettivamente *interi*, *a virgola mobile*, *booleani*, *stringa* e in generale i tipi di dati strutturati sono tutte classi e le istanze di tali classi sono i valori (*oggetti*) con cui lavoriamo.

Per capire questo concetto utilizziamo la funzione `type` che restituisce il tipo di una variabile. Proviamo il seguente programma:

```
a = 10
b = 3.5
c = True
print(type(a),type(b),type(c),sep="\n")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
<class 'int'>
<class 'float'>
<class 'bool'>
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Tutte e tre le variabili sono elementi di classi predefinite

#### 5.2.1.3 La funzione `id()`

La funzione `id()` ritorna l'identità di una variabile (oggetto)[4].

Esaminiamo il seguente codice:

```
a = 1
b = 2
c = b
```

[4] https://docs.python.org/3/library/functions.html#id

```
print(id(a),id(b),id(c),sep="\n")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
125680728670448
125680728670480
125680728670480
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Le variabili *a* e *b* hanno due identificativi differenti
- ➔ Le variabili *b* e *c* hanno lo stesso identificativo perchè condividono la stessa zona di memoria, a e b sono la "stessa cosa"

Per quanto riguarda le variabili `b` e `c` si sarebbe potuto utilizzare l'operatore di identità `is` (cfr. 4.2.6) per capire che sono lo stesso oggetto.

Per far si che la funzione `id()` restituisca un valore sempre differente, tra oggetti diversi, si utilizza la locazione di memoria dove l'oggetto stesso è allocato[5].

#### 5.2.1.4 Tipi immutabili e mutabili

Estratto da:"Il manuale di riferimento di Python"[6]
"Il valore di alcuni oggetti può cambiare. **Gli oggetti il cui valore può cambiare vengono detti *mutabili*; gli oggetti il cui valore non può cambiare vengono detti *immutabili***. (Il valore di un oggetto contenitore immutabile che contiene un riferimento ad un oggetto mutabile può cambiare quando cambia il valore di quest'ultimo; comunque il contenitore viene ancora considerato immutabile, poiché la collezione di oggetti che contiene non può essere cambiata. Perciò immutabile non è strettamente equivalente ad un valore non modificabile, la differenza è più sottile). La mutabilità di un oggetto viene determinata dal suo tipo; per esempio numeri, stringhe e tuple sono immutabili, mentre dizionari e liste sono mutabili."

Questo breve estratto descrive sinteticamente ma esaustivamente il concetto di mutabilità e immutabilità delle variabili nel linguaggio Python.

L'introduzione di questo concetto nasce da diverse esigenze quali, p.e., sicurezza del codice (previene modifiche accidentali in aree di memoria condivise), ottimizzazione della memoria (gli immutabili possono essere memorizzati in cache e riutilizzati risparmiando così accessi in memoria) , etc,etc.

Può risultare bizzarro avere il tipo int come immutabile in quanto la manipolazione dei valori numerici è fondamentale in qualsiasi procedura.
Analizziamo questo esempio per comprendere meglio cosa avviene:

[5] Il manuale di riferimento di Python - Guido van Rossum - Python Software Foundation
[6] Il manuale di riferimento di Python - Guido van Rossum - Python Software Foundation

```
a = 1
print("id prima modifica:", id(a))
a += 1
print("id dopo  modifica:", id(a))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
id prima modifica: 132330608918768
id dopo  modifica: 132330608918800
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La variabile a dopo la modifica ha un identificativo differente
- ➔ Questo significa che la variabile iniziale non è stata modificata ma è stata allocata un'altra zona di memoria per ospitare il risultato dell'operazione.
- ➔ La prima zona di memoria, se non più referenziata da nessun'altra variabile verrà pulita dal *garbage collector* dell'interprete Python per renderla disponibile ad altre elaborazioni.

Eseguiamo un codice analogo per un tipo di dati mutabile come la lista (questa struttura informativa l'affronteremo nel capitolo delle strutture dati (Cap. 7.2)

```
a = [1, 2]
print("id prima modifica:", id(a))
a += [3]
print("id dopo  modifica:", id(a))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
id prima modifica: 130634231324480
id dopo  modifica: 130634231324480
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La variabile `a` è una lista numerica
- ➔ Andiamo ad aggiungere un altro elemento alla lista con l'operatore `+=`
- ➔ Notiamo che l'identificativo della variabile non si è modificato, indice del fatto che la lista è un tipo dati (struttura dati) *mutabile*.

#### 5.2.1.5 Passaggio di parametri in una funzione Python

Dopo tutti questi paragrafi di approfondimenti e di precisazioni possiamo affrontare la modalità di passaggio parametri alle funzioni.

Visto che le variabili Python sono tutte degli oggetti ci aspettiamo che il passaggio avvenga per riferimento.

A conferma di quanto affermato, eseguiamo questo semplice programma:

```
def fun(a):
    print(id(a))

a=1
print(id(a))
fun(a)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
123845066080496
123845066080496
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Andiamo a richiamare una funziona che passa un intero a una funzione
- ➔ Stampando l'identificativo della variabile passata ci rendiamo conto che è lo stesso identificativo della variabile all'interno della funzione.
- ➔ Deduciamo che il parametro è stato passato per riferimento.

In generale possiamo affermare che:

**Passaggio di parametri:** Il passaggio di parametri in una funzione Python avviene sempre per riferimento.

Ora dobbiamo capire come si comporta la variabile passata alla funzione. Per comprendere questo dobbiamo aver compreso la differenza tra variabile *mutabile* e *immutabile* (cfr. 5.2.1.4).

Aiutiamoci con il seguente codice:

```
def fun(a):
    print("id(a) prima modifica:", id(a))
    a += 1
    print("id(a) dopo  modifica:", id(a))
    print("Valore a dopo modifica:", a)

a = 1
print("id(a) prima chiamata:", id(a))
fun(a)
print("id(a) dopo  chiamata:", id(a))
print("Valore a dopo funzione:", a)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
id(a) prima chiamata: 130883385295088
id(a) prima modifica: 130883385295088
```

```
id(a) dopo  modifica: 130883385295120
Valore a dopo modifica: 2
id(a) dopo  chiamata: 130883385295088
Valore a dopo funzione: 1
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Notiamo come la variabile `a` sia stata passata per riferimento
- ➔ All'interno della funzione la variabile a viene modificata e quindi anche il suo identificativo viene ad essere diverso
- ➔ Dal ritorno della funzione la variabile a non ha subito modifiche, infatti l'identificativo della funzione è lo stesso che aveva prima della chiamata.

Il comportamento della variabile `a` nel codice è dovuto al fatto che la variabile `a` è un tipo immutabile.

**Passaggio di parametri variabili immutabili:** possiamo assimilare il comportamento del passaggio di parametri di una variabile immutabile come se questa fosse passata per valore.

# 5.3 Approfondimenti sul passaggio di parametri

## 5.3.1 Parametri con valori predefiniti

Riprendiamo la funzione di calcolo dell'area di un rettangolo. Diamo dei valori predefiniti ai parametri formali.

Proviamo il seguente codice:

```
def areaRettangolo(base = 2, altezza = 4):
     return base*altezza

print("Area 1 = ", areaRettangolo())
print("Area 2 = ", areaRettangolo(5))
print("Area 3 = ", areaRettangolo(5,5))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Area 1 =  8
Area 2 =  20
Area 3 =  25
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Se si omettono i valori per i parametri, nella chiamata della funzione, l'interprete non segnala un errore ma valorizza il parametro con il valore predefinito.

### 5.3.2 Argomenti denominati

Il Python ci dà la possibilità di inserire i valori dei parametri attuali in ordine differente da quello dei parametri formali.

Proviamo il seguente codice:

```
def areaRettangolo(base, altezza, profondita):
     return base*altezza*profondita

print("Area 1 = ", areaRettangolo(3,profondita=4,altezza=5))
print("Area 2 = ", areaRettangolo(profondita=4,altezza=5,base=3))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Area 1 =  60
Area 2 =  60
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Nella chiamata della funzione abbiamo inserito i nomi dei parametri formali valorizzandoli ai valori voluti.
- ➔ Scrivendo i nomi dei parametri formali possiamo evitare di scriverli nel corretto ordine, cioè l'ordine indicato nella lista dei parametri formali della funzione.
- ➔ Gli argomenti denominati devono essere inseriti dopo gli argomenti posizionali della funzione. Questo significa che se avessi chiamato la funzione in questa maniera: `print("Area 3 = ", areaRettangolo(altezza=5,profondita=3,5))` avremmo ottenuto il seguente errore: `SyntaxError: positional argument follows keyword argument`

Praticamente ogni *argomento denominato* in una chiamata ha la seguente forma:

```
nomeParametro = valore
```

Un'utilità degli argomenti denominati è quello di aumentare la leggibilità del codice, si consideri p.e. le chiamate a funzioni con molti argomenti.

### 5.3.3 Numero variabile di argomenti

Il Python permette di gestire un numero di argomenti indefinito come parametri formali a una funzione.

Proviamo il seguente codice:

```
def media(*parametri):
    somma=0

    for val in parametri:
```

```
        somma+=val

    return somma/len(parametri)

print("Media 1 = ", media(1,2,3))
print("Media 2 = ", media(1,2,3,4,5,6))
print("Media 3 = ", media())
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Media 1 =  2.0
Media 2 =  3.5
Traceback (most recent call last):
  File "provafun.py", line 11, in <module>
    print("Media 3 = ", media())
  File "provafun.py", line 7, in media
    return somma/len(parametri)
ZeroDivisionError: division by zero
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Nella definizione della funzione abbiamo utilizzato il parametro `*parametri` per indicare che la funzione può ricevere un numero qualsiasi di argomenti.
- ➔ `parametri` è una *tupla* come si può capire stampando `type(parametri)`. La struttura informativa tupla verrà affrontata nel capitolo 7 delle Strutture Dati
- ➔ Si possono inserire anche 0 argomenti, il programma dà errore in quanto si effettua una divisione per zero in quanto la lunghezza della tupla inserita è, in questo caso, proprio zero. Naturalmente una situazione del genere deve essere opportunamente gestita dal programmatore.

La nostra funzione potrebbe avere uno o più argomenti prima della lista arbitraria di argomenti.
Diamo un'occhiata al seguente codice:

```
def media(a,b,*parametri):
    somma=a+b;

    for val in parametri:
        somma+=val

    return somma/(len(parametri)+2)

print("Media 1 = ", media(-1,-2,1,2,3))
print("Media 3 = ", media(-1,-2))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Media 1 =  0.6
```

```
Media 3 =  -1.5
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Nella lista dei parametri formali sono presenti due argomenti e in coda la lista degli argomenti variabili
- ➔ I primi due argomenti devono essere necessariamente presenti pena un errore dell'interprete.
- ➔ Il funzionamento della lista variabile di argomenti è analoga a quanto visto nel precedente esempio: la lista può essere presente con un numero arbitrario di elementi o non essere presente.

### 5.3.4 Numero variabile di argomenti denominati

Il Python permette anche di gestire un numero indefinito di argomenti denominati (keyword arguments) come parametri formali di una funzione.

Proviamo il seguente codice:

```
def descriviPersona(**dati):
    for chiave, valore in dati.items():
        print(chiave, "->", valore)

descriviPersona(nome="Marco", età=30)
print("---")
descriviPersona(nome="Giulia", città="Torino",
professione="ingegnere")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
nome -> Marco
età -> 30
---
nome -> Giulia
città -> Torino
professione -> ingegnere
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Nella definizione della funzione abbiamo utilizzato il parametro **dati per indicare che la funzione può ricevere un numero qualsiasi di argomenti denominati.
- ➔ dati è un dizionario, come si può capire stampando type(dati). La struttura informativa dizionario verrà affrontata nel capitolo 7 delle Strutture Dati.
- ➔ Ogni argomento denominato passato nella chiamata diventa una coppia chiave-valore nel dizionario: il nome del parametro è la chiave, il valore passato è il valore associato.
- ➔ Si possono inserire anche 0 argomenti denominati; in questo caso il dizionario dati

risulterà vuoto e il ciclo for non verrà eseguito, senza generare alcun errore.

Anche in questo caso, la nostra funzione potrebbe avere uno o più parametri ordinari prima della lista arbitraria di argomenti denominati.

Diamo un'occhiata al seguente codice:

```
def descriviPersona(nome, **altriDati):
    print("Nome:", nome)
    for chiave, valore in altriDati.items():
        print(chiave, "->", valore)

descriviPersona("Luca", età=25, città="Milano")
descriviPersona("Anna")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Nome: Luca
età -> 25
città -> Milano
Nome: Anna
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Nella lista dei parametri formali è presente un argomento ordinario (nome) e in coda la lista degli argomenti denominati variabili (**altriDati).
- ➔ Il parametro nome deve essere necessariamente presente, pena un errore dell'interprete.
- ➔ Il funzionamento della lista variabile di argomenti denominati è analogo a quanto visto per *parametri: il dizionario può contenere un numero arbitrario di coppie chiave-valore oppure essere vuoto.
- ➔ A differenza di *parametri, con **altriDati l'ordine con cui vengono passati gli argomenti denominati nella chiamata non ha alcuna importanza, proprio perché ciascuno di essi viene identificato dal proprio nome (chiave) e non dalla posizione.

Infine, è possibile combinare nella stessa funzione parametri posizionali, parametri con valori predefiniti, *args e **kwargs, purché vengano rispettati questo preciso ordine nella definizione:

```
def funzione(pos1, pos2, default=valore, *args, **kwargs):
    ...
```

Ecco un semplice esempio che combina tutti e quattro i tipi di parametri: posizionali, con valore predefinito, *args e **kwargs:

```
def funzione(pos1, pos2, default=10, *args, **kwargs):
    print("pos1 =", pos1)
    print("pos2 =", pos2)
```

```
    print("default =", default)
    print("args =", args)
    print("kwargs =", kwargs)

funzione(1, 2)
print("---")
funzione(1, 2, 100, 200, 300)
print("---")
funzione(1, 2, 100, 200, 300, extra1="ciao", extra2="mondo")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
pos1 = 1
pos2 = 2
default = 10
args = ()
kwargs = {}
---
pos1 = 1
pos2 = 2
default = 100
args = (200, 300)
kwargs = {}
---
pos1 = 1
pos2 = 2
default = 100
args = (200, 300)
kwargs = {'extra1': 'ciao', 'extra2': 'mondo'}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ pos1 e pos2 sono parametri posizionali: vengono valorizzati sempre con i primi due argomenti passati nella chiamata.
- ➔ default è un parametro con valore predefinito: se nella chiamata è presente un terzo argomento posizionale, questo sovrascrive il valore predefinito (come accade nella seconda e terza chiamata, dove default diventa 100).
- ➔ *args raccoglie, sotto forma di tupla, tutti gli argomenti posizionali in eccesso rispetto a pos1, pos2 e default.
- ➔ **kwargs raccoglie, sotto forma di dizionario, tutti gli argomenti denominati passati nella chiamata che non corrispondono a nessuno dei parametri già definiti.
- ➔ L'ordine dei parametri nella definizione della funzione (posizionali, poi default, poi *args, infine **kwargs) non è casuale: è l'unico ordine ammesso dal Python, pena un errore di sintassi.

## 5.4 Regole di visibilità

Ogni variabile ha una sua *visibilità* (*scope*) che determina dove può essere usato nel flusso

del programma. In quella particolare porzione di flusso la variabile è "visibile" e quindi può essere utilizzata.

Esistono due tipi di visibilità:
- ➔ Visibilità locale
- ➔ Visibilità globale

### 5.4.1 Visibilità locale

Quando definiamo una variabile all'interno di una funzione, questa è definita localmente alla funzione stessa, in altre parole ha *visibilità locale*. Al di fuori della funzione la variabile non potrà essere più referenziata. Cioè la variabile non sarà più visibile quando la funzione ritorna al suo chiamante.

Proviamo a scopo di esempio il seguente programma

```
def fun(a,b):
    somma = a+b
    print("Somma = ",somma)
    return somma

print(fun(2,3))
print("Somma = ",somma)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Somma =  5
5
Traceback (most recent call last):
    File "/home/massimo/provafun.py", line 7, in
<module>
    print("Somma = ",somma)
NameError: name 'somma' is not defined
```

Dall'analisi del codice e dell'output osserviamo che:
- ➔ Abbiamo richiamato una funzione che esegue la somma dei due numeri passati come parametri
- ➔ All'interno della funzione utilizziamo una variabile `somma` che stampiamo
- ➔ Stampando la stessa variabile al di fuori della funzione otteniamo un errore di nome non definito

Nell'esempio sopra riportato la variabile `somma` è una variabile locale che ha una visibilità all'interno della funzione `fun`, in altre parole il suo *scope* è la funzione `fun`.

### 5.4.2 Visibilità globale

Una variabile definita al di fuori di una funzione ha una visibilità globale, cioè si può referenziare sia all'interno della funzione che all'esterno.

Proviamo il seguente programma

```
var = 10
def fun():
     print("2 var = ", var)
     var=var+1
     print("3 var = ", var)

print("1 var = ",var)
fun()
print("4 var = ",var)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
1 var =  10
Traceback (most recent call last):
  File "/home/massimo/provafun.py", line 8, in <module>
    fun()
  File "/home/massimo/provafun.py", line 3, in fun
    print("2 var = ", var)
UnboundLocalError:  local  variable  'var'  referenced  before
assignment
```

Dall'analisi del codice e dell'output osserviamo che:

➔ La variabile globale `var` viene definita e inizializzata prima di tutte le altre istruzioni

➔ `var` viene stampata correttamente nel programma principale

➔ La prima istruzione della funzione fun manda in errore l'interprete Python, l'errore indica che l'interprete considera var come variabile locale e si produce errore in quanto risulta non essere prima inizializzata

Il codice presentato ci fa comprendere che Python all'interno della funzione `fun` considera `var` come variabile locale. Questo succede in quanto stiamo cercando di modificare la variabile locale all'interno della funzione. L'istruzione di assegnazione fa si che l'interprete vada a creare una variabile locale alla funzione di nome `var`.
Se andassimo a eseguire il codice senza l'istruzione `var = var + 1` otterremo il seguente output:

```
1 var =  10
2 var =  10
3 var =  10
4 var =  10
```

Quindi le variabili globali non si possono modificare all'interno delle funzioni.

Per superare questo problema proviamo a eseguire il seguente codice:

```
var = 10
def fun():
    global var
    print("2 var = ", var)
    var=var+1
    print("3 var = ", var)

print("1 var = ",var)
fun()
print("4 var = ",var)
```

Otteniamo il seguente output:

```
1 var =  10
2 var =  10
3 var =  11
4 var =  11
```

Dall'analisi del codice e dell'output osserviamo che:

➔ La keyword `global` informa l'interprete Python che la variabile var è globale

➔ La variabile `var`, ora considerata globale, viene a essere modificata all'interno della funzione

➔ Il nuovo valore della variabile `var` viene correttamente stampato dal programma principale

Approfondiamo il comportamento delle variabili locali e globali: assegnamo alla variabile `var` il valore -2 e cancelliamo la keyword `global`. Abbiamo quindi il seguente programma:

```
var = 10
def fun():
    var=-2
    print("2 var = ", var)
    var=var+1
    print("3 var = ", var)

print("1 var = ",var)
fun()
print("4 var = ",var)
```

Eseguendolo otteniamo il seguente output:

```
1 var =  10
2 var =  -2
3 var =  -1
4 var =  10
```

Dall'analisi del codice e dell'output osserviamo che:
➔ La variabile `var` viene modificata all'interno della funzione
➔ Il nuovo valore della variabile `var` non viene stampato nel programma principale, in quanto `var` viene considerata come variabile locale all'interno della funzione e questa "nasconde" la variabile globale con nome omonimo.

## 5.5 Approfondimenti sulla funzione print

Sino a questo momento abbiamo utilizzato la funzione `print` per stampare semplicemente del testo e delle variabili su una riga dello standard output (solitamente il monitor).

Ora vedremo che possiamo effettuare delle stampe più sofisticate utilizzando i parametri formali che la funzione `print` accetta.

### 5.5.1 `Print` accetta un numero variabile di parametri

Proviamo a eseguire il seguente codice:

```
var1 = 10
var2 = "stringa"
var3 = 5.6
print("var1 = ",var1," var2 = ",var2," var3 = ",var3)
```

Otteniamo il seguente output:

```
var1 =  10  var2 =  stringa  var3 =  5.6
```

Dall'analisi del codice e dell'output osserviamo che:
➔ L'istruzione `print` è una funzione che accetta un numero variabile di argomenti
➔ Tutti gli argomenti devono essere separati da una virgola
➔ Gli argomenti possono essere di diverso tipo e non esiste un ordine predefinito
➔ Gli argomenti passati alla `print` vengono stampati tutti sulla stessa riga

Possiamo utilizzare le sequenze di escape per modificare il comportamento descritto nell'ultimo punto. Ad esempio vogliamo stampare var1 e var2 sulla prima riga e var3 a capo sulla seconda riga. Inoltre vogliamo distanziare va1 da var2 con una tabulazione.

Proviamo quindi a eseguire il seguente codice:

```
var1 = 10
var2 = "stringa"
var3 = 5.6
print("var1 = ",var1,"\tvar2 = ",var2,"\nvar3 = ",var3)
```

Otteniamo il seguente output:

```
var1 =  10      var2 =  stringa
var3 =  5.6
```

Dall'analisi del codice e dell'output osserviamo che:
➔ Con la stessa istruzione `print` abbiamo stampato su più righe. Questo è successo grazie al fatto che abbiamo utilizzato la sequenza di escape: `\n` che inserisce sullo standard output una *newline*.
➔ Notiamo che utilizzando la sequenza di escape: `\t` andiamo a inserire sullo standard output una tabulazione

#### 5.5.1.1 Sulle sequenze di escape

La seguente tabella riporta le sequenze di escape che possono essere utilizzate, tra le più comuni come visto precedentemente a quelle meno utilizzate.[7]

| Sequenza di escape | Significato |
|---|---|
| `\\` | Backslash (\) |
| `\'` | Carattere di quotatura singola (') |
| `\"` | Carattere di quotatura doppia (") |
| `\a` | ASCII Bell (BEL) |
| `\b` | ASCII Backspace (BS) |
| `\f` | ASCII Formfeed (FF) |
| `\n` | ASCII Linefeed (LF) |
| `\N{name}` | Carattere chiamato name nel database Unicode (solamente in Unicode) |
| `\r` | ASCII Carriage Return (CR) |
| `\t` | ASCII Tab orizzontale (TAB) |

[7] https://docs.python.it/html/ref/strings.html

| | |
|---|---|
| \uxxxx | Carattere con valore esadecimale a 16 bit xxxx (valido solo per Unicode) |
| \Uxxxxxxxx | Carattere con valore esadecimale a 32 bit xxxxxxxx (valido solo per Unicode) |
| \v | ASCII Tab verticale (VT) |
| \ooo | Carattere ASCII con valore ottale ooo |
| \xhh | Carattere ASCII con valore esadecimale hh |

Ci sono casi in cui non vogliamo che le stringhe seguano le sequenze di escape. In questi casi, generalmente vogliamo preservare i *backslash*. Alcune delle situazioni in cui ciò potrebbe essere utile sono:

- ➔ La stringa contiene un percorso di rete o locale
- ➔ La stringa contiene una pathname
- ➔ La stringa contiene una regular expression

Possiamo usare due metodi per prevenire l'interpretazione delle sequenze di escape[8]:

**Primo metodo:**
Raddoppiando i *backslash* consente di inibire l'interpretazione della sequenza di escape. Riprendiamo l'esempio precedente e modifichiamolo come segue:

```
var1 = 10
var2 = "stringa"
var3 = 5.6
print("var1 = ",var1,"\\tvar2 = ",var2,"\\nvar3 = ",var3)
```

Eseguendolo otteniamo il seguente output:

```
var1 =  10 \tvar2 =  stringa \nvar3 =  5.6
```

Come si può notare il primo *backslash* impedisce l'azione della sequenza di escape della tabulazione e del formfeed.

**Secondo metodo:**
Possiamo utilizzare le lettere **`r`** e **`R`** davanti alla stringa per inibire automaticamente l'interpretazione delle sequenze di escape. Il costrutto `r"..."` o `R"..."` sta per: *raw string*.
Riprendiamo di nuovo l'esempio precedente e modifichiamolo come segue:

```
var1 = 10
```

[8] https://www.geeksforgeeks.org/preventing-escape-sequence-interpretation-in-python/

```
var2 = “stringa”
var3 = 5.6
print("var1 = ",var1,r”\tvar2 = “,var2,R”\nvar3 = “,var3)
```

Eseguendolo otteniamo il seguente output:

```
var1 =  10 \tvar2 =  stringa \nvar3 =  5.6
```

Abbiamo così impedito che le sequenze di escape vengano interpretate. Questo secondo metodo è più comodo se trattiamo delle pathname o comunque in generale dei percorsi che solitamente utilizzano molti *backslash*.

### 5.5.2 Parametro formale `end`

Possiamo utilizzare il parametro facoltativo `end` per organizzare le nostre stampe sullo standard output.

Proviamo a eseguire il seguente codice:

```
print("Stampa su prima riga",end="\n")
print("Stampa su seconda riga")
print("Stampa su terza riga",end="\t")
print("dopo tab su terza riga ",end=" XXX ")
print("ancora su terza riga ")
print("Fine esempio su quarta riga ")
```

Otteniamo il seguente output:

```
Stampa su prima riga
Stampa su seconda riga
Stampa su terza riga    dopo tab su terza riga  XXX ancora su terza riga
Fine esempio su quarta riga
```

Dall’analisi del codice e dell’output osserviamo che:

➔ Il contenuto del parametro `end` di print viene accodato alla stringa che viene stampata

➔ Se omesso il valore di default è `“\n”` cioè *newline*.

➔ Il valore di `end` può essere una qualsiasi stringa.

➔ Il valore di `end` puo’ anche essere una qualsiasi sequenza di escape ( cfr. 5.5.5.1 )

### 5.5.3 Parametro formale `sep`

Abbiamo visto come con la funzione `print` possiamo stampare più valori su una stessa riga, è sufficiente inserire tali valori come argomenti ( cfr. 5.5.1 ).
Possiamo indicare con il parametro `sep` un *separatore* tra i vari campi della `print`.

```
campo1 = 2
campo2 = "string"
campo3 = 5.03
print(campo1,campo2,campo3)
print(campo1,campo2,campo3,sep=" ")
print(campo1,campo2,campo3,sep=" | ")
print(campo1,campo2,campo3,sep="\t")
print(campo1,campo2,campo3,sep="\n")
```

Otteniamo il seguente output:

```
2 string 5.03
2 string 5.03
2 | string | 5.03
2       string  5.03
2
string
5.03
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Il contenuto del parametro `sep` di print viene accodato alla fine di ogni variabile che viene stampata

➔ Se omesso il valore di default è " " cioè uno spazio.

➔ Il valore di `sep` può essere una qualsiasi stringa.

➔ Il valore di `sep` puo' anche essere una qualsiasi sequenza di escape ( cfr. 5.5.5.1 )

## 5.5.4 Formattazione dell'output alla maniera del linguaggio C

Questo paragrafo è utile soprattutto per chi ha già un'esperienza di programmazione in linguaggio C, nell'utlilizzo della printf() celeberrima funzione della libreria standard del linguaggio C, che permette di stampare sullo standard output.

- Le operazioni di formattazione qui descritte presentano delle particolarità che portano a una serie di errori comuni (come la mancata visualizzazione di tuple e dizionari correttamente). L'uso delle stringhe letterali formattate[9] (f-strings), l'interfaccia di str.format()[10] o le template string[11] possono aiutare a evitare questi errori.[12]

L'operatore `%` può essere utilizzato anche per la formattazione delle stringhe. Interpreta l'argomento sinistro in modo molto simile a un formato in stile printf() come nelle stringhe del linguaggio C da applicare all'argomento destro. In Python non esiste la funzione printf()

[9] https://docs.python.org/3.9/reference/lexical_analysis.html#f-strings
[10] https://docs.python.org/3.9/library/stdtypes.html#str.format
[11] https://docs.python.org/3.9/library/string.html#template-strings
[12] https://docs.python.org/3.9/library/stdtypes.html#old-string-formatting

ma la funzionalità dell'antico printf è contenuta in Python. A tale scopo, l'operatore modulo % viene sovraccaricato dalla classe string per eseguire la formattazione delle stringhe. Pertanto, viene spesso chiamato operatore modulo stringa (o talvolta anche chiamato modulo).
L'operatore modulo stringa ( `%` ) è ancora disponibile in Python(3.x) ed è ampiamente utilizzato. Ma questo vecchio stile di formattazione verosimilmente cadrà nell'obsolescenza nelle future versioni del linguaggio..

I tipi dei segnaposto di formato sono:

| Tipo | Significato |
|---|---|
| `'d'` | Decimale intero con segno. |
| `'i'` | Decimale intero senza segno. |
| `'o'` | Valore ottale firmato. |
| `'u'` | Tipo obsoleto: è identico a 'd'. |
| `'x'` | Esadecimale con segno (minuscolo). |
| `'X'` | Esadecimale con segno (maiuscolo). |
| `'e'` | Formato esponenziale in virgola mobile (minuscolo). |
| `'E'` | Formato esponenziale in virgola mobile (maiuscolo). |
| `'f'` | Formato decimale a virgola mobile. |
| `'F'` | Formato decimale a virgola mobile. |
| `'g'` | Formato in virgola mobile. Utilizza il formato esponenziale minuscolo se l'esponente è inferiore a -4 o non inferiore a precisione, altrimenti il formato decimale. |
| `'G'` | Formato in virgola mobile. Utilizza il formato esponenziale maiuscolo se l'esponente è inferiore a -4 o non inferiore a precisione, altrimenti il formato decimale. |
| `'c'` | Carattere singolo (accetta una stringa di caratteri intera o singola). |
| `'r'` | String (converte qualsiasi oggetto Python usando `repr()`[13]). |
| `'s'` | String (converte qualsiasi oggetto Python usando `str()`[14]). |

[13] https://docs.python.org/3.9/library/functions.html#repr
[14] https://docs.python.org/3.9/library/stdtypes.html#str

| | |
|---|---|
| 'a' | String (converte qualsiasi oggetto Python usando `ascii()`[15]). |
| '%' | Nessun argomento viene convertito, risulta in un '%' carattere nel risultato. |

Vediamo un esempio che utilizza questa formattazione:

```
# stampa un valore intero e uno float
print("intero : %2d, float : %5.2f" % (1, 05.333))

# stampa solo valori interi
print("primo int : %3d, secondo int : %2d" % (240, 120))

# stampa un valore ottale
print("%7.3o" % 25)

# stampa un numero in forma esponenziale
print("%10.3E" % (356.08977))
```

Otteniamo il seguente output:

```
intero :  1, float :  5.33
primo int : 240, secondo int : 120
    031
 3.561E+02
```

Dall'analisi del codice e dell'output osserviamo che:

- L'operatore modulo rappresenta l'operatore che permette di formattare per la stampa la tupla presente a destra dell'operatore utilizzando la stringa di formattazione presente sulla sinistra dell'operatore
- Il primo segnaposto "%2d" viene utilizzato per il primo componente della nostra tupla, ovvero l'intero 1. Il numero verrà stampato con 2 caratteri. Poiché 1 è costituito solo da una cifra, l'output viene riempito con 1 spazi vuoti iniziali.
- Il secondo "%5.2f" è una descrizione del formato per un numero float. Come altri segnaposto, viene introdotto con il carattere %. Questo è seguito dal numero totale di cifre che la stringa dovrebbe contenere. Questo numero include il punto decimale e tutte le cifre, cioè prima e dopo il punto decimale.
- La terza stampa non presenta nei valori le parentesi tonde, questo indica che non è una tupla. Quindi se abbiamo solo un valore possiamo indifferentemente utilizzare direttamente il valore o indicarlo come unico elemento di una tupla
- Il nostro numero float 05.333 deve essere formattato con 5 caratteri. La parte decimale del numero o della precisione è impostata su 2, ovvero il numero che segue il "." nel nostro segnaposto. Infine, l'ultimo carattere "f" del nostro segnaposto sta per "float".

[15] https://docs.python.org/3.9/library/functions.html#ascii

➔ La sintassi generale per un segnaposto di formato è:

```
%[flags][larghezza][.precision]tipo
```

che è identica ai segnaposto di formato della printf() del linguaggio C.

## 5.5.5 Formattazione dell'output utilizzando le f-string

### 5.5.5.1 Cosa sono le f-string?

Le f-string sono delle stringhe letterali formattate, cioè è un valore letterale stringa con prefisso `'f'` o `'F'`. La cosa interessante è che queste stringhe possono contenere campi di sostituzione, che sono costituiti da espressioni delimitate da parentesi graffe `{}`. I valori letterali stringa hanno sempre un valore costante, le stringhe formattate sono in realtà espressioni valutate in fase di esecuzione.[16]

Le f-string sono state introdotte dalla versione 3.6 del Python.

### 5.5.5.2 L'uso delle f-string con la funzione `print`

Esaminiamo il seguente codice:

```
valore = 10
print(f"Valore: {valore}")
print(F"Valore con 3 cifre {valore:03}")
print(f"Valore con 3 \'posizioni\' {valore:3}")
numero = 10.456
print(f"Numero con 2 cifre decimali {numero: .2f}")
print(f"Numero con 10 cifre decimali {numero: .10f}")
print(f"Numero: {numero:e}")
print(f"Numero: {numero:.3e}")
```

Otteniamo il seguente output:

```
Valore: 10
Valore con 3 cifre 010
Valore con 3 'posizioni'  10
Numero con 2 cifre decimali  10.46
Numero con 10 cifre decimali  10.4560000000
Numero: 1.045600e+01
Numero: 1.046e+01
```

[16] https://docs.python.org/3.9/reference/lexical_analysis.html#f-strings

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Definamo una f-string sia utilizzando `'f'` che `'F'`
- ➔ Possiamo inserire all'interno delle graffe delle variabili che poi verranno valutate nel momento in cui si effettuerà la stampa.
- ➔ I caratteri speciali come i singoli apici vengono inseriti utilizzando i backslash (\)
- ➔ Possiamo formattare un numero intero indicando le cifre con cui deve essere stampato (se inferiore si aggiungono zeri nella posizione meno significativa) oppure si puo' indicare il numero di spazi che deve occupare (utile per l'incolonnamento). Se il numero di cifre è inferiore al numero di posizioni si inseriscono sulla sinistra degli spazi.
- ➔ Se utilizziamo il suffisso:`"e"` otteniamo la notazione scientifica (rappresentazione esponenziale) del numero. Abbiamo anche la possibilità di indicare il numero di cifre decimali da stampare.

Possiamo valutare all'interno delle parentesi graffe anche delle espressioni. A tal proposito esaminiamo il seguente codice:

```
def somma(a,b):
    return a+b

n1=10
n2=20
print(f"n1 incrementato di 2: {n1+2}")
print(f"n1/n2 {n1/n2:.2f}")
print(f"n1+n2={somma(n1,n2)}")
print(f"{somma(n1,n2)=}")
```

Eseguendo il programma si ottiene il seguente output:

```
n1 incrementato di 2: 12
n1/n2 0.50
n1+n2=30
somma(n1,n2)=30
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Tra le `{}` possiamo inserire una qualsiasi espressione, questa verrà valutata prima di essere inserita nella stringa
- ➔ Possiamo richiamare anche delle funzioni definite dall'utente.
- ➔ L'ultima istruzione, in cui si aggiunge:`"="` alla fine del richiamo della funzione permette di "autodocumentare" l'espressione stessa. E' una funzionalità inserita con la versione 3.8 del linguaggio. Può essere utilizzata congiuntamente alla formattazione del valore numerico restituito.

Se avessimo la necessità di stampare dei valori numerici utilizzando una base numerica differente dalla 10 possiamo utilizzare il seguente codice per effettuare le conversioni:

```
num = 10
# esadecimale
print(f"{num:x}")
# ottale
print(f"{num:o}")
# binario
print(f"{num:b}")
print(f"{num:8b}")
print(f"{num:08b}")
# decimale
print(f"{num:d}")
```

Eseguendo il programma si ottiene il seguente output:

```
a
12
1010
    1010
00001010
10
```

Dall'analisi del codice e dell'output osserviamo che:

- La prima stampa converte il valore decimale 10 memorizzato in un in valore esadecimale utilizzando la notazione:"`:x`" all'interno della f-string
- Per stampare in base ottale utilizziamo la notazione:"`:o`" all'interno della f-string
- La notazione:":b" viene utilizzata per stampare in formato binario un numero decimale
- Possiamo aggiungere un suffisso all'indicazione della base numerica che permette di inserire dei blank iniziali oppure degli 0 iniziali (come visto in precedenza per le f-string con i numeri di interi degli esempi visti precedentemente)
- L'ultimo esempio mostra come "`:d`" indichi la stampa del numero decimale, naturalmente non è necessario inserirlo in quanto è la formattazione di default. come già visto

Infine consideriamo il comportamento nella formattazione delle stringhe con il seguente codice:

```
str1 = 'a'
num2 = 15
str3 = 'abc'
str4 = 'abcd'
print(f'{str1:>10}')
print(f'{num2:>10}')
print(f'{str3:<10}')
```

```
print(f'{str4:>10}')
```

Lanciando questo codice otteniamo sullo standard output quanto segue:

```
         a
        15
abc
      abcd
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Notiamo come il carattere ">" e "<" permetta di giustificare a destra o a sinistra l'espressione tra le parentesi graffe.

## 5.6 Sequenze e dizionari come argomento di funzioni

Questo paragrafo è utile affrontarlo solo dopo aver letto il capitolo 7 sulle strutture dati del Python.

In questo paragrafo daremo per scontata la conoscenza delle strutture informative come la stringa, la lista, la tupla e il dizionario. Se non sai di cosa stiamo parlando allora salta questo paragrafo e vai al capitolo 7 dove si introducono questi argomenti.

Ricordiamo che le stringhe e le tuple sono strutture immutabili, quindi non possiamo variare gli elementi che le compongono. Viceversa le liste e i dizionari sono mutabili, quindi possiamo modificare le componenti che le compongono.

È importare ricordare questa differenza per comprendere il meccanismo del passaggio come parametro a una funzione di una di queste strutture informative.

Gli esempi che abbiamo visto nei precedenti paragrafi ci hanno fatto comprendere che i parametri a una funzione python sono passati sempre "per riferimento". Ebbene anche questa informazione è da tenere bene in mente per comprendere questo paragrafo.

Cominciamo a considerare la categoria delle sequenze *immutabili*.

### 5.6.1 Stringhe e Tuple

Le stringhe e le tuple sono strutture che non possono essere modificate.

Osserviamo prima di tutto che in Python, **gli argomenti vengono sempre passati per riferimento all'oggetto** (cfr. 5.2.1.5). Tuttavia, per le strutture immutabili come le stringhe e le tuple, ogni modifica crea internamente un nuovo oggetto, lasciando invariato l'originale. Questo comportamento può essere confuso con un passaggio "per valore", ma tecnicamente non è corretto utilizzare questa terminologia in Python.

Proviamo ora il seguente codice:

```
def funStringa(str):
    str="ciao"
    return str

def funTupla(tupla):
    tupla=tupla+(100,200)
    return tupla

str="pippo"
print(str)
print(funStringa(str))
print(str)
print('------------')

t=(1,2,3,4)
print(t)
print(funTupla(t))
print(t)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
pippo
ciao
pippo
------------
(1, 2, 3, 4)
(1, 2, 3, 4, 100, 200)
(1, 2, 3, 4)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Nella funzione che accetta come parametro formale la stringa, questa viene riassegnata. La modifica consiste nella riscrittura completa della stringa stessa. Il valore restituito dalla funzione è la nuova stringa. Comprendiamo che, sebbene in Python i parametri siano passati per riferimento, quando riassegniamo completamente una variabile immutabile, creiamo un nuovo oggetto in memoria. Infatti la stringa originale non risulta affetta da nessuna variazione dopo l'esecuzione della funzione.
- ➔ Nella funzione che accetta come parametro formale la tupla si ha un comportamento analogo a quanto accaduto nella funzione della stringa. funTupla non modifica la tupla originale, ma crea una nuova tupla risultante dalla concatenazione della tupla passata come parametro con un'altra tupla definita nella funzione stessa. Lo scope della nuova tupla è quello della funzione.
- ➔ Possiamo concludere che sia per la stringa che per la tupla, essendo tipi immutabili, qualsiasi operazione che sembrerebbe modificarle in realtà crea nuovi oggetti in memoria. Questo comportamento può sembrare simile al passaggio per valore, ma in Python tutti i parametri sono passati per riferimento (cfr. 5.2.1.5).

### 5.6.2 Liste

Le liste sono strutture che possono essere modificate, quindi ci aspettiamo che riesca a modificare gli elementi di una lista all'interno di una funzione.

Proviamo il seguente codice:

```
def funLista(lista):
    for i in range(len(lista)):
        lista[i] *=2
    return lista

l=[1,2,3,4]
print(funLista(l))
print(l)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
[2, 4, 6, 8]
[2, 4, 6, 8]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La lista `lista` viene modificata all'interno della funzione andando a raddoppiare ogni suo elemento.
- ➔ La lista `l` parametro attuale della funzione `funlista( )` viene a sua volta modificato, come si evince dalla stampa di l subito dopo la chiamata della funzione
- ➔ Visto che la lista viene modificata allora è inutile far "ritornare" la lista alla funzione.

Questo codice ci conferma come **le liste vengano passate per riferimento (cioè per indirizzo)**, il lettore può provare a stampare l'identificativo della lista per avere conferma di ciò (cfr. 5.1.2.3). Poi il fatto che la lista sia mutabile permette ai suoi elementi di mantenere le modifiche effettuate all'interno della funzione.

Ma per comprendere ancora meglio quello che accade proviamo a modificare il sorgente nel seguente modo:

```
def funLista(lista):
    lista = lista + [10]
    for i in range(len(lista)):
        lista[i] *=2
    return lista

l=[1,2,3,4]
print(funLista(l))
print(l)
```

Praticamente abbiamo aggiunto una riga (quella sottolineata) che va ad aggiungere un elemento in coda alla lista.

Lanciamo il programma, l'output stampato a video è il seguente:

```
[2, 4, 6, 8, 20]
[1, 2, 3, 4]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La lista `lista` viene modificata all'interno dello scope della funzione `funLista( )`
- ➔ La lista `l` non mantiene le modifiche della lista `lista`, il comportamento è assimilabile a quello di un passaggio per valore.

Il comportamento differente è generato dall'istruzione che abbiamo inserito nella seconda riga (riga sottolineata):

```
lista = lista + [10]
```

Questa riga va a creare una copia del parametro formale lista che comporta una visibilità locale alla sola funzione della variabile stessa.

Per ovviare a questo problema non dobbiamo effettuare nessun assegnamento al parametro formale, pena l'effettuazione della copia dei valori con conseguente visibilità solo locale delle modifiche.

Nel caso specifico potremmo andare a utilizzare il metodo append :

```
lista.append(10)
```

oppure andare a riassegnare il valore di l con il risultato restituito dalla funzione:

```
l = funLista(l)
```

O ancora andare a effettuare l'assegnazione `lista = lista + [10]` solo dopo aver modificato gli elementi della lista stessa visto che prima dell'assegnazione `l` e `lista` coincidono.
Naturalmente quest'ultima modifica non verrà riportata nella lista `l`.

### 5.6.3 Dizionari

I dizionari sono strutture che possono essere modificate, quindi ci aspettiamo che si riesca a modificare gli elementi di un dizionario all'interno di una funzione.

Proviamo il seguente codice:

```
def funDizionario(dizionario):
   dizionario['k3']='v3'
   for key in dizionario:
```

```
        dizionario[key] = 'X'
    return dizionario

diz={'k1':'v1','k2':'v2'}
print(funDizionario(diz))
print(diz)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
{'k1': 'X', 'k2': 'X', 'k3': 'X'}
{'k1': 'X', 'k2': 'X', 'k3': 'X'}
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Notiamo che si ha lo stesso identico comportamento ottenuto con la lista.

Concludiamo che **il dizionario viene passato a una funzione per riferimento (cioè per indirizzo)** e non per valore. Questo codice ci conferma come **il dizionario venga passato per riferimento (cioè per indirizzo)**, il lettore può provare a stampare l'identificativo del dizionario per avere conferma di ciò (cfr. 5.1.2.3). Poi il fatto che il dizionario sia mutabile permette ai suoi elementi di mantenere le modifiche effettuate all'interno della funzione.

Se, pero', all'interno della funzione andassimo a riassegnare il dizionario con un'assegnazione avente lo stesso nome del parametro formale, otterremo una copia del dizionario di partenza e quindi la creazione di un oggetto locale alla sola funzione.
Lasciamo al lettore la verifica di questo comportamento.

### 5.6.4 Insiemi

Gli insiemi sono strutture che possono essere modificate (sono mutabili), quindi ci aspettiamo che sia possibile modificare gli elementi di un insieme all'interno di una funzione.

Proviamo il seguente codice:

```
def funInsieme(insieme):
    insieme.add(100)
    insieme.discard(2)
    return insieme

s={1,2,3,4}
print(funInsieme(s))
print(s)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
{1, 3, 4, 100}
```

```
{1, 3, 4, 100}
```

Dall'analisi del codice e dell'output osserviamo che:

- L'insieme `insieme` viene modificato all'interno della funzione: viene aggiunto l'elemento 100 e rimosso l'elemento 2.
- L'insieme `s`, parametro attuale della funzione `funInsieme()`, viene a sua volta modificato, come si evince dalla stampa di `s` subito dopo la chiamata della funzione.
- Visto che l'insieme viene modificato, anche in questo caso è inutile far "ritornare" l'insieme dalla funzione.

Questo codice ci conferma come **gli insiemi vengano passati per riferimento (cioè per indirizzo)**, esattamente come già visto per le liste e per i dizionari. Il fatto che l'insieme sia mutabile permette ai suoi elementi di mantenere le modifiche effettuate all'interno della funzione.

Ma, come già visto per le liste, proviamo a modificare il sorgente nel seguente modo:

```
def funInsieme(insieme):
    insieme = insieme | {100}
    insieme.discard(2)
    return insieme

s={1,2,3,4}
print(funInsieme(s))
print(s)
```

Praticamente abbiamo sostituito la riga che aggiungeva l'elemento con `add()` con una riga (quella sottolineata) che effettua l'unione dell'insieme con un altro insieme contenente il solo elemento 100.
Lanciamo il programma, l'output stampato a video è il seguente:

```
{1, 3, 4, 100}
{1, 2, 3, 4}
```

Dall'analisi del codice e dell'output osserviamo che:

L'insieme insieme viene modificato all'interno dello scope della funzione `funInsieme()`.
L'insieme `s` non mantiene le modifiche dell'insieme `insieme`: il comportamento è assimilabile a quello di un passaggio per valore.

Il comportamento differente è generato, ancora una volta, dall'istruzione che abbiamo inserito nella seconda riga (riga sottolineata):

```
insieme = insieme | {100}
```

Questa riga, così come già visto con l'operatore `+` per le liste, va a creare un nuovo oggetto `insieme`, distinto da quello passato come parametro, che comporta una visibilità locale alla sola funzione della variabile stessa.

Per ovviare a questo problema non dobbiamo effettuare nessun assegnamento al parametro formale, pena la creazione di un nuovo oggetto con conseguente visibilità solo locale delle modifiche.

Nel caso specifico potremmo andare a utilizzare i metodi che modificano l'insieme *in place*, come `add()`, `discard()`, `remove()`, `update()`, oppure andare a riassegnare il valore di `s` con il risultato restituito dalla funzione:

```
s = funInsieme(s)
```

Concludiamo quindi che anche **l'insieme**, come la lista e il dizionario, **viene passato a una funzione per riferimento (cioè per indirizzo)** e non per valore. Il medesimo principio generale visto per le liste vale anche per questa struttura dati: qualunque operatore o metodo che riassegna l'intera variabile (come l'operatore `|`, `union()` usato con riassegnazione, o simili) rompe il legame con l'oggetto originale. Mentre i metodi che agiscono direttamente sull'oggetto (come `add()` o `discard()`) mantengono le modifiche visibili anche al di fuori della funzione.

## 5.7 Suggerimenti del tipo degli argomenti e del valore restituito

Sappiamo che il python è un linguaggio debolmente tipizzato, questo significa, tra l'altro, che non è necessario dichiarare i tipi delle variabili utilizzate. Infatti, come abbiamo visto nei precedenti paragrafi, non abbiamo mai dichiarato il tipo degli argomenti e il tipo del valore di ritorno di una funzione.
Usiamo la PEP 3107[17] per "descrivere" il tipo di variabile passata come parametro alla funzione e il tipo di variabile restituita.

Per una migliore leggibilità del codice possiamo indicare il tipo di dato di un parametro di una funzione o del valore da lei restituito.

Esaminiamo il seguente esempio:

```
def areaRettangolo(base: float, altezza: float ) -> float:
     return base*altezza

print("Area 1 = ", areaRettangolo(3.2,4.5))
print("Area 2 = ", areaRettangolo(5,4))
print("Area 3 = ", areaRettangolo("A",3))
print("Area 4 = ", areaRettangolo("A","B"))
```

[17] s://peps.python.org/pep-3107/http

Lanciamo il programma, l'output stampato a video è il seguente:

```
Area 1 =  14.4
Area 2 =  20
Area 3 =  AAA
Traceback (most recent call last):
  File "/ProvePython/area.py", line 8, in <module>
    print("Area4 = ", areaRettangolo("A", "B"))
  File "/ProvePython/area.py", line 2, in areaRettangolo
    return base*altezza
TypeError: can't multiply sequence by non-int of type 'str'
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Suggeriamo il tipo di dato andando a inserire il tipo del dato subito dopo il nome del parametro seguito da ":"
- ➔ Il tipo del valore di ritorno viene indicato subito dopo la lista dei parametri aggiungendo il simbolo "->" seguito dal tipo
- ➔ Il tipo inserito è solo un suggerimento, infatti se chiamiamo la funzione senza seguire tale suggerimento l'interprete Python non produce alcun errore se non quelli che nascono da un utilizzo non corretto degli operatori con operandi non consoni (nell'esempio si vede che non si può moltiplicare due oggetti di tipo stringa!)

# 6. Input da tastiera

## 6.1 Inserimento di stringhe

Scriviamo il seguente programma:

```
print ("Chi sei?")
risposta = input()
print ("Benvenuto ",risposta)
print ("Inserisci N ")
N = input()
print ("N = ",N)
```

Lanciamo il programma (che chiamiamo per comodità inserisci.py) , l'output stampato a video è il seguente:

```
Chi sei?
Neo
Benvenuto  Neo
Inserisci N
61
N =  61
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'istruzione deputata ad inserire stringhe alfanumeriche e valori numeri è input()
- ➔ Non appena l'interprete python incontra un'istruzione di input si blocca e aspetta un inserimento dati da parte dell'utente. Lo standard input è la tastiera.

**N.B.** Si può reindirizzare lo standard input con il comando di ridirezione:"<". Eseguendo da console Linux il precedente programma :

```
$ python inserisci.py < input.txt [invio]
```

dove input.txt è un file che contiene 2 righe:

```
input.txt:
     Pippo
     21
```

Otteniamo il seguente output:

```
Chi sei?
Pippo
Benvenuto  Pippo
Inserisci N
21
N =  21
```

Altro esempio in cui cercheremo di rendere più “gradevole” l'inserimento dati da standard input :

```
a = input(“Inserisci a: “)
b = input(“Inserisci b: “)
print (“a+b = “,a+b)
```

Lanciamo il programma, l’output stampato a video è il seguente:

```
Inserisci a: 2
Inserisci b: 45
a+b =  245
```

Dall’analisi del codice e dell’output osserviamo che:

- ➔ L'istruzione input può avere come argomento una stringa che può rendere più user-friendly l'inserimento del dato.
- ➔ L'inserimento avviene sulla stessa riga della stringa stampata dall'istruzione input.
- ➔ Il risultato dell’operazione di somma è errato, al posto di essere 47 è 245. In questo caso il problema non è che il Python non è in grado di eseguire le operazioni aritmetiche (!) ma l’operatore + non sta effettuando la somma ma la concatenazione di stringhe. Questo perchè la funzione `input` restituisce sempre una stringa e mai un numero. Si consideri che nella versione 2.x del Python si avevano due funzioni per l’inserimento da standard input, una che serviva per le stringhe e l’altra che serviva per i numeri (reali e interi)

L’inserimento dei numeri interi si ottiene effettuando una trasformazione delle stringhe inserite con la funzione `input`.

## 6.2 Inserimento di valori numerici

Abbiamo visto che la funzione input permette di inserire soltanto delle stringhe.
Riprendiamo il precedente programma con alcune modifiche:

```
a = input("Inserisci a: ")
a = int(a)
b = float(input("Inserisci b: "))
print ("a = ",a)
print ("b = ",b)
print ("a+b = ",a+b)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Inserisci a: 2
Inserisci b: 3.2
a =  2
b =  3.2
a+b =  5.2
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `int` permette di convertire la stringa in un numero intero
- ➔ La funzione `float` permette di convertire la stringa in un numero floating point (reale)
- ➔ Le funzioni `int` e `float` possono essere applicate a delle stringhe o a funzioni che restituiscono stringhe

Quindi con `int` e `float` andiamo a effettuare una conversione su una stringa, la quale deve necessariamente contenere un valore numerico valido.

Consideriamo il seguente programma:

```
print(int("ciao"))
print(float("ciao"))
print(float("2"))
print(int("2.2"))
print(float(3))
print(int(3.3))
```

Eseguite le singole righe commentando le righe che producono errori oppure lanciate direttamente i comandi utilizzando l'interfaccia interattiva. Abbiamo utilizzato questa ultima modalità per comodità. Otteniamo in output:

```
>>> print(int("ciao"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'ciao'
>>> print(float("ciao"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: 'ciao'
>>> print(float("2"))
2.0
>>> print(int("2.2"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '2.2'
>>> print(float(3))
```

```
3.0
>>> print(int(3.3))
3
```

Esaminando l' output osserviamo che:

- ➔ Le prime due istruzioni producono un `valueError` in quanto si cerca di convertire in un valore numerico una stringa alfabetica
- ➔ La terza e quarta istruzione fa comprendere che la funzione `int` puo' generare un errore se il suo argomento è una stringa che contiene un numero floating point.
- ➔ La quinta e sesta istruzione ci fanno comprendere che gli argomenti di `int` e `float` possono anche essere dei valori numerici. La funzione in questo caso andrà a convertire un valore reale in uno intero e, viceversa, un intero in uno reale.

## 6.3 Inserimento di più valori sulla stessa riga

Lo sviluppatore spesso desidera che un utente inserisca più valori o input in una riga. In C++/C l'utente può prendere più input in una riga usando la funzione standard `scanf` .
In Python input può permettere di prendere in input un unico valore per linea. Quello che andiamo a leggere dallo standard input viene restituito dalla funzione input come una stringa.
Per aggirare il problema possiamo pensare di chiedere all'utente di inserire tutto su una linea e poi con un'opportuna elaborazione andiamo a estrarre dalla stringa inserita i campi che definiscono il nostro input.

Consideriamo il seguente programma:

```
a,b = input("Inserisci due stringhe separate da spazio: ").split()
print("a: ", a)
print("b: ", b)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Inserisci due stringhe separate da spazio: str1 str2
a:  str1
b:  str2
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Si utilizza il metodo di `split()` che permette di trasformare una stringa in una lista suddividendola in più elementi. Gli elementi vengono individuati grazie al separatore indicato come argomento di split(). Se non è indicato nulla viene considerato come separatore uno spazio.
- ➔ Le variabili a e b sono a loro volte delle stringhe

Se volessimo inserire dei valori numerici possiamo effettuare un cast come indicato nel precedente paragrafo.

Quindi possiamo inserire nel precedente programma le seguenti linee di codice:

```
a = float(a)
b = float(b)
```

Questo nel caso volessimo chiedere due variabili float in input.

Possiamo utilizzare la funzione map per eseguire tutto con una sola linea di istruzione. A questo proposito consideriamo il seguente programma:

```
a,b = map(float,input("Inserisci due numeri separati da una
virgola ").split(“,”))
print("a: ", a)
print("b: ", b)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Inserisci due numeri separati da una virgola 3.5,4.5
a:  3.5
b:  4.5
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `map` permette di applicare la funzione **float** sugli elementi inseriti andando a effettuare il cast simultaneo su tutti i numeri inseriti
- ➔ Split ha come argomento una virgola, quindi utilizza come separatore degli elementi della stringa inserita, la virgola.

# 7. Strutture dati

Una struttura dati è un'entità usata per organizzare un insieme di dati all'interno della memoria del computer, ed eventualmente per memorizzarli in una memoria di massa. La scelta delle strutture dati da utilizzare è strettamente legata a quella degli algoritmi; per questo, spesso essi vengono considerati insieme. Infatti, la scelta della struttura dati influisce inevitabilmente sull'efficienza computazionale degli algoritmi che la manipolano. La struttura dati è un metodo di organizzazione dati, quindi prescinde da ciò che è effettivamente contenuto. [18]

Il Python implementa cinque strutture dati:

- ➔ stringhe,
- ➔ liste,
- ➔ tuple,
- ➔ dizionari,
- ➔ insiemi .

## 7.1 Stringhe

Abbiamo già utilizzato le stringhe negli esempi visti precedentemente.
Ora cerchiamo di focalizzare il nostro studio su cosa si intende per stringa e come viene gestita dal Python.

Una stringa è una sequenza ordinata di caratteri.

È il più semplice esempio di struttura dati in Python. Sulle stringhe sono consentite operazioni specifiche di concatenazione e di accesso agli elementi. Scopriremo che sono in realtà degli oggetti che esprimono attributi e metodi propri.

La stringa è una struttura dati immutabile, cioè non è permesso eliminare o modificare uno o più elementi della stessa.

### 7.1.1 Creare una stringa

Proviamo il seguente programma:

```
str1 = "Prima stringa"
print(str1)
str2 = 'Seconda stringa'
print(str2)
str3 = """Terza stringa"""
```

[18] https://it.wikipedia.org/wiki/Struttura_dati

```
print(str3)
str4 = "'Quarta' stringa"
print(str4)
str5 = '"Quinta" stringa'
print(str5)
str6 = "Sesta\" stringa"
print(str6)
```

Lanciamo il programma, in uscita otteniamo:

```
Prima stringa
Seconda stringa
Terza stringa
'Quarta' stringa
"Quinta" stringa
Sesta" stringa
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Una stringa viene definita da una sequenza di caratteri alfanumerici racchiusi tra doppi o singoli apici
- ➔ Una stringa viene anche definita da una sequenza di caratteri racchiusi tra tre doppi apici o singoli apici. Questa notazione serve a definire una docstring, che è una stringa che documenta la funzionalità di una classe, un metodo, una funzione o un modulo. Affronteremo questo argomento quando studieremo le classi.
- ➔ Se volessimo inserire degli apici all'interno della stringa dobbiamo delimitare la stringa con doppi apici e viceversa se volessimo inserire dei doppi apici nella stringa.
- ➔ Se volessimo inserire doppi apici all'interno di una stringa delimitata da doppi apici possiamo farlo utilizzando \" cioè usiamo una sequenza di escape. Analogamente se volessimo inserire un singolo apice in una stringa delimitata da singoli apici. Studieremo più in dettaglio le sequenze di escape quando approfondiremo la formattazione delle stringhe.

### 7.1.2 Accedere agli elementi di una stringa

Esaminiamo il seguente programma:

```
str1 = "Stringa"
carattere = str1[1]
print(carattere)
print(str1[0])
print(len(str1))
print(str1[-1], str1[-len(str1)])
i = 1
print(str1[i+1])
for i in range(len(str1)):
```

```
    print(i,": ",str1[i])
```

Lanciamo il programma, in uscita otteniamo:

```
t
S
7
a S
r
0 :  S
1 :  t
2 :  r
3 :  i
4 :  n
5 :  g
6 :  a
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo accedere ai caratteri della stringa indicando la loro posizione numerica all'interno delle parentesi quadre
- ➔ La prima posizione è 0
- ➔ La funzione `len` restituisce il numero di caratteri all'interno della stringa
- ➔ Indici negativi indicano la posizione dei caratteri a partire dall'ultimo
- ➔ L'indice di un carattere può essere il risultato di un'espressione
- ➔ Possiamo scandire l'intera stringa utilizzando un ciclo `for` e l'istruzione `range`

A proposito dell'ultimo punto, possiamo scandire una stringa anche nel seguente modo. Esaminiamo il seguente programma:

```
stringa = "Stringa"
for carattere in stringa :
    print(carattere,": ",ord(carattere))
```

Lanciamo il programma, in uscita otteniamo:

```
S :  83
t :  116
r :  114
i :  105
n :  110
g :  103
a :  97
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo scorrere l'intera stringa utilizzando il costrutto `for - in` , questo è possibile in quanto la stringa è un oggetto *iterabile*.

➔ La funzione `ord` restituisce il corrispondente codice numerico nella tabella *Unicode*. La funzione inversa è `chr` che accetta come argomento un codice numerico e restituisce il carattere corrispondente nella codifica *Unicode*.

### 7.1.3 Operazioni tra stringhe

Esaminiamo il seguente programma:

```
str1 = "Stringa1"
str2 = "Stringa2"
strconc = str1 + str2
print(strconc)
strrep = str1 *3
print(strrep)
```

Lanciamo il programma, in uscita otteniamo:

```
Stringa1Stringa2
Stringa1Stringa1Stringa1
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ In Python è possibile concatenare più stringhe tra di loro e assegnare la concatenazione a un'altra stringa
- ➔ L'operatore di concatenazione è +
- ➔ L'operatore * permette replicare n volte la stringa operando creando così una nuova stringa

### 7.1.4 Modificare i caratteri di una stringa

Esaminiamo il seguente programma:

```
str1 = "Stringa"
str1[0] = 'X'
print(str1)
```

Lanciamo il programma, in uscita otteniamo:

```
Traceback (most recent call last):
  File "/home/massimo/str.py", line 2, in <module>
    str1[0] = 'X'
TypeError: 'str' object does not support item
assignment
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il programma ha generato un errore nella linea 2
- ➔ Il problema consiste nel fatto che non possiamo modificare un carattere della stringa
- ➔ Questo significa che le stringhe sono *immutabili*, quindi non possiamo modificarle.

Ora esaminiamo il seguente programma:

```
str1 = "Stringa"
print(str1)
str1 = "Altra"
print(str1)
str1 += " suffisso"
print(str1)
str1 *= 3
print(str1)
```

Lanciamo il programma, in uscita otteniamo:

```
Stringa
Altra
Altra suffisso
Altra suffissoAltra suffissoAltra suffisso
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Sembrerebbe che si possa sotituire completamente il contenuto di una stringa. Ma in realtà essendo la stringa una struttura dati immutabile, non modifichiamo il contenuto di una stringa, ma riassegniamo la variabile a un nuovo oggetto creato in memoria.

  * Dal punto di vista della gestione della memoria, è utile visualizzare cosa accade: Per l'immutabilità: una volta che l'oggetto `"Stringa"` è creato, non può mai cambiare. Per la riassegnazione: quando scrivi `str1 = "Altra"`, non stai "sovrascrivendo" le lettere dentro il primo oggetto; stai semplicemente dicendo alla variabile `str1` di puntare a un nuovo indirizzo di memoria dove risiede la parola `"Altra"`.

- ➔ Possiamo utilizzare la concatenazione utilizzando l'operatore **+=**
- ➔ Possiamo utilizzare la replicazione di stringhe utilizzando l'operatore `*=`

  * Anche in questo caso precisiamo che il fatto che la stringa sia un tipo immutabile porta a dire che operatori come `+=` o `*=` non alterano la stringa originale, ma generano una nuova istanza che sostituisce il riferimento precedente nella variabile.

### 7.1.5 Estrazione di sottostringhe

Esaminiamo il seguente programma:

```
str1 = "Stringa"
sottostringa = str1[2:4]
print(sottostringa)
print(str1[0:1])
print(str1[:4])
print(str1[5:])
print(str1[-3:-1])
```

Lanciamo il programma, in uscita otteniamo:

```
ri
S
Stri
ga
ng
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ È possibile specificare sezioni di stringa per ottenere una nuova stringa.
- ➔ La nuova sottostringa viene identificata attraverso l'indice iniziale e l'indice finale separati da `:` e racchiuse tra parentesi quadre.
- ➔ L'estremo superiore dell'intervallo non appartiene alla sottostringa
- ➔ Se manca l'estremo inferiore si prendono tutti i caratteri sino all'estremo superiore
- ➔ Se manca l'estremo superiore si prendono tutti i caratteri dall'estremo inferiore sino a fine stringa
- ➔ È possibile specificare indici negativi che specificano le posizioni a partire dall'ultimo carattere della stringa (dal fondo della stringa)

### 7.1.6 Confrontare le stringhe

Esaminiamo il seguente programma:

```
str1 = "ABC"
str2 = "abc"
str3 = "ABC"
str4 = "ABCD"
str5 = "ABD"
print(str1 == str2)
print(str1 > str2)
print(str1 == str3)
print(str1 > str4)
print(str1 > str5)
```

Lanciamo il programma, in uscita otteniamo:

```
False
False
True
False
False
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'operatore == restituisce True se tutti gli elementi sono uguali
- ➔ Si considera l'ordinamento dei caratteri secondo la posizione nella tabella Unicode (p.e. caratteri maiuscoli sono "più piccoli" dei caratteri minuscoli)
- ➔ Se la lunghezza delle due stringhe differisce l'operatore > o < ci dà informazioni sulla differenza della lunghezza

### 7.1.7 Appartenenza a una stringa

Esaminiamo il seguente programma:

```
str1 = "Stringa"
print ('o' in str1)
print ('ing' in str1)
```

Lanciamo il programma, in uscita otteniamo:

```
False
True
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'operatore in permette di capire se una sottostringa è contenuta in una stringa
- ➔ Questo è un'operatore di membership che abbiamo incontrato nel capitolo 4.2.5

## 7.2 Liste

Una lista è una struttura di dati che contiene una raccolta ordinata di elementi, ovvero è possibile memorizzare una sequenza di elementi in un elenco.
L'elenco degli elementi deve essere racchiuso tra parentesi quadre. Dopo aver creato un elenco, puoi aggiungere, rimuovere o cercare elementi nell'elenco. Poiché possiamo aggiungere e rimuovere elementi, diciamo che la lista è un tipo di dati mutabile , ovvero questo tipo può essere modificato.

### 7.2.1 Creare una lista

Proviamo il seguente programma:

```
lista = [ -10, ‘ciao’, 34.6 , True]
print(lista)
```

Lanciamo il programma, in uscita otteniamo:

```
[-10, 'ciao', 34.6, True]
```

Dall’analisi del codice e dell’output osserviamo che:

- La lista viene definita da un elenco di elementi separati da virgole e racchiusi tra parentesi quadre
- Gli elementi possono essere di differente tipo, nell’esempio abbiamo degli interi, dei floating point, delle stringhe e dei booleani

### 7.2.2 Accedere agli elementi di una lista

Esaminiamo il seguente programma:

```
lista = [ -10, ‘ciao’, 34.6 , True]
print(lista[0], lista[1], lista[3])
print(len(lista))
print(lista[-4], lista[-1])
i = 1
print(lista[i+1])
for i in range(len(lista)):
     print(i,”: “,lista[i])
```

Lanciamo il programma, in uscita otteniamo:

```
-10 ciao True
4
-10 True
34.6
0 :  -10
1 :  ciao
2 :  34.6
3 :  True
```

Dall’analisi del codice e dell’output osserviamo che:

- Possiamo accedere agli elementi della lista indicando la loro posizione numerica all’interno delle parentesi quadre
- La prima posizione è 0
- La funzione `len` restituisce il numero di elementi all’interno della lista
- Indici negativi indicano la posizione degli elementi a partire dall’ultimo
- L’indice di un elemento può essere il risultato di un’espressione
- Possiamo scandire l’intera lista utilizzando un ciclo `for` e l’istruzione `range`

Come già osservato per la scansione delle stringhe, possiamo scandire una lista anche nel seguente modo. Esaminiamo il seguente programma:

```
lista = [ -10, 'ciao', 34.6 , True]
for elemento in lista :
     print(elemento)
```

Lanciamo il programma, in uscita otteniamo:

```
-10
ciao
34.6
True
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo scorrere l'intera lista utilizzando il costrutto `for - in` , questo è possibile in quanto la lista è un oggetto *iterabile*. Naturalmente questo è il modo più "naturale" in Python per effettuare tale operazione.

### 7.2.3 Modificare gli elementi di una lista

Esaminiamo il seguente programma:

```
lista = [ -10, 'ciao', 34.6 , True]
lista[0] = 5
lista[1] = 6
lista[2] = 'pippo'
print(lista)
```

Lanciamo il programma, in uscita otteniamo:

```
[5, 6, 'pippo', True]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo modificare i singoli elementi della lista
- ➔ Siamo liberi di modificare anche il tipo di elemento
- ➔ Questo significa che le liste sono *mutabili*

### 7.2.4 Modificare le liste

Esaminiamo il seguente programma:

```
lista = []
lista += [3]
```

```
lista += ['c',2.3]
print(lista)
lista2 = [10, 30]
lista3 = lista + lista2
print(lista3)
lista4 = lista2 * 3
print(lista4)
```

Lanciamo il programma, in uscita otteniamo:

```
[3, 'c', 2.3]
[3, 'c', 2.3, 10, 30]
[10, 30, 10, 30, 10, 30]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo aggiungere elementi a una lista
- ➔ Possiamo aggiungere un singolo elemento ma dobbiamo "trasformarlo" in una lista di un unico elemento.
- ➔ Possiamo concatenare gli elementi di una lista a un'altra lista
- ➔ L'operatore * permette replicare n volte la lista creando così una nuova lista

### 7.2.5 Confrontare le liste

Esaminiamo il seguente programma:

```
l1 = [1, 2, 3]
l2 = [1, 2, 3]
l3 = [1, 2, 3, 4]
print( l1 == l2 )
print( l1==l3 )
print( l1 < l3 )
print( l3 >= l2 )
```

Lanciamo il programma, in uscita otteniamo:

```
True
False
True
True
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'operatore == restituisce True se tutti gli elementi sono uguali e se ce ne sono in egual numero
- ➔ Se le lunghezze delle due liste differiscono l'operatore > o < ci dà informazioni sulla

differenza della lunghezza. (Questo avviene solo nel caso in cui gli elementi corrispondenti sono uguali)

### 7.2.6 Alcuni metodi specifici delle liste

Affrontiamo ora una veloce carrellata sui metodi peculiari dell'oggetto lista.
la lista ci offre i seguenti metodi che possiamo proficuamente utilizzare nelle nostre applicazioni:

1. `index`
2. `count`
3. `list`
4. `append`
5. `extend`
6. `insert`
7. `clear`
8. `pop`
9. `remove`
10. `reverse`
11. `sort`

I primi due li incontreremo nel paragrafo *7.4 Operazioni sulle sequenze* ( cfr. 7.4 ) quindi tratteremo i successivi.
(Attenzione `list` non è un metodo delle liste ma una funzione che accetta oggetti complessi come parametri)

#### 7.2.6.1 Funzione `list()`

Questa funzione permette di creare una lista vuota, oppure una lista avente come elementi gli attributi di un oggetto complesso.

Esaminiamo il seguente programma:

```
lista1 = list()
print(lista1)
lista2 = list("stringa")
print(lista2)
```

Lanciamo il programma, in uscita otteniamo:

```
[]
['s', 't', 'r', 'i', 'n', 'g', 'a']
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `list()` senza alcun parametro formale, permette di creare una lista vuota.

- ➔ Il funzione `list()` senza parametri formali è analoga all'istruzione: `l = []` che serve per inizializzare una lista vuota
- ➔ Se alla funzione `lista()` passiamo come parametro un oggetto complesso, questo viene trasformato in una lista i cui elementi sono gli elementi dell'oggetto stesso. P.e. se passiamo una stringa otteniamo una lista che ha come elementi i caratteri della stringa.

### 7.2.6.2 Metodi `append()`, `extend()` e `insert()`

Questi tre metodi permettono di incrementare la numerosità di una lista andando a inserire dei nuovi elementi.

Esaminiamo il seguente programma:

```
lista = [8, -3, 15, -2]
lista.append(10)
print(lista)
lista.extend([30,40])
print(lista)
lista.insert(2,-1)
print(lista)
```

Lanciamo il programma, in uscita otteniamo:

```
[8, -3, 15, -2, 10]
[8, -3, 15, -2, 10, 30, 40]
[8, -3, -1, 15, -2, 10, 30, 40]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `append()` permette di inserire un elemento al fondo della lista.
- ➔ Il metodo `append()` è analogo all'operazione `lista += [ele]` che va ad accodare l'elemento `ele` in fondo alla lista.
- ➔ Il metodo `extend()` permette di concatenare la lista inserita come argomento in coda alla lista che richiama il metodo. In pratica 'estenda' la lista con la lista presente nell'argomento del metodo.
- ➔ Il metodo `insert()` permette di inserire un elemento nella lista nella posizione indicata dal primo parametro.
- ➔ la sintassi del metodo `insert()` è la seguente:

  `insert(posizione, elemento)`

  dove: `posizione` è la posizione in cui si vuole inserire l'elemento
  `elemento` è il valore che si vuole inserire.
  Se `posizione` è maggiore di `len(lista)` allora `elemento` viene inserito in coda

### 7.2.6.3 Metodi `clear()`, `pop()` e `remove()`

Questi tre metodi permettono di decrementare la numerosità di una lista andando a rimuovere gli elementi della lista stessa.

Esaminiamo il seguente programma:

```
lista = [8, -3, 15, -2]
print(lista.pop())
print(lista)
lista.remove(-3)
print(lista)
lista.clear()
print(lista)
```

Lanciamo il programma, in uscita otteniamo:

```
-2
[8, -3, 15]
[8, 15]
[]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `pop()` permette di rimuovere l'ultimo elemento inserito (ordinamento LIFO). Restituisce l'elemento rimosso.
- ➔ Il metodo `remove(ele)` consente di rimuovere l'elemento `ele` indicato come parametro al metodo stesso. Se l'elemento non esiste allora l'interprete Python genera un messaggio di errore di tipo `ValueError`
- ➔ Il metodo `clear()` rimuove tutti gli elementi della lista

### 7.2.6.4 Metodi `sort()` e `reverse()`

Questi due metodi permettono di modificare l'ordine degli elementi all'interno della lista.

Esaminiamo il seguente programma:

```
lista = [8, -3, 15, -2]
print(lista)
lista.sort()
print(lista)
lista.reverse()
print(lista)
```

Lanciamo il programma, in uscita otteniamo:

```
[8, -3, 15, -2]
[-3, -2, 8, 15]
[15, 8, -2, -3]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `sort()` permette di ordinare gli elementi della lista

➔ Il metodo `reverse()` inverte le posizioni degli elementi della lista

In generale la sintassi del metodo `sort` è la seguente:

```
sort(reverse=True|False, key=myFunc)
```

dove:

`reverse` indica se l'ordinamento deve essere crescente o decrescente

`key`

#### 7.2.6.4.1 Funzione `sorted()` e `reversed()`

Il metodo `reverse()` restituisce la lista invertita. A questo proposito esiste la funzione `reversed()` che restituisce un iteratore che itera sugli elementi invertiti della sequenza fornita come parametro formale.

Come esempio si esamini il seguente codice:

```
lista = [8, -3, 15, -2]
for ele in reversed(lista):
   print(ele,end=" ")
print(lista)
```

lanciandolo si ottiene il seguente eloquente output:

```
-2 15 -3 8
[8, -3, 15, -2]
```

Quanto visto per il metodo `reverse()` vale anche per il metodo sort, quindi troviamo una funzione `sorted()` analoga alla funzione `reverted()`.

## 7.3 Tuple

Una tupla in Python è una sequenza *immutabile* di elementi eterogenei che ammette duplicati, perciò possiamo mettere gli oggetti che vogliamo, anche di tipi diversi, e anche ripetuti.
E' del tutto simile alla lista eccezion per il fatto che la lista è mutabile mentre la tupla no.

### 7.3.1 Creare una tupla

Proviamo il seguente programma:

```
tupla = 'Pippo', -4 , 3.6
print(tupla)
altraTupla = (3,'ciao')
print(altraTupla)
tuplaVuota = ()
print(tuplaVuota)
print(len(tupla))
```

```
print(len(altraTupla))
print(len(tuplaVuota))
```

Lanciamo il programma, in uscita otteniamo:

```
('Pippo', -4, 3.6)
(3, 'ciao')
()
3
2
0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo creare una tupla andando a elencare i suoi elementi separandoli con delle virgole
- ➔ Nella creazione della tupla gli elementi possono essere racchiusi tra parentesi tonde
- ➔ L'output di una tupla sarà sempre rappresentata come successione di elementi separati dalle virgole e racchiusi tra parentesi tonde.
- ➔ Possiamo creare la tupla vuota aprendo e chiudendo le parentesi tonde senza andare a inserire alcun elemento all'interno delle stesse
- ➔ Utilizziamo la funzione `len` per conoscere il numero di elementi all'interno della tupla

In generale è preferibile creare una tupla utilizzando le parentesi tonde per questioni di chiarezza del codice.

#### 7.3.1.1 Creare una tupla con un singolo elemento

La creazione di una tupla che contenga un singolo elemento non è banale.
Proviamo il seguente programma:

```
tuplaFalsa = (4)
print(tuplaFalsa)
print(type(tuplaFalsa))
tuplaVera = (4,)
print(tuplaVera)
print(type(tuplaVera))
print(len(tuplaVera))
altraTuplaVera = 4,
print(altraTuplaVera)
print(type(altraTuplaVera))
print(len(altraTuplaVera))
```

Lanciamo il programma, in uscita otteniamo:

```
4
<class 'int'>
```

```
(4,)
<class 'tuple'>
1
(4,)
<class 'tuple'>
1
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Per creare una tupla con un solo elemento dobbiamo inserire la virgola
- ➔ Se omettiamo la virgola otteniamo il tipo di dato dell'elemento della tupla che volevamo costruire
- ➔ L'istruzione `type` ci restituisce il tipo di dato del suo argomento.
- ➔ Possiamo creare una tupla con un singolo elemento aggiungendo subito dopo la virgola anche senza l'utilizzo delle parentesi tonde.

### 7.3.2 Accedere agli elementi di una tupla

Esaminiamo il seguente programma:

```
tupla = ( -10, 'ciao', 34.6 , True)
print(tupla[0], tupla[1], tupla[3])
print(len(tupla))
print(tupla[-4], tupla[-1])
i = 1
print(tupla[i+1])
for i in range(len(tupla)):
     print(i,": ",tupla[i])
```

Lanciamo il programma, in uscita otteniamo:

```
-10 ciao True
4
-10 True
34.6
0 :  -10
1 :  ciao
2 :  34.6
3 :  True
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo accedere agli elementi della tupla indicando la loro posizione numerica all'interno di parentesi quadre
- ➔ La prima posizione è 0
- ➔ Indici negativi indicano la posizione degli elementi a partire dall'ultimo
- ➔ L'indice di un elemento può essere il risultato di un'espressione
- ➔ Possiamo scandire l'intera tupla utilizzando un ciclo `for` e l'istruzione `range`

Si noti che il programma lanciato è esattamente lo stesso utilizzato per il paragrafo analogo sulle liste. Quindi si accede agli elementi della tupla nello stesso identico modo con cui si accede agli elementi di una lista.
A riguardo dell'ultimo punto dell'analisi dell'output, cioè della scansione di tutti gli elementi di una tupla con un ciclo iterativo, è importante far notare che, solitamente, su una tupla non si accede ai suoi elementi in questa modalità. L'accesso avviene indicando direttamente l'elemento della tupla oppure gli elementi vengono acceduti nel momento in cui vengono assegnati a una tupla di variabili.

Per chiarire quest'ultimo concetto si provi il seguente programma:

```
tupla = (8 , 'ciao',  True)
a,b,c = tupla
print(a,type(a))
print(b,type(b))
print(c,type(c))
```

Lanciandolo otteniamo il seguente output:

```
8 <class 'int'>
ciao <class 'str'>
True <class 'bool'>
```

Dall'analisi del codice e dell'output osserviamo che abbiamo effettuato un' assegnazione multipla di tutti gli elementi della tupla `tupla` a 3 singole variabili di tipi differenti.

#### 7.3.2.1 Esempio notevole di assegnazione di tuple

Un'applicazione interessante di questa peculiarità dell'assegnazione di più variabili in un'unica istruzione la si ottiene con il problema dello scambio dei valori tra due variabili.
Supponiamo di avere la variabile X e la variabile Y, per effettuare lo scambio di valori dobbiamo utilizzare una variabile di supporto Z e quindi eseguire le seguenti istruzioni:

```
X = 2
Y = 1
print(X," ",Y)
Z = X
X = Y
Y = Z
print(X," ",Y)
```

Lanciando il programma otteniamo il seguente output:

```
2    1
1    2
```

Abbiamo ottenuto lo scambio effettivo dei valori contenuti nelle due variabili.

Proviamo a scambiare i valori lavorando con le tuple.
Si provi il seguente programma:

```
X = 2
Y = 1
print(X," ",Y)
X,Y = Y,X
print(X," ",Y)
```

Lanciando il programma otteniamo il seguente output:

```
2   1
1   2
```

Abbiamo ottenuto lo stesso identico output ma al posto di scrivere 3 istruzioni abbiamo scritto una unica istruzione e abbiamo aumentato la comprensibilità del codice.

#### 7.3.2.2 Funzioni che restituiscono tuple

Le funzioni solitamente restituiscono un oggetto di un determinato tipo di dati oppure restituiscono nulla.
In linguaggio Python possiamo far restituire più oggetti contemporaneamente utilizzando le tuple.

Consideriamo il seguente programma che richiama una funzione che calcola sia l'area di un rettangolo che il perimetro:

```
def rettangolo(base, altezza):
    return base*altezza, 2*base+2*altezza

a=2
b=3
print(rettangolo(a,b))
area,perimetro = rettangolo(4,6)
print("area= ",area," perimetro= ",perimetro)
```

Lanciando il programma otteniamo il seguente output:

```
(6, 10)
area=  24  perimetro=  20
```

La funzione *rettangolo* è come se ritornasse 2 valori: l'area del rettangolo e il suo perimetro. Nell'esempio vediamo come stampare direttamente i due valori dii ritorno, oppure come assegnarli prima a due variabili e successivamente stamparli.

### 7.3.3 Modificare gli elementi di una tupla

Esaminiamo il seguente programma:

```
tupla = ( -10, 'ciao', 34.6 , True )
tupla[0] = 5
print(tupla)
```

Lanciamo il programma, in uscita otteniamo:

```
Traceback (most recent call last):
  File "prova.py", line 2, in <module>
    tupla[0] = 5
TypeError:  'tuple'  object  does  not  support  item
assignment
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il programma ha generato un errore nella linea 2
- ➔ Il problema consiste nel fatto che non possiamo modificare un elemento della tupla
- ➔ Questo significa che le tuple sono *immutabili* a differenza delle liste che sono *mutabili*

#### 7.3.3.1 Modificare gli elementi di un oggetto in una tupla

Abbiamo visto nel paragrafo precedente che non possiamo modificare gli elementi che compongono una tupla. Ma se un elemento di una tupla è un oggetto complesso, p.e. una lista, cosa succede se cerchiamo di modificare gli elementi di quell'oggetto complesso?

A tal proposito esaminiamo il seguente programma:

```
tupla = (2,'q',[10,20,30])
lista = tupla[2]
print(lista)
print(lista[1])
print(tupla[2][1])
tupla[2][1]=-10
print(tupla)
```

Lanciamo il programma, in uscita otteniamo:

```
[10, 20, 30]
20
20
(2, 'q', [10, -10, 30])
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il terzo elemento della tupla è una lista a cui posso accedere utilizzando il riferimento `tupla[2]`
- ➔ Il secondo elemento della lista contenuta nella tupla è riferibile utilizzando `tupla[2][1]`
- ➔ Se provo a modificare il secondo elemento della lista contenuta nella tupla, noto che l'assegnazione va a buon fine

Questo esempio ci fa comprendere come gli elementi di una tupla sia immutabili, ma gli elementi interni a singoli componenti della tupla possono essere modificabili se l'oggetto è un mutabile.
Ne consegue che le tuple possono contenere oggetti mutabili

Attenzione: non posso eseguire l'istruzione: `tupla[2] += [2,3,4]` oppure `tupla[2] = [2,3,4]` in quanto in questa maniera stiamo cercando di modificare un elemento della tupla che per definizione è immutabile.

### 7.3.4 Modificare le tuple

Esaminiamo il seguente programma:

```
tupla = ()
tupla += (3,)
tupla += ('c',2.3)
print(tupla)
tupla2 = (10, 30)
tupla3 = tupla + tupla2
print(tupla3)
tupla4 = tupla2 * 3
print(tupla4)
```

Lanciamo il programma, in uscita otteniamo:

```
(3, 'c', 2.3)
(3, 'c', 2.3, 10, 30)
(10, 30, 10, 30, 10, 30)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo aggiungere un singolo elemento ma dobbiamo "trasformarlo" in una tupla di un unico elemento.
- ➔ Possiamo concatenare gli elementi di una tupla a un'altra tupla
- ➔ L'operatore `*` è l'operatore di replicazione, permette replicare n volte la tupla creando così una nuova tupla

Si fa notare che questo è lo stesso identico comportamento della lista.

### 7.3.5 Concatenare tuple e liste

Esaminiamo il seguente programma:

```
lista = [2,3]
lista += ('c',2.3)
print(type(lista))
print(lista)
tupla = (2,3)
tupla += ['c',2.3]
print(tupla)
```

Lanciamo il programma, in uscita otteniamo:

```
<class 'list'>
[2, 3, 'c', 2.3]
Traceback (most recent call last):
  File "prova.py", line 6, in <module>
    tupla += ['c',2.3]
TypeError: can only concatenate tuple (not "list") to tuple
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo aggiungere elementi a una lista concatenando una tupla
- ➔ Gli elementi della tupla concatenata diventano i nuovi elementi della lista
- ➔ Non possiamo concatenare una lista a una tupla
- ➔ Alle tuple possiamo solo concatenare altre tuple

## 7.4 Operazioni sulle sequenze

In questo capitolo andremo a studiare alcune funzioni speciali che possono essere utilizzate con grande profitto sul tipo dati sequenza cioè sulle stringhe, tuple e liste (alcune di queste anche sui dizionari e insiemi)

### 7.4.1 Ricerca di un elemento

La ricerca di un elemento in una sequenza viene effettuato con il metodo `index` .

Esaminiamo il seguente programma:

```
lista = [10,3,7,1,9,3,2,4,6,8]
tupla = (10,3,7,1,9,3,2,4,6,8)
stringa = "abdc45hyydsa"
print(lista.index(3))
print(tupla.index(3))
print(stringa.index('d'))
```

```
print(lista.index(3,4))
print(lista.index(3,4,10))
print(lista.index(-2))
```

Lanciamo il programma, in uscita otteniamo:

```
1
1
2
5
5
Traceback (most recent call last):
  File "/home/massimo/PythonProjects/prova.py", line 10, in
<module>
    print(lista.index(-2))
ValueError: -2 is not in list
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ `index` ci permette di effettuare la ricerca dell'elemento inserito nell'argomento
- ➔ `index` funziona nella stessa identica maniera sia per le *liste* che per le *tuple* che per le *stringhe*
- ➔ la sintassi generale del metodo è la seguente:

  `index(elemento, indiceIniziale, indiceFinale)`

  dove: elemento è l'elemento della sequenza da ricercare

  `indiceIniziale` è la posizione all'interno della sequenza da cui viene iniziata la ricerca

  `indiceFinale` è la posizione all'interno della sequenza a cui si arriva nella ricerca (indice finale escluso)
- ➔ se l'elemento ricerca non esiste si produce un errore del tipo `ValueError`

Per evitare di produrre un errore possiamo utilizzare gli operatori di membership `in` e `not in` (paragrafo 4.2.5).

Esaminiamo il seguente programma:

```
lista = [10,3,7,1,9,3,2,4,6,8]
print(1 in lista)
print(-2 in lista)
print(-2 not in lista)
```

Lanciamo il programma, in uscita otteniamo:

```
True
False
True
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ gli operatori di membership possono essere impiegati nella ricerca di un elemento in una sequenza ma non riportano l'esatta posizione. Danno solo una indicazione sulla presenza o meno dell'elemento nella sequenza
- ➔ se l'elemento non è presenta nella sequenza non nasce un ValueError ma si ha il valore booleano `False` restituito dal test di membership
- ➔ naturalmente il codice si riferisce alla sola *lista* per brevità, ma quanto detto è valido anche per le *tuple* e le *stringhe*

### 7.4.2 Il numero di occorrenze di un elemento

Un'altra informazione che risulta essere molto interessante conoscere da una sequenza è sapere quante volte è presente un elemento, cioè quante sono le sue occorrenze nella sequenza stessa. Questa operazione è svolta dal metodo `count`.

Esaminiamo il seguente programma:

```
lista = [10,3,7,1,9,3,2,4,6,8]
tupla = (10,3,7,1,9,3,2,4,6,8)
stringa = "abdc45hyydsa"
print(lista.count(3))
print(tupla.count(3))
print(stringa.count('d'))
print(lista.count(-2))
print(tupla.count(-2))
print(stringa.count('z'))
```

Lanciamo il programma, in uscita otteniamo:

```
2
2
2
0
0
0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ `count` ci permette di effettuare la ricerca del numero di occorrenze dell'elemento inserito nell'argomento
- ➔ `count` funziona nella stessa identica maniera sia per le *liste* che per le *tuple* che per le *stringhe*
- ➔ se l'elemento cercato non esiste allora il conteggio è pari a 0, non restituisce un errore come il metodo `index` visto precedentemente

INSERIRE ESEMPIO RICERCA OCCORRENZA SOTTOSEQUENZA

### 7.4.3 Estrazioni di sotto-sequenze

Possiamo estrarre da una sequenze delle sotto-sequenze analogamente a quanto già visto per le estrazioni delle sotto-stringhe (paragrafo 7.1.5)

Esaminiamo il seguente programma:

```
lista = [10,3,7,1,9,3,2,4,6,8]
tupla = (10,3,7,1,9,3,2,4,6,8)
stringa = "abdc45hyydsa"
print(lista[2:4])
print(tupla[2:4])
print(stringa[2:4])
print(lista[4:])
print(tupla[4:])
print(stringa[4:])
print(lista[:4])
print(tupla[:4])
print(stringa[:4])
print(lista[4])
print(tupla[4])
print(stringa[4])
print(lista[-4:])
print(tupla[-4:])
print(stringa[-4:])
print(lista[:-4])
print(tupla[:-4])
print(stringa[:-4])
```

Lanciamo il programma, in uscita otteniamo:

```
[7, 1]
(7, 1)
dc
[9, 3, 2, 4, 6, 8]
(9, 3, 2, 4, 6, 8)
45hyydsa
[10, 3, 7, 1]
(10, 3, 7, 1)
abdc
9
9
4
[2, 4, 6, 8]
(2, 4, 6, 8)
ydsa
[10, 3, 7, 1, 9, 3]
```

```
(10, 3, 7, 1, 9, 3)
abdc45hy
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Le parentesi quadre ci permettono di individuare gli slices sia in una *lista* che in una *tupla* che in una *stringa* nella stessa identica maniera.
- ➔ La sintassi generale per selezionare uno slice è la seguente:

```
sequenza[ini:fin]
```

dove: `ini` è l'indice iniziale che punta al primo elemento dello slice
`fin` è l'indice finale che punta all'ultimo elemento dello slice

`ini` e `fin` sono facoltativi, se si omette `ini` si considera come primo elemento quello di posizione 0, se si omette `fin` si considera come ultimo elemento quello in posizione finale. Paradossalmente si possono omettere tutti e due gli indici come: `sequenza[:]` in questo caso lo slice considerato coinciderà con l'intera sequenza.

- ➔ Se `ini` o `fin` assumono valori negativi il conteggio della posizione dell'elemento avviene dalla posizione opposta. p.e. se `ini = -4` allora si andrà a considerare come primo elemento dello slice il quart'ultimo elemento della sequenza.

L'utilizzo dell'estrazione delle sotto-sequenze puo' essere ulteriormente complicato.

Esaminiamo il seguente programma:

```
stringa = "abdcdefghilmn"
print(stringa[2:8:2])
print(stringa[::])
print(stringa[::3])
print(stringa[::-1])
print(stringa[::-2])
```

Lanciamo il programma, in uscita otteniamo:

```
ddf
acfin
nmlihgfedcdba
nlhfdda
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Per semplicità abbiamo considerato solo la sequenza `stringa`, ma quanto visto vale esattamente anche per le `liste` e le `tuple`.
- ➔ la sintassi generale del metodo è la seguente:

```
sequenza[ini:fin:step]
```

in cui notiamo che si può inserire un terzo indice che rappresenta il passo con cui

andiamo a prelevare gli elementi tra `ini` e `fin`

- ➔ Step negativo ci permette di andare a prelevare gli elementi in ordine inverso. Con `step = -1` tutta la sequenza risulta scritta al “contrario”.

### 7.4.4* Ricerca di sottosequenze

Oltre alla ricerca di un singolo elemento [cfr. 7.4.1], può capitare di voler cercare una sottosequenza, ovvero una porzione ordinata di elementi (una sottostringa, una sottolista o una sottotupla) all'interno di una sequenza più grande.

Esaminiamo il seguente programma che mette a confronto il comportamento dei diversi tipi:

```
stringa = "abcd efgh ilmn"
lista = [10, 3, 7, 1, 9, 3, 2, 4]
tupla = (10, 3, 7, 1, 9, 3, 2, 4)

# Ricerca su stringa
print("efgh" in stringa)
print(stringa.index("efgh"))
print(stringa.find("efgh"))

# Ricerca su lista e tupla
print([3, 7, 1] in lista)
print((3, 7, 1) in tupla)

print(stringa.find("efgz"))
print(stringa.index("efgz"))
```

Lanciamo il programma, in uscita otteniamo:

```
True
5
5
False
False
-1
Traceback (most recent call last):
  File "/home/massimo/PythonProjects/prova.py", line 15, in
<module>
    print(stringa.index("efgz"))
          ^^^^^^^^^^^^^^^^^^^^^
ValueError: substring not found
```

Dall’analisi del codice e dell’output osserviamo che:

- ➔ nelle stringhe la ricerca di una sottosequenza è un'operazione nativa: l'operatore `in` e il metodo `index` riconoscono una serie di caratteri consecutivi.
- ➔ nelle liste e nelle tuple, gli operatori di ricerca non "vedono" le sottosequenze. Il

risultato è `False` perché Python cerca l'intera sottolista [3, 7, 1] come se fosse un singolo elemento della lista principale (un caso di lista annidata), non come una successione di numeri.

➔ per le sole stringhe esiste anche il metodo `find("sottostringa")` che, a differenza di `index`, restituisce -1 invece di un errore se la sequenza non viene trovata.

➔ Sintassi del metodo di stringa `find()`:

```
find(sottostringa, inizio, fine))
```

dove:

`sottostringa`: la sottostringa da cercare all'interno della stringa principale
`inizio` (facoltativo): l'indice iniziale da cui iniziare la ricerca.
`fine` (facoltativo): l'indice finale in cui la ricerca deve interrompersi.

Valore di ritorno:

- Restituisce il primo indice della sottostringa se viene trovato all'interno dell'intervallo specificato.
- Restituisce -1 se la sottostringa non viene trovata.

Si puo' notare come sia molto simile al metodo index tranne per il comportamento che si ha quando la sottostringa non viene trovata.

#### 7.4.4.1 Ricerca tramite slicing

Per effettuare una ricerca di una sottosequenza all'interno di una lista o di una tupla, dobbiamo utilizzare la tecnica dello slicing (cfr 7.4.3) all'interno di un ciclo. Lo slicing ci permette di estrarre "fette" della sequenza per confrontarle con ciò che stiamo cercando.

Esaminiamo il seguente programma:

```
lista = [10, 3, 7, 1, 9, 3, 2, 4]
bersaglio = [3, 7, 1]
n = len(bersaglio)
trovato = False

for i in range(len(lista) - n + 1):
    fetta = lista[i : i + n]
    if fetta == bersaglio:
        print("Sottosequenza trovata all'indice:", i)
        trovato = True
        break

if not trovato:
    print("Sottosequenza non trovata")
```

Lanciamo il programma, in uscita otteniamo:

```
Sottosequenza trovata all'indice: 1
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ lo slicing (`lista[i : i + n]`) estrae una porzione della lista della stessa lunghezza della sequenza cercata.
- ➔ facendo scorrere l'indice i con un ciclo, possiamo confrontare ogni possibile "fetta" della lista con il nostro bersaglio.
- ➔ questa tecnica è universale: funziona esattamente allo stesso modo per liste, tuple e stringhe, permettendo di superare i limiti dei metodi standard sulle sequenze di oggetti.
- ➔ lo slicing non produce mai un errore di tipo `IndexError` se i limiti superano la lunghezza della lista, rendendo il confronto sicuro.

#### 7.4.4.2 Confronto tra i vari metodi di ricerca delle sottosequenze

Lo slicing (`[inizio:fine:passo]`) è lo strumento universale che ci permette di "estrarre" porzioni di qualsiasi sequenza per confrontarle.
Abbiamo compreso che è fondamentale per implementare una ricerca di sottosequenze su liste e tuple, dove i metodi pronti mancano.

Esaminiamo la seguente tabella riassuntiva e comparativa:

| Operazione | Stringhe | Liste | Tuple |
|---|---|---|---|
| Ricerca elemento | `index() / in` | `index() / in` | `index() / in` |
| Ricerca sottosequenza | Nativa (`"ab" in "abc"`) | Manuale (via slicing) | Manuale (via slicing) |
| Metodo find() | Disponibile | Non disponibile | Non disponibile |
| Slicing | `s[1:4]` (restituisce stringa) | `l[1:4]` (restituisce lista) | `t[1:4]` (restituisce tuple) |

## 7.4.5* Il numero di occorrenze di una sottosequenza

Dopo aver visto come contare un singolo elemento in una sequenza, è utile capire come contare quante volte una sottosequenza si ripete all'interno di una sequenza più grande. Come per la ricerca (cfr. 7.4.4) , anche in questo caso il comportamento di Python varia tra stringhe e sequenze come liste e tuple.

Esaminiamo il seguente programma:

```
stringa = "abababa"
lista = [1, 2, 1, 2, 1, 2]
tupla = (1, 2, 1, 2, 1, 2)
sotto_lista = [1, 2]
```

```
# Conteggio su stringa
print(stringa.count("aba"))

# Conteggio su lista e tupla (comportamento standard)
print(lista.count(sotto_lista))
print(tupla.count((1, 2)))

# Conteggio manuale su lista tramite slicing
conteggio = 0
n = len(sotto_lista)
for i in range(len(lista) - n + 1):
    if lista[i : i + n] == sotto_lista:
        conteggio += 1
print(conteggio)
```

Lanciamo il programma, in uscita otteniamo:

```
2
0
0
3
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ il metodo `count` applicato alle stringhe è in grado di contare intere sottostringhe.
  **Nota:** Il conteggio delle stringhe è di tipo "non sovrapposto". In "abababa", la sottostringa "aba" viene contata 2 volte (la prima e l'ultima), ignorando quella che inizierebbe al centro perché i caratteri sono già stati conteggiati.
- ➔ nelle liste e nelle tuple, il metodo `count` cerca la sottosequenza come se fosse un unico elemento (un oggetto annidato). Poiché nella nostra lista non c'è un elemento che sia a sua volta una lista [1, 2], il risultato è 0.
- ➔ **Soluzione:** per contare quante volte una sottosequenza appare in una lista o tupla, dobbiamo procedere manualmente utilizzando la tecnica della "finestra scorrevole" con lo slicing (come visto nella ricerca delle sootosequenze par. 7.4.4), incrementando una variabile contatore ogni volta che troviamo una corrispondenza.

## 7.5 Funzioni di utilità sulle sequenze

In questo capitolo andremo a studiare alcune funzioni speciali che possono essere utilizzate con grande profitto sul tipo dati sequenza cioè sulle stringhe, tuple e liste (alcune di queste anche sui dizionari e insiemi).
Esamineremo le seguenti funzioni:

1. len
2. min e max
3. filter
4. map

5. zip

La prima funzione l'abbiamo già incontrata quindi tratteremo le successive.

### 7.5.1 `min()` e `max()`

Queste funzioni sono molto utili perchè permettono di individuare facilmente i valori massimi e minimi di una sequenza.

Esaminiamo il seguente programma:

```
lista = [1, 2, 4, 3]
tupla = (1, 2, 4, 3)
stringa = "abdc"
str2 = "xyz"
str3 = "lm"
vuota = []

print(min(lista),max(lista))
print(min(tupla),max(tupla))
print(min(stringa),max(stringa))
print(min(stringa,str2,str3),max(stringa,str2,str3))
print(min(stringa,str2,str3,key=len),max(stringa,str2,str3,key=len
))
print(max(vuota,default=1))
print(max(vuota))
```

Lanciamo il programma, in uscita otteniamo:

```
1 4
1 4
a d
abcd xyz
lm abcd
1
Traceback (most recent call last):
  File "prova.py", line 14, in <module>
    print(max(vuota))
ValueError: max() arg is an empty sequence
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Le funzioni `min()` e `max()` restituiscono il valore rispettivamente minimo e massimo della sequenza inserita come parametro alla funzione stessa.
- ➔ Le funzioni accettano un parametro predefinito key che rappresenta la funzione rispetto alla quale si devono eseguire i confronti tra gli elementi dati in input alla funzione. Se non inserita si considera come default il confronto > o < (a seconda che si consideri la funzione max o min). Nel caso in cui si inserisce come funzione len, allora si puo' notare che la funzione segue la ricerca del massimo o del minimo

a seconda della lunghezza dell'oggetto inserito. p.e. nel caso delle stringhe a seconda del numero di caratteri che compongono le stringhe confrontate.

- ➔ Nel caso si inserisse come argomento una sequenza vuota l'interprete genera un errore di tipo `ValueError`. Per evitare questo errore si può utilizzare il parametro predefinito default che viene restituito nel caso in cui si inserissero delle sequenze vuote.
- ➔ In generale le funzioni `max()` e `min()` accettano come parametri di input degli oggetti iterabili (le sequenze ovviamente lo sono)
- ➔ la sintassi della funzione `max()` e `min()` è la seguente:

```
max(iterabile, *iterabile, key, default)
```

dove:

`iterabile` è la sequenza (o in generale un'iterabile) su cui si vuole ricercare il massimo o il minimo

`*iterabile` è l'insieme di oggetti di numero indefinito su cui si vuole ricercare il massimo o il minimo

`key` è la funzione che si vuole utilizzare per ricercare il massimo o il minimo

`default` è il valore che viene restituito quando gli iterabili sono vuoti

### 7.5.2 `filter()`

La funzione `filter()` applica una funzione a ogni elemento di una sequenza restituendo una sequenza che contiene come elementi soltanto quelli che rendono vera la funzione applicata.

Esaminiamo il seguente programma:

```
# funzioni che filtrano le vocali
def funLista(carattere):
   vocali = ['a', 'e', 'i', 'o', 'u']
   return carattere not in vocali

def funTupla(carattere):
   vocali = ('a', 'e', 'i', 'o', 'u')
   return carattere not in vocali

def funStringa(carattere):
   vocali = "aeiou"
   return carattere not in vocali

# Sequenze
lista = ['e', 's', 'e', 'm', 'p', 'i', 'o']
tupla = ('e', 's', 'e', 'm', 'p', 'i', 'o')
stringa = 'esempio'

# filtraggio delle sequenze
filtered = filter(funLista, lista)
print(type(filtered))
```

```
for consonante in filtered:
   print(consonante,end = " ")
print()
filtered = filter(funTupla, tupla)
for consonante in filtered:
   print(consonante,end = " ")
print()
filtered = filter(funStringa, stringa)
for consonante in filtered:
   print(consonante,end = " ")
```

Lanciamo il programma, in uscita otteniamo:

```
<class 'filter'>
s m p
s m p
s m p
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `filter()` agisce nella stessa identica maniera con le diverse sequenze.
- ➔ È necessario che la funzione che `filter()` applica agli elementi della sequenza considerata debba restituire `True` o `False`.
- ➔ Il tipo restituito dalla funzione `filter()` è un tipo specifico della classe `filter`. Si comprende che è un oggetto iterabile.
- ➔ la sintassi della funzione `filter()` è la seguente:

  `filter(funzione, sequenza)`

  dove:

  `funzione` è la funzione che viene applicata a tutti gli elementi della sequenza, restituisce una tipo booleano

  `sequenza` è la sequenza che si vuole elaborare

**Curiosità:** si può settare la funzione a None, questo permette di eliminare dalla sequenza i valori nulli.
A questo proposito esaminiamo il seguente codice:

```
lista=[1,0,"","c",[1,2],[],(2,3),()]

filtered = filter(None,lista)

for ele in filtered:
   print(ele,end=" ")
```

Lanciamo il programma, in uscita otteniamo:

```
1 c [1, 2] (2, 3)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `filter()` con una funzione nulla (`None`) permette di eliminare tutti i valori nulli

### 7.5.3 map()

La funzione `map()` applica una funzione a ogni elemento di una sequenza restituendo una sequenza che contiene gli elementi risultanti dall'applicazione della funzione passata a `map` come parametro.

Visto che il comportamento di map() è analogo per tutte le sequenze, limitiamo l'esempio alla lista. Quindi esaminiamo il seguente programma:

```
def fun(num):
    return num*num
def fun2(num1,num2):
    return num1+num2
def funPrecSucc(num):
    return num-1,num,num+1

lista=[1, 2, 3, 4]
lista1=[5, 6, 7, 8, 9]
mapped = map(fun,lista)
print(type(mapped))
for ele in mapped:
    print(ele,end=" ")
print()
for ele in map(fun2,lista,lista1):
    print(ele,end=" ")
print()
for ele in map(funPrecSucc,lista):
    print(ele)
```

Lanciamo il programma, in uscita otteniamo:

```
<class 'map'>
1 4 9 16
6 8 10 12
(0, 1, 2)
(1, 2, 3)
(2, 3, 4)
(3, 4, 5)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `map()` restituisce una sequenza i cui elementi sono il risultato dell'applicazione della funzione passata come parametro a `map()`.

- ➔ Il tipo restituito dalla funzione `map()` è un tipo specifico della classe `map`. Si comprende che è un oggetto iterabile.
- ➔ La funzione che map() applica può restituire qualsiasi oggetto. Nel codice che stiamo analizzando la funzione `fun2()` restituisce una tupla con 3 elementi
- ➔ Possiamo inserire più di una sequenza da elaborare. In questo caso la funzione che si applica deve avere un numero di parametri pari al numero di sequenze che si è inserito come parametri attuali a `map()`. Tali sequenze devono avere identico numero di elementi, se fossero differenti `map()` andrà a considerare solo gli accoppiamenti che considerano gli elementi di tutte le sequenze.
- ➔ la sintassi generale della funzione `map()` è la seguente:

  `map(funzione, seq`$_1$`,seq`$_2$`,...,seq`$_n$`)`

  dove:

  `funzione` è la funzione che viene applicata a tutti gli elementi della sequenza

  `seq`$_i$ rappresentano le sequenze che si vogliono elaborare

### 7.5.4 `zip()`

La funzione `zip()` accetta come parametro delle sequenze (e anche dei dizionari nell'accezione di 'sequenze' estratte dai dizionari)[19] e restituisce un oggetto iterabile di tuple formate dai rispettivi elementi delle sequenze inserite come parametri alla funzione stessa.

Esaminiamo il seguente programma:

```
lista=[1, 2, 3, 4]
stringa="abcd"
zipped=zip(lista,stringa)
print(type(zipped))
print(zipped)
for ele in zipped:
   print(ele)
for ele in zip(lista):
```

[19] Si intende che come parametro si può inserire un dizionario, in questo caso zip esamina la sequenza delle chiavi del dizionario vs le altre sequenze inserite come argomenti di zip.
Codice di esempio:

```
diz={"c1":"v1","c2":"v2","c3":"v3"}
lista=[1,4,-2,7]

for e in zip(diz,lista):
    print(e)
```

Lanciamo il programma, in uscita otteniamo:

```
('c1', 1)
('c2', 4)
('c3', -2)
```

```
print(ele)
```

Lanciamo il programma, in uscita otteniamo:

```
<class 'zip'>
<zip object at 0x7f6dcee01940>
(1, 'a')
(2, 'b')
(3, 'c')
(4, 'd')
(1,)
(2,)
(3,)
(4,)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il tipo restituito dalla funzione `zip()` è un tipo specifico della classe `zip`. Si comprende che è un oggetto iterabile.
- ➔ La funzione `zip()` restituisce un iteratore di tuple con ogni tupla avente elementi di tutte le sequenze inserite come parametri. Le sequenze devono avere lo stesso numero di elementi, in caso contrario si considerano solo gli accoppiamenti possibili componibili con tutte le sequenze inserite, in questo caso il numero di tuple è identico al numero di elementi della sequenza con il minor numero di elementi.
- ➔ La funzione `zip()` applicata a una sola sequenza restituisce un iteratore di tuple che ha come elementi una tupla che contiene un singolo elemento.
- ➔ la sintassi generale della funzione `zip()` è la seguente:

  `zip(`$seq_1$`,`$seq_2$`,...,`$seq_n$`)`
  dove:

  $seq_i$ rappresentano le sequenze che si vogliono "zippare"

## 7.6 Dizionari

I dizionari sono una struttura informativa non sequenziale, cioè non è possibile stabilire un' ordine tra gli oggetti che memorizza al suo interno (come, invece, avviene per le strutture informative di tipo sequenza che abbiamo incontrato al 7.1, 7.2 e 7.3).

Definiamo come *dizionario* una collezione non ordinata di coppie *chiave-valore*. Le chiavi sono univoche all'interno della collezione e mappano i valori loro associati. Per fare un parallelismo è esattamente come succede per l'associazione delle parole (le chiavi) alle loro definizione (i valori).

Un dizionario può anche essere visto come una generalizzazione della lista in cui le chiavi possono solamente essere un intervallo di interi consecutivi a partire da 0.[20]

[20] https://pellacini.di.uniroma1.it/teaching/fondamenti12/lecture07.html

### 7.6.1 Creare un dizionario

Proviamo il seguente programma:

```
diz1 = {}
print(diz1)
diz2 = {'pippo':'0113456732', 2: 'numero',
False: 'paperino',5:2 }
print(diz2)
diz3 = {'pippo':'0113456732', 'pippo': 'numero'}
print(diz3)

print(len(diz1))
print(len(diz2))
print(len(diz3))
```

Lanciamo il programma, in uscita otteniamo:

```
{}
{'pippo': '0113456732', 2: 'numero', False: 'paperino', 5: 2}
{'pippo': 'numero'}
0
4
1
```

Dall'analisi del codice e dell'output osserviamo che:

- Definiamo un *dizionario* andando a definire i suoi elementi all'interno di parentesi graffe aperte e chiuse
- Il *dizionario* vuoto viene definito da due parentesi graffe aperte e chiuse
- Il *dizionario* con le coppie *chiave-valore* viene definito andando a elencare tali coppie scrivendo prima la chiave e poi il valore preceduto da: "**:**". Tutte le coppie vengono separate utilizzando la: "**,**"
- Le chiavi e i valori possono essere di qualsiasi tipo anche all'interno di uno stesso dizionario. Quindi il dizionario memorizza oggetti eterogenei tra di loro.
- I valori possono essere duplicati all'interno del dizionario
- Le chiavi non possono essere duplicate all'interno del dizionario, se questo dovesse accadere viene considerata solo l'ultima coppia chiave-valore inserita. Non viene quindi generato un'errore (Python interpreta il codice come un "aggiornamento" del valore associato a quella particolare chiave)
- La funzione `len( )` fornisce la lunghezza del dizionario intendendo il numero di coppie *chiave-valore* presenti nello stesso

### 7.6.2 Accedere agli elementi di un dizionario

Proviamo il seguente programma:

```
diz ={'k1':'v1', 'k2':'v2','k3':'v3'}
print(diz['k2'])
for chiave in diz:
   print(chiave,":",diz[chiave])
print("-------")
for chiave, valore in diz.items():
   print(chiave,":",valore)
```

Lanciamo il programma, in output otteniamo:

```
v2
k1 : v1
k2 : v2
k3 : v3
-------
k1 : v1
k2 : v2
k3 : v3
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo accedere al valore associato ad una chiave inserendo la chiave all'interno di parentesi quadre utilizzando come indicatore il nome del dizionario
- ➔ Il dizionario è un oggetto iterabile che restituisce a ogni iterazione la chiave. Sfruttando questo comportamento possiamo scandire tutti gli elementi di un dizionario utilizzando l'istruzione `for`
- ➔ Il metodo `items( )` del dizionario permette di ottenere un oggetto iterabile che restituisce a ogni iterazione la coppia chiave-valore, in questo modo si possono scandire tutti gli elementi del dizionario in maniera più versatile.

### 7.6.3 Modificare gli elementi di un dizionario

Proviamo il seguente programma:

```
diz ={'k1':'v1', 'k2':'v2','k3':'v3'}
print(diz['k2'])
diz['k2']=3
print(diz['k2'])
```

Lanciamo il programma, in output otteniamo:

```
v2
3
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo modificare i singoli valori associati a una determinata chiave
- ➔ Siamo liberi di modificare anche il tipo del valore

### 7.6.4 Modificare i dizionari

Non possiamo effettuare le operazioni di concatenazione e in generale di modificazione delle strutture dati che abbiamo incontrato nel tipo sequenza. Ma possiamo comunque modificare il numero degli elementi di un dizionario aumentandolo o dimunendolo.

Esaminiamo il seguente programma:

```
diz ={'k1':'v1', 'k2':'v2','k3':'v3'}
diz['k4']='v4'
print(diz)
print(len(diz))
print(diz.pop('k1'))
print(diz)
del(diz['k4'])
print(diz)
diz={'n1':1, 'n2':1,'n3':1}
print(diz)
diz.clear()
print(diz)
```

Lanciamo il programma, in uscita otteniamo:

```
{'k1': 'v1', 'k2': 'v2', 'k3': 'v3', 'k4': 'v4'}
4
v1
{'k2': 'v2', 'k3': 'v3', 'k4': 'v4'}
{'k2': 'v2', 'k3': 'v3'}
{'n1': 1, 'n2': 1, 'n3': 1}
{}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo aggiungere un'associazione chiave-valore andando a effettuare un'*assegnazione* all'elemento del dizionario di chiave *nuovachiave* del *nuovovalore* utilizzando la seguente sintassi:

```
diz[nuovachiave] = nuovovalore
```

In questo modo verrà aggiunta l'associazione *nuovachiave* - *nuovovalore* al dizionario *diz*. Ovviamente la lunghezza del dizionario rifletterà la modifica.

- Il metodo `pop( )` viene utilizzato per estrarre una chiave dal dizionario, in questa maniera la chiave indicata come argomento viene rimossa. Se non si specifica come argomento una chiave esistente viene generato un eccezione di tipo `KeyError`.
  La sintassi del metodo pop è la seguente:

  ```
  .pop(chiave,keyerror)
  ```

  dove:

  `chiave` è la chiave che si vuole rimuovere
  `keyerror` è il valore che viene restituito se non viene trovata la chiave

- Un altro metodo per rimuovere un' associazione da un dizionario è quello di utilizzare la funzione `del` . Questa rimuove la chiave e il valore senza restituire nulla. Se la chiave non dovesse esistere viene generato un errore del tipo `KeyError`
- Possiamo sovrascrivere un dizionario con un altro dizionario
- Abbiamo la possibilità di rimuovere tutti gli elementi di un dizionario o utilizzando il metodo clear( ) oppure andando a sovrascrivere il dizionario con il dizionario vuoto `{ }`.

### 7.6.5 Confrontare i dizionari

Possiamo utilizzare gli operatori di confronto per confrontare tra loro due dizionari. Non ha senso utilizzare gli operatori `>` o `<` in quanto il dizionario non contiene oggetti ordinati quindi non si può stabilire una relazione di ordinamento tra i diversi dizionari.

Esaminiamo il seguente programma:

```
diz1 ={'k1':'v1','k2':'v2', 'k3':'v3'}
diz2 ={'k2':'v2','k1':'v1', 'k3':'v3'}
diz3 ={'k2':'v1','k1':'v1', 'k3':'v3'}
print( diz1 == diz2 )
print( diz1 == diz3 )
```

Lanciamo il programma, in uscita otteniamo:

```
True
False
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'operatore == viene utilizzato per effettuare confronti tra i dizionari
- ➔ Se due dizionari contengono le stesse associazioni ma in ordine differente risultano comunque uguali
- ➔ Se hanno chiavi o valori differenti risultano diversi

### 7.6.5 Ricerca di elementi nel dizionario

Gli algoritmi di ricerca risultano essere molto utili non solo per le sequenze ma anche per i dizionari.
Abbiamo già incontrato due metodi per ricercare una chiave in un dizionario:

- ➔ Accesso diretto al valore associato alla chiave, se inserisco una chiave che non esiste l'interprete mi restituisce un KeyError
- ➔ Utilizzo del metodo `pop( )`, il secondo parametro predefinito mi permette di gestire l'assenza della chiave ricercata. Attenzione che con questo metodo ottengo la rimozione dell'associazione dal dizionario

Esistono degli altri metodi, esaminiamo a tale proposito il segente codice:

```
diz ={'k1':'v1', 'k2':'v2','k3':'v3'}
print('k1' in diz)
print('k7' in diz)
print('v1' in diz)
print('v7' in diz)
print('k1' and 'k2' in diz)
print(diz.get('k1'))
print(diz.get('k7'))
print(diz.get('k7',0))
```

Lanciamo il programma, in uscita otteniamo:

```
True
False
False
False
True
v1
None
0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Gli operatori di membership `in` e `not in` ( paragrafo 4.2.5 ) possono essere utilizzati proficuamente per ricercare elementi nel dizionario
- ➔ Con gli operatori di membership si possono ricercare più chiavi utilizzando gli operatori booleani
- ➔ Si possono ricercare solo le chiavi e non i valori (ovviamente altrimenti sarebbe una

sorta di lista)

- ➔ Il metodo `get( )` permette di reperire il valore associato alla chiave passata come parametro al metodo stesso. Se la chiave non esiste nel dizionario viene restituito: "`None`"
- ➔ la sintassi generale del metodo `get()` è la seguente:

```
get(chiave, default)
```

dove:

`chiave` è la chiave che si vuole ricercare nel dizionario
`default` è un parametro predefinito che indica quale valore restituire nel caso in cui la chiave ricercata non esista.

### 7.6.6 Alcuni metodi specifici dei dizionari

Affrontiamo ora una veloce carrellata sui metodi peculiari dell'oggetto dizionario.
Il dizionario ci offre i seguenti metodi che possiamo proficuamente utilizzare nelle nostre applicazioni:

```
clear
get
pop
items
del
popitem
setdefault
update
keys
values
```

I primi cinque li abbiamo già incontrati quindi tratteremo i successivi.
(Attenzione `del` non è un metodo dei dizionari ma una funzione che accetta come argomento un elemento di un dizionario)

#### 7.6.6.1 Metodo popitem( )

Esaminiamo il seguente programma:

```
diz1 ={'k1':'v1','k2':'v2', 'k3':'v3'}
keyValue=diz1.popitem()
print(keyValue)
print(diz1)
```

Lanciamo il programma, in uscita otteniamo:

```
('k3', 'v3')
```

```
{'k1': 'v1', 'k2': 'v2'}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `popItem( )` estrae l'ultima coppia chiave-valore inserita (secondo l'ordine LIFO).
- ➔ tale estrazione implica l'effettiva eliminazione della coppia dal dizionario
- ➔ Nel caso in cui il dizionario fosse vuoto, verrà generato un errore di tipo keyError.

### 7.6.6.2 Metodo setdefault( )

Esaminiamo il seguente programma:

```
diz1 ={'k1':'v1','k2':'v2', 'k3':'v3'}
print(diz1.setdefault('k3'))
print(diz1.setdefault('k4'))
print(diz1)
print(diz1.setdefault('k5','default'))
print(diz1)
```

Lanciamo il programma, in uscita otteniamo:

```
v3
None
{'k1': 'v1', 'k2': 'v2', 'k3': 'v3', 'k4': None}
default
{'k1': 'v1', 'k2': 'v2', 'k3': 'v3', 'k4': None, 'k5': 'default'}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `setdefault( )` accetta come parametro una chiave del dizionario. Se questa esiste nel dizionare viene restituito il corrispettivo valore.
- ➔ Se la chiave non esiste viene inserita nel dizionario associata al valore `None`.
- ➔ Se viene specificato il secondo parametro (facoltativo) e la chiave non dovesse esistere allora la chiave verrà inserita nel dizionario con il valore indicato nel secondo parametro inserito.

### 7.6.6.3 Metodo update( )

Esaminiamo il seguente programma:

```
diz1 ={'k1':'v1','k2':'v2'}
diz1.update({'k4':'v4'})
print(diz1)
diz1.update({'k5':'v5','k6':'v6'})
print(diz1)
diz1.update(k1='VV1')
print(diz1)
```

```
diz1.update(k1='VVV11',k2='VVV2')
print(diz1)
diz1.update(k10='v10')
print(diz1)
diz1.update(k11)
print(diz1)
```

Lanciamo il programma, in uscita otteniamo:

```
{'k1': 'v1', 'k2': 'v2', 'k4': 'v4'}
{'k1': 'v1', 'k2': 'v2', 'k4': 'v4', 'k5': 'v5', 'k6': 'v6'}
{'k1': 'VV1', 'k2': 'v2', 'k4': 'v4', 'k5': 'v5', 'k6': 'v6'}
{'k1': 'VVV11', 'k2': 'VVV2', 'k4': 'v4', 'k5': 'v5', 'k6': 'v6'}
{'k1': 'v01', 'k2': 'v02', 'k4': 'v4', 'k5': 'v5', 'k6': 'v6',
'k10': 'v10'}
Traceback (most recent call last):
  File "prova.py", line 15, in <module>
    diz1.update('k11')
NameError: name 'k11' is not defined
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `update( )` permette di effettuare un aggiornamento del dizionario andando ad aggiungere delle coppie *chiave-valore*.
- ➔ Possiamo inserire come parametri al metodo una singola coppia o un insieme di coppie. Praticamente è possibile inserire un'intero dizionario in aggiunta a quello già presente.
- ➔ Se specifichiamo come argomenti le chiavi del dizionario, possiamo aggiornare i valori di tali chiavi.
- ➔ Se specifichiamo come argomento una chiave che non esiste accoppiata a un valore, tale chiave verrà inserita con il valore specificato. In questo caso, se la chiave non esiste, verrà generato un errore dall'interprete.

### 7.6.6.4 Metodi keys( ) e values( )

Esaminiamo il seguente programma:

```
diz1 ={'k1':'v1','k2':'v2','k3':'v3'}
for k in diz1.keys():
    print(k)
print('----------')
for v in diz1.values():
    print(v)
```

Lanciamo il programma, in uscita otteniamo:

```
k1
k2
```

```
k3
----------
v1
v2
v3
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `keys( )` è analogo al metodo items( ) ma restituisce un oggetto iterabile che contiene tutte le chiavi del dizionario.
- ➔ Il metodo `values( )` è analogo al metodo items( ) ma restituisce un oggetto iterabile che contiene tutti i valori del dizionario.

## 7.7 Insiemi

Un insieme è una raccolta di dati non ordinata, iterabile, modificabile e senza elementi duplicati. Visto che i dati collezionati da un insieme non sono ordinati allora non possiamo accedere agli elementi utilizzando gli indici come facciamo con le liste.

Gli elementi di un insieme possono essere solo oggetti immutabili.

### 7.7.1 Creare un insieme

Proviamo il seguente programma:

```
set1 = set()
print(set1)
set2 = {'pippo',3, 4.5,3,3, True}
print(set2)
set3 = {}
print(set3)
set4 = set([1,2,1,4])
print(set4)

print(type(set1))
print(type(set2))
print(type(set3))
print(type(set4))

print(len(set1))
print(len(set2))
print(len(set4))

set5 = {1,[1,2,3],"stringa"}
```

Lanciamo il programma, in uscita otteniamo:

```
set()
{'pippo', True, 3, 4.5}
{}
{1, 2, 4}
<class 'set'>
<class 'set'>
<class 'dict'>
<class 'set'>
0
4
3
Traceback (most recent call last):
  File "/path/prova.py", line 18, in <module>
    set5 = {1,[1,2,3],"stringa"}
TypeError: unhashable type: 'list'
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Definiamo un *insieme* andando a definire i suoi elementi all'interno di parentesi graffe aperte e chiuse
- ➔ L' *insieme* vuoto non viene definito da due parentesi graffe aperte e chiuse con all'interno nulla, perchè in questo modo si defnisce un dizionario vuoto.
- ➔ L' *insieme* vuoto si definisce utilizzando la keyword `set()`, all'interno delle parentesi tonde non andiamo a inserire nulla.
- ➔ L' *insieme* non vuoto viene definito utilizzando un elencazione di oggetti separati da: "`,`".
- ➔ Se si inserisce lo stesso oggetto questo viene considerato una sola volta.
- ➔ Gli oggetti possono essere di differente tipo. Quindi l'*insieme* memorizza oggetti eterogenei tra di loro.
- ➔ Possiamo creare un *insieme* utilizzando la funzione `set()`, passando come parametro un unico argomento che deve essere un oggetto *iterabile* (che non contiene oggetti *mutabili*). Verrà creato un *insieme* i cui elementi sono gli elementi dell'oggetto, come nell'esempio in cui si è passato come parametro alla funzione `set()` una *lista*.
- ➔ Gli oggetti devono essere *immutabili*, altrimenti l'interprete Python restituisce un errore del tipo: "`TypeError: unhashable type:`"
- ➔ La funzione `len( )` fornisce la lunghezza dell'*insieme*, cioè intendendo il numero di elementi presenti nello stesso

### 7.7.2 La funzione `frozenset()`

Gli insieme non possono contenere oggetti *mutabili*, ma sono di per sè *mutabili*, cioè possiamo modificarli durante la loro vita.
Questo significa che un *insieme* non può essere un elemento di un altro *insieme*, in altre parole un insieme non può contenere un altro insieme. Questa è una regola che non va d'accordo con la definizione di insieme della matematica, per tale motivo esiste un tipo di insieme che è immutabile, il *frozenset*.

Esaminiamo il seguente codice:

```
set1 = {1,2,3}
print(set1)
set2 = frozenset([1,2,3])
print(type(set2))
print(set2)
set3 = {set2,4}
print(set3)
set4 = {set1,4}
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3}
<class 'frozenset'>
frozenset({1, 2, 3})
{frozenset({1, 2, 3}), 4}
Traceback (most recent call last):
  File "/home/massimo/PythonProjects/prova.py", line 8, in
<module>
    set4 = {set1,4}
TypeError: unhashable type: 'set'
```

Dall'analisi del codice e dell'output osserviamo che:

- La funzione `frozenset()` permette di creare un insieme *immutabile*, è necessario passargli come unico parametro un oggetto iterabile, analogamente alla funzione `set()`.
- Il *fronzenset* viene stampato sullo standard output come un insieme che è argomento della funzione `frozenset()`.
- Il *frozenset* creato può divenire elemento di un insieme
- Un *insieme* non può essere elemento di un altro insieme in quanto otteniamo il noto errore di "`TypeError: unhashable type`", in quanto l'insieme è un oggetto *mutabile*.

### 7.7.3 Accedere agli elementi di un insieme

Come abbiamo accennato nell'introduzione agli insiemi, questi non avendo un ordinamento sui propri elementi non possono essere acceduti utilizzando gli indici. Però l'insieme è un iterabile, quindi possiamo accedere ai suoli elementi iterando tutto l'insieme.

Esaminiamo il seguente programma:

```
set1 = {21,2,"ciao", 3.4, False}
print (set1)
for ele in set1:
    print(ele)
```

```
print(2 in set1)
```

Lanciamo il programma, in uscita otteniamo:

```
{False, 2, 3.4, 21, 'ciao'}
False
2
3.4
21
ciao
True
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Possiamo utilizzare il ciclo for per "scansionare" tutto l'insieme ed estrarre i suoi elementi
- ➔ Utilizziamo gli operatori di membership per verificare se un oggetto appartiene a un dato insieme.

## 7.7.4 Operazioni insiemistiche sugli insiemi

Possiamo effettuare diverse operazioni tra gli insieme:

- ➔ unione
- ➔ intersezione
- ➔ differenza
- ➔ differenza simmetrica

### 7.7.4.1 Unione

L'unione tra insiemi crea un nuovo insieme i cui elementi sono l'unione degli insiemi di partenza.
Si esamini il seguente codice:

```
set1 = {1,2,3}
set2 = {3,4,5}
print (set1 | set2)
print (set1.union(set2))
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `union()` consente di effettuare l'operazione di unione tra l'insieme chiamante e gli insiemi argomento del metodo
- ➔ Al posto del metodo `union()` è possibile utilizzare l'operatore di unione " | ".

#### 7.7.4.2 Intersezione

L'intersezione tra insiemi crea un nuovo insieme i cui elementi sono comuni agli elementi degli insiemi di partenza.
Si esamini il seguente codice:

```
set1 = {1,2,3}
set2 = {3,4,5}
print (set1 & set2)
print (set1.intersection(set2))
```

Lanciamo il programma, in uscita otteniamo:

```
{3}
{3}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `intersection()` consente di effettuare l'operazione di intersezione tra l'insieme chiamante e gli insiemi argomento del metodo
- ➔ Al posto del metodo `intersection()` è possibile utilizzare l'operatore di intersezione "`&`".

#### 7.7.4.3 Differenza

La differenza tra insiemi crea un nuovo insieme i cui elementi del primo insieme non sono presenti negli altri insiemi presi in considerazione.
Si esamini il seguente codice:

```
set1 = {1,2,3}
set2 = {3,4,5}
print (set1 - set2)
print (set1.difference(set2))
```

Lanciamo il programma, in uscita otteniamo:

```
{1,2}
{1,2}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `difference()` consente di effettuare l'operazione di differenza tra l'insieme chiamante e gli insiemi argomento del metodo
- ➔ Al posto del metodo `difference()` è possibile utilizzare l'operatore di differenza "`-`".

#### 7.7.4.4 Differenza simmetrica

La differenza simmetrica tra insiemi crea un nuovo insieme i cui elementi del primo

insieme non sono presenti nell'altro insieme e quelli dell'altro insieme non presenti nel primo insieme.
Si esamini il seguente codice:

```
set1 = {1,2,3}
set2 = {3,4,5}
print (set1 ^ set2)
print (set1.symmetric_difference(set2))
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 4, 5}
{1, 2, 4, 5}
```

Dall'analisi del codice e dell'output osserviamo che:

- Il metodo `symmetric_difference()` consente di effettuare l'operazione di differenza simmetrica tra l'insieme chiamante e l'altro insieme argomento del metodo.
- Al posto del metodo `symmetric_ifference()` è possibile utilizzare l'operatore di differenza simmetrica "^".

## 7.7.5 Modificare un insieme

Abbiamo a disposizione metodi per modificare l'*insieme* attraverso i singoli elementi oppure tutto l'insieme operando con altri insiemi.
Naturalmente stiamo parlando di *insiemi mutabili*.

Tratteremo i seguenti metodi:

- add
- pop
- remove
- discard
- clear

### 7.7.5.1 Il metodo `add()`

Questo metodo permette di aggiungere un nuovo elemento all'*insieme* considerato.
Si esamini il seguente codice:

```
set1 = {1,2,3}
print (set1)
set1.add(4)
print (set1)
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3}
{1, 2, 3, 4}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `add()` permette di inserire un elemento nell'*insieme* considerato. L'argomento del metodo è l'elemento che si vuole inserire.

### 7.7.5.2 Il metodo `pop()`

Questo metodo permette di rimuovere il primo elemento inserito nell'*insieme* considerato. Si esamini il seguente codice:

```
set1 = {1,2,3}
print (set1)
print (set1.pop())
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3}
1
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `pop()` permette di rimuovere un elemento dall'*insieme* considerato.
- ➔ Non necessita di argomenti e restituisce l'elemento rimosso.

### 7.7.5.3 Il metodo `remove()`

Questo metodo permette di rimuovere uno specifico elemento dall'*insieme* considerato.

Si esamini il seguente codice:

```
set1 = {1,2,3}
print (set1)
print (set1.remove(2))
print (set1)
print (set1.remove(10))
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3}
None
{1, 3}
Traceback (most recent call last):
  File "/path/prova.py", line 5, in <module>
    print (set1.remove(10))
KeyError: 10
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `remove()` permette di rimuovere un elemento dall'*insieme* considerato.
- ➔ È necessario inserire come argomento l'elemento che si desidera rimuovere.
- ➔ Il metodo non restituisce nulla
- ➔ Se l'elemento che si passa al metodo non appartiene all'*insieme*, viene generato un errore di tipo `KeyError`.

#### 7.7.5.4 Il metodo `discard()`

Questo metodo permette di rimuovere uno specifico elemento dall'*insieme* considerato. È analogo al metodo `remove()`.

Si esamini il seguente codice:

```
set1 = {1,2,3}
print (set1)
print (set1.discard(2))
print (set1)
print (set1.discard(10))
print (set1)
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3}
None
{1, 3}
None
{1, 3}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `discard()` permette di rimuovere un elemento dall'*insieme* considerato.
- ➔ È necessario inserire come argomento l'elemento che si desidera rimuovere.
- ➔ Il metodo non restituisce nulla
- ➔ Se l'elemento che si passa al metodo non appartiene all'*insieme*, NON viene generato un errore di tipo `KeyError` come invece accade con il metodo `remove()`.

#### 7.7.5.5 Il metodo `clear()`

Questo metodo consente di rimuovere tutti gli elementi dall'*insieme* considerato. Trasforma un *insieme* in un *insieme* vuoto.

Si esamini il seguente codice:

```
set1 = {1,2,3}
print (set1)
print (set1.clear())
print (set1)
```

Lanciamo il programma, in uscita otteniamo:

```
{1, 2, 3}
None
set()
```

Dall'analisi del codice e dell'output osserviamo che:
- ➔ Il metodo `clear()` svuota l'*insieme* considerato.
- ➔ Il metodo non accetta parametri e non restituisce nulla.

## 7.7.6 Confrontare gli insiemi

Possiamo effettuare diverse operazioni di confronto tra gli insiemi:
- ➔ disgiunzione
- ➔ sottoinsieme
- ➔ sovrainsieme

### 7.7.6.1 Disgiunzione

Due insieme sono disgiunti tra di loro quando non hanno elementi in comune.
Si esamini il seguente codice:

```
set1 = {1,2,3}
set2 = {3,4,5}
print (set1.isdisjoint(set2))
```

Lanciamo il programma, in uscita otteniamo:

```
False
```

Dall'analisi del codice e dell'output osserviamo che:
- ➔ Il metodo `isdisjoint()` consente di testare se due insiemi sono disgiunti. Restituisce `True` se lo sono o `False` se non lo sono.

### 7.7.6.2 Sottoinsieme

Un insieme è un sottoinsieme di un altro se tutti i suoi elementi sono contenuti nell'altro insieme.
Si esamini il seguente codice:

```
set1 = {1,2}
set2 = {1,2,3}
print (set1.issubset(set2))
print (set1 < set2)
print (set1 <= set2)
```

Lanciamo il programma, in uscita otteniamo:

```
True
True
True
```

Dall'analisi del codice e dell'output osserviamo che:

- Il metodo `issubset()` consente di verificare se l'insieme chiamante è un sottoinsieme dell'argomento del metodo.
- Al posto del metodo `issubset()` è possibile utilizzare l'operatore "<".
- È possibile utilizzare l'operatore: "<=" che lavora anche con sottoinsiemi impropri

#### 7.7.6.2 Sovrainsieme

Un insieme è un sovrainsieme di un altro se contiene tutti gli elementi dell'altro insieme.
Si esamini il seguente codice:

```
set1 = {1,2}
set2 = {1,2,3}
print (set2.issuperset(set1))
print (set2 > set1)
print (set2 >= set1)
```

Lanciamo il programma, in uscita otteniamo:

```
True
True
True
```

Dall'analisi del codice e dell'output osserviamo che:

- Il metodo `issuperset()` consente di verificare se l'insieme chiamante è un sovrainsieme dell'argomento del metodo.
- Al posto del metodo `issuperset()` è possibile utilizzare l'operatore ">".
- È possibile utilizzare l'operatore: ">=" che lavora anche con sottoinsiemi impropri

# 8. Classi e Oggetti

## 8.1 La programmazione orientata agli oggetti[21]

Python è stato sin dalla sua nascita un linguaggio orientato agli oggetti. Per tale motivo creare classi e oggetti è del tutto naturale.
Per leggere è comprendere questo capitolo è necessario avere una "infarinatura" di base su cosa sia un oggetto e come venga creato da una classe e in generale come approcciare alla programmazione orientata agli oggetti.

Il supporto alla programmazione orientata agli oggetti del Python è molto semplice: si creano le *classi* che poi verranno utilizzate per creare gli *oggetti* (che sono le i*stanze della classe* detti anche *oggetti istanza*).

Un'istanza di una classe (da ora in poi *oggetto*) può avere un qualsiasi numero di *attributi* (dati membro, in parole povere le variabili appartenenti all'oggetto) e *metodi* (funzioni membro, in parole povere funzioni appartenenti all'oggetto che operano sugli attributi dello stesso).

È possibile derivare una nuova classe derivandola da una o più classi. La nuova classe figlia è detta *sottoclasse* ed eredita gli attributi e i metodi delle classi genitrici. La sottoclasse può anche ridefinire o aggiungere nuovi attributi o metodi.

Dopo questa brevissima introduzione che ci serve più che altro per utilizzare un linguaggio comune, passiamo a vedere come utilizzare l'OOP (Object Oriented Programming) in Python.

## 8.2 Creazione di una classe e la sua istanziazione

Consideriamo come esempio la creazione di una classe che descrive un punto sul piano cartesiano.

[21] Python 2.1 Tutto&Oltre D.Brueck-S.Tanner Ed. Apogeo

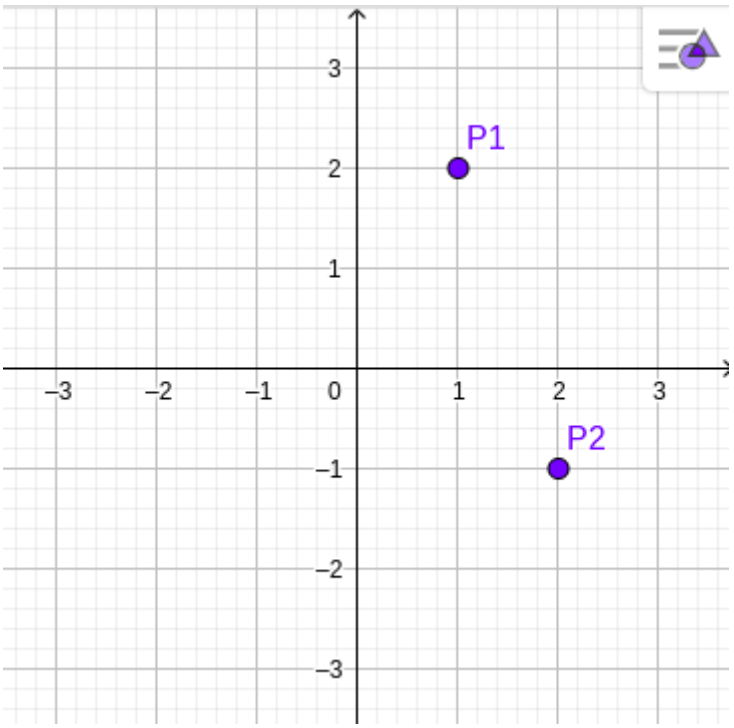


Fig.1

P1 e P2 sono due oggetti punti che hanno ognuno due coordinate x e y che le collocano in una posizione ben determinata sul piano cartesiano XY raffigurato in Fig.1

L'oggetto P è descritto da una classe. Tale oggetto è caratterizzato da due attributi che rappresentano le coordinate cartesiane del punto stesso.

Formalizziamo il codice Python che descriva una classe Punto che abbia due attributi X e Y.

Esaminiamo il seguente programma:

```
class Punto(object):
   """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
   def __init__(self,X,Y):
       self.X = X
       self.Y = Y
   def stampa(self):
       return str(self.X)+" "+str(self.Y)

p1 = Punto(2,3)
print(p1.__doc__)
print(p1)
print(p1.X, p1.Y)
print(p1.stampa())
```

Lanciamo il programma, in uscita otteniamo:

```
    Classe Punto di esempio
- Appunti 'Python Primi Passi' -
<__main__.Punto object at 0x7f98922c5fd0>
2 3
2 3
```

Dall'analisi del codice e dell'output osserviamo che:

- L'istruzione che ci permette di definire una classe è `class`
- La stringa multilinea definita subito dopo l'istruzione `class` definisce la stringa di descrizione della classe stessa. Viene memorizzata nell'attributo `__doc__`
- La classe viene istanziata andando a richiamare il nome della classe seguito dalle parentesi tonde in cui si vanno a inserire i valori richiesti dal costruttore della classe
- Il costruttore della classe è definito da `__init__` che ha sempre il parametro `self` seguito dai parametri che si vogliono richiedere. Nell'esempio i due parametri aggiuntivi sono i valori che andranno a inizializzare gli attributi definiti nella classe.
- Dopo il costruttore possiamo definire il corpo dei metodi della classe

Gli attributi sono organizzati come in un dizionario, esaminiamo il seguente programma:

```
class Punto(object):
   """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
   def __init__(self,X,Y):
       self.X = X
       self.Y = Y
   def stampa(self):
       return str(self.X)+" "+str(self.Y)

p1 = Punto(2,3)
print(p1.__dict__)
p1.altroAttributo=1
print(p1.__dict__)
del(p1.Y)
print(p1.__dict__)
p2=Punto(4,2)
print(p2.__dict__)
```

Lanciamo il programma, in uscita otteniamo:

```
{'X': 2, 'Y': 3}
{'X': 2, 'Y': 3, 'altroAttributo': 1}
{'X': 2, 'altroAttributo': 1}
{'X': 4, 'Y': 2}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'attributo privato __dict__ contiene tutti gli attributi della classe organizzati in un dizionario
- ➔ Si possono aggiungere nuovi attributi effettuando una semplice assegnazione
- ➔ Gli attributi possono essere cancellati da un oggetto. Se si istanzia di nuovo la classe, l'oggetto risultante non sarà affetto dalle modifiche di attributi dell'oggetto precedente.

## 8.3 Creazione dei metodi di una classe

Riprendiamo l'esempio del punto sul piano cartesiano visto nel paragrafo precedente. Vogliamo arricchire la classe con i metodi getter e setter che permettono di modificare e di verificare il valore degli attributi dell'oggetto. Quindi richiamando i metodi setter di X e Y andiamo a modificare il loro valore, richiamando i metodi getter di X e Y andiamo a leggere il loro valore.

Consideriamo il seguente codice :

```
class Punto(object):
    """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
    def __init__(self,X,Y):
        self.X = X
        self.Y = Y
    def stampa(self):
        return str(self.X)+" "+str(self.Y)
    def getX(self):
        return self.X
    def getY(self):
        return self.Y
    def setX(self,val):
        self.X = val
    def setY(self,val):
        self.Y = val


p1 = Punto(2,3)
print(p1.X, p1.Y)
print(p1.getX(),p1.getY())
p1.setX(4)
p1.setY(1)
print(p1.X, p1.Y)
```

Lanciamo il programma, in uscita otteniamo:

```
2 3
2 3
4 1
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ I metodi devono avere sempre almeno un parametro formale di nome `self`. In realtà potremmo chiamare questo parametro anche con un altro nome (non è quindi una parola chiave), l'importante è passarlo. È l'analogo `this` di Java che è una parola chiave del linguaggio e rappresenta l'istanza della classe (l'oggetto).
- ➔ I metodi getter e setter "funzionano" esattamente come i linguaggi C++ e Java, ma essendo metodi di un linguaggio interpretato avviene, come per le funzioni, che non è necessario indicare il tipo di valore di ritorno e il tipo dei parametri formali passati.

### 8.3.1 Il metodo __str__ di Object

Ricordiamo che tutte le classi Python ereditano dalla classe standard `Object`. Questo comportamento lo ritroviamo nei vari linguaggi di programmazione che implementano il paradigma della programmmazione orientata agli oggetti.

Nel paragrafo precedente abbiamo visto il metodo `stampa()` che restituisce una stringa che contiene i valori degli attributi del punto. Possiamo pensare di ottenere la stessa funzionalità andando a stampare direttamente l'istanza della classe che si sta considerando.

Questo, p.e. , in linguaggio Java si implementa andando a effettuare un *override* del metodo `toString()` della classe standard da cui ereditano tutte le classi Java.
In linguaggio Python si effettua la stessa identica cosa andando a effettuare un override dell'analogo metodo che si chiama `__str__()`

Consideriamo il seguente codice :

```
class Punto(object):
   """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
   def __init__(self,X,Y):
       self.X = X
       self.Y = Y
   def __str__(self):
       return "("+str(self.X)+","+str(self.Y)+")"

p1 = Punto(2,3)
print(p1)
```

Lanciamo il programma, in uscita otteniamo:

```
(2,3)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `__str__()` viene inserito nel codice come se definissimo un nuovo metodo. Non andiamo a indicare che viene effettuato un override
- ➔ Gli underscore presenti prima e dopo il nome indicano che il metodo è un metodo *privato*. Parleremo dei modificatori *pubblico*, *privato* e *protetto* in un prossimo capitolo.

### 8.3.2 Metodi che accettano oggetti della classe

Consideriamo i due punti P1 e P2 sul piano cartesiano, come rappresentato in Figura 2.

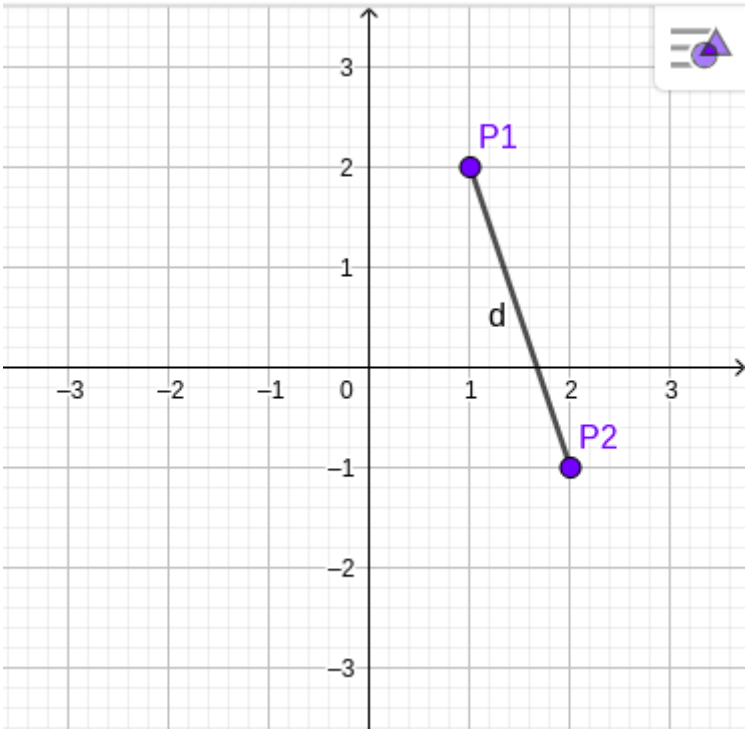



Fig. 2

Vogliamo calcolare la distanza del punto P1 dal punto P2.

Siano $P1=(x_1, y_1)$ e $P2=(x_2, y_2)$ due punti del piano cartesiano allora la distanza d si calcola tramite la formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Per calcolare la distanza andiamo a implementare un metodo `getDistance()` che accetta come parametro formale un oggetto di tipo punto. Tale metodo rappresenta la modellizzazione dell'interazione tra un punto P1 con un punto P2.

Consideriamo il seguente codice :

```
class Punto(object):
```

```
    """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
    def __init__(self,X,Y):
        self.X = X
        self.Y = Y
    def getDistance(self, P):
        return ((self.X - P.X)**2 + (self.Y - P.Y)**2)**0.5
p1 = Punto(0,0)
p2 = Punto(0,3)
print(p1.getDistance(p2))
print(p2.getDistance(p1))
```

Lanciamo il programma, in uscita otteniamo:

```
3.0
3.0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `getDistance()` accetta come parametro attuale un oggetto di tipo Punto.
- ➔ Tale metodo restituisce la distanza dei due punti, quindi un valore di tipo `float`

### 8.3.3 Metodi che restituiscono oggetti della classe

Implementiamo un metodo della classe Punto che restituisce un nuovo oggetto Punto. Vogliamo costruire un metodo che restituisca un punto che rappresenta la traslazione del punto di partenza di una certa quantità.

Sia `P1 = (x1,y1)` e `t = fattore di traslazione` allora il nuovo punto P2 avrà le seguenti coordinate:

$$P2 = ( x_1+t , y_1+t )$$

Andiamo a implementare un metodo translatePoint`()` che accetta come parametro formale il fattore di traslazione t e restituisce un oggetto di tipo punto.

Dopo queste premesse possiamo considerare il seguente codice:

```
class Punto(object):
    """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
    def __init__(self,X,Y):
        self.X = X
        self.Y = Y
    def translatePoint(self, t):
        return Punto(self.X + t,self.Y + t)
```

```
    def __str__(self):
        return "("+str(self.X)+","+str(self.Y)+")"

p1 = Punto(2,3)
p2=p1.translatePoint(2)
print("Punto iniziale: ",p1)
print("Punto traslato: ",p2)
```

Lanciamo il programma, in uscita otteniamo:

```
Punto iniziale:  (2,3)
Punto traslato:  (4,5)
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `translatePoint()` accetta come parametro formale un numero che rappresenta la traslazione del punto sul piano cartesiano. Tale metodo restituisce un oggetto di tipo `Punto`
- ➔ La creazione del punto da restituire avviene richiamando il costruttore della classe attraverso `Punto()`
- ➔ Il valore di ritorno del metodo `translatePoint()` viene assegnato a una variabile che, per effetto di questa assegnazione, viene a divenire di tipo `punto`.

### 8.3.4 Il codice della classe Punto completa

```
class Punto(object):
  """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
  def __init__(self,X,Y):
      self.X = X
      self.Y = Y
  def stampa(self):
      return str(self.X)+" "+str(self.Y)
  def getX(self):
      return self.X
  def getY(self):
      return self.Y
  def setX(self,val):
      self.X = val
  def setY(self,val):
      self.Y = val
  def __str__(self):
      return "("+str(self.X)+","+str(self.Y)+")"
  def getDistance(self, P):
      return ((self.X - P.X)**2 + (self.Y - P.Y)**2)**0.5
  def translatePoint(self, t):
      return Punto(self.X + t,self.Y + t)
```

### 8.3.5 * Una piccola digressione teorica sui metodi getter e setter che ci porterà a parlare delle `proprietà` di un oggetto.[22]

I metodi getter e setter, noti anche come accessori e mutatori, rispettivamente, sono una parte fondamentale del paradigma della programmazione orientata agli oggetti, in particolare nel contesto dell'incapsulamento. Questi metodi forniscono un modo controllato per accedere (getting) e per modificare (setting) i valori degli attributi privati in una classe.
Utilizzando i metodi getter e setter, viene aggiunto un livello di astrazione tra gli attributi interni della classe e il codice esterno che interagisce con tali attributi. Questo approccio non solo protegge i dati, ma offre anche flessibilità nel modo in cui i dati sono accessibili e modificabili.

#### 8.3.5.1 Metodi getter

I metodi getter vengono utilizzati per accedere o recuperare il valore dell'attributo privato di un oggetto senza accedere direttamente all'attributo stesso.
Ciò consente di nascondere la rappresentazione interna di un attributo al mondo esterno. I metodi getter, in genere, non modificano i dati; lo restituiscono e basta.
In Python, come abbiamo già avuto modo di vedere, i metodi getter sono definiti come qualsiasi altro metodo, ma per convenzione, i loro nomi iniziano con `get` seguito dal nome dell'attributo che sono progettati per restituire.

Analizziamo il seguente codice di esempio per il metodo getter:

```
class Person(object):
     def __init__ (self, name):
          self.__name = name # Attributo privato
     def get_name(self): # metodo getter
          return self.__name
```

#### 8.3.5.2 Metodi setter

I metodi setter forniscono un modo per modificare il valore dell'attributo privato di un oggetto. A differenza dell'accesso alla modifica diretta dell'attributo, l'utilizzo di un metodo setter consente la convalida o la trasformazione dei dati prima che vengano assegnati all'attributo. Ciò garantisce che solo dati validi e correttamente formattati vengano memorizzati, in questo modo manteniamo l'integrità dello stato dell'oggetto.
In Python, come abbiamo già avuto modo di vedere, i metodi setter sono definiti in modo simile ad altri metodi ma in genere, per convenzione, i loro nomi iniziano con `set` seguito dal nome dell'attributo che sono progettati per modificare.

Analizziamo il seguente codice di esempio per il metodo setter:

[22] Object Oriented Programming with Python for beginners, Sam Campbell, 2024

```
class Person(object):
     def __init__(self, name):
          self.__name = name # Attributo privato
     def set_name(self, name): # Metodo setter
          if isinstance(name, str) and len(name) > 0: #validazione
               self.__name = name
          else:
               raise ValueError("Invalid name")
```

#### 8.3.5.3 ** Cosa è un decoratore

In pochissime e povere parole un decoratore python è una funzione che ne avvolge un'altra, aggiungendo funzionalità senza modificare il codice originale della funzione decorata.
In pratica, è un modo per estendere o modificare il comportamento di una funzione esistente, aggiungendo codice prima o dopo l'esecuzione della funzione stessa.

Scendendo un po' più nel dettaglio possiamo dire che:

- ➔ Un decoratore accetta una funzione come argomento e restituisce una nuova funzione modificata.
- ➔ Utilizza la sintassi `@decoratore` sopra la definizione della funzione che si vuole decorare.
- ➔ Questo approccio rende il codice più pulito e modulare, evitando la duplicazione di codice.
- ➔ ** I decoratori sono un'applicazione del concetto di funzioni di prima classe in Python, dove le funzioni possono essere passate come argomenti, restituite da altre funzioni e assegnate a variabili.

#### 8.3.5.4 Property decorator

Python fornisce un modo più elegante di definire getter e setter utilizzando il decoratore di proprietà (@property decorator) e il corrispondente decoratore setter.
Questo approccio consente di utilizzare direttamente il nome dell'attributo (come se fosse pubblico) mentre si richiama ancora il getter sottostante, in questo modo può sembrare che l'attributo sia stato definito pubblico.

Analizziamo il seguente codice di esempio per il metodo setter:

```
class Person(object):
    def __init__ (self, name):
        self.__name = name

    @property
    def name(self): # Getter method
        return self.__name

    @name.setter
    def name(self, name): # Setter method
        if isinstance(name, str) and len(name) > 0:
```

```
            self.__name = name
        else:
            raise ValueError("Invalid name")
```

Utilizzando questo approccio, è possibile interagire con l'attributo `name` come se fosse un attributo *regolare*, ma i metodi `getter` e `setter` sono chiamati automaticamente.

Analizziamo il seguente codice di esempio che richiama i metodi getter e setter tramite la definizione mediante decoratori riportata precedentemente:

```
p = Person("Carlo")
print(p.name) # Richiama il getter
p.name = "Carla" # Richiama il setter
print(p.name) # Richiama di nuovo il getter
```

Lanciando il codice otteniamo il seguente output:

```
Carlo
Carla
```

I metodi `getter` e `setter` sono essenziali per l'implementazione dell'incapsulamento in OOP. Forniscono un accesso controllato agli attributi di un oggetto, facilitando la convalida, la trasformazione e il mantenimento dell'integrità dell'oggetto.

L'utilizzo del @property decorator in Python semplifica la dichiarazione di questi metodi, rendendo il codice più pulito e più pytonico.

#### 8.3.5.5 Il codice della classe Punto con le properties

Riprendiamo il codice della classe Punto completa visto nel capitolo 8.3.4.
Inserendo il *decoratore* `@property` otterremo:

```
class Punto(object):
  """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
  def __init__(self,X,Y):
      self.__X = X
      self.__Y = Y
  def stampa(self):
      return str(self.X)+" "+str(self.Y)
  @property
  def X(self):
      return self.__X
  @property
  def Y(self):
      return self.__Y
  @X.setter
```

```
    def X(self,val):
        self.__X = val
    @Y.setter
    def Y(self,val):
        self.__Y = val
    def __str__(self):
        return "("+str(self.X)+","+str(self.Y)+")"
    def getDistance(self, P):
        return ((self.X - P.X)**2 + (self.Y - P.Y)**2)**0.5
    def translatePoint(self, t):
        return Punto(self.X + t,self.Y + t)
```

In grassetto sono indicate le modifiche.
Gli attributi X e Y in questo codice sono stati implementati privati.

## 8.4 Ereditarietà

Il paradigma della programmazione orientata agli oggetti prevede il concetto di ereditarietà. Il linguaggio Python implementa il concetto di ereditarietà, in questo paragrafo affronteremo questi concetti.

### 8.4.1 Classe di esempio Cerchio

Come esempio guida consideriamo la rappresentazione di un cerchio su un piano cartesiano. Il cerchio viene individuato univocamente attraverso il punto che ne definisce il centro e dal suo raggio. P.e. se il centro è nel punto A=(2,2) e il raggio è 1 allora abbiamo la situazione rappresentata in figura 3 .

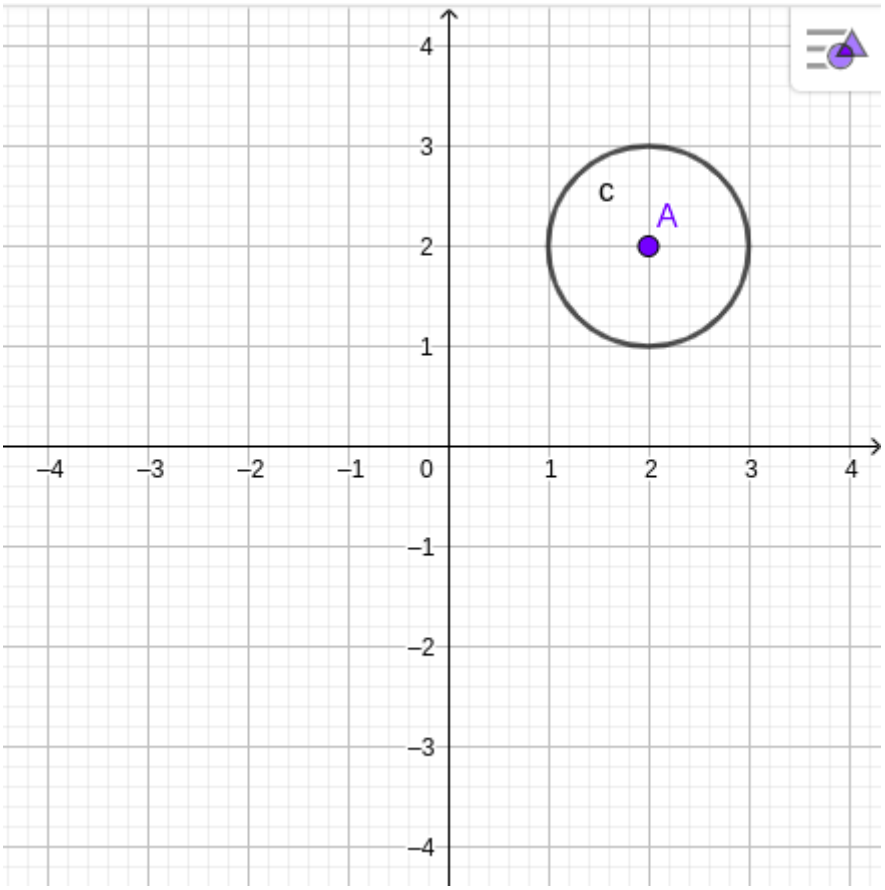


Fig.3

Implementiamo il codice Python per descrivere l'oggetto cerchio attraverso i suoi attributi Centro e Raggio e inseriamo il metodo per il calcolo della superficie. Il Centro verrà rappresentato da un oggetto di tipo `Punto` visto nei precedenti paragrafi.

Il codice che implementa la classe cerchio potrebbe essere il seguente:

```
class Cerchio(object):
    def __init__(self,centro,raggio):
        self.centro = centro
        self.raggio = raggio
    def getCentro(self):
        return self.centro
    def setCentro(self,punto):
        self.centro = punto
    def getRaggio(self):
        return self.raggio
    def setRaggio(self,val):
        self.raggio = val
    def __str__(self):
        return "C= "+str(self.centro)+" r= "+str(self.raggio)
    def getSuperficie(self):
        pigreco=3.1415
        return pigreco*self.raggio**2
```

Aggiungiamo il codice per provare la nostra classe `Cerchio`:

```
c1=Cerchio(Punto(2,2),1)
print(c1)
print(c1.getSuperficie())
```

Otteniamo il seguente output:

```
C= (2,2) r= 1
3.1415
```

Dall'analisi del codice e dell'output osserviamo che:

- La classe `Cerchio` definisce due attributi: `centro` e `raggio`
- Non viene esplicitato che centro deve essere un oggetto di tipo Punto così come, solitamente, non viene esplicitato il tipo di una variabile in uno script Python
- Il metodo `getSuperficie()` restituisce il calcolo dell'area del cerchio.

Il secondo punto della precedente lista di commenti ci può far pensare che al posto di un oggetto di tipo Punto potremo inserire, ad esempio, una stringa!
Proviamo il seguente codice:

```
c2=Cerchio("str",2)
print(c2)
print(c2.getSuperficie())
```

Otteniamo il seguente output:

```
C= str r= 2
12.566
```

Dall'analisi del codice e dell'output osserviamo che:

- Al posto di passare un tipo `Punto` per definire il valore del centro del cerchio, abbiamo passato una stringa. L'interprete non ha dato errori.
- Il calcolo della superficie del cerchio è corretto in quanto nella formula non è presente il centro del cerchio, quindi non interviene l'oggetto errato che è stato inserito nella creazione dell'oggetto Cerchio c2.

Avremmo sicuramente ottenuto un errore nel caso in cui avessimo utilizzato il centro inserito per effettuare dei calcoli o avessimo utilizzato dei metodi della classe Punto che utilizzavano direttamente gli attributi della classe.

### 8.4.2 La sottoclasse Cilindro

Ora vogliamo modellizzare un cilindro. Sappiamo che il cilindro è un solido di rotazione che ha come base un cerchio. In figura 4 riportiamo un cilindro che ha come base il

cerchio di figura 3- di altezza 2.

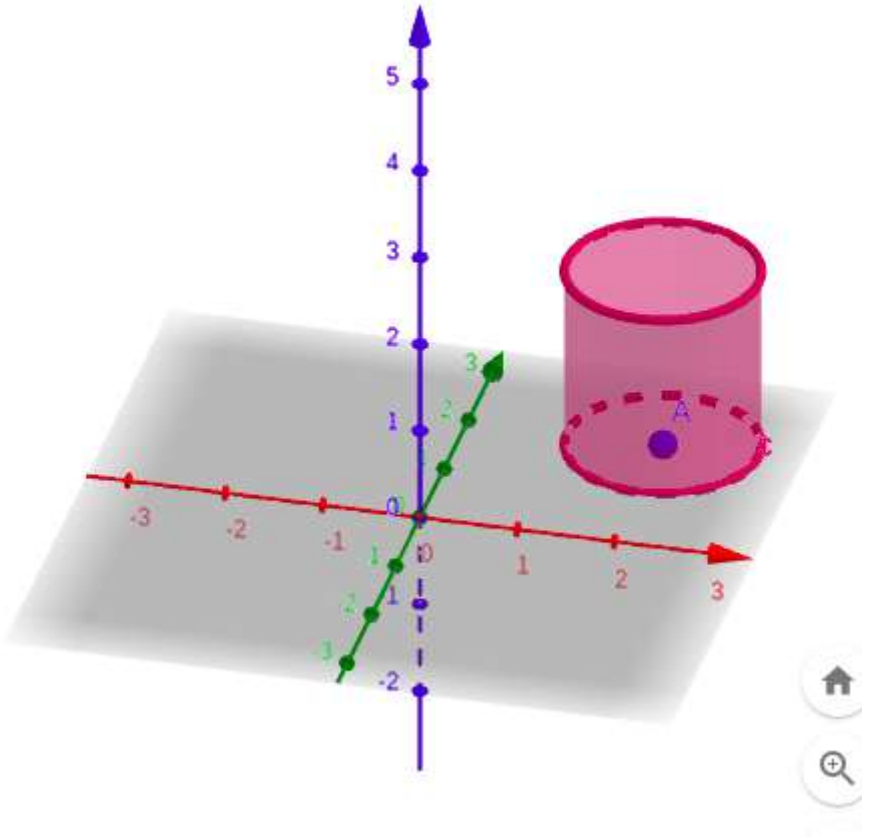



Fig.4

Modellizziamo la figura geometrica Cilindro.
Innanzitutto individuiamo gli attributi: un cilindro è individuato da una base e da un'altezza. Quindi potremmo pensare di costruire la classe `Cilindro` analogamente a quanto fatto per la classe `Cerchio` in cui un attributo era un oggetto di tipo `Punto`.

Affrontiamo il problema da un'altra prospettiva, potremmo pensare di estendere, ampliare la classe Cerchio aggiungendo attributi e metodi che andranno a caratterizzare la nuova classe. Tale classe nascerà dalla classe esistente `Cerchio` ma avrà caratteristiche specifiche che la andranno a differenziare dalla classe di partenza.

Questa operazione concettuale si chiama **ereditarietà** ed è una caratteristica fondamentale della programmazione orientata agli oggetti.

La nuova classe `Cilindro` avrà un nuovo attributo, l'*altezza*, che insieme al centro e al raggio della base individuano univocamente un cilindro nello spazio cartesiano a tre dimensioni.
Vogliamo *ridefinire* il metodo di stampa delle informazioni dell'oggetto e soprattutto il metodo di calcolo della superficie della figura geometrica. Inoltre andremo ad aggiungere un metodo per il calcolo del volume della figura geometrica.

Le classi derivate da una classe generatrice attraverso il meccanismo dell'ereditarietà

vengono chiamate `sottoclassi`.

Consideriamo il seguente codice che formalizza la classe `Cilindro` derivata dalla classe `Cerchio`:

```
class Cilindro(Cerchio):
   def __init__(self,centro,raggio,altezza):
      super().__init__(centro,raggio)
      self.altezza = altezza
   def getAltezza(self):
       return self.altezza
   def setAltezza(self,val):
       self.altezza = val
   def __str__(self):
       return "Cilindro: "+super().__str__()+" h= "+str(self.
altezza)
   def getSuperficie(self):
       pigreco=3.1415
       return 2*pigreco*self.raggio*self.altezza +
2*super().getSuperficie()
   def getVolume(self):
       return super().getSuperficie()*self.altezza
```

Anche in questo caso aggiungiamo il codice per provare la nostra classe `Cilindro`:

```
c1=Cilindro(Punto(2,2),1,2)
print(c1)
print(c1.raggio)
print(c1.getCentro())
print(c1.getSuperficie())
print(c1.getVolume())
```

Otteniamo il seguente output:

```
Cilindro: C= (2,2) r= 1 h= 2
1
(2,2)
18.849
6.283
```

Dall'analisi del codice e dell'output osserviamo che:

- La classe `Cilindro` eredita dalla classe `Cerchio` in quanto nella definizione della classe, nella prima riga del codice, tra le parentesi tonde vi è indicata la classe da cui eredita.
- Nella definizione della classe vi è riportato solo l'attributo aggiuntivo, l'altezza, mentre gli altri attributi non si riscrivono in quanto sono già presenti nella classe padre detta anche **superclasse**.

- ➔ Gli attributi della superclasse sono visibili nella classe derivata come se fossero stati lì definiti.
- ➔ Il costruttore della classe `Cilindro` richiama il costruttore della superclasse utilizzando la parola chiave `super()` seguita dal metodo costruttore `__init__()`
- ➔ I metodi `__str__()` e `getSuperficie()` sono stati ridefiniti, è stato effettuato un **overriding** dei metodi della superclasse.
- ➔ Il metodo `getVolume()` è un nuovo metodo che è presente solo nella classe `Cilindro`.

#### 8.4.2.1 Piccola digressione matematica

Questo paragrafo nasce come precisazione del significato delle figure geometriche qui utilizzate. Non è necessario leggerlo perché non aggiunge nulla alla discussione sull'ereditarietà implementata del Python.

Dal momento che stiamo costruendo una classe che descrive un solido da una classe che descrive una figura piana, dobbiamo considerare che il perimetro della figura piana si estende a una superficie della figura solida, così come una superficie di una figura piana si estende a un volume di un solido.

Questo ragionamento porterebbe a effettuare un'override del metodo del perimetro verso il metodo della superficie di una figura solida e analogamente l'override del metodo di una superficie verso il metodo del volume di una figura solida.

Quindi nell'esempio della classe `Cilindro` il metodo `getSuperficie` doveva essere sovrascritto sul metodo `getVolume` (o `getSuperficie` su tre dimensioni).

Come detto prima questa è solo una piccola precisazione prettamente matematica.

## 8.5 Polimorfismo

Il *polimorfismo* è un altro concetto fondamentale della programmazione orientata agli oggetti. Il termine deriva dal greco e significa "avere molte forme". Nel contesto della programmazione, il *polimorfismo* permette di utilizzare un'interfaccia unica per entità di diverso tipo. In altre parole, consente di trattare oggetti di classi diverse in modo uniforme, purché implementino gli stessi metodi.

Python, essendo un linguaggio a *tipizzazione dinamica*, implementa naturalmente il *polimorfismo*. Non è necessario dichiarare esplicitamente interfacce o utilizzare parole chiave specifiche come in altri linguaggi.

Ora analizziamo il seguente codice:

```
class Figura(object):
    """    Classe base per le figure geometriche
    - Appunti 'Python Primi Passi' - """
    def __init__(self):
        pass
```

```
    def getArea(self):
        pass
    def __str__(self):
        return "Figura geometrica generica"

class Rettangolo(Figura):
    def __init__(self, base, altezza):
        self.base = base
        self.altezza = altezza
    def getArea(self):
        return self.base * self.altezza
    def __str__(self):
       return f"Rettangolo: base={self.base},
altezza={self.altezza}"

class Triangolo(Figura):
    def __init__(self, base, altezza):
        self.base = base
        self.altezza = altezza
    def getArea(self):
        return (self.base * self.altezza) / 2
    def __str__(self):
     return f"Triangolo: base={self.base}, altezza={self.altezza}"

class Cerchio(Figura):
    def __init__(self, centro, raggio):
        self.centro = centro
        self.raggio = raggio
    def getArea(self):
        pigreco = 3.1415
        return pigreco * self.raggio**2
    def __str__(self):
     return f"Cerchio: centro={self.centro}, raggio={self.raggio}"

# Definisco questa funzione per dimostrare il polimorfismo
def stampaArea(figura):
    print(f"Area {figura}: {figura.getArea()}")

# Programma principale
r1 = Rettangolo(4, 5)
t1 = Triangolo(4, 5)
c1 = Cerchio(Punto(0,0), 3)

stampaArea(r1)
stampaArea(t1)
stampaArea(c1)
```

Lanciamo il programma, in uscita otteniamo:

```
Area Rettangolo: base=4, altezza=5: 20
Area Triangolo: base=4, altezza=5: 10.0
Area di Cerchio: centro=(0,0), raggio=3: 28.2735
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo definito una classe base `Figura` con un metodo `getArea()` che serve da interfaccia comune.
- ➔ Le classi `Rettangolo`, `Triangolo` e `Cerchio` ereditano da `Figura` e implementano ciascuna la propria versione del metodo `getArea()`.
- ➔ La funzione `stampaArea()` accetta un parametro di tipo `Figura` ma in realtà può ricevere qualsiasi oggetto che implementi il metodo *`getArea()`*.
- ➔ Python non verifica il tipo dell'oggetto passato, ma solo che abbia il metodo richiesto. Questo è noto come "*`duck typing`*": "se cammina come un'anatra e starnazza come un'anatra, allora è un'anatra".

### 8.5.1 Polimorfismo con operatori

Un'altra forma di *polimorfismo* in Python è rappresentata dalla possibilità di ridefinire il comportamento degli operatori standard del linguaggio. Questo si ottiene implementando dei metodi speciali, chiamati "*metodi magici*" o "*dunder methods*" (double underscore methods). Useremo diffusamente questi metodi nel paragrafo 8.7.4

Riprendiamo la classe Punto e implementiamo l'operatore `+` per sommare due oggetti punto:

```
class Punto(object):
    """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y
    def __str__(self):
        return f"({self.X},{self.Y})"
    def __add__(self, other):
        return Punto(self.X + other.X, self.Y + other.Y)

p1 = Punto(1, 2)
p2 = Punto(3, 4)
p3 = p1 + p2

print(f"p1 = {p1}")
print(f"p2 = {p2}")
print(f"p1 + p2 = {p3}")
```

Lanciamo il programma, in uscita otteniamo:

```
p1 = (1,2)
```

```
p2 = (3,4)
p1 + p2 = (4,6)
```

Dall'analisi del codice e dell'output osserviamo che:

- Il metodo `__add__` permette di definire il comportamento dell'operatore `+` quando applicato a oggetti della classe Punto.
- Quando scriviamo `p1 + p2`, Python traduce questa espressione in `p1.__add__(p2)`.
- Il risultato è un nuovo oggetto Punto con coordinate pari alla somma delle coordinate dei punti originali.

Tratteremo diffusamente questi metodi speciali nel paragrafo 8.7.4.

### 8.5.2 Polimorfismo con collezioni eterogenee

Un'applicazione comune del polimorfismo è l'uso di collezioni che contengono oggetti di tipi diversi.

Riprendendo l'esempio delle figure geometriche:

```
figure = [Rettangolo(4, 5), Triangolo(4, 5), Cerchio(Punto(0,0),
3)]

area_totale = 0
for figura in figure:
    area = figura.getArea()
    print(f"Area di {figura}: {area}")
    area_totale += area

print(f"Area totale di tutte le figure: {area_totale}")
```

Lanciamo il programma, in uscita otteniamo:

```
Area di Rettangolo: base=4, altezza=5: 20
Area di Triangolo: base=4, altezza=5: 10.0
Area di Cerchio: centro=(0,0), raggio=3: 28.2735
Area totale di tutte le figure: 58.2735
```

Dall'analisi del codice e dell'output osserviamo che:

- Abbiamo creato una lista contenente oggetti di tre classi diverse: `Rettangolo`, `Triangolo` e `Cerchio`.
- Nonostante le diverse implementazioni, possiamo trattare tutti questi oggetti in modo uniforme, iterando attraverso la lista e chiamando il metodo `getArea()` su ciascuno.
- Python determina dinamicamente quale versione del metodo `getArea()`

chiamare in base al tipo effettivo dell'oggetto, vediamo così il *polimorfismo* in azione.

Il *polimorfismo* è un aspetto cruciale della programmazione orientata agli oggetti che permette di scrivere codice più flessibile e riutilizzabile.

In Python, grazie alla *tipizzazione dinamica*, il *polimorfismo* è implementato in modo particolarmente semplice e naturale.

## 8.6 I modificatori di visibilità

Vari linguaggi orientati agli oggetti come C++, Java, Python controllano le modifiche all'accesso che vengono utilizzate per limitare l'accesso alle variabili e ai metodi della classe. La maggior parte dei linguaggi di programmazione ha tre forme di modificatori di accesso, che sono *Public* , *Protected* e *Private* in una classe.

Python usa il simbolo '_' per determinare il controllo di accesso per un membro dati specifico o una funzione membro di una classe. Gli specificatori di accesso in Python hanno un ruolo importante da svolgere nella protezione dei dati da accessi non autorizzati e nell'impedire che vengano sfruttati.

La dichiarazione di attributi e metodi privati e pubblici garantisce il principio dell'*incapsu*lamento e dell'*information hiding* dei dati.

Una classe in Python ha tre tipi di modificatori di accesso:

- ➔ Modificatore di accesso pubblico
- ➔ Modificatore di accesso protetto
- ➔ Modificatore di accesso privato

### 8.6.1 Modificatore di accesso pubblico

I membri pubblici (attributi e metodi dichiarati in una classe) sono accessibili dall'esterno della classe. L'oggetto istanza della classe che si sta utilizzando è necessario per invocare un metodo pubblico.

Tutti i membri in una classe Python sono pubblici per impostazione predefinita. È possibile accedere a qualsiasi membro dall'esterno dell'ambiente della classe.

Consideriamo la seguente classe di esempio:

```
class Studente(object):
  def __init__(self,nome,eta):
      self.nome = nome  # attributo di istanza
      self.eta = eta    # attributo di istanza
```

```
  def giorniEta(self):
      return self.eta * 365
```

Possiamo accedere agli attributi della classe Studente utilizzando il seguente codice:

```
studente1 = Studente("Carlo",20)
print(studente1.nome,studente1.eta)
studente1.nome = "Mario"
studente1.eta = 21
print(studente1.nome,studente1.eta)
print(studente1.giorniEta())
```

Lanciamo il programma, in uscita otteniamo:

```
Carlo 20
Mario 21
7665
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Riusciamo a referenziare e modificare gli attributi `nome` ed `eta` utilizzando direttamente l'oggetto e il nome dell'attributo come fatto in tutti gli esempi visti sino a ora . Si parla in questo caso di membri pubblici della classe.
- ➔ Attraverso l'oggetto studente1 riusciamo a richiamare il metodo della classe `giorniEta`. Anche questo è un membro pubblico della classe `Studente`.
- ➔ Quindi se non indichiamo nulla tutti i membri della classe risultano essere pubblici.

### 8.6.2 Modificatore di accesso protetto

I membri protetti di una classe sono accessibili dall'interno della classe e sono disponibili anche per le sue sottoclassi. Nessun altro ambiente è autorizzato ad accedervi. Ciò consente alle risorse specifiche della classe padre di essere ereditate dalla classe figlia.

La convenzione di Python per rendere protetta una variabile di istanza consiste nell'aggiungervi un prefisso _ (singolo underscore). Ciò impedisce effettivamente l'accesso a meno che non provenga da una sottoclasse.

Utilizziamo come esempio il codice visto nel paragrafo precedente:

```
class Studente(object):
  def __init__(self,nome,eta):
      self._nome = nome  # attributo di istanza
      self._eta = eta    # attributo di istanza
  def giorniEta(self):
      return self._eta * 365
```

Possiamo accedere agli attributi della classe Studente utilizzando il seguente codice:

```
studente1 = Studente("Carlo",20)
print(studente1.nome)
```

Lanciamo il programma, in uscita otteniamo:

```
Traceback (most recent call last):
  File "esempio1.py", line 11, in <module>
    print(studente1.nome)
AttributeError: 'Studente' object has no attribute 'nome'
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'attributo nome non è più referenziato
- ➔ Se si cerca di referenziarlo si ottiene un errore di tipo `AttributeError`

Fare attenzione al fatto che avremmo potuto referenziare l'attributo con `_nome`. Si lascia al lettore la prova. Naturalmente il programmatore dovrebbe astenersi dall'utilizzare direttamente i membri che abbiano un underscore come prefisso in quanto sono membri protetti e quindi utilizzabili solo dalla classe stessa o dalle sue sottoclassi.

Python non ha alcun meccanismo che limiti efficacemente l'accesso a qualsiasi attributo o metodo di istanza, prescrive solo questa convenzione di anteporre al nome dell' attributo/metodo un carattere di sottolineatura singolo per emulare il comportamento degli specificatori di accesso protetto. Dà un forte suggerimento di non toccarlo dall'esterno della classe. Qualsiasi tentativo in tal senso porterà a un `AttributeError`.

### 8.6.3 Modificatore di accesso privato

Per quanto riguarda la dichiarazione di un membro privato. l'interprete Python prescrive Il doppio carattere di sottolineatura __ per rendere un'attributo o un metodo privato.

Nel caso del doppio underscore, l'interprete Python esegue la manipolazione dei nomi delle variabili private, ogni membro con un doppio underscore verrà cambiato in `_object._class__variable`. Quindi, è ancora possibile accedervi dall'esterno della classe, ma la pratica dovrebbe essere astenuta.

A supporto di questo comportamento si propone nuovamente la classe del precedente paragrafo andando, però, a sostituire `_nome` e `_eta` con `__nome` e `__eta`, per indicare che sono attripbuti privati:

```
class Studente(object):
  def __init__(self,nome,eta):
      self.__nome = nome  # attributo di istanza
      self.__eta = eta    # attributo di istanza
```

```
    def giorniEta(self):
        return self.__eta * 365
```

Istanziamo la classe e stampiamo sullo standard output tutti gli attributi e i metodi della classe Studente utilizzando il seguente codice:

```
studente1 = Studente("Carlo",20)
print(dir(studente1))
print("\n"+studente1._Studente__nome)
```

Lanciamo il programma, in uscita otteniamo:

```
['_Studente__eta', '_Studente__nome', '__class__',
'__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__gt__',
'__hash__', '__init__', '__init_subclass__', '__le__',
'__lt__', '__module__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__str__', '__subclasshook__', '__weakref__', 'giorniEta']

Carlo
```

Dall'analisi del codice e dell'output osserviamo che:

- In grassetto sono indicati i nomi degli attributi che sono stati scritti aggiungendo il prefisso dei doppi underscore. Si nota che il nome dell'attributo ha come prefisso il nome della classe preceduto da un singolo underscore.
- Possiamo sempre accedere direttamente all'attributo utilizzando il nome modificato dall'interprete. p.e. nel nostro esempio possiamo riferici all'attributo nome utilizzando: `studente1._Studente__nome`

# 8.7 Importare le classi

## 8.7.1 Utilizzare le classi definite in altri sorgenti

Nel mondo della programmazione Python, la suddivisione del codice in moduli separati è una pratica che favorisce ordine e riusabilità.
Immaginate di avere un codice che gestisce punti e cerchi in un piano cartesiano. Per una gestione ordinata, potremmo definire le classi `Punto` e `Cerchio` in un modulo separato e utilizzarle in altre parti del progetto.
Scopriamo come realizzare tutto ciò, mantenendo il nostro codice chiaro e modulare.

Prima però definiamo cosa intendiamo per *modulo*.

### 8.7.1.1 Cosa è un modulo

Un modulo in Python è semplicemente un file con estensione `.py` che contiene definizioni di funzioni, variabili e classi.
Come abbiamo accennato sopra separare il codice in più moduli ci permette di organizzarlo meglio e di favorire il riutilizzo delle componenti.

Vediamo ora come definire le classi `Punto` e `Cerchio` in un modulo e come importarle in un altro.

### 8.7.2 Il nostro primo modulo: `geometria.py`

Riprendiamo gli esempi delle classi Punto e Cerchio visti nei precedenti paragrafi. Supponiamo di avere un modulo chiamato `geometria.py`, dove definiamo le classi `Punto` e `Cerchio`.
In questo file andiamo a copiare la definizione della classe `Punto` e della classe `Cerchio` viste prima. riportiamo di seguito il codice per comodità:

```
# geometria.py
class Punto(object):
    """Classe che rappresenta un punto nel piano cartesiano."""

    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    def stampa(self):
        return str(self.X) + " " + str(self.Y)

    def getX(self):
        return self.X

    def getY(self):
        return self.Y

    def setX(self, val):
        self.X = val

    def setY(self, val):
        self.Y = val

    def __str__(self):
        return "(" + str(self.X) + "," + str(self.Y) + ")"

    def getDistance(self, P):
        return ((self.X - P.X)**2 + (self.Y - P.Y)**2)**0.5

    def translatePoint(self, t):
        return Punto(self.X + t, self.Y + t)
```

```
class Cerchio(object):
      """Classe che rappresenta un cerchio con un centro e un
raggio."""

    def __init__(self, centro, raggio):
        self.centro = centro
        self.raggio = raggio

    def getCentro(self):
        return self.centro

    def setCentro(self, punto):
        self.centro = punto

    def getRaggio(self):
        return self.raggio

    def setRaggio(self, val):
        self.raggio = val

    def __str__(self):
                return "C= " + str(self.centro) + " r= " +
str(self.raggio)

    def getSuperficie(self):
        pigreco = 3.1415
        return pigreco * self.raggio**2

# Fine di geometria.py
```

In questo modulo, `geometria.py`, abbiamo definito le classi `Punto` e `Cerchio`, con metodi per manipolare le coordinate di un punto e per calcolare la superficie di un cerchio. Se eseguissimo questo codice vedremmo che a video non verrebbe stampato nulla, non verrebbe nemmeno eseguita alcuna operazione in quanto il modulo contiene solo la definizione delle classi, ma non l'istanziazione e l'utilizzo delle stesse.

### 8.7.3 Importare classi da altri moduli

Vediamo ora come utilizzare le classi `Punto` e `Cerchio` definite in `geometria.py` in un altro modulo, per esempio `main.py`.
Questo è il file in cui vogliamo utilizzare queste classi per creare oggetti e lavorare con essi. In altre parole è il programma che andrà a eseguire la logica del nostro algoritmo che utilizza le classi definite in `geometria.py`.

Scriviamo quindi il seguente codice:

```
# main.py
from geometria import Punto, Cerchio
```

```
# Creiamo un'istanza della classe Punto
p = Punto(3, 4)
print("Il punto p:", p)

# Creiamo un'istanza della classe Cerchio
c = Cerchio(p, 5)
print("Il cerchio c:", c)
print("Superficie del cerchio:", c.getSuperficie())
```

Lanciamo il programma, in uscita otteniamo:

```
Il punto p: (3,4)
Il cerchio c: C= (3,4) r= 5
Superficie del cerchio: 78.53750000000001
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Con l'istruzione `from geometria import Punto, Cerchio` indichiamo all'interprete Python di cercare il modulo `geometria.py` nella directory corrente e di importare solo le classi `Punto` e `Cerchio` definite in esso.
- ➔ Dopo aver importato le classi di interesse creiamo le istanze di queste classi e utilizziamo i loro metodi. In questo caso calcoliamo la superficie di un cerchio di raggio 5 e di centro con coordinate (3,4)

Naturalmente se il modulo non esistesse otterremo il seguente errore:

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    from geometria_nonesiste import Punto, Cerchio
ModuleNotFoundError: No module named 'geometria_nonesiste'
```

Abbiamo fatto la supposizione che il file che contiene il programma principale si chiama `main.py`, quindi l'errore avviene alla riga 2 di tale file.

### 8.7.4 Importare tutto da un modulo

Se volessimo importare tutto il contenuto di un modulo, potremmo usare l'operatore `*`. Questo approccio, tuttavia, è sconsigliato per evitare conflitti di nome.

Esaminiamo il seguente codice, identico a quello riportato precedentemente in cui è stata modificata l'istruzione di import.

```
# main.py
from geometria import *
```

```
p = Punto(1, 2)
c = Cerchio(p, 10)
print("Il cerchio:", c)
print("Superficie del cerchio:", c.getSuperficie())
```

Lanciando il programma notiamo che si ottengono lo stesso identico output del programma precedente, infatti:

```
Il punto p: (3,4)
Il cerchio c: C= (3,4) r= 5
Superficie del cerchio: 78.53750000000001
```

L'istruzione `from geometria import *` importa tutte le classi, funzioni e variabili definite nel modulo `geometria.py`.
Sebbene questo metodo sia comodo, potrebbe non essere chiaro quali elementi siano stati importati, rendendo il codice più difficile da leggere e mantenere.

Approfondiremo questa problematica nel capitolo 10 “Importazione dei moduli”.

### 8.7.5 Importazione con la direttiva `alias`

Per evitare conflitti di nome o per abbreviare i nomi dei moduli, è possibile utilizzare la direttiva `alias`:

```
# main.py
import geometria as geo

p = geo.Punto(0, 0)
c = geo.Cerchio(p, 2)
print("Il cerchio con centro in", c.getCentro(), "e raggio",
c.getRaggio())
```

In questo esempio, abbiamo importato il modulo `geometria` con l'alias `geo`. Ora possiamo accedere alle classi `Punto` e `Cerchio` tramite `geo.Punto` e `geo.Cerchio`, rendendo il codice più compatto e leggibile.

### 8.7.6 Gestire il percorso dei moduli

In progetti più complessi, i moduli possono essere organizzati in *sottodirectory*.
Possiamo importare moduli da sottodirectory usando il percorso relativo. Immaginiamo che `geometria.py` si trovi in una sottodirectory chiamata `forme`:

```
progetto/
|-- main.py
```

```
|-- forme/
    |-- geometria.py
```

In `main.py`, importeremmo `Punto` e `Cerchio` come segue:

```
# main.py
from forme.geometria import Punto, Cerchio

p = Punto(5, 5)
c = Cerchio(p, 3)
print(c)
print("Superficie del cerchio:", c.getSuperficie())
```

Qui, stiamo specificando il percorso completo per accedere al modulo `geometria` all'interno della sottodirectory `forme`.

### 8.7.7 Consigli pratici

Alcuni piccoli consigli per gestire meglio il nostro codice:

- ➔ Create moduli con uno scopo specifico e ben definito
- ➔ Scegliete nomi significativi per i vostri moduli
- ➔ Documentate le vostre classi e metodi con *docstring*
- ➔ Importate solo ciò che vi serve
- ➔ Organizzate i file in cartelle quando il progetto cresce

## 8.8 Un esempio notevole: classe Frazione

### 8.8.1 Il progetto

Vogliamo costruire una libreria replica della libreria `fractions`[23] presente nella standard library del linguaggio Python.
Questa libreria gestisce i numeri razionali sia in forma decimale che in forma frazionaria.

A titolo di esempio si esegua il seguente codice:

```
from fractions import Fraction
r1=Fraction(1,2)
r2=Fraction(1,3)
print(r1," ",r2)
somma=r1+r2
print("somma= ", somma)
prodotto=r1*r2
print("prodotto= ", prodotto)
print(r1+r1)
```

[23] https://docs.python.org/3/library/fractions.html

Se eseguiamo il programma otteniamo il seguente output:

```
1/2   1/3
somma=  5/6
prodotto=  1/6
<class 'fractions.Fraction'>

1
```

Dall'analisi del codice e dell'output osserviamo che:

- La libreria `fractions` mette a disposizione una classe `Fraction` che permette di istanziare dei numeri razionali sotto forma di frazioni.
- Sugli oggetti di questa classe possono essere effettuate delle operazioni aritmetiche e il risultato risulta essere sempre un oggetto `Fraction`
- La classe esegue degli algoritmi che effettuano la semplificazione delle frazioni e il riconoscimento delle frazioni apparenti (nell'esempio. ½ + ½ = 2/2 = 1)

### 8.8.2 La classe `Frazione`

Partiamo a costruire la classe andando a definire la classe `Frazione`. Questa, in prima battuta, avrà due attributi:

- Numeratore
- Denominatore

Sono due numeri che definiscono la frazione, p.e. se numeratore vale 5 e il denominatore vale 8 allora la frazione definita è ⅝ .

Naturalmente sia numeratore che denominatore sono numeri interi.

Dobbiamo pensare anche alla rappresentazione stringa dell'oggetto della classe, quindi andremo a effettuare l'override del metodo `__str__` (come visto nel paragrafo 8.3.1) per avere una stampa dell'oggetto che ricordi una frazione.

Quindi una prima bozza della classe potrebbe essere la seguente:

```
class Frazione(object):
    """    Classe Frazione
- Appunti 'Python Primi Passi' - """
    def __init__(self,numeratore,denominatore):
        self.__numeratore = numeratore
        self.__denominatore = denominatore
    def getNumeratore(self):
        return self.__numeratore
    def getDenominatore(self):
        return self.__denominatore
    def setNumeratore(self,val):
```

```
        self.__numeratore = val
    def setDenominatore(self,val):
        self.__denominatore = val
    def __str__(self):
                                                                    return
str(self.__numeratore)+"/"+str(self.__denominatore)

r1=Frazione(1,2)
r2=Frazione(2,1)
r3=Frazione(2,2)
r4=Frazione(1,0)
r5=Frazione(0,0)

print(r1)
print(r2)
print(r3)
print(r4)
print(r5)
```

Lanciando il programma si ottiene il seguente output:

```
1/2
2/1
2/2
1/0
0/0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo definito la classe Frazione secondo quanto detto precedentemente. Il costruttore della classe inizializza il numeratore e il denominatore e l'override di __str__ produce una rappresentazione dell'oggetto istanziato che ricorda una frazione.
- ➔ Sono stati inseriti anche i metodi getter e setter dei due attributi della classe

Notiamo, dall'output, che qualcosa non funziona correttamente. La classe :

- ➔ Non effettua la semplificazione delle frazioni
- ➔ Non riconosce le frazioni apparenti
- ➔ Il denominatore può assumere valore 0 portando a una non coerenza matematica

Il primo problema verrà risolto nel prossimo paragrafo in quanto dobbiamo codificare un algoritmo di semplificazione.

Il secondo problema viene risolto implementando la semplificazione e il riconoscimento del denominatore unitario. Ad esempio se la frazione è 4/2 è una frazione apparente, semplificando otteniamo 2/1 riconoscendo il denominatore unitario possiamo stampare a video soltanto il numeratore.

Il problema del denominatore nullo si risolve effettuando un semplice test, eventualmente se il denominatore è 0 si può forzare la frazione ad avere denominatore 1. Il comportamento più corretto sarebbe quelo di sollevare un'eccezione, ma per ora saltiamo questa soluzione.

Con le modifiche il nuovo codice risulta essere il seguente:

```
class Frazione(object):
    """    Classe Frazione
- Appunti 'Python Primi Passi' - """
    def __init__(self,numeratore = 0,denominatore = 1):
        # Eliminazione incongruenza divisore 0
        if(denominatore == 0):
            denominatore=1
        self.__numeratore = numeratore
        self.__denominatore = denominatore
    def getNumeratore(self):
        return self.__numeratore
    def getDenominatore(self):
        return self.__denominatore
    def setNumeratore(self,val):
        self.__numeratore = val
    def setDenominatore(self,val):
        self.__denominatore = val
    def __str__(self):
         if self.__denominatore == 1:
            ratio =  str(self.__numeratore)
         else:
          ratio=
          str(self.__numeratore)+"/"+str(self.__denominatore)
         return ratio

r1=Frazione(1,2)
r2=Frazione(2,1)
r3=Frazione(1,0)
r4=Frazione(4)
r5=Frazione()

print(r1)
print(r2)
print(r3)
print(r4)
print(r5)
```

Lanciando il programma si ottiene il seguente output:

```
1/2
2
```

```
1
4
0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo risolto il problema delle frazioni apparenti modificando il metodo `__str__`
- ➔ Se il denominatore che viene passato al costruttore è nullo, viene posto a 1
- ➔ Nel costruttore i parametri `numeratore` e `denominatore` sono parametri di default, in questa maniera se non si inserisce il denominatore si ha come oggetto un numero intero (cioè con un denominatore uguale a 1) mentre se non si inserisce nulla si ha come oggetto il numero 0.

## 8.8.3 Il metodo di semplificazione della frazione

Andiamo a progettare e implementare il metodo di semplificazione dell'oggetto appartenente alla classe Frazione.
Questo è un metodo molto importante in quanto se non semplifichiamo la frazione rischiamo di avere dei numeri enormi mano a mano che si seguono dei calcoli.

### 8.8.3.1 Piccolo background teorico

Cosa significa semplificare una frazione, cioè ridurla ai minimi termini?
Senza addentrarci troppo in definizioni e teoremi diciamo che una frazione viene semplificata ai minimi termini quando dividiamo ambo il numeratore e il denominatore per il loro *massimo comune divisore*[24].
In altre parole sia `f` oggetto appartenente alla classe `Frazione`, cioè

$$(1) \qquad f = \frac{N}{D}$$

dove $N, D \in Z$ insieme dei numeri interi relativi

e sia

$$(2) \qquad m = M.C.D.(N, D)$$

il massimo comune divisore tra N e D

allora la frazione f si riduce ai minimi termine nel seguente modo:

$$(3) \qquad f = \frac{N/m}{D/m}$$

cioè si dividono ambo i membri della frazione per l'M.C.D. trovato.

[24] https://it.wikipedia.org/wiki/Massimo_comun_divisore

### 8.8.3.2 Calcolo dell'M.C.D. in Python

Esistono diversi algoritmi per calcolare il massimo comune divisore tra due numeri interi, il più utilizzato è il metodo di Euclide[25]. Non andremo a implementare questo algoritmo ma utilizzeremo la funzione di calcolo dell'MCD che ci fornisce la libreria math della standard library di Python: `gcd()`[26]. Tale funzione dalla versione 3.9 di Python accetta un numero indefinito di parametri, mentre prima accettava solo due numeri.

Quindi la (2) diventa:

$$m = math.gcd(N, D) \qquad \text{(2bis)}$$

ne consegue che la (3) risulta facilmente calcolabile all'interno di un metodo che si può chiamare `semplifica()`.

### 8.8.3.3 Il codice della classe con il metodo `semplifica()`

A questo punto siamo in grado di introdurre il metodo di semplificazione delle frazioni.
Si riprende il codice precedente con l'aggiunta del nuovo metodo. Il codice aggiuntivo è evidenziato su sfondo grigio:

```
import math

class Frazione(object):
    """    Classe Frazione
- Appunti 'Python Primi Passi' - """
    def __init__(self,numeratore = 0,denominatore = 1):
        # Eliminazione incongruenza divisore 0
        if(denominatore == 0):
            denominatore=1
        self.__numeratore = numeratore
        self.__denominatore = denominatore
    def getNumeratore(self):
        return self.__numeratore
    def getDenominatore(self):
        return self.__denominatore
    def setNumeratore(self,val):
        self.__numeratore = val
    def setDenominatore(self,val):
        self.__denominatore = val
    def __str__(self):
        if self.__denominatore == 1:
           ratio =  str(self.__numeratore)
        else:
```

[25] https://it.wikipedia.org/wiki/Algoritmo_di_Euclide
[26] https://docs.python.org/3/library/math.html#math.gcd

```
        ratio= str(self.__numeratore)+"/"+str(self.__denominatore)
        return ratio
    def semplifica(self):
        mcd=math.gcd(self.getNumeratore(),self.getDenominatore())
        return Frazione(int(self.getNumeratore()/mcd), \
                        int(self.getDenominatore()/mcd))

r1=Frazione(10,2)
r2=r1.semplifica()
print(r1)
print(r2)
```

Lanciando il programma si ottiene il seguente output:

```
10/2
5
```

Dall'analisi del codice e dell'output osserviamo che:

- Per utilizzare la funzione che calcola l'M.C.D. abbiamo dovuto importare il modulo `math`
- Il metodo `semplifica()` restituisce un nuovo oggetto `Frazione` che rappresenta la frazione semplificata dell'oggetto chiamante.

## 8.8.4 Introduciamo le operazioni tra oggetti frazione

### 8.8.4.1 Descrizione della funzionalità

Siamo arrivati ad avere una classe che ci permette di rappresentare i numeri razionali sotto forma di frazione. Tra i vari metodi della classe, ne abbiamo anche uno che ci permette di semplificare la frazione qualora non fosse ridotta ai minimi termini.

È già un ottimo risultato ottenuto con una manciata di righe di Python!

Ora vogliamo introdurre le operazioni tra gli oggetti della classe Frazione, in altre parole se abbiamo una frazione `r1` e una frazione `r2` vogliamo poter effettuare, p.e., la somma `r1 + r2` e vogliamo che il risultato sia ancora rappresentato come una frazione.

Ci poniamo l'obiettivo di implementare le quattro operazioni aritmetiche:

- Addizione
- Prodotto
- Sottrazione
- Divisione

Possiamo senz'altro pensare di implementare 4 metodi che eseguano le operazioni sopra indicate.
Ad esempio per l'addizione potremo costruire un metodo `addizione(altraFrazione)`

da utilizzare nel seguente modo:

```
r1 = Frazione(1,2)
r2 = Frazione(1,3)
somma = r1.addizione(r2)
```

Sicuramente questo stralcio di codice potrà assolvere al compito di eseguire le quattro operazioni aritmetiche, fornendo come risultati altri oggetti Frazione.
Ma, in realtà, quello che noi vogliamo è poter eseguire le operazioni nel seguente modo:

```
r1 =  Frazione(1,2)
r2 = Frazione(1,3)
somma = r1 + r2
prodotto = r1 * r2
sottrazione = r1 - r2
divisione = r1 / r2
```

Quindi vogliamo utilizzare gli operatori aritmetici standard.

### 8.8.4.2 Overloading dell'operatore +

Per utilizzare gli operatori aritmetici dobbiamo effettuare un overloading degli operatori aritmetici che vogliamo utilizzare.

Consideriamo l'operatore + sappiamo che se abbiamo due interi, utilizzando questo operatore otteniamo la loro somma.

Come esempio consideriamo il seguente codice:

```
a = 1
b = 2
print(a+b)
```

Lanciando il programma si ottiene il seguente output:

```
3
```

Ma se consideriamo il seguente codice:

```
a = "ciao"
b = " Pippo"
print(a+b)
```

Lanciando otteniamo in output:

```
ciao Pippo
```

Quindi il comportamento dell'operatore + è differente a seconda dei tipi di dati su cui va a operare.
In questo specifico caso l'operatore è stato sovraccaricato dalla classe degli interi (`int`) e dalla classe delle stringhe (`str`).

Per effettuare l'overloading dell'operatore + viene richiamato il metodo privato `__add__(self, other)`[27] (questa tipologia di metodi viene anche denominata *metodi magici*).

Facciamo un semplice esempio riprendendo la classe Punto vista nel capitolo 8.2 (per comodità abbiamo sostituito il metodo `stampa` con il metodo `__str__`)

```
class Punto(object):
   """    Classe Punto di esempio
- Appunti 'Python Primi Passi' - """
   def __init__(self,X,Y):
       self.X = X
       self.Y = Y
   def __str__(self):
       return str(self.X)+" "+str(self.Y)
```

Questa classe definisce un punto sul piano cartesiano, ma non ci interessa comprendere lo scopo della classe, piuttosto vogliamo effettuare l'overloading dell'operatore di somma andando a implementare la somma cartesiana di due punti.

Definiamo cosa intendiamo per somma di due punti:
Siano P1(X1,Y1) e P2(X2,Y2) due punti qualsiasi sul piano cartesiano, definiamo la loro somma come:

$$S = P1 + P2$$

dove S è ancora un punto sul piano cartesiano e le sue coordinate sono la somma delle rispettive coordinate di P1 e P2:

$$S(Xs, Ys),\ dove\ Xs = X1 + X2 \ \ e \ \ Ys = Y1 + Y2$$

Secondo questa definizione potremmo costruire il metodo di somma nel seguente modo:

```
   def __add__(self,P2):
       Xs= self.X + P2.X
       Ys= self.Y + P2.Y
       return Punto(Xs,Ys)
```

Se aggiungiamo questo metodo al precedente codice e andiamo a istanziare due punti e poi a farne la somma aggiungendo questo ulteriore codice di prova della classe così modificata:

[27] https://docs.python.org/3/reference/datamodel.html#object.__add__

```
P1 = Punt(2,3)
P2 = Punto(3,4)
print(P1+P2)
```

Lanciando il programma di esempio otteniamo in output:

```
5 7
```

Quindi in questa maniera abbiamo effettuato un overloading dell'operatore utilizzando il *metodo magico* a lui riservato.

### 8.8.4.3 Overloading degli operatori in Python

Abbiamo visto nei capitoli introduttivi tutti i vari operatori del linguaggio Python, esistono diverse tipologie che si distinguono per numero di operandi (unari o binari), per funzionalità etc,etc (cfr cap. 4.2).

Ora vediamo alcuni operatori e quali siano i *metodi magici* loro associati, per una lista completa si rimanda alla documentazione ufficiale[28]

| Operatore | Espressione | Metodo magico | Utilizzo |
|---|---|---|---|
| Addizione | o1 + o2 | __add__(self, other) | o1.__add__(o1) |
| Sottrazione | o1 - o2 | __sub__(self, other) | o1.__sub__(o1) |
| Moltiplicazione | o1 * o2 | __mul__(self, other) | o1.__mul__(o1) |
| Potenza | o1 ** o2 | __pow__(self, other) | o1.__pow__(o1) |
| Divisione | o1 / o2 | __truediv__(self, other) | o1.__truediv__(o1) |
| Divisione modulare | o1 % o2 | __mod__(self, other) | o1.__mod__(o2) |
| Minore | o1 < o2 | __lt__(self, other) | o1.__lt__(o1) |
| Minore o uguale | o1 <= o2 | __le__(self, other) | o1.__le__(o1) |
| Maggiore | o1 > o2 | __gt__(self, other) | o1.__gt__(o1) |
| Maggiore o uguale | o1 >= o2 | __ge__(self, other) | o1.__ge__(o1) |
| Uguale | o1 == o2 | __eq__(self, other) | o1.__eq__(o1) |
| Diverso | o1 != o2 | __ne__(self, other) | o1.__ne__(o1) |

[28] https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types

Attenzione: non è possibile modificare il numero di operandi in un'operatore.

#### 8.8.4.3 Implementazione delle quattro operazioni

A questo punto siamo in grado di implementare le quattro operazioni aritmetiche andando a effettuare l'overloading dei seguenti *metodi magici*:

- ➔ `__add__` (addizione)
- ➔ `__mul__` (prodotto)
- ➔ `__sub__` (sottrazione)
- ➔ `__truediv__` (divisione)

##### 8.8.4.3.1 Moltiplicazione

Partiamo dal codice della moltiplicazione. Andremo ad aggiungere il seguente metodo al codice della classe `Frazione`:

```
def __mul__(self, altraFrazione):
     num=self.getNumeratore() * altraFrazione.getNumeratore()
     den=self.getDenominatore()*altraFrazione.getDenominatore()
     return Frazione(num, den).__semplifica()__
```

Il codice di questo metodo è abbastanza semplice, si vanno a moltiplicare i rispettivi denominatori e numeratori dei due oggetti frazione impegnati nell'operazione. La frazione risultante viene semplificata prima di essere restituita.

##### 8.8.4.3.2 Divisione

La divisione è molto simile alla moltiplicazione in quanto dividere due frazioni R1 e R2 significa moltiplicare R1 per il reciproco di R2. Si poteva, quindi, optare per andare a richiamare il metodo di moltiplicazione andando a invertire la seconda frazione. Si è preferito implementare completamente il metodo nel seguente modo:

```
def __truediv__(self, altraFrazione):
     num=self.getNumeratore() * altraFrazione.getDenominatore()
     den=self.getDenominatore() * altraFrazione.getNumeratore()
     return Frazione(num, den).__semplifica()__
```

Come si può notare il codice è praticamente identico a quello della moltiplicazione, l'unica differenza consiste nel fatto che si è invertito esplicitamente la seconda frazione.

##### 8.8.4.3.3 Addizione

L'operazione di addizione è leggermente più complessa in quanto per sommare due frazioni è necessario calcolare il minimo comune multiplo tra i denominatori e poi andare a effettuare delle operazioni sui numeratori, cioè è necessario "rapportare" il numeratore al nuovo denominatore comune trovato .

Per il calcolo dell'm.c.m. si è fatto uso della specifica funzione esposta nel modulo `math`:

`lcm(*integers)`[29]. Si riporta di seguito il codice che implementa il metodo di addizione:

```
   def __add__(self, altraFrazione):

den=math.lcm(self.getDenominatore(),altraFrazione.getDenominatore(
))
num=self.getNumeratore()*int(den/self.getDenominatore())+ \
altraFrazione.getNumeratore()*int(den/altraFrazione.getDenominator
e())
    return Frazione(num, den).__semplifica()__
```

Naturalmente anche in questo caso la frazione risultante viene semplificata con il metodo `__semplifica()__`.

Può sembrare inutile effettuare il cast a intero nell'operazione di divisione che vede coinvolto il numeratore, questo in quanto il risultato di tale divisione sarà sicuramente un intero. Ma in realtà l'interprete restituisce un valore floating point! Ad esempio se il risultato della divisione è 4 il risultato sarà 4.0 e questo porta ad avere problemi con la funzione di calcolo dell'M.C.D. che accetta solo interi.

#### 8.8.4.3.4 Sottrazione

L'operazione di sottrazione è molto simile a quella di addizione, come si può dedurre analizzando il seguente codice:

```
   def __sub__(self, altraFrazione):
den=math.lcm(self.getDenominatore(),altraFrazione.getDenominatore(
))
num=self.getNumeratore()*int(den/self.getDenominatore())- \
altraFrazione.getNumeratore()*int(den/altraFrazione.getDenominator
e())
    return Frazione(num, den).__semplifica__()
```

### 8.8.5 Implementazione del confronto tra frazioni

Siamo arrivati al punto in cui la nostra classe è parecchio completa, siamo in grado di eseguire le quattro operazioni aritmetiche, possiamo semplificare e rappresentiamo i nostri oggetti secondo la rappresentazione frazionaria dei numeri razionali.
Manca però ancora un tassello molto importante, non abbiamo definito un operatore di confronto che ci permetta di definire se due oggetti della classe `Frazione` sono uguali o no. In realtà sull'insieme dei numeri razionali sussiste una relazione d'ordine e quindi manca anche la definizione di un tale operatore, p.e. maggiore o minore, ma tralasceremo l'implementazione di tali operatori.

Il *metodo magico* che vogliamo sovraccaricare è `__eq__` che è relazionato all'operatore `==`.

[29] https://docs.python.org/3/library/math.html#math.lcm

Potrebbe venirci in mente che questo operatore funzioni correttamente con glii oggetti della nostra classe. Per verificare questa ipotesi potremmo provare il seguente codice:

```
r1 = Frazione(4,5)
r2 = Frazione(4,5)
if(r1==r1):
    print("uguali")
else:
    print("diversi")

if(r1==r2):
    print("uguali")
else:
    print("diversi")
```

Se lo lanciamo otterremo in output:

```
uguali
diversi
```

Questo in quanto l'operatore == confronta la posizione di memoria che l'oggetto occupa, quindi nel primo caso si ha effettivamente una risposta positiva ma non perchè loggetto è uguale a se stesso, ma solo perchè l'oggetto r1 è nella stessa posizione di memoria di se stesso!
Infatti se verifichiamo l'uguaglianza tra la frazione r1 e la frazione r2, che sono identiche, la verifica fallisce perchè i due oggetti non occupano, ovviamente, le stesse locazioni di memoria.

Quindi da queste prove abbiamo compreso che risulta necessario effettuare un overloading di == sulla classe `Frazione`.

Due frazioni sono uguali (*equivalenti*) se riducendole ai minimi termini presentano stesso numeratore e stesso denominatore, questa definizione ci porta a implementare l'overriding del metodo `__eq__` nel seguente modo:

```
def __eq__(self,altraFrazione):
     f1=self.__semplifica__()
     f2=altraFrazione.__semplifica__()
     return f1.getNumeratore()==f2.getNumeratore() and \
            f1.getDenominatore() == f2.getDenominatore()
```

`f1` e `f2` sono le frazioni semplificate, la funzione ritorna `True` se numeratori e denominatori delle semplificate sono identici.

Per provare tale metodo possiamo usare un codice analogo a quello visto precedentemente per provare che l'operatore == non funzionava correttamente per le nostre frazioni:

```
r1 = Frazione(4,5)
r2 = Frazione(8,10)
if(r1==r1):
    print("uguali")
else:
    print("diversi")

if(r1!=r2):
    print("uguali")
else:
    print("diversi")
```

Se lo lanciamo otterremo in output:

```
uguali
diversi
```

Il primo test fornisce una risposta affermativa, infatti le due frazioni sono equivalenti. Per quanto riguarda il secondo test, quello con diverso, questi fallisce in quanto non è stato effettuato l'overloading del *metodo magico* `__ne__` , si lascia al lettore il compito di implementarlo come utile esercizio.

#### 8.8.5.1 Metodo alternativo

Possiamo riscrivere il metodo `__eq__` in maniera più ottimizzata evitando di effettuare la semplificazione delle due frazioni che risulta molto onerosa in termini computazionali.

Il metodo consiste nell'osservare che se due frazioni sono uguali allora è vera la seguente eguaglianza:

$$(4) \qquad \frac{N_1}{D_1} = \frac{N_2}{D_2}$$

$$(4') \qquad \text{dove } N_1, D_1, N_2, D_2 \in Z \text{ e } D_1 \neq 0, D_2 \neq 0$$

possiamo scrivere la (4) nel seguente modo:

$$(5) \qquad N_1 D_2 = N_2 D_1$$

Quindi possiamo affermare che se la (5) è vera allora le due frazioni sono uguali.
Da queste considerazioni possiamo riscrivere l'override al medoto `__eq__` nel seguente modo:

```
    def __eq__(self,altraFrazione):
    return self.getNumeratore()*altraFrazione.getDenominatore() \
== self.getDenominatore()*altraFrazione.getNumeratore()
```

Al posto di semplificare le due frazioni e confrontare rispettivamente i numeratori e i denominatori, si va a effettuare il confronto tra la moltiplicazione di due numeri interi.

### 8.8.6 Il metodo `getDecimalValue()`

Un'implementazione alternativa del metodo `__eq__` è quello di calcolare il valore decimale della frazione e successivamente confrontare i valori in floating point che derivano dalla conversione. Per fare questo possiamo implementare un nuovo metodo `getDecimalValue()` che restituisce banalmente il calcolo della divisione del numeratore con il denominatore dell'oggetto `Frazione` considerato.

Una possibile implementazione potrebbe essere la seguente:

```
    def getDecimalValue(self):
         return self.__numeratore / self.__denominatore
```

Un tale metodo potrebbe essere utile anche per conoscere effettivamente il valore decimale della frazione e quindi è utile renderlo pubblico.

### 8.8.7 Il codice completo della classe `Frazione`

Riportiamo ora il codice della classe Frazione completo di tutte le modifiche sinora fatte.

```
import math

class Frazione(object):
    """    Classe Frazione
- Appunti 'Python Primi Passi' - """
    def __init__(self,numeratore = 0,denominatore = 1):
        # Eliminazione incongruenza divisore 0
        if(denominatore == 0):
            denominatore=1
        self.__numeratore = numeratore
        self.__denominatore = denominatore
    def getNumeratore(self):
        return self.__numeratore
    def getDenominatore(self):
        return self.__denominatore
    def setNumeratore(self,val):
        self.__numeratore = val
    def setDenominatore(self,val):
        self.__denominatore = val
    def getDecimalValue(self):
        return self.__numeratore / self.__denominatore
    def __semplifica__(self):
```

```
        mcd=math.gcd(self.getNumeratore(),self.getDenominatore())
        return Frazione(int(self.getNumeratore()/mcd),
int(self.getDenominatore()/mcd))
    def __str__(self):
        if self.__denominatore == 1:
            ratio =  str(self.__numeratore)
        else:
            ratio=
str(self.__numeratore)+"/"+str(self.__denominatore)
        return ratio
    def __mul__(self, altraFrazione):
        num=self.getNumeratore() * altraFrazione.getNumeratore()
        den=self.getDenominatore() *
altraFrazione.getDenominatore()
        return Frazione(num, den).__semplifica__()
    def __truediv__(self, altraFrazione):
        num=self.getNumeratore() * altraFrazione.getDenominatore()
        den=self.getDenominatore() * altraFrazione.getNumeratore()
        return Frazione(num, den).__semplifica__()
    def __add__(self, altraFrazione):

den=math.lcm(self.getDenominatore(),altraFrazione.getDenominatore(
))

num=self.getNumeratore()*int(den/self.getDenominatore())+ \

altraFrazione.getNumeratore()*int(den/altraFrazione.getDenominator
e())
            return Frazione(num, den).__semplifica__()
    def __sub__(self, altraFrazione):

den=math.lcm(self.getDenominatore(),altraFrazione.getDenominatore(
))

num=self.getNumeratore()*int(den/self.getDenominatore())- \

altraFrazione.getNumeratore()*int(den/altraFrazione.getDenominator
e())
            return Frazione(num, den).__semplifica__()
    def __eq__(self,altraFrazione):
        f1=self.__semplifica__()
        f2=altraFrazione.__semplifica__()
        return f1.getNumeratore()==f2.getNumeratore() and \
               f1.getDenominatore() == f2.getDenominatore()
```

# 8.9 Classi astratte

## 8.9.1 Introduzione alle classi astratte

Nel panorama della programmazione orientata agli oggetti, le classi astratte rappresentano un concetto fondamentale, in quanto permettono di definire strutture di dati e comportamenti comuni per una famiglia di classi correlate. Se le classi sono come le mappe che guidano la creazione di oggetti, le classi astratte sono come i modelli di quelle mappe, stabilendo regole e interfacce che altre mappe (classi) devono seguire.

In Python, utilizziamo il modulo '`abc`' (*Abstract Base Classes*) per definire queste entità astratte.

## 8.9.2 Definizione di una classe astratta

**Classe astratta:** E' una classe che non è progettata per essere istanziata direttamente, ma solo per essere usata come classe base da una o più sottoclassi.[30]

Quindi una classe astratta non può essere istanziata direttamente. Serve da modello per altre classi che devono implementare i metodi definiti nella classe astratta. Queste classi figlie devono fornire le implementazioni concrete dei *metodi* dichiarati *astratti*.

**Metodo astratto:** E' un metodo della classe astratta che deve essere sovrascritto in ciascuna sottoclasse.[31]

In alcuni linguaggi di programmazione le classi astratte vengono nominate come *classi virtuali*.

Vediamo come funziona in pratica.

Immaginiamo di voler creare un sistema per diversi tipi di animali. Tutti gli animali hanno alcune caratteristiche comuni, come il fatto di poter emettere un verso. Tuttavia, il modo in cui emettono questo verso varia. Vediamo come una classe astratta può aiutarci a definire questa struttura comune.

```
from abc import ABC, abstractmethod

class Animale(ABC):
    """
    Classe astratta che rappresenta un animale.
    """

    @abstractmethod
    def emetti_verso(self):
```

[30] Object Oriented Python - Irv Kalb - No Starch press - 2022
[31] Object Oriented Python - Irv Kalb - No Starch press - 2022

```
        """
        Metodo astratto che deve essere implementato da ogni
classe derivata.
        """
        pass
```

In questo esempio, `Animale` è una classe astratta. Il metodo `emetti_verso()` è definito come astratto utilizzando il decoratore `@abstractmethod`. Questo implica che ogni sottoclasse di `Animale` deve implementare questo metodo.

### 8.9.3 Precisazioni sulla definizione

Una classe in Python è astratta quando:

- ➔ Eredita da ABC (`Abstract Base Class`) senza definire metodi astratti
- ➔ Usa il decoratore `@abstractmethod` su almeno un metodo

La differenza principale sta nel comportamento:

- ➔ Una classe che eredita solo da ABC può essere istanziata direttamente
- ➔ Una classe con almeno un metodo *astratto* non può essere istanziata finché tutti i suoi metodi *astratti* non vengono implementati nelle sottoclassi (vedi par. 8.8.4.1)

Analizziamo il seguente codice:

```
from abc import ABC

# Creiamo una classe astratta senza metodi astratti
class Esempio(ABC):
    def metodo(self):
        print("ciao")

# Istanziamo la classe Esempio
obj = Esempio()
obj.metodo()
```

Se lanciamo il programma otteniamo in output:

```
Ciao
```

Analizzando il codice e l'output possiamo notare:

- ➔ Possiamo creare una classe che eredita da ABC senza alcun metodo astratto
- ➔ Possiamo istanziare la classe "teoricamente" astratta senza ricevere alcun errore
- ➔ Possiamo utilizzare tutti i metodi concreti definiti nella classe

Quindi, mentre ereditare da ABC rende una classe tecnicamente "astratta", per ottenere il vero comportamento di una classe astratta (che non può essere istanziata) è necessario includere almeno un metodo astratto.

L'uso di ABC senza metodi astratti è più comune quando si vuole definire un'interfaccia comune o un comportamento base, ma non si vuole forzare l'implementazione di metodi specifici nelle sottoclassi.

### 8.9.4 Implementazione delle classi concrete

Ora, possiamo creare delle classi concrete che ereditano da `Animale` e forniscono un'implementazione specifica del metodo `emetti_verso`.

```
class Cane(Animale):
    """
    Classe concreta che rappresenta un cane.
    """

    def emetti_verso(self):
        return "Bau!"

class Gatto(Animale):
    """
    Classe concreta che rappresenta un gatto.
    """

    def emetti_verso(self):
        return "Miao!"
```

Le classi `Cane` e `Gatto` estendono `Animale` e implementano il metodo `emetti_verso()` con i rispettivi versi. Queste classi possono ora essere istanziate e utilizzate per creare oggetti specifici.

#### 8.9.4.1 Tentativo di istanziare una classe astratta

Se provassimo a creare un'istanza della classe `Animale`, l'interprete Python ci impedirà di farlo, perché una classe astratta non può essere istanziata.

```
try:
    animale = Animale()
except TypeError as e:
    print(e)
```

Lanciando questo codice otteniamo il seguente output:

```
Can't instantiate abstract class Animale with abstract
method emetti_verso
```

Questo codice genererà un errore, perché la classe `Animale` contiene un metodo *astratto* che non è stato implementato.

### 8.9.5 A cosa servono le classi astratte?

Le classi astratte sono utili per definire un'interfaccia comune che deve essere seguita da tutte le classi derivate. Questo approccio è particolarmente utile in progetti di grandi dimensioni, dove si vuole garantire che tutte le sottoclassi implementino determinati metodi.
Ad esempio, in un'applicazione grafica, si potrebbe avere una classe astratta `FiguraGeometrica` con un metodo `calcola_area()`. Tutte le figure geometriche (cerchio, quadrato, triangolo) dovrebbero poi implementare questo metodo.

```
from abc import ABC, abstractmethod

class FiguraGeometrica(ABC):
    """
    Classe astratta che rappresenta una figura geometrica.
    """

    @abstractmethod
    def calcola_area(self):
        """
        Metodo astratto per calcolare l'area della figura.
        """
        pass

class Cerchio(FiguraGeometrica):
    """
    Classe concreta che rappresenta un cerchio.
    """

    def __init__(self, raggio):
        self.raggio = raggio

    def calcola_area(self):
        import math
        return math.pi * self.raggio ** 2

class Rettangolo(FiguraGeometrica):
```

```
    """
    Classe concreta che rappresenta un rettangolo.
    """

    def __init__(self, larghezza, altezza):
        self.larghezza = larghezza
        self.altezza = altezza

    def calcola_area(self):
        return self.larghezza * self.altezza
```

In questo codice, `FiguraGeometrica` definisce l'interfaccia per calcolare l'area, mentre `Cerchio` e `Rettangolo` forniscono le implementazioni specifiche per tale calcolo.

### 8.9.6 Verifica dell'implementazione dei metodi

Python ci permette anche di verificare se una classe ha implementato tutti i metodi astratti della classe base astratta, utilizzando il decoratore `@abstractmethod` per prevenire la creazione di classi incomplete.

```
class Triangolo(Figura Geometrica):
    pass

try:
    triangolo = Triangolo()
except TypeError as e:
    print(e)    #Output: Can't instantiate abstract class Triangolo
                #with abstract methods calcola_area
```

Lanciando questo codice otteniamo il seguente output:

```
Can't instantiate abstract class Triangolo with abstract
method calcola_area
```

In questo caso, l'interprete Python solleva un errore perché nella classe `Triangolo` il metodo `calcola_area()` non è stato implementato.

Attenzione al fatto che questo esempio è differente da quello riportato nel par. 8.8.4.1 in quanto in questo caso abbiamo una classe che estende una classe astratta e non è essa stessa una classe astratta come nell'esempio del paragrafo citato.

### 8.9.7 Metodi concreti in classi base astratte

Le classi *concrete* contengono solo metodi *concreti* (normali) mentre le classi *astratte* possono contenere sia metodi *concreti* che metodi *astratti*.

La classe concreta fornisce un'implementazione di metodi astratti, la classe base astratta può anche fornire un'implementazione invocando i metodi tramite `super()`.

Esaminiamo il seguente codice di esempio in cui si invoca un metodo utilizzando `super()`:

```
from abc import ABC, abstractmethod

class A(ABC):
    def metodo1(self):
        print("Abstract Base Class")

    @abstractmethod
    def metodo2(self):
        print("Abstract method in Abstract Base Class")

class C(A):
    def metodo1(self):
        super().metodo1()
        print("subclass ")

    def metodo2(self):
        super().metodo2()
        print("subclass ")

# Istanziamo la classe concreta C
r = C()
r.metodo1()
r.metodo2()
```

Lanciando questo codice otteniamo il seguente output:

```
Abstract Base Class
subclass
Abstract method in Abstract Base Class
subclass
```

Dall'analisi del codice e dell'output notiamo che:

- La classe astratta contiene un metodo *concreto* `metodo1()` che viene sovrascritto dalla classe concreta che estende la classe *astratta*. Il metodo utilizza `super()` per richiamare il metodo `metodo1()` originale.
- Attenzione: la classe concreta C potrebbe, anche, non sovrascrivere il metodo concreto `metodo1()` della classe astratta.
- Nella classe astratta è definito un metodo *astratto* che viene implementato nella classe *concreta*.
- Il metodo astratto viene ridefinito utilizzando, attraverso `super()`,

l'implementazione originale presente nella classe *astratta*.

➔ Deduciamo che possiamo invocare i metodi definiti nelle classi astratte utilizzando `super()`. I metodi possono essere *concreti* (`metodo1()`) ma anche *astratti* (`metodo2()`)

### 8.9.8 Un esempio avanzato: *gestione dei pagamenti*

Vediamo un esempio più avanzato che mostra come utilizzare le classi astratte per gestire diversi tipi di pagamenti in un sistema di e-commerce:

```
from abc import ABC, abstractmethod

class MetodoPagamento(ABC):
    """
    Classe astratta che rappresenta un metodo di pagamento.
    """

    @abstractmethod
    def paga(self, importo):
        """
        Metodo astratto per effettuare un pagamento.

        importo: Importo da pagare
        """
        pass

class CartaDiCredito(MetodoPagamento):
    """
    Classe che rappresenta un pagamento con carta di credito.
    """

    def paga(self, importo):
        """
        Effettua un pagamento con carta di credito.

        importo: Importo da pagare
        """
        print(f"Pagamento di {importo}€ effettuato con carta di
credito.")

class PayPal(MetodoPagamento):
    """
    Classe che rappresenta un pagamento con PayPal.
    """

    def paga(self, importo):
        """
        Effettua un pagamento con PayPal.
```

```
        importo: Importo da pagare
        """
        print(f"Pagamento di {importo}€ effettuato con PayPal.")

# Esempio di utilizzo
def processa_pagamento(metodo, importo):
    """
    Processa un pagamento utilizzando il metodo di pagamento
specificato.

    metodo: Istanza di una classe derivata da MetodoPagamento
    importo: Importo da pagare
    """
    metodo.paga(importo)

# Esempio di utilizzo: creo funzione di pagamento
carta = CartaDiCredito()
paypal = PayPal()

processa_pagamento(carta, 100)
processa_pagamento(paypal, 200)
```

Lanciando questo codice otteniamo il seguente output:

```
Pagamento di 100€ effettuato con carta di credito.
Pagamento di 200€ effettuato con PayPal.
```

Dall'analisi del codice e dell'output possiamo notare che:

- La classe astratta `MetodoPagamento` definisce un metodo astratto `paga()`.
- Le classi `CartaDiCredito` e `PayPal` implementano questo metodo.
- La funzione `processa_pagamento()` accetta un'istanza di `MetodoPagamento` e un `importo`, e chiama il metodo `paga()` sull'istanza fornita.
- Questo approccio ci permette di aggiungere facilmente nuovi metodi di pagamento senza modificare il codice esistente.

### 8.9.9. Vantaggi delle classi astratte

Le classi astratte offrono diversi vantaggi:

- **Strutturazione del codice**: Le classi astratte ci aiutano a definire una struttura chiara e coerente per il nostro codice, stabilendo un contratto che le sottoclassi devono seguire.

- **Riusabilità del codice**: Promuovono la riusabilità del codice, permettendo di definire un comportamento comune in un'unica classe base astratta.

- ➔ **Manutenzione facilitata**: Riducono la duplicazione del codice e rendono più facile la manutenzione e l'espansione del progetto.

- ➔ **Polimorfismo**: Facilitano il polimorfismo, permettendo di trattare oggetti di diverse classi derivate in modo uniforme tramite la loro classe base comune.

## 8.10 * Ereditarietà multipla

Nella programmazione orientata agli oggetti abbiamo già visto che una classe può essere creata a partire da un'altra classe. Quando una classe nuova deriva da una classe già esistente, essa può riutilizzare attributi e metodi della classe originaria, senza doverli riscrivere da zero.

Questo meccanismo prende il nome di ereditarietà.

In Python possiamo avere:

- ➔ ereditarietà singola, quando una classe figlia deriva da una sola classe padre;
- ➔ ereditarietà multipla, quando una classe figlia deriva da due o più classi padre.

L'ereditarietà è molto utile perché permette di organizzare il codice in modo più ordinato, riusabile e vicino al modo in cui descriviamo gli oggetti del mondo reale.

### 8.10.1 Ereditarietà singola: un breve ripasso

Prima di parlare di ereditarietà multipla, è utile richiamare il funzionamento dell'ereditarietà singola.

Pensiamo a una classe Animale che descrive comportamenti comuni a molti animali, come mangiare e dormire. Possiamo poi creare una classe Cane che eredita questi comportamenti e aggiunge un comportamento proprio, ad esempio abbaiare.

Consideriamo il seguente codice:

```
class Animale(object):
    """Classe Animale di esempio"""

    def mangia(self):
        print("Sto mangiando.")

    def dorme(self):
        print("Sto dormendo.")


class Cane(Animale):
    """Classe Cane di esempio"""

    def abbaia(self):
```

```
        print("Bau bau!")


rex = Cane()
rex.mangia()
rex.dorme()
rex.abbaia()
```

Lanciamo il programma, in uscita otteniamo:

```
Sto mangiando.
Sto dormendo.
Bau bau!
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `Cane` deriva dalla classe `Animale`;
- l'oggetto `rex`, creato come istanza di `Cane`, può usare i metodi `mangia`() e `dorme`() ereditati da `Animale`;
- la classe `Cane` può anche definire nuovi metodi, come `abbaia`();
- la relazione tra `Cane` e `Animale` può essere letta come: "*un cane è un animale*".

Ricordiamo che in Python, per indicare che una classe eredita da un'altra, si scrive il nome della classe padre tra parentesi tonde dopo il nome della classe figlia:

```
class Cane(Animale):
    pass
```

### 8.10.2 Dall'ereditarietà singola all'ereditarietà multipla

L'ereditarietà singola è sufficiente in molti casi, ma non sempre un oggetto può essere descritto derivandolo da una sola categoria (classe).

Pensiamo, per esempio, a un'anatra. Un'anatra può essere descritta come un animale che:

- sa volare;
- sa nuotare;
- sa starnazzare.

In Python potremmo rappresentare queste capacità con classi separate:

- una classe `Volante`, che fornisce il metodo `vola()`;
- una classe `Nuotatore`, che fornisce il metodo `nuota()`;

- una classe `Anatra`, che eredita entrambe le capacità.

In questo caso la classe `Anatra` non eredita da una sola classe, ma da due classi contemporaneamente.

```
class Volante(object):
    """Classe che fornisce la capacità di volare"""

    def vola(self):
        print("Sto volando in alto nel cielo!")

class Nuotatore(object):
    """Classe che fornisce la capacità di nuotare"""

    def nuota(self):
        print("Sto nuotando nell'acqua!")

class Anatra(Volante, Nuotatore):
    """Classe Anatra: può volare, nuotare e starnazzare"""

    def starnazza(self):
        print("Qua qua!")

donald = Anatra()
donald.vola()
donald.nuota()
donald.starnazza()
```

Lanciamo il programma, in uscita otteniamo:

```
Sto volando in alto nel cielo!
Sto nuotando nell'acqua!
Qua qua!
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe Anatra eredita sia dalla classe Volante sia dalla classe Nuotatore;
- per indicare più classi padre, esse vengono scritte tra parentesi tonde e separate da virgole;
- l'oggetto donald, istanza di Anatra, può utilizzare:
  - il metodo `vola()`, ereditato da `Volante`;
  - il metodo `nuota()`, ereditato da `Nuotatore`;
  - il metodo `starnazza()`, definito direttamente nella classe `Anatra`.

La sintassi generale dell'ereditarietà multipla è quindi la seguente:

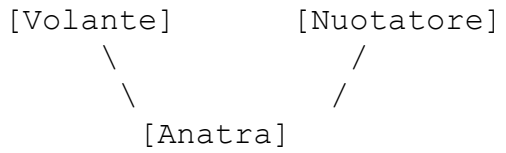
```
class ClasseFiglia(ClassePadre1, ClassePadre2, ... ,ClassePadreN):
    pass
```

In questo modo la classe figlia ottiene attributi e metodi da tutte le classi padre indicate.

### 8.10.3 A cosa serve l'ereditarietà multipla

L'ereditarietà multipla permette a una classe di combinare comportamenti provenienti da fonti diverse.

Riprendendo l'esempio precedente, possiamo immaginare la classe `Anatra` come punto di incontro tra due capacità:




La classe `Anatra` possiede quindi:

- ➔ il comportamento di chi può volare;
- ➔ il comportamento di chi può nuotare;
- ➔ eventuali comportamenti propri, come starnazzare.

Questo meccanismo evita di dover copiare più volte lo stesso codice. Se il metodo `vola()` viene definito nella classe `Volante`, tutte le classi che ereditano da `Volante` potranno usarlo senza riscriverlo.

#### 8.10.4 Le problematiche dell'ereditarietà multipla

L'ereditarietà multipla è uno strumento potente, ma deve essere usato con attenzione. Se le classi diventano troppe o troppo collegate tra loro, il codice può diventare difficile da capire e da mantenere.

I due problemi principali sono:

1. conflitto tra metodi con lo stesso nome;
2. problema del diamante.

##### 8.10.4.1 Problema del conflitto tra metodi con lo stesso nome

Supponiamo che due classi padre abbiano un metodo con lo stesso nome. Se una classe figlia eredita da entrambe, quale metodo verrà eseguito quando chiamiamo quel metodo?

Consideriamo il seguente codice:

```
class Telefono(object):
    """Classe Telefono di esempio"""

    def accendi(self):
        print("Schermo del telefono acceso.")


class Fotocamera(object):
    """Classe Fotocamera di esempio"""

    def accendi(self):
        print("Obiettivo fotocamera aperto.")


class Smartphone(Telefono, Fotocamera):
    pass


telefono_nuovo = Smartphone()
telefono_nuovo.accendi()
```

Lanciamo il programma, in uscita otteniamo:

```
Schermo del telefono acceso.
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ sia la classe `Telefono` sia la classe `Fotocamera` definiscono un metodo chiamato `accendi()`;
- ➔ la classe `Smartphone` eredita da entrambe;
- ➔ Python non produce un errore;
- ➔ viene eseguito il metodo `accendi()` della classe `Telefono`.

Perché succede questo?

Il linguaggio Python segue un ordine preciso per cercare i metodi nelle classi padre. In questo caso, nella definizione:

```
class Smartphone(Telefono, Fotocamera):
     pass
```

la classe `Telefono` è stata scritta per prima. Python quindi cerca il metodo prima in `Telefono` e, poiché lo trova, esegue quello.

Se cambiassimo l'ordine delle classi padre:

```
class Smartphone(Fotocamera, Telefono):
    pass
```

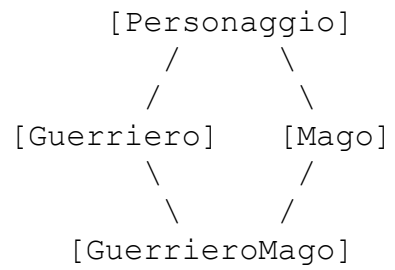
verrebbe eseguito il metodo `accendi()` della classe `Fotocamera`.

Questo comportamento può essere utile, ma può anche introdurre errori difficili da individuare se non si conosce l'ordine con cui Python risolve i metodi.

### 8.10.4.2 Il problema del diamante

Il cosiddetto *problema del diamante* si verifica quando due classi ereditano dalla stessa classe base, e una quarta classe eredita da entrambe.

La struttura grafica ricorda la forma di un diamante:




In questo caso:

- `Guerriero` eredita da `Personaggio`;
- `Mago` eredita da `Personaggio`;
- `GuerrieroMago` eredita sia da `Guerriero` sia da `Mago`.

Consideriamo il seguente codice:

```
class Personaggio(object):
    """Classe Personaggio di esempio"""

    def saluta(self):
        print("Sono un personaggio generico.")


class Guerriero(Personaggio):
    """Classe Guerriero di esempio"""

    def saluta(self):
        print("Sono un forte Guerriero!")
```

```
class Mago(Personaggio):
    """Classe Mago di esempio"""

    def saluta(self):
        print("Sono un saggio Mago!")


class GuerrieroMago(Guerriero, Mago):
    pass


eroe = GuerrieroMago()
eroe.saluta()
```

Lanciamo il programma, in uscita otteniamo:

```
Sono un forte Guerriero!
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `Personaggio` definisce il metodo `saluta()`;
- la classe `Guerriero` ridefinisce il metodo `saluta()`;
- anche la classe `Mago` ridefinisce il metodo `saluta()`;
- la classe `GuerrieroMago` eredita da entrambe;
- chiamando `eroe.saluta()`, Python deve scegliere quale versione del metodo eseguire. In questo caso viene eseguito il metodo definito in `Guerriero`, perché `Guerriero` compare per primo nella lista delle classi padre.

Il problema del diamante non è necessariamente un errore, ma mostra come l'ereditarietà multipla possa creare ambiguità quando più classi della gerarchia definiscono metodi con lo stesso nome.

### 8.10.4.3 L'MRO: Method Resolution Order

Per decidere quale metodo eseguire in caso di ambiguità, Python utilizza una lista ordinata chiamata `MRO`, cioè *Method Resolution Order*,cioè l'ordine di risoluzione dei metodi.

L'`MRO` indica l'ordine con cui Python cerca un metodo nelle classi della gerarchia.

Possiamo vedere l'`MRO` di una classe usando l'attributo speciale `__mro__` oppure il metodo `mro()`.

Consideriamo il seguente codice:

```
print(GuerrieroMago.__mro__)
```

Lanciando il programma, otteniamo un output simile al seguente:

```
(<class '__main__.GuerrieroMago'>, <class
'__main__.Guerriero'>, <class '__main__.Mago'>, <class
'__main__.Personaggio'>, <class 'object'>)
```

Dall'analisi dell'output osserviamo che:

- Python cerca il metodo prima nella classe `GuerrieroMago`;
- poi nella classe `Guerriero`;
- poi nella classe `Mago`;
- poi nella classe `Personaggio`;
- infine nella classe `object`, che è la classe base di tutte le classi Python.

Nel caso del metodo `saluta()`, Python trova subito il metodo in `Guerriero`, quindi esegue quello.

Possiamo controllare l'`MRO` anche per la classe `Smartphone` vista in precedenza:

```
print(Smartphone.__mro__)
```

Output:

```
(<class '__main__.Smartphone'>, <class
'__main__.Telefono'>, <class '__main__.Fotocamera'>, <class
'object'>)
```

Questo spiega perché, chiamando:

```
telefono_nuovo.accendi()
```

venga eseguito il metodo `accendi()` di `Telefono` e non quello di `Fotocamera`.

### 8.10.5 L'uso di `super()` per far cooperare le classi

La funzione integrata `super()` permette a una classe di richiamare il metodo della classe successiva secondo l'ordine definito dall'`MRO`.

L'uso di `super()` è particolarmente utile nell'*ereditarietà multipla* perché consente di far collaborare più classi evitando duplicazioni.

Consideriamo il seguente codice:

```
class Componente(object):
    """Classe Componente di esempio"""

    def avvia(self):
        print("Inizializzazione base...")


class Schermo(Componente):
    """Classe Schermo di esempio"""

    def avvia(self):
        super().avvia()
        print("Schermo acceso.")


class Tastiera(Componente):
    """Classe Tastiera di esempio"""

    def avvia(self):
        super().avvia()
        print("Tastiera collegata.")


class Computer(Schermo, Tastiera):
    """Classe Computer di esempio"""

    def avvia(self):
        super().avvia()
        print("Computer pronto all'uso!")

pc = Computer()
pc.avvia()
```

Lanciamo il programma, in uscita otteniamo:

```
Inizializzazione base...
Tastiera collegata.
Schermo acceso.
Computer pronto all'uso!
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `Computer` eredita da `Schermo` e `Tastiera`;
- tutte e tre le classi `Schermo`, `Tastiera` e `Computer` ridefiniscono il metodo `avvia()`;

- ➔ ogni metodo chiama `super().avvia()` prima di stampare il proprio messaggio;
- ➔ Python segue l'`MRO` e chiama ogni classe una sola volta;
- ➔ l'output mostra che la parte "base" viene eseguita prima, poi vengono eseguiti i messaggi delle classi intermedie e infine quello della classe `Computer`.

L'uso di `super()` rende il codice più ordinato e aiuta a evitare chiamate multiple o dimenticate nella gerarchia.

### 8.10.6 I *Mixin*: una buona pratica di design

Per usare l'ereditarietà multipla in modo più sicuro, in Python si utilizza spesso il pattern dei *Mixin*.

Nei linguaggi di programmazione orientati agli oggetti , un mixin (o mix-in ) è una classe che contiene metodi utilizzabili da altre classi senza che queste debbano essere la classe padre di tali altre classi. I mixin sono talvolta descritti come "inclusi" piuttosto che "ereditati".[32]

In sintesi un *Mixin* è una classe che:

1. non rappresenta un oggetto reale a sé stante;
2. di solito non viene istanziata direttamente;
3. fornisce una funzionalità specifica e ben delimitata;
4. non ha uno stato interno complesso.

In pratica, un *Mixin* serve ad "aggiungere" una capacità a una classe senza creare una gerarchia troppo complicata.

Per esempio, possiamo creare un *Mixin* che aggiunge a una classe la capacità di serializzare i propri dati in formato JSON.

A tale scopo consideriamo il seguente esempio:

```
import json

class JSONSerializerMixin(object):
    """Mixin che aggiunge la serializzazione JSON"""

    def serializza_in_json(self):
        return json.dumps(self.__dict__)

class Utente(object):
    """Classe Utente di esempio"""

    def __init__(self, nome, email):
```

[32] https://en.wikipedia.org/wiki/Mixin

```
        self.nome = nome
        self.email = email

class UtenteEsportabile(Utente, JSONSerializerMixin):
    """Utente che può essere esportato in JSON"""
    pass

u = UtenteEsportabile("Mario", "mario@example.com")
print(u.serializza_in_json())
```

Lanciamo il programma, in uscita otteniamo:

```
{"nome": "Mario", "email": "mario@example.com"}
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `JSONSerializerMixin` fornisce un solo metodo: `serializza_in_json()`;
- il metodo utilizza `self.__dict__`, cioè il dizionario degli attributi dell'oggetto, già visto negli esempi sulla classe `Punto`;
- la classe `Utente` rappresenta un oggetto reale, con `nome` ed `email`;
- la classe `UtenteEsportabile` unisce la classe `Utente` con il *Mixin*;
- l'oggetto `u` possiede sia gli attributi di `Utente` sia il metodo di serializzazione fornito dal *Mixin*.

Questo approccio è molto usato perché permette di aggiungere funzionalità senza creare gerarchie profonde o confuse.

#### 8.10.6.1 Un altro esempio di *Mixin* senza uso di librerie esterne

Nel precedente esempio abbiamo usato un *Mixin* per serializzare un oggetto in formato `JSON`. Ora vediamo un esempio ancora più semplice, senza importare alcuna libreria.

Vogliamo aggiungere a classi diverse un metodo capace di descrivere automaticamente lo stato di un oggetto, cioè i suoi attributi. Invece di riscrivere questo metodo in ogni classe, possiamo creare un *Mixin*.

Consideriamo il seguente codice:

```
class DescrizioneMixin(object):
    """Mixin che aggiunge una descrizione automatica
dell'oggetto"""

    def descrizione(self):
        parti = []
        for chiave, valore in self.__dict__.items():
```

```
            parti.append(chiave + ": " + str(valore))
        return "; ".join(parti)

class Persona(object):
    """Classe Persona di esempio"""

    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

class PersonaDescrivibile(Persona, DescrizioneMixin):
    """Persona con descrizione automatica"""
    pass

class Automobile(object):
    """Classe Automobile di esempio"""

    def __init__(self, modello, colore):
        self.modello = modello
        self.colore = colore

class AutomobileDescrivibile(Automobile, DescrizioneMixin):
    """Automobile con descrizione automatica"""
    pass


p = PersonaDescrivibile("Anna", 23)
a = AutomobileDescrivibile("Cinquecento", "rossa")

print(p.descrizione())
print(a.descrizione())
```

Lanciamo il programma, in uscita otteniamo:

```
nome: Anna; eta: 23
modello: Cinquecento; colore: rossa
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `DescrizioneMixin` non rappresenta un oggetto reale;
- la classe `DescrizioneMixin` non viene istanziata direttamente;
- il *Mixin* fornisce una sola funzionalità aggiuntiva: il metodo `.descrizione()`;
- il metodo `.descrizione()` utilizza l'attributo speciale `self.__dict__`, già visto quando abbiamo analizzato gli attributi della classe Punto;
- il *Mixin* può essere aggiunto a classi diverse, come `Persona` e `Automobile`;
- non viene usata nessuna libreria esterna, il codice utilizza solo meccanismi base di Python.

Questo esempio mostra bene l'idea dei *Mixin*: una classe leggera che aggiunge una piccola funzionalità riutilizzabile, senza creare gerarchie complesse.

Possiamo pensare al *Mixin* come a una "etichetta" che possiamo applicare a una classe esistente. Nel nostro caso, l'etichetta è: `DescrizioneMixin` e permette a qualsiasi classe di ottenere il metodo: `.descrizione()` senza doverlo ridefinire ogni volta.

### 8.10.7 Quando usare l'ereditarietà multipla

L'*ereditarietà multipla* può essere molto utile, ma non deve essere usata sempre. Prima di usarla, è opportuno chiedersi se la relazione tra le classi sia davvero una relazione di tipo "*è un*".

Per esempio:

```
class Anatra(Volante, Nuotatore):
    pass
```

può avere senso perché un'anatra:

- ➔ è un oggetto volante, nel senso che può volare;
- ➔ è un oggetto nuotatore, nel senso che può nuotare.

In altri casi, invece, l'*ereditarietà multipla* può rendere il codice più difficile da leggere.

### 8.10.8 Ereditarietà e composizione

Nella programmazione orientata agli oggetti esistono due relazioni importanti:

- ➔ relazione **IS-A**, cioè "*è un*";
- ➔ relazione **HAS-A**, cioè "*ha un*".

L'*ereditarietà* è adatta soprattutto alla relazione **IS-A** (come visto nel precedente paragrafo)

Per esempio:

```
Un cane è un animale.
Un'anatra è un animale volante e nuotatore.
```

La *composizione*, invece, è più adatta alla relazione **HAS-A**.

Per esempio:

```
Un'automobile ha un motore.
```

```
Un carrello ha un calcolatore di prezzi.
```

La composizione si può realizzare inserendo un oggetto dentro un altro come attributo.

Consideriamo il seguente esempio:

```
class Motore(object):
    """Classe Motore di esempio"""

    def avvia(self):
        print("Il motore si avvia.")

class Automobile(object):
    """Classe Automobile di esempio"""

    def __init__(self):
        self.motore = Motore()

    def accendi(self):
        self.motore.avvia()
        print("L'automobile è pronta.")

auto = Automobile()
auto.accendi()
```

Lanciamo il programma, in uscita otteniamo:

```
Il motore si avvia.
L'automobile è pronta.
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `Automobile` non eredita da `Motore`;
- la classe `Automobile` possiede un attributo `motore`;
- l'oggetto `Automobile` usa l'oggetto `Motore` al suo interno.

Questo approccio è spesso più semplice e più chiaro rispetto all'uso dell'*ereditarietà multipla*.

### 8.10.9 Regole d'oro per l'ereditarietà multipla

Per usare bene l'*ereditarietà multipla*, è utile seguire alcune regole pratiche:

1. Usare l'ereditarietà multipla principalmente per i *Mixin*
2. Evitare gerarchie troppo profonde o intrecciate
3. Controllare l'`MRO` quando si usano più classi padre
4. Usare `super()` quando serve collaborazione tra classi
5. Preferire la composizione quando la relazione non è "*è un*"

### 1. Usare l'ereditarietà multipla principalmente per i Mixin

Aggiungere classi "leggere", che forniscono uno o pochi metodi di utilità, è generalmente il modo più sicuro di sfruttare l'*ereditarietà multipla*.

Esempio:

```
class ClassePrincipale(object):
    pass

class FunzioneAggiuntivaMixin(object):
    pass

class ClasseCompleta(ClassePrincipale, FunzioneAggiuntivaMixin):
    pass
```

### 2. Evitare gerarchie troppo profonde o intrecciate

Se una classe eredita da molte classi, e queste classi ereditano a loro volta da altre classi, il codice può diventare difficile da leggere, da capire e da correggere.

Una gerarchia semplice è quasi sempre preferibile a una gerarchia complessa.

### 3. Controllare l'`MRO` quando si usano più classi padre

In caso di metodi con lo stesso nome, è importante sapere in quale ordine l'interprete Python cerca i metodi.

Possiamo farlo con:

```
print(NomeClasse.__mro__)
```

oppure:

```
print(NomeClasse.mro())
```

### 4. Usare `super()` quando serve collaborazione tra classi

`super()` aiuta a chiamare i metodi delle classi padre seguendo l'`MRO` e può evitare duplicazioni o dimenticanze.

### 5. Preferire la composizione quando la relazione non è "*è un*"

Se la relazione tra due classi è "*ha un*", spesso è meglio usare la *composizione*.

Esempio:

```
Un carrello ha un calcolatore di prezzi.
```

In questo caso conviene scrivere qualcosa come:

```
self.calcolatore_prezzi = CalcolatorePrezzi()
```

anziché far ereditare `Carrello` da `CalcolatorePrezzi`.

## 8.10.10 Riepilogo

In questo paragrafo abbiamo visto che:

- l'*ereditarietà singola* permette a una classe di ereditare da una sola classe padre;
- l'*ereditarietà multipla* permette a una classe di ereditare da due o più classi padre;
- *l'ereditarietà multipla* si scrive elencando le classi padre tra parentesi tonde, separate da virgole;
- l'*ereditarietà multipla* può causare ambiguità quando più classi definiscono metodi con lo stesso nome;
- il *problema del diamante* si verifica quando due classi intermedie ereditano dalla stessa classe base e una quarta classe eredita da entrambe;
- L'interprete Python risolve le ambiguità usando l'`MRO`, cioè l'*ordine di risoluzione dei metodi*;
- l'`MRO` può essere ispezionato con `__mro__` o con il metodo `mro()`;
- `super()` permette di chiamare i metodi delle classi padre in modo ordinato;
- i *Mixin* sono classi leggere usate per aggiungere funzionalità specifiche;
- quando possibile, è preferibile usare l'*ereditarietà multipla* per i *Mixin* e usare la *composizione* quando la relazione tra oggetti è di tipo "*ha un*".

# 8.11 I metodi statici

Nei paragrafi precedenti abbiamo visto che i metodi di una classe rappresentano il comportamento degli oggetti e che, quando definiamo un metodo di istanza, il primo parametro è normalmente `self`, cioè l'oggetto su cui il metodo opera.

Tuttavia non sempre una funzione definita dentro una classe ha bisogno di accedere a un oggetto specifico. In alcuni casi ci serve una funzione che sia logicamente collegata alla classe, ma che non debba leggere o modificare gli attributi di una particolare istanza.

Per questi casi abbiamo a nostra disposizione i *metodi statici*.

### 8.11.1 Che cos'è un metodo statico

Un *metodo statico* è una funzione definita all'interno di una classe che non riceve automaticamente né l'oggetto corrente, cioè `self`, né la classe, cioè `cls`.

Questo significa che un *metodo statico*:

- ➔ non può accedere direttamente agli attributi dell'istanza;
- ➔ non può modificare lo stato di un oggetto;
- ➔ non può modificare direttamente lo stato della classe;
- ➔ riceve solo i parametri che noi indichiamo esplicitamente.

Un *metodo statico* è utile quando vogliamo raggruppare dentro una classe una funzione di utilità che riguarda quella classe, ma che non dipende dai dati di un oggetto specifico.

In altre parole, un *metodo statico* è una funzione che "vive" dentro la classe, ma che non ha bisogno di un oggetto particolare per essere eseguita.

### 8.11.2 Come si definisce un metodo statico

Per definire un *metodo statico* si usa il decoratore `@staticmethod`, posizionato subito prima della definizione del metodo.

La sintassi è la seguente:

```
class NomeClasse(object):
    @staticmethod
    def nome_metodo(parametri):
        ...
```

A differenza dei metodi di istanza, qui non compare il parametro `self`.

### 8.11.3 Un primo esempio semplice

Consideriamo una classe `Operazioni` che contiene un *metodo statico* capace di sommare due numeri.

```
class Operazioni(object):
    """Classe di esempio per metodi statici"""

    @staticmethod
    def somma(a, b):
        return a + b
```

```
print(Operazioni.somma(2, 3))

op = Operazioni()
print(op.somma(4, 5))
```

Lanciamo il programma, in uscita otteniamo:

```
5
9
```

Dall'analisi del codice e dell'output osserviamo che:

- il metodo `somma()` è definito all'interno della classe `Operazioni`;
- il metodo è preceduto dal decoratore `@staticmethod`;
- il metodo non riceve il parametro `self`;
- il metodo può essere chiamato direttamente tramite la classe: `Operazioni.somma(2, 3)`
- il metodo può essere chiamato anche tramite un'istanza: `op.somma(4, 5)`. Tuttavia, poiché il metodo non usa dati dell'oggetto, la chiamata più naturale è quella fatta tramite la classe.

### 8.11.4 Differenza tra metodo di istanza e metodo statico

Per capire meglio la differenza, consideriamo la classe `Rettangolo`.

Un rettangolo ha due attributi: `base` e `altezza`. Il metodo di istanza `.area()` usa questi attributi per calcolare l'area dell'oggetto.

Possiamo però anche inserire un *metodo statico* che calcoli l'area a partire da due valori passati come parametri, senza bisogno di avere un oggetto rettangolo.

Consideriamo il seguente codice:

```
class Rettangolo(object):
    """Classe Rettangolo di esempio"""

    def __init__(self, base, altezza):
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

    @staticmethod
```

```
    def area_da_misure(base, altezza):
        return base * altezza

r = Rettangolo(3, 4)

print(r.area())
print(Rettangolo.area_da_misure(5, 6))
```

Lanciamo il programma, in uscita otteniamo:

```
12
30
```

Dall'analisi del codice e dell'output osserviamo che:

- il metodo `area()` è un *metodo di istanza*;
- il metodo `area()` riceve automaticamente `self`;
- il metodo `area()` legge gli attributi `self.base` e `self.altezza`;
- il metodo `area_da_misure()` è un *metodo statico;*
- il metodo `area_da_misure()` non riceve `self`;
- il metodo `area_da_misure()` usa solo i parametri `base` e `altezza` passati dall'esterno;
- il metodo `area_da_misure()` può essere chiamato senza creare un oggetto `Rettangolo`.

Possiamo quindi dire che:

**Metodo di istanza**: lavora su un oggetto specifico, sui suoi attributi.
**Metodo statico**: lavora su dati passati come parametri.

### 8.11.5 Un esempio collegato alla classe Punto

Riprendiamo la classe `Punto` utilizzata nei paragrafi precedenti. Sappiamo che un punto è descritto da due coordinate `X` e `Y`.

Nei paragrafi precedenti abbiamo definito un metodo `.getDistance()` capace di calcolare la distanza tra due oggetti `Punto`. Ora possiamo aggiungere un *metodo statico* che calcoli la distanza direttamente tra quattro coordinate numeriche, senza dover creare due oggetti `Punto`.

Consideriamo il seguente codice:

```
class Punto(object):
    """Classe Punto di esempio"""

    def __init__(self, X, Y):
```

```
        self.X = X
        self.Y = Y

    @staticmethod
    def distanza_tra_coordinate(x1, y1, x2, y2):
        return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5

    def distanza_da(self, altro):
        return Punto.distanza_tra_coordinate(self.X, self.Y,
altro.X, altro.Y)

    def __str__(self):
        return "(" + str(self.X) + "," + str(self.Y) + ")"

p1 = Punto(0, 0)
p2 = Punto(0, 3)
print(Punto.distanza_tra_coordinate(0, 0, 0, 3))
print(p1.distanza_da(p2))
print(p1.distanza_tra_coordinate(0, 0, 4, 0))
```

Lanciamo il programma, in uscita otteniamo:

```
3.0
3.0
4.0
```

Dall'analisi del codice e dell'output osserviamo che:

- il metodo `.distanza_tra_coordinate()` è un *metodo statico*;
- il metodo non riceve `self`;
- il metodo non conosce gli attributi interni di un oggetto `Punto`;
- il metodo riceve quattro parametri numerici: `x1, y1, x2, y2`;
- il metodo può essere chiamato direttamente tramite la classe:
  `Punto.distanza_tra_coordinate(0, 0, 0, 3)`
- il metodo può anche essere chiamato tramite un'istanza:
  `p1.distanza_tra_coordinate(0, 0, 4, 0)`
- il *metodo di istanza* `.distanza_da()` riutilizza il *metodo statico* passando gli attributi dell'oggetto corrente e dell'altro punto.

Questo esempio mostra bene una situazione comune: il calcolo della distanza è logicamente legato alla classe `Punto`, ma può essere eseguito anche senza possedere un oggetto specifico, purché vengano fornite le coordinate necessarie.

### 8.11.6 A cosa servono i metodi statici

I *metodi statici* servono principalmente a inserire dentro una classe funzioni di utilità

correlate alla classe stessa.

Per esempio, una classe potrebbe contenere metodi statici per:

- ➔ validare un dato;
- ➔ convertire un valore;
- ➔ eseguire un calcolo;
- ➔ creare una stringa formattata;
- ➔ verificare una condizione;
- ➔ fornire un servizio utile a più metodi della classe.

Nel caso della classe `Punto`, il *metodo statico*: `distanza_tra_coordinate()` è un servizio utile perché rappresenta un calcolo geometrico legato al concetto di punto, anche se non richiede necessariamente un oggetto `Punto`.

Un altro esempio potrebbe essere un *metodo statico* per controllare se una coordinata è valida:

```
class Punto(object):
    """Classe Punto di esempio"""

    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    @staticmethod
    def coordinata_valida(valore):
        return isinstance(valore, (int, float))

print(Punto.coordinata_valida(5))
print(Punto.coordinata_valida("x"))
```

Lanciamo il programma, in uscita otteniamo:

```
True
False
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ il metodo `.coordinata_valida()` è statico;
- ➔ il metodo non ha bisogno di un oggetto `Punto`;
- ➔ il metodo controlla se il valore passato è un numero intero o un numero decimale;
- ➔ il metodo può essere richiamato direttamente dalla classe.

### 8.11.7 Quando conviene usare un metodo statico

Un *metodo statico* conviene quando:

1. la funzione è legata logicamente alla classe;
2. la funzione non ha bisogno di leggere o modificare attributi dell'istanza;
3. la funzione non ha bisogno di accedere allo stato della classe;
4. vogliamo evitare di lasciare funzioni isolate fuori dalla classe;
5. vogliamo rendere più chiaro il codice raggruppando funzioni utility dentro la classe appropriata.

Per esempio, se abbiamo una classe `Data` per gestire date, potremmo inserire un metodo statico per controllare se un anno è bisestile. A tale scopo esaminiamo il seguente codice:

```
class Data(object):
    """Classe Data di esempio"""

    @staticmethod
    def anno_bisestile(anno):
        return anno % 4 == 0 and (anno % 100 != 0 or anno % 400 ==
0)

print(Data.anno_bisestile(2024))
print(Data.anno_bisestile(2023))
```

Lanciamo il programma, in uscita otteniamo:

```
True
False
```

In questo caso il metodo è legato alla classe `Data`, ma non ha bisogno di un oggetto `Data` specifico.

### 8.11.8 Pro e contro dei metodi statici

Come ogni strumento della programmazione orientata agli oggetti, anche i *metodi statici* hanno vantaggi e svantaggi. È importante conoscerli per usarli in modo appropriato.

I **principali vantaggi** dei metodi statici sono i seguenti:

1. Raggruppano funzioni utili dentro la classe
2. Permettono di non creare oggetti inutili
3. Rendono il codice più ordinato
4. Sono facili da testare
5. Favoriscono il riutilizzo

Analizziamo i singoli vantaggi:

### 1. Raggruppano funzioni utili dentro la classe

Se una funzione è strettamente legata a una classe, ha senso inserirla dentro quella classe.

Per esempio:
`Punto.distanza_tra_coordinate(...)` è più leggibile rispetto a una funzione esterna chiamata: `distanza_tra_coordinate(...)`

### 2. Permettono di non creare oggetti inutili

Se una funzione non dipende dallo stato di un oggetto, non è necessario creare un'istanza solo per usarla.

Per esempio:
`Rettangolo.area_da_misure(5, 6)` evita di creare un oggetto `Rettangolo` solo per calcolare un'area.

### 3. Rendono il codice più ordinato

Invece di avere molte funzioni sparse nel programma, possiamo raggrupparle nelle classi a cui appartengono logicamente.

### 4. Sono facili da testare

Poiché un metodo statico riceve parametri espliciti e non dipende dallo stato interno di un oggetto, spesso è semplice da provare.

Per esempio:
`Punto.coordinata_valida(5)` restituisce sempre `True` se il valore è valido, indipendentemente da qualsiasi oggetto.

### 5. Favoriscono il riutilizzo

Un *metodo statico* può essere usato sia dalla classe sia da eventuali metodi interni.

Per esempio:

```
class Punto(object):
    @staticmethod
    def distanza_tra_coordinate(x1, y1, x2, y2):
        return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5

    def distanza_da(self, altro):
        return Punto.distanza_tra_coordinate(self.X, self.Y,
altro.X, altro.Y)
```

Il *metodo statico* viene riutilizzato dal metodo di istanza.

Gli **svantaggi** principali sono invece i seguenti:

1. Non possono accedere a `self`
2. Non possono accedere direttamente alla classe
3. Possono essere usati male
4. Possono ridurre i vantaggi dell'OOP
5. Possono creare confusione

Analizziamo i singoli svantaggi:

### 1. Non possono accedere a `self`

Un *metodo statico* non riceve `self`, quindi non può leggere o modificare gli attributi dell'oggetto.

Per esempio, questo codice è sbagliato:

```
class Punto(object):
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    @staticmethod
    def metodo_sbagliato():
        return self.X
```

Il nome `self` non esiste all'interno del *metodo statico*, perché non viene passato automaticamente.

### 2. Non possono accedere direttamente alla classe

Un *metodo statico* non riceve nemmeno `cls`, cioè la classe. Se abbiamo bisogno di accedere alla classe, in Python si usa normalmente un metodo di classe con il *decoratore* `@classmethod`.

### 3. Possono essere usati male

Se una classe contiene troppi *metodi statici* e pochi attributi o *metodi di istanza*, rischia di diventare un semplice contenitore di funzioni.

In quel caso potrebbe essere più appropriato creare un modulo Python con funzioni normali, invece di creare una classe.

### 4. Possono ridurre i vantaggi dell'OOP

La programmazione orientata agli oggetti si basa molto su oggetti, stato e comportamento. Un *metodo statico*, non avendo accesso allo stato dell'oggetto, è meno

“orientato agli oggetti” rispetto a un *metodo di istanza*.

#### 5. Possono creare confusione

Uno studente potrebbe pensare che un *metodo statico* possa accedere agli attributi dell’oggetto, ma non è così. Bisogna quindi usare con chiarezza *metodi statici* e *metodi di istanza*.

### 8.11.9 Errore comune: usare `self` in un *metodo statico*

Un errore frequente è provare a usare `self` dentro un *metodo statico* come se fosse un metodo normale.

Consideriamo il seguente codice:

```
class Punto(object):
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    @staticmethod
    def distanza_sbagliata():
        return self.X
```

Questo codice produce un errore, perché `self` non è disponibile in un *metodo statico*.

La versione corretta deve ricevere esplicitamente i dati necessari:

```
class Punto(object):
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    @staticmethod
    def distanza_tra_coordinate(x1, y1, x2, y2):
        return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5
```

In questo modo il *metodo statico* non cerca di accedere a un oggetto, ma lavora solo con i valori che gli vengono passati.

### 8.11.10 Metodi statici e metodi di classe

Per completezza, è utile accennare alla differenza tra *metodi statici* e *metodi di classe*.

Python offre due *decoratori* nominalmente simili ma funzionalmente differenti:

- ➔ `@staticmethod`
- ➔ `@classmethod`

Un *metodo statico* non riceve né `self` né `cls`.

```
@staticmethod
def metodo_statico():
    pass
```

Un *metodo di classe* riceve come primo parametro `cls`, cioè la classe.

```
@classmethod
def metodo_di_classe(cls):
    pass
```

I *metodi di classe* possono accedere alla classe e sono utili, per esempio, quando vogliamo creare oggetti usando la classe stessa. (cfr. 8.11.10.1)

I *metodi statici*, invece, sono più adatti quando vogliamo solo una funzione di utilità che non deve interagire né con l'oggetto né con la classe.

### 8.11.10.1 * Approfondimento: i metodi di classe

Un *metodo di classe* è un metodo che non lavora direttamente su un oggetto specifico, ma lavora sulla classe stessa. A differenza di un metodo statico, un metodo di classe riceve automaticamente come primo parametro la classe a cui appartiene. Questo parametro, per convenzione, si chiama `cls`.

La sintassi generale è la seguente:

```
class NomeClasse(object):
    @classmethod
    def nome_metodo(cls, parametri):
        ...
```

Il *decoratore* usato è quindi: `@classmethod`

Mentre un *metodo statico* può essere visto come una semplice funzione utilità collocata dentro una classe, un metodo di classe può essere visto come una funzione che conosce la classe e può operare su di essa.

**Focus:** uno degli usi più comuni dei metodi di classe è la creazione di oggetti in modi alternativi rispetto al costruttore principale.

Consideriamo il seguente codice:

```
class Punto(object):
    """Classe Punto di esempio"""
```

```
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    @classmethod
    def origine(cls):
        return cls(0, 0)

    def __str__(self):
        return "(" + str(self.X) + "," + str(self.Y) + ")"


p = Punto.origine()

print(p)
print(type(p))
```

Lanciamo il programma, in uscita otteniamo:

```
(0,0)
<class '__main__.Punto'>
```

Dall'analisi del codice e dell'output osserviamo che:

- il metodo `.origine()` è definito con il decoratore `@classmethod`;
- il metodo `.origine()` riceve automaticamente il parametro `cls`;
- il parametro `cls` rappresenta la classe `Punto`, quindi la chiamata `cls(0, 0)` equivale, in questo caso, alla chiamata: `Punto(0, 0)`
- il metodo non riceve `self`, quindi non sta lavorando su un oggetto già esistente;
- il metodo restituisce una nuova istanza di `Punto`;
- il metodo può essere chiamato direttamente sulla classe: `Punto.origine()`

Questo tipo di metodo è utile quando vogliamo fornire un modo alternativo per creare un oggetto. Di fatti se volessimo istanziare la classe `Punto` nell'origine `(0,0)` dovremmo scrivere:

```
p = Punto(0, 0)
```

Utilizzando il metodo di classe definito nel precedente codice, potremmo scrivere:

```
p = Punto.origine()
```

rendendo il codice più leggibile.

**** Elenco principali utilizzi dei metodi di classe**
**1. Fornire costruttori alternativi**
Come visto poco sopra

### 2. Creare oggetti con valori predefiniti
Può essere utile quando vogliamo creare rapidamente un oggetto con alcuni valori già fissati.

```
@classmethod
def punto_centrale(cls)
   return cls(0, 0)
```

### 3. Leggere o modificare attributi della classe
Un *metodo di classe* può accedere alla classe tramite `cls`, quindi può lavorare su dati condivisi dalla classe stessa.

```
   class Contatore(object):
       oggetti_creati = 0

       def __init__(self):
           Contatore.oggetti_creati += 1

       @classmethod
       def numero_oggetti(cls):
           return cls.oggetti_creati
```

Quindi tenere traccia di informazioni comuni a tutte le istanze.

### 4. Creare oggetti a partire da dati in formato diverso
Un *metodo di classe* può ricevere una stringa, una lista, un dizionario o un altro formato e trasformarlo in un oggetto.

```
   @classmethod
   def da_stringa(cls, testo):
       x, y = testo.split(",")
       return cls(int(x), int(y))
```

In questo modo potremmo scrivere:

```
  p = Punto.da_stringa("3,4")
```

### 5. Favorire l'ereditarietà
Poiché `cls` indica la classe effettivamente chiamata, un `metodo di classe` può essere ereditato dalle sottoclassi e continuare a funzionare correttamente.
Ad esempio se una classe `PuntoColorato` eredita da `Punto`, un `metodo di classe` scritto bene può creare un oggetto della classe corretta senza dover scrivere nuovamente il metodo. In altre parole possiamo scrivere un metodo di classe in `Punto`, questo funzionerà anche come *metodo di classe* della classe `PuntoColorato` (si provi a utilizzare il metodo `.origine()` creato nei precedenti esempi)

In tutti questi casi, il punto essenziale è uno: un metodo di classe è utile quando la funzione deve lavorare sulla classe e non su un singolo oggetto specifico.

## 8.11.11 Esempio riassuntivo

Per riassumere, consideriamo la classe `Matematica` che contiene solo metodi statici. Naturalmente abbbaimo già imparato a capire che questa classe potrebbe essere senz'altro sostituita da un modulo che contenga le stesse funzioni dei *metodi statici* implementati in essa.

```
class Matematica(object):
    """Classe di esempio per metodi statici"""

    @staticmethod
    def somma(a, b):
        return a + b

    @staticmethod
    def quadrato(n):
        return n ** 2

    @staticmethod
    def area_rettangolo(base, altezza):
        return base * altezza

print(Matematica.somma(3, 4))
print(Matematica.quadrato(5))
print(Matematica.area_rettangolo(2, 6))
```

Lanciamo il programma, in uscita otteniamo:

```
7
25
12
```

Dall'analisi del codice e dell'output osserviamo che:

- la classe `Matematica` non ha bisogno di un costruttore;
- la classe contiene solo funzioni di utilità;
- tutti i metodi sono statici;
- i metodi possono essere chiamati direttamente tramite la classe;
- non viene creato alcun oggetto;
- ogni metodo riceve solo i parametri necessari.

### 8.11.12 Riepilogo

Abbiamo visto che:

- un *metodo statico* è una funzione definita dentro una classe;
- un *metodo statico* viene definito usando il decoratore `@staticmethod`;
- un *metodo statico* non riceve automaticamente `self`;
- un *metodo statico* non riceve automaticamente `cls`;
- un *metodo statico* non può accedere direttamente agli attributi dell'oggetto;
- un *metodo statico* non può modificare lo stato dell'istanza;
- un *metodo statico* può essere chiamato tramite la classe;
- un *metodo statico* può anche essere chiamato tramite un'istanza, ma spesso non è

la scelta più chiara;
- ➔ un *metodo statico* è utile per funzioni di utilità legate logicamente alla classe;
- ➔ i *metodi statici* aiutano a organizzare il codice e a evitare funzioni sparse nel programma;
- ➔ l'uso eccessivo dei metodi statici può allontanare il codice dallo stile orientato agli oggetti;
- ➔ se una funzione deve usare gli attributi dell'oggetto, conviene usare un metodo di istanza;
- ➔ se una funzione deve usare la classe, conviene usare un metodo di classe.

Possiamo riassumere la differenza principale con la seguente tabella:

| Tipo di metodo | Riceve automaticamente | Può accedere all'oggetto | Può accedere alla classe | Uso tipico |
|---|---|---|---|---|
| Metodo di istanza | `self` | Sì | Non direttamente | Operare sugli attributi dell'oggetto |
| Metodo di classe | `cls` | No | Sì | Operazioni legate alla classe |
| Metodo statico | Nessuno | No | No | Funzioni utility indipendenti dallo stato |

In definitiva, il *metodo statico* è uno strumento semplice ma molto utile. Va usato quando vogliamo lasciare una funzione dentro la classe per motivi di ordine e chiarezza, senza però collegarla allo stato di un oggetto specifico.

# 9. Gestione degli errori

In Python la gestione degli errori è essenziale per garantire che il programma possa gestire situazioni impreviste senza terminare in modo anomalo.

La gestione degli errori viene implementata utilizzando le *eccezioni*. Le *eccezioni* in Python ci permettono di intercettare e gestire situazioni anomale che possono verificarsi durante l'esecuzione di un programma, come ad esempio la divisione per zero o l'accesso a una variabile non definita.

## 9.1 Cosa sono le eccezioni

Le eccezioni sono oggetti che rappresentano errori o condizioni anomale che possono verificarsi durante l'esecuzione di un programma, quando accadono viene generato un evento che interrompe il normale flusso di un programma.
Quando si verifica un'eccezione, il controllo del programma passa al gestore dell'eccezione, se presente.

Ecco un esempio semplice di un'eccezione che si verifica durante una divisione per zero:

```
# Esempio di divisione per zero
numeratore = 10
denominatore = 0
risultato = numeratore / denominatore
print(risultato)
```

Eseguendo il programma otteniamo il seguente output:

```
Traceback (most recent call last):
  File "/home/massimo/prova", line 4, in <module>
    risultato = numeratore / denominatore
ZeroDivisionError: division by zero
```

Dall'analisi del codice e dell'output osserviamo che:

- Nell'ultima riga del programma si esegue l'istruzione che genera l'eccezione `ZeroDivisionError`.
- Appena l'interprete incontra l'eccezione interrompe il flusso del programma generando un errore sullo standard output.
- Le istruzioni successive a quella che ha generato l'eccezione non vengono eseguite.

Quando si verifica un errore, viene generata un'eccezione che può essere gestita utilizzando i costrutti:

- `try`,
- `except`,

- ➔ `else`,
- ➔ `finally`.

## 9.2 Struttura di base per la gestione delle eccezioni

La struttura di base per la gestione delle eccezioni la possiamo osservare nel seguente codice:

```
# Struttura di base del costrutto try - except
try:
    # Codice che potrebbe generare un'eccezione
    pass
except ExceptionType as e:
    # Codice per gestire l'eccezione
    pass
else:
    # Codice da eseguire se non si verifica alcuna eccezione
    pass
finally:
    # Codice da eseguire indipendentemente dal verificarsi
dell'eccezione
    pass
```

Esaminiamo un esempio:

```
try:
    x = 10 / 0
except ZeroDivisionError as err:
    print("Errore: divisione per zero \n",err)
else:
    print("Divisione eseguita correttamente.")
finally:
    print("Esecuzione del blocco finally.")
```

Quando lanciamo il programma, l'output stampato a video è il seguente:

```
Errore: divisione per zero.
 division by zero
Esecuzione del blocco finally.
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il blocco `try` contiene il codice che potrebbe generare un'eccezione, in questo caso la divisione di un numero per zero.
- ➔ Il blocco `except` cattura e gestisce l'eccezione specificata (`ZeroDivisionError`).
- ➔ Memorizziamo l'errore nella variabile `err` la label che descrive l'eccezione (non è obbligatorio farlo)

- ➔ Il blocco `else` viene eseguito solo se non si verifica alcuna eccezione.
- ➔ Il blocco `finally` viene eseguito sempre, indipendentemente dal verificarsi dell'eccezione.

### 9.2.1 Il blocco `finally`

Il blocco `finally` viene eseguito indipendentemente dal fatto che si verifichi un'eccezione o meno. È utile per eseguire codice di pulizia, come la chiusura di file o la disconnessione da un database.

Ecco un esempio che utilizza finally per chiudere un file:

```
fh = None
try:
    fh = open("filechenonesiste.txt", "r")
    contenuto = fh.read()
except FileNotFoundError:
    print("Errore: file non trovato")
finally:
    if fh :
        fh.close()
        print("File chiuso correttamente")
```

Eseguendo il programma otteniamo il seguente output (se il file non esiste):

```
Errore: file non trovato
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Cerchiamo di aprire un file che non esiste
- ➔ La `open()` genera un'eccezione `FileNotFoundError` e quindi la variabile `contenuto` non viene istanziata.
- ➔ Nel blocco finally andiamo a verificare se il file è stato aperto controllando che la variabile fh non sia inizializzata. Se fosse inizializzata allora significherebbe che il file è stato aperto e quindi si deve chiudere.

Con il blocco `finally` si effettuano quelle operazioni di chiusura che liberano la memoria o comunque ripristinano le condizioni corrette per continuare a seguire il flusso dell'algoritmo implementato dal programma. Queste operazioni sono generalmente dette di *clean-up*.

**N.B.** Solitamente per i file si utilizza l'istruzione `with` per essere sicuri di aver chiuso correttamente tutti i flussi (cfr. capitolo 4.1.7 "L'istruzione with per la gestione dei file" del volume n.2 )

### 9.2.2 Il blocco `else`

Il blocco `else` viene eseguito solo se non si verifica alcuna eccezione nel blocco `try`. È

utile per eseguire codice solo nel caso in cui non venga sollevata l'eccezione..

Ecco un esempio che utilizza `else` per stampare il risultato di una divisione solo se non si verifica un'eccezione:

```
try:
    numeratore = 10
    denominatore = 2
    risultato = numeratore / denominatore
except ZeroDivisionError:
    print("Errore: divisione per zero non consentita")
else:
    print(f"Risultato: {risultato}")
```

Eseguendo il programma otteniamo il seguente output:

```
Risultato: 5.0
```

Dall'analisi del codice e dell'output osserviamo che:

- L'operazione aritmetica eseguita nel blocco `try` non genera alcuna eccezione
- Viene eseguita l'istruzione contenuta nel blocco `else` e non in quello `except`

## 9.3 Eccezioni comuni

In Python, esistono diverse eccezioni predefinite che possono essere sollevate durante l'esecuzione di un programma. Di seguito sono elencate alcune delle eccezioni più comuni:

- `ZeroDivisionError`: sollevata quando si tenta di dividere un numero per zero.
- `IndexError`: sollevata quando si tenta di accedere a un indice inesistente di una sequenza.
- `KeyError`: sollevata quando si tenta di accedere a una chiave inesistente in un dizionario.
- `ValueError`: sollevata quando una funzione riceve un argomento con il tipo corretto ma valore non appropriato.
- `TypeError`: sollevata quando un'operazione o una funzione è applicata a un oggetto di tipo inappropriato.

Potete trovare una lista esaustiva delle eccezioni nella documentazione ufficiale[33].

## 9.4 Utilizzare più blocchi `except`

È possibile gestire diverse tipologie di eccezioni utilizzando più blocchi `except`. Questo ci permette di catturare e gestire vari tipi di errori separatamente.

[33] https://docs.python.org/3/library/exceptions.html

Esaminiamo un esempio:

```
try:
    x = int(input("Inserisci un numero: "))
    y = 10 / x
except ValueError as err:
    print("Errore: input non valido. Inserisci un numero intero.")
except ZeroDivisionError as err:
    print("Errore: divisione per zero.")
else:
    print(f"Risultato: {y}")
finally:
    print("Fine programma di calcolo.")
```

Quando lanciamo il programma, l'output stampato a video varia a seconda dell'input, otteniamo i seguenti tre casi:

## Caso 1: Input non valido

```
Inserisci un numero: abc
Errore: input non valido. Inserisci un numero intero.
Fine programma di calcolo.
```

## Caso 2: Divisione per zero

```
Inserisci un numero: 0
Errore: divisione per zero.
Fine programma di calcolo.
```

## Caso 3: Input valido

```
Inserisci un numero: 2
Risultato: 5.0
Fine programma di calcolo.
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Utilizziamo due blocchi `except` per catturare e gestire due diverse eccezioni: `ValueError` e `ZeroDivisionError`. La prima è generata nel caso in cui l'utente inserisca un valore non intero e la seconda nel caso venga inserito uno zero.
- ➔ Il blocco `else` viene eseguito solo se non si verifica alcuna eccezione e quindi

viene utilizzato per stampare il risultato dell'operazione.

➔ Il blocco `finally` viene eseguito sempre, indipendentemente dal verificarsi dell'eccezione e serve le operazioni di *clean-up*.

## 9.5 Utilizzare più eccezioni in un unico blocco `except`

È possibile gestire diverse tipologie di eccezioni utilizzando un solo blocco `except`. Questo ci permette di catturare diversi tipi di errori raggruppando la loro gestione.

Prendiamo l'esempio fatto nel precedente paragrafo e modifichiamolo in maniera tale da avere un unico `except`:

```
try:
    x = int(input("Inserisci un numero: "))
    y = 10 / x
except (ValueError,ZeroDivisionError) as err:
    print("Errore: ",err,"\n")
else:
    print(f"Risultato: {y}")
finally:
    print("Fine programma di calcolo.")
```

Quando lanciamo il programma, l'output stampato a video varia a seconda dell'input, otteniamo i seguenti tre casi:

### Caso 1: Input non valido

```
Inserisci un numero: abc
Errore:  invalid literal for int() with base 10: 'abc'
Fine programma di calcolo.
```

### Caso 2: Divisione per zero

```
Inserisci un numero: 0
Errore:  division by zero
Fine programma di calcolo.
```

### Caso 3: Input valido

```
Inserisci un numero: 2
Risultato: 5.0
```

```
Fine programma di calcolo.
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Utilizziamo un unico blocco `except` per catturare e gestire due diverse eccezioni: `ValueError` e `ZeroDivisionError`. La prima è generata nel caso in cui l'utente inserisca un valore non intero e la seconda nel caso venga inserito uno zero.
- ➔ In questo caso la stampa dell'errore risulta differenziata in quanto si utilizza la label del dettaglio dell'errore fornita dal gestore delle eccezioni di python.
- ➔ Naturalmente nulla vieta che il programmatore gestisca le diverse eccezioni allo stesso modo o che utilizzi una logica all'interno del blocco `except` per differenziare il comportamento a seconda dell'errore. Ovviamente in questo ultimo caso la gestione di più eccezioni in un'unica `except` non ha più molto senso.
- ➔ Il blocco `else` viene eseguito solo se non si verifica alcuna eccezione e quindi viene utilizzato per stampare il risultato dell'operazione.
- ➔ Il blocco `finally` viene eseguito sempre, indipendentemente dal verificarsi dell'eccezione e serve le operazioni di *clean-up*.

## 9.6 Sollevare eccezioni

In Python, è possibile sollevare manualmente un'eccezione utilizzando l'istruzione `raise`. Questo è utile quando vogliamo segnalare un errore specifico all'interno del nostro programma. La sintassi per sollevare un'eccezione è la seguente:

```
raise ExceptionType("Messaggio di errore")
```

Vogliamo elaborare la gestione degli errori all'interno di una funzione che ha come scopo quello di filtrare il valore che ha come suo argomento, se il valore è negativo allora solleva un'eccezione.
Esaminiamo il seguente codice:

```
def controlla_valore(x):
    if x < 0:
        raise ValueError("Il valore non può essere negativo.")
    return x

try:
    controlla_valore(-1)
except ValueError as err:
    print(f"Errore: {err}")
```

Quando lanciamo il programma otteniamo in output:

```
Errore: Il valore non può essere negativo.
```

Dall'analisi del codice e dell'output osserviamo che:

- La funzione `controlla_valore()` ha lo scopo di segnalare i valori negativi passati come argomento. In questo caso si deve sollevare un'eccezione che verrà gestita dal programma chiamante la funzione stessa.
- Tale funzione solleva un'eccezione `ValueError` se il valore del suo argomento è negativo utilizzando l'istruzione `raise`.
- La sintassi dell'istruzione `raise` è la seguente:

```
raise ExceptionType("Messaggio di errore")
```

- Nel blocco `try` si va a richiamare la funzione con un argomento negativo, questa provoca la comparsa dell'errore di tipo `ValueError` con la label di descrizione personalizzata del messaggio di errore.
- Nel blocco `except` viene gestita l'eccezione sollevata dalla funzione `controlla_valore()` con la stampa del messaggio di errore personalizzato.

## 9.7 Definire eccezioni personalizzate

Finora abbiamo considerato le eccezioni predefinite, ma Python ci permette di definire anche eccezioni personalizzate. Si possono definire queste eccezioni creando nuove classi che ereditano dalla classe base `Exception`.
Questo è utile quando vogliamo creare eccezioni specifiche per il nostro programma.

Rielaboriamo l'esempio visto nel precedente paragrafo in cui andavamo a sollevare un'eccezione quando a una funzione di controllo passavamo come argomento un valore negativo.
Esaminiamo il seguente codice:

```
class ErrorePersonalizzato(Exception):
    pass

def controlla_valore(x):
    if x < 0:
        raise ErrorePersonalizzato("Il valore non può essere
negativo.")
    return x

try:
    controlla_valore(-1)
except ErrorePersonalizzato as err:
    print(f"Errore: {err}")
```

Quando lanciamo il programma, l'output stampato a video è il seguente:

```
Errore: Il valore non può essere negativo.
```

Dall'analisi del codice e dell'output osserviamo che:

- Abbiamo definito una nuova eccezione `ErrorePersonalizzato` che eredita dalla classe `Exception`.

- ➔ La funzione `controlla_valore()` solleva un'eccezione `ErrorePersonalizzato` se il valore di x è negativo.
- ➔ Il blocco `try` tenta di eseguire la funzione `controlla_valore()` con un valore negativo.
- ➔ Il blocco `except` cattura e gestisce l'eccezione `ErrorePersonalizzato`.

per approfondimenti su come organizzare il codice delle eccezioni personalizzate si rimanda alla documentazione ufficiale[34].

### 9.7.1 Esempio di gestione eccezioni personalizzate

Per completare il paragrafo riportiamo un esempio pensato per un sistema di gestione di un conto bancario, in cui implementiamo una piccola gerarchia di eccezioni.

Si analizzi il seguente codice:

```
from datetime import datetime

class ErroreConto(Exception):
    """Eccezione base per tutti gli errori relativi al conto."""
    def __init__(self, messaggio, valore=None):
        super().__init__(messaggio)
        self.messaggio = messaggio
        self.valore = valore
        self.timestamp = datetime.now().strftime("%d/%m/%Y
%H:%M:%S")

    def dettagli(self):
        """Restituisce una descrizione formattata dell'errore."""
        info = f"[{self.timestamp}] {self.messaggio}"
        if self.valore is not None:
            info += f" (valore ricevuto: {self.valore})"
        return info

class SaldoNegativoError(ErroreConto):
    """Eccezione specifica: si tenta di impostare un saldo
negativo."""
    def __init__(self, valore):
        super().__init__("Il saldo non può essere negativo.",
valore)

class FondiInsufficientiError(ErroreConto):
    """Eccezione specifica: prelievo superiore al saldo
disponibile."""
    def __init__(self, saldo, importo_richiesto):
```

[34] https://docs.python.it/html/tut/node10.html

```
        messaggio = "Fondi insufficienti per completare il
prelievo."
        super().__init__(messaggio, importo_richiesto)
        self.saldo = saldo
        self.importo_richiesto = importo_richiesto
        self.differenza = importo_richiesto - saldo

    def dettagli(self):
        info = super().dettagli()
        info += f" | Saldo disponibile: {self.saldo}€, mancano
{self.differenza}€"
        return info

class ContoBancario:
    def __init__(self, saldo=0):
        self.saldo = saldo

    def preleva(self, importo):
        if importo < 0:
            raise SaldoNegativoError(importo)
        if importo > self.saldo:
            raise FondiInsufficientiError(self.saldo, importo)
        self.saldo -= importo
        return self.saldo

# Programma principale in cui utilizzo le classi
conto = ContoBancario(saldo=100)

try:
    conto.preleva(150)
except FondiInsufficientiError as err:
    print(err.dettagli())
except SaldoNegativoError as err:
    print(err.dettagli())
except ErroreConto as err:
    print(err.dettagli())
```

Se eseguiamo il programma, otteniamo a video il seguente output:

```
[04/07/2026 19:15:26] Fondi insufficienti per completare il
prelievo. (valore ricevuto: 150) | Saldo disponibile: 100€,
mancano 50€
```

Dall'analisi del codice e dell'output osserviamo che:

- Abbiamo creato una *gerarchia di eccezioni*: `ErroreConto` è la classe base, da cui derivano `SaldoNegativoError` e `FondiInsufficientiError`. Questo permette di catturare eccezioni specifiche oppure, con un `except ErroreConto`,

catturare qualsiasi errore relativo al conto in un colpo solo.

- ➔ Utilizzo di *attributi personalizzati:* ogni eccezione porta con sé dati utili (valore, saldo, differenza, timestamp), non solo un messaggio testuale.
- ➔ Abbiamo effettuato l'override del costruttore `__init__`: ogni sottoclasse costruisce un messaggio specifico e salva informazioni extra, chiamando `super().__init__()` per non perdere il comportamento standard della classe `Exception`.
- ➔ Utilizzo del metodo personalizzato `dettagli()`: mostra che una classe di eccezione può avere metodi propri, non solo attributi. Risulta utile per formattare log, messaggi per l'utente finale o report di errore.
- ➔ Attenzione all'ordine degli except: si intercettano prima le eccezioni più specifiche e poi quella più generica (`ErroreConto`), altrimenti quella generica catturerebbe tutto prima che le altre abbiano la possibilità di farlo.

Questo pattern (eccezione base + sottoclassi specifiche con dati propri) è molto comune in progetti reali, ad esempio nelle librerie che gestiscono richieste di rete, validazione di form, o accesso a database.

## 9.8 Utilizzare le eccezioni per il controllo del flusso

Le eccezioni possono essere utilizzate anche per controllare il flusso di esecuzione del programma. Ad esempio, possiamo utilizzare le eccezioni per gestire situazioni in cui una funzione fallisce e dobbiamo riprovare o eseguire un'azione alternativa.
Esaminiamo un esempio:

```
def leggi_file(nome_file):

    try:
        with open(nome_file, 'r') as file:
            retValue = file.read()
    except FileNotFoundError:
        print(f"Errore: il file {nome_file} non esiste.")
        retValue = None
    return retValue

contenuto = leggi_file("filechenonesiste.txt")
if contenuto is None:
    print("Esecuzione alternativa in caso di errore.")
```

Quando lanciamo il programma, l'output stampato a video è il seguente:

```
Errore: il file filechenonesiste.txt non esiste.
Esecuzione alternativa in caso di errore.
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione `leggi_file()` tenta di aprire e leggere un file.
- ➔ Se il file non esiste, viene sollevata e gestita un'eccezione `FileNotFoundError`.

- ➔ In caso di errore, viene eseguita un'azione alternativa.
- ➔ La funzione `leggi_file()` restituisce due tipi di valori differenti tra loro: nel caso in cui il file venga letto andrà a restituire una stringa con il contenuto del file. Nel caso in cui si abbia l'eccezione andrà a restituire None che indicherà al programma chiamante che è avvenuto un errore nella lettura

**N.B.** Naturalmente questo non è l'unico modo per segnalare un comportamento errato all'interno di una funzione. Se si vuole mantenere un codice pulito e quindi avere sempre lo stesso tipo di ritorno da una funzione si potrebbe restituire una tupla che contiene un codice di errore e la stringa contenuto del file. In questo modo il programma chiamante va a testare il codice di errore e se riscontra errore allora esegue un flusso alternativo.

Vediamo come lo stesso programma potrebbe essere riscritto:

```
def leggi_file(nome_file):
    codiceErrore = False
    returnValue =""
    try:
        with open(nome_file, 'r') as file:
            returnValue = file.read()
    except FileNotFoundError:
        print(f"Errore: il file {nome_file} non esiste.")
        codiceErrore = True
    return codiceErrore,returnValue

err,contenuto = leggi_file("filechenonesiste.txt")
if err :
    print("Esecuzione alternativa in caso di errore.")
```

Quando lanciamo il programma, l'output stampato a video è il seguente:

```
Errore: il file filechenonesiste.txt non esiste.
Esecuzione alternativa in caso di errore.
```

Dall'analisi del codice e dell'output osserviamo che:

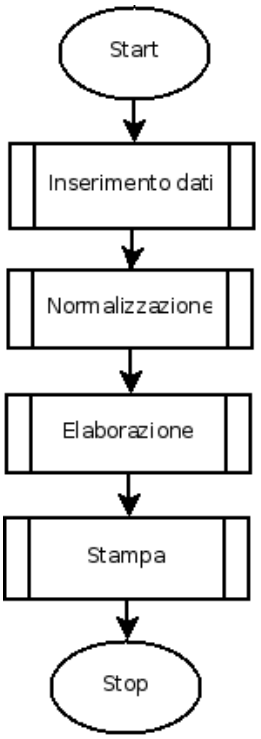
- ➔ L'output che otteniamo è lo stesso, come è la stessa la logica del programma chiamante.
- ➔ La funzione `leggi_file()` restituisce una tupla il cui primo elemento indica se è presente l'errore. Se presente esegue una logica alternativa altrimenti gestisce il contenuto del file letto.

# 10. Importazione moduli

## 10.1 Introduzione

Quando si affronta un problema si utilizza una metodologia che fornisce gli strumenti per arrivare alla sua soluzione. Per esempio la metodologia *top-down* affronta il problema suddividendolo in problemi più piccoli, questi, a loro volta, vengono suddivisi in altri e così via, sino ad arrivare a dei problemi la cui soluzione è banale. Mettendo insieme tutti gli algoritmi risolutivi trovati si compone la soluzione al problema di partenza. Questo è un metodo ricorsivo che viene diffusamente usato per scrivere algoritmi.

Utilizzando la descrizione di un algoritmo mediante flow-chart, possiamo rappresentare con dei blocchi i sottoproblemi in cui abbiamo diviso il nostro problema di partenza. Il seguente macro-flow ne è un esempio per un semplice generico problema:




Nell'esempio possiamo notare quattro macroblocchi:

- Inserimento dati, in cui andiamo a specificare l'algoritmo che permette di alimentare con i dati di input la nostra procedura;
- Normalizzazione, è una sorta di pre-elaborazione, potrebbe essere utile nel caso in cui dobbiamo elaborare una serie di dati "sporchi". p.e. potremmo andare ad eliminare dai dati quei valori che non sono consoni ai valori di input da noi aspettati.
- Elaborazione, è l'algoritmo di elaborazione vera e propria. Ovviamente a seconda dei casi potrà assumere nomi diversi, l'importante è che il nome descriva l'elaborazione che l'algoritmo andrà a fare. p.e. se abbiamo la necessità di integrare

numericamente una funzione reale allora il nome di questo macro-blocco potrebbe essere:"integrazione numerica".

➔ Stampa, è la fase di emissione dei risultati, quasi sempre conviene tenerla separata dalla fase elaborativa pura.

Ogni macro-blocco può essere a sua volta spezzato in altri macro-blocchi. Dopo questo processo di analisi, ogni macro-blocco verrà descritto da un flow-chart in cui si specifica in dettaglio l'algoritmo.
Nella fase di codifica un macro-blocco verrà rappresentato mediante una funzione che esprime l'algoritmo contenuto nel macro-blocco stesso.

P.e. la codifica che rappresenta il macroflow sopra riportato potrebbe essere:

```
# definizione delle funzioni descritte dal macro-flow
def inserimento_dati(..,...,...):
     ...
     ...
     return var
def normalizzazione(..,...,...):
     ...
     ...
     return var
def elaborazione(..,...,...):
     ...
     ...
     return var
def stampa(..,...,...):
     ...
     ...
     return var

# programma principale
# definizione delle variabili
...
...
# chiamata delle funzioni
a=inserimento_dati(...,...,...)
b=normalizzazione(...,...,...)
c=elaborazione(...,...,...)
d=stampa(...,...,...)
```

Analizzando il codice osserviamo che il programma principale è costituito soltanto dalle chiamate alle funzioni che codificano gli algoritmi dei macro-blocchi. Generalmente questa è la struttura della codifica che genera un macro-flow come quello qui descritto.
N.B.: i puntini nella codifica rappresentano tutte le parti che si sono deliberatamente omesse per non appesantire troppo la codifica di esempio, ma che, necessariamente, in una codifica reale non possono essere tralasciate.

## 10.2 I moduli del Python

Alcune volte risulta comodo utilizzare parti di codifiche scritte per una specifica soluzione, per risolvere altri problemi. Per esempio riutilizzare la procedura di normalizzazione o di stampa del precedente esempio potrebbe far risparmiare diverso tempo nello sviluppo totale del problema preso in esame.
Un modo molto semplice per far questo è quello di copiare la parte di codice che interessa. In questo modo, però, si va incontro alle diverse problematiche che un'operazione del genere comporta, basti pensare solo alla complessità di gestione delle modifiche di una procedura se presente in più codifiche contemporaneamente.

Il Python risolve questo problema introducendo l'uso dei moduli.
Si deve seguire la seguente procedura per utilizzare i moduli per l'esempio riportato precedentemente:

- ➔ Si scriva il codice di una o più funzioni in un file sorgente separato
- ➔ Si scriva il sorgente del programma principale come nella codifica precedente omettendo il corpo delle funzioni.
- ➔ Si aggiungano i comandi di importazione delle funzioni nel sorgente del programma principale.

Si hanno i seguenti files sorgenti:

*inserimento.py* :
```
# definizione delle funzioni
def inserimento_dati(..,...,...):
     ...
     ...
     return var
def funzione_dati(..,...,...):
     ...
     ...
     return var
...
...
```

*preelaborazioni.py* :
```
# definizione delle funzioni

def normalizzazione(..,...,...):
     ...
     ...
     return var
def medie(..,...,...):
     ...
     ...
     return var
...
...
```

*elaborazioni.py* :

```
# definizione delle funzioni
def elaborazione(..,...,...):
     ...
     ...
     return var
def elaborazione2(..,...,...):
     ...
     ...
     return var
...
...
```

*stampe.py* :

```
# definizione delle funzioni
def stampa(..,...,...):
     ...
     ...
     return var
def stampa2(..,...,...):
     ...
     ...
     return var
...
...
```

*main.py* :

```
# programma principale
# importazione dei moduli
import inserimento, preelaborazioni, elaborazioni, stampe

# definizione delle variabili
...
...
# chiamata delle funzioni
a=inserimento.inserimento_dati(...,...,...)
b=preelaborazioni.normalizzazione(...,...,...)
c=elaborazioni.elaborazione(...,...,...)
d=stampe.stampa(...,...,...)
```

Analizzando il codice osserviamo che:

- ➔ il programma principale è costituito soltanto dalle definizioni di variabili e dalle chiamate alle funzioni presenti nei moduli
- ➔ le funzioni all'interno dei moduli sono definite in maniera standard
- ➔ non è necessario che all'interno di un modulo sia presente solo una funzione
- ➔ nel programma principale appare l'istruzione import che permette di effettuare l'importazione del codice da un file sorgente

- ➔ Nel programma principale le funzioni vengono chiamate apponendo come suffisso il nome del modulo a cui la funzione appartiene seguito da un punto (p.e. stampe.stampa(...,...,...) ). Si noti che questa sintassi richiama la sintassi dei linguaggi orientati agli oggetti.
- ➔ l'istruzione import può essere inserita in qualsiasi posizione del programma principale, l'importante è che sia prima della chiamata alla funzione che contiene

## 10.3 L'istruzione import

Il comando che indica al Python di importare un modulo (cioè un file sorgente) è import, questa può essere scritta nei seguenti modi:

a) `import` *nomemodulo*
b) `from` *nomemodulo* `import` *nomefunzione1*,*nomefunzione2*,...
c) `from` *nomemodulo* `import *`

Il primo caso lo abbiamo esaminato nel precedente paragrafo, in questo caso le chiamate alle funzioni prevedono un suffisso composto dal nome del modulo a cui la funzione appartiene seguito da un punto. Con questa istruzione si va a caricare in memoria tutte le funzioni contenute nel modulo importato. Questo potrebbe dare problemi di memoria e di performance della macchina su cui gira lo script.

Nel secondo caso (b) si va a specificare quale funzione vogliamo caricare dal modulo preso in considerazione. Utilizzando questa sintassi dobbiamo modificare la chiamata alla funzione, non si porrà più il suffisso ma si scriverà solo il nome della funzione. Si consideri il seguente programma che è la riscrittura di main.py:

*main.py* :
```
# programma principale
# importazione dei moduli
from inserimento import inserimento_dati
from preelaborazioni import normalizzazione
from elaborazioni import elaborazione
from stampe import stampa

# definizione delle variabili
...
...
# chiamata delle funzioni
a=inserimento_dati(...,...,...)
b=normalizzazione(...,...,...)
c=elaborazione(...,...,...)
d=stampa(...,...,...)
```

In questa maniera la chiamata alle funzioni presenti nel modulo risulta essere meno tediosa.

Nel terzo caso (c) si vanno ad importare tutte le funzioni presenti all'interno del modulo. La

presenza del `from` permette di eseguire la chiamata alle funzioni in maniera più comoda senza dover inserire il suffisso del modulo. Bisogna far attenzione a non abusare di questa istruzione perchè si può rischiare di richiamare moduli che presentano lo stesso nome di funzioni e questo potrebbe portare a conflitti. Per chiarire quanto detto si esaminino i seguenti sorgenti:

*modulo1.py*:
```
def fun():
        return "modulo1"
```

*modulo2.py*:
```
def fun():
        return "modulo2"
```

*main.py*:
```
from modulo1 import *
from modulo2 import *
print (fun())
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
modulo2
```

Dall'analisi del codice e dell'output osserviamo che le istruzioni di import hanno generato una ridefinizione della funzione f*un()* , l'ultimo modulo importato impone la propria funzione (nel caso dell'esempio *modulo2*)

# 11. I packages

## 11.1 Packages e moduli

Prima di cominciare a lavorare con i packages, è importante capire la differenza fondamentale tra packages e moduli.
Un modulo in Python è un singolo file contenente codice Python, mentre un package è una collezione di moduli organizzati in una gerarchia di directory. Possiamo pensare a un package come a una cartella che contiene più moduli correlati, aiutandoci a organizzare il codice in modo più strutturato e manutenibile.

Per illustrare questo concetto esaminiamo un semplice esempio::

```
# Un semplice modulo chiamato calcolatrice.py
def somma(a, b):
    return a + b

def sottrai(a, b):
    return a - b
```

Questo file rappresenta un modulo, come abbiamo visto nel precedente capitolo (cfr. Capitolo 10 )

Se abbiamo più moduli tra loro correlati e vogliamo organizzarli in un'unica "entità" andiamo a creare un package che li contiene.

Per esempio se abbiamo dei moduli che ci mettono a disposizione oggetti per il calcolo matematico, potremmo pensare di organizzarli secondo la seguente struttura di directory:

```
strumenti_matematici/
    └── __init__.py
    └── calcolatrice.py
    └── geometria/
        └── __init__.py
        └── cerchio.py
        └── rettangolo.py
```

In questo esempio abbiamo che il package `geometria` è annidato nel package `strumenti_matematici`.
Nel prossimo paragrafo vedremo come creare un tale package.

## 11.2 Creazione dei packages

Per creare un package in Python, dobbiamo seguire questi passaggi essenziali:

1. Creare l'albero di directories che conterrà il package
2. Creare un file `__init__.py` nella directory root del package
3. Aggiungere i moduli (`file.py`) nell'albero delle directories

Come esempio riprendiamo il package `strumenti_matematici` iniziato a vedere nel precedente paragrafo:

1. Si deve costruire sul filesystem l'albero delle directories a partire dalla root strumenti_matematici.

2. Nella root del package andiamo a creare il file __init__.py.

File sorgente: strumenti_matematici/__init__.py

```
# Questo file può essere vuoto o contenere codice di
# inizializzazione del package
```

3. Ora si andranno a memorizzare i file sorgenti che costituiscono il package.

File sorgente: strumenti_matematici/calcolatrice.py

```
def somma(a, b):
    return a + b

def moltiplica(a, b):
    return a * b
```

File sorgente: strumenti_matematici/geometria/cerchio.py

```
import math

def area(raggio):
    return math.pi * raggio ** 2

def circonferenza(raggio):
    return 2 * math.pi * raggio
```

File sorgente: strumenti_matematici/geometria/rettangolo.py

```
def area(lunghezza, larghezza):
    return lunghezza * larghezza

def perimetro(lunghezza, larghezza):
    return 2 * (lunghezza + larghezza)
```

## 11.3 Utilizzo dei packages

Una volta creato un package, possiamo utilizzarlo in diversi modi.

Esaminiamo vari metodi di importazione lanciando il seguente programma:

```
# Importare funzioni specifiche da un modulo nel package
from strumenti_matematici.calcolatrice import somma, moltiplica
print(somma(5, 3))
print(moltiplica(4, 2))

# Importare un intero modulo dal package
from strumenti_matematici import calcolatrice
print(calcolatrice.somma(5, 3))
print(calcolatrice.moltiplica(4, 2))

# Importare da un package annidato
from strumenti_matematici.geometria import cerchio
print(cerchio.area(5))
print(cerchio.circonferenza(5))
```

Lanciamo  il programma, l'output stampato a video è il seguente:

```
8
8
8
8
78.53981633974483
31.41592653589793
```

Dall'analisi del codice e dell'output osserviamo che:

- Il file `__init__.py` marca una directory come package Python
- Possiamo annidare packages all'interno di altri packages (come il package `geometria` dentro `strumenti_matematici`)
- Possiamo importare funzioni specifiche o interi moduli usando diverse sintassi di importazione ( analogamente a quanto visto nel capitolo 10 sull'importazione dei moduli cfr. Capitolo 10 )

## 11.4 Configurazione del package con `init.py`

Il file `__init__.py` può essere più che un semplice marcatore di directory come package.

Può anche svolgere le seguenti funzioni:

- ➔ Inizializzare variabili a livello di package
- ➔ Importare funzioni comunemente utilizzate per renderle direttamente disponibili
- ➔ Definire cosa viene importato con `from package import *`

Sempre all'interno del nostro semplice esempio andiamo a personalizzare l' `__init__.py` nel seguente modo:

```
# strumenti_matematici/__init__.py

# Il punto indica un'importazione relativa

from .calcolatrice import somma, moltiplica
from .geometria.cerchio import area as area_cerchio
from .geometria.rettangolo import area as area_rettangolo

# Definisce cosa viene importato con from strumenti_matematici
import *
__all__ = ['somma', 'moltiplica', 'area_cerchio',
'area_rettangolo']
```

Con questo nuovo `__init__.py` possiamo usare il package in modo più conveniente:

```
# Importare direttamente dal package
from strumenti_matematici import somma, area_cerchio

print(somma(5, 3))
print(area_cerchio(5))
```

Eseguendo il programma otteniamo il seguente output:

```
8
78.53981633974483
```

## 11.5 Importazioni assolute e relative

Quando lavoriamo con i packages, possiamo utilizzare due tipi di importazioni:

- ➔ Importazioni assoluta
- ➔ Importazione relativa

Con l'*importazione assoluta* si specifica il percorso completo dalla radice del progetto. Così come si può vedere nel seguente codice di esempio:

```
from strumenti_matematici.geometria.cerchio import area
```

Con l'importazione relativa si specifica il percorso relativo al modulo corrente.
Così come si può vedere nel seguente codice di esempio:

```
from ..geometria.cerchio import area  # Sali di due livelli, poi
entra in geometria/cerchio
```

Possiamo osservare che:

- ➔ Le *importazioni assolute* sono più chiare ed esplicite
- ➔ Le *importazioni relative* possono essere più comode durante il *refactoring* o lo spostamento del codice
- ➔ Un singolo punto (.) si riferisce al package corrente
- ➔ Due punti (..) si riferiscono al package genitore
- ➔ Tre punti (...) si riferiscono al package nonno

La scelta tra importazioni assolute e relative dipende spesso dalla struttura del progetto e dalle preferenze personali, ma le importazioni assolute sono generalmente raccomandate per una migliore leggibilità e manutenibilità.

Ricordiamo che i packages ci aiutano a organizzare il codice in modo logico e manutenibile, rendendo più facile riutilizzare il codice in diversi progetti e condividerlo con altri. Sono uno strumento fondamentale in Python per creare applicazioni ben strutturate e scalabili.

# 12. Paradigmi funzionali e gestione efficiente della memoria

Studieremo le espressioni lambda, le comprehension, gli iteratori e i generatori

## 12.1 Perché abbiamo bisogno di nuovi strumenti

Fino a questo punto abbiamo attraversato le collezioni di dati (liste, stringhe, dizionari, tuple, insiemi) quasi esclusivamente con il ciclo `for` e con istruzioni esplicite come `.append()`. Questo approccio funziona ed è quello con cui abbiamo preso confidenza.

Ma man mano che i nostri programmi crescono, e soprattutto man mano che i dati da elaborare crescono, ci accorgiamo di due problemi:

- ➔ il codice scritto *a cicli* tende a diventare lungo e ripetitivo, anche per operazioni concettualmente semplici;
- ➔ se la collezione di dati è molto grande, tenerla tutta in memoria (in RAM) può diventare un problema serio.

Python offre alcuni strumenti pensati apposta per affrontare questi due problemi: le **espressioni lambda**, le **comprehension**, gli **iteratori** e i **generatori**. Non si tratta di un semplice "modo più corto" di scrivere un `for`, bensì cambia il modo in cui pensiamo al problema e alla sua risoluzione.
Vediamoli uno alla volta, con lo stesso approccio pratico che abbiamo usato fino ad ora: scriviamo il programma, osserviamo l'output, commentiamo cosa è successo.

## 12.2 Le espressioni lambda

### 12.2.1 Funzioni "usa e getta"

Sappiamo definire funzioni utilizzando la parola chiave `def`. Esistono situazioni in cui abbiamo bisogno di una funzione semplicissima, che useremo una volta sola, magari solo per passarla come argomento a un'altra funzione. In questi casi scrivere un `def` completo, dargli un nome, per poi usarlo una sola volta, è scomodo. Per questo Python mette a disposizione le *funzioni anonime*, dette *espressioni lambda*.

Il nome *lambda* non è casuale: deriva dal *lambda-calcolo*, un sistema formale ideato negli anni '30 dal matematico *Alonzo Church* per studiare in modo astratto il concetto di funzione e di calcolo[35]. Python ne ha preso in prestito il nome e l'idea, adattandola a una

[35] https://it.wikipedia.org/wiki/Lambda_calcolo

sintassi molto pratica.

La sintassi generale è la seguente:

```
lambda parametri: espressione
```

Una lambda ha alcune regole molto rigide, diverse da quelle di una funzione normale:

- non ha un nome proprio (da cui il termine *anonima*), anche se possiamo assegnarla a una variabile se vogliamo riutilizzarla;
- il corpo è una sola espressione, non un blocco di istruzioni;
- non possiamo usare `return`, pass, cicli o altre istruzioni, il risultato dell'espressione viene restituito automaticamente.

Come primo esempio esaminiamo il seguente codice:

```
# Una lambda che calcola il quadrato di un numero
quadrato = lambda x: x**2

print(quadrato(5))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
25
```

Dall'analisi del codice e dell'output osserviamo che:

- `lambda x: x**2` definisce una *funzione anonima* che prende un parametro (`x`) e restituisce `x**2`, senza bisogno di scrivere `def` e `return`;
- qui l'abbiamo assegnata alla variabile quadrato, quindi in questo caso specifico si comporta come una normale funzione, richiamabile con `quadrato(5)`;

**ATTENZIONE**: assegnare una *lambda* a una variabile, come abbiamo fatto qui, è utile solo a scopo didattico, per mostrarne il funzionamento. Nella pratica, se una funzione deve avere un nome ed essere riutilizzata più volte, conviene definirla con `def`, che è più leggibile e permette anche di scrivere un corpo su più righe. Il vero punto di forza delle *lambda* si vede quando le passiamo direttamente come argomento ad un'altra funzione, senza doverle assegnare a una variabile.

### 12.2.2 *Lambda* insieme a `map()` e `filter()`

Le *lambda* raramente si usano da sole: il loro habitat naturale è dentro le cosiddette *funzioni di ordine superiore*, cioè funzioni che accettano altre funzioni come argomento.
Le due più comuni sono:

- ➔ `map(funzione, iterabile)`: applica `funzione` a ogni elemento di *iterabile* e restituisce un *iteratore* con i risultati;
- ➔ `filter(funzione, iterabile)`: tiene solo gli elementi di *iterabile* per cui `funzione` restituisce `True`.

Le abbiamo già incontrate nei precedenti capitoli: la prima insieme alla funzione `input()` (cfr. cap. 6.3) e la seconda nelle funzioni di utilità sulle sequenze (cfr. cap. 7.5)

Scriviamo il seguente programma:

```
# Lista di temperature espresse in gradi Celsius
t_celsius = [20.0, 25.5, 30.0, 35.5, 40.0]

# Conversione in gradi Fahrenheit tramite map() e una lambda
t_fahrenheit = list(map(lambda c: (c * 9/5) + 32, t_celsius))

# Sole temperature > 90 gradi Fahrenheit tramite filter()
t_elevate = list(filter(lambda f: f > 90.0, t_fahrenheit))

print("Temperature in Fahrenheit:", t_fahrenheit)
print("Temperature superiori a 90°F:", t_elevate)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Temperature in Fahrenheit: [68.0, 77.9, 86.0, 95.9, 104.0]
Temperature superiori a 90°F: [95.9, 104.0]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Ricordiamo che `map()` e `filter()` restituiscono un oggetto rispettivamente della classe `map` e della classe `filter`;
- ➔ `map()` non modifica la lista di partenza. Il programma ne crea una nuova attraverso la funzione `list()`;
- ➔ la *lambda* `lambda c: (c * 9/5) + 32` sostituisce, in una sola riga, quella che altrimenti sarebbe stata una funzione `def converti(c): return (c * 9/5) + 32`;
- ➔ `filter()` richiede una funzione che restituisca `True` o `False`, solo gli elementi per cui la condizione è vera vengono mantenuti;
- ➔ usare `map()` e `filter()` con *lambda* evita di "sporcare" lo spazio dei nomi con funzioni che useremo una volta sola.

Si consiglia di usare le *lambda* solo per operazioni davvero minime, come in questo esempio. Se il corpo della funzione comincia a diventare complicato, conviene tornare a un `def` con un nome parlante. In questo modo il codice guadagna in leggibilità molto più di quanto perda in compattezza.

## 12.3 La sintassi delle comprehension

### 12.3.1 *List comprehension*

Le *list comprehension* sono probabilmente lo strumento più usato tra quelli che vedremo in questo capitolo. Permettono di costruire una lista nuova a partire da un iterabile, in una sola riga, con una sintassi che ricorda la notazione matematica degli insiemi.

La sintassi generale è:

```
[espressione for variabile in iterabile if condizione]
```

La parte `if` è opzionale.

Per capire bene come funziona, conviene "smontare" un ciclo `for` tradizionale e osservare come le sue parti si ricompongono in una *comprehension*.

**Ciclo tradizionale**:

```
lista_risultato = []
for item in collezione:
    if condizione(item):
        lista_risultato.append(trasformazione(item))
```

Equivalente in **list comprehension**:

```
lista_risultato = [trasformazione(item) for item in collezione if condizione(item)]
```

Notiamo che l'ordine delle parti cambia: la *trasformazione*, che nel ciclo tradizionale è posionata alla fine (dentro `.append()`), nella *comprehension* viene scritta per prima.

Partiamo con un esempio molto semplice e riportiamo la *list comprehension* e il ciclo for tradizionale, ecco il codice:

```
# Metodo tradizionale con ciclo for
quadrati_tradizionale = []
for x in range(1, 6):
    quadrati_tradizionale.append(x**2)

# Metodo con list comprehension
quadrati_comprehension = [x**2 for x in range(1, 6)]

print("Metodo tradizionale:", quadrati_tradizionale)
print("List comprehension:", quadrati_comprehension)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Metodo tradizionale: [1, 4, 9, 16, 25]
List comprehension: [1, 4, 9, 16, 25]
```

Dall'analisi del codice e dell'output osserviamo che:

- I due metodi producono lo stesso identico risultato;
- La *list comprehension* racchiude tutto in una sola riga, racchiusa tra parentesi quadre `[]`;
- Le `list comprehension` sono generalmente più veloci dei cicli tradizionali, perché il ciclo viene eseguito direttamente a livello di bytecode dall'interprete C.

Consideriamo un esempio pù complesso: implementiamo una procedura che "appiattisce" una matrice bidimensionale portandola a una sola dimensione e una procedura che filtra i valori di una lista a seconda del logaritmo. In quest'ultimo caso utilizzeremo l'operatore `:=` (*walrus operator*), questo è stato introdotto in Python 3.8[36]. Quindi esaminiamo il seguente codice:

```
import math

# Procedura 1: appiattimento di una matrice (lista di liste)
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
matrice_appiattita = [elemento for riga in matrice for elemento in
riga]

# Procedura 2: uso dell'operatore walrus (:=) per non ricalcolare
il logaritmo due volte
valori_sorgente = [2.0, 5.0, 10.0, 20.0]
logaritmi_filtrati = [
    log_val for x in valori_sorgente
    if (log_val := math.log(x)) > 1.5
]

print("Matrice appiattita:", matrice_appiattita)
print("Logaritmi significativi (> 1.5):", logaritmi_filtrati)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Matrice appiattita: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Logaritmi significativi (> 1.5): [1.6094379124341003,
2.302585092994046, 2.995732273553991]
```

Dall'analisi del codice e dell'output osserviamo che:

[36] https://docs.python.org/3/whatsnew/3.8.html

- ➔ quando in una *comprehension* scriviamo più cicli `for` uno di seguito all'altro, questi vengono eseguiti nello stesso ordine, da sinistra a destra, con cui li scriveremmo annidando dei `for` tradizionali. Per questo l'appiattimento della matrice funziona: prima scorriamo le righe, poi, per ogni riga, scorriamo gli elementi;
- ➔ l'operatore `:=` ci permette di assegnare un valore a una variabile (`log_val`) e, contemporaneamente, di usarlo dentro la condizione `if`. Senza il *walrus operator* avremmo dovuto calcolare `math.log(x)` due volte, una nel filtro e una nell'espressione;

#### 12.3.1.1* Considerazioni sulla velocità delle list comprehension

Le *list comprehension* sono generalmente più veloci dei cicli tradizionali, perché il ciclo viene eseguito direttamente a livello di *bytecode* dall'interprete C.
Il codice Python viene tradotto in bytecode (un linguaggio intermedio di istruzioni molto semplici). Questo bytecode viene eseguito da *CPython*, che è l'interprete standard di Python, scritto appunto in linguaggio C .

Proviamo a misuare i tempi di esecuzione di due loop uno effettuato con un ciclo `for` tradizionale e l'altro effettuato con una *comprehension*.
Per fare questa prova utilizzeremo il modulo `timeit` che ci "offre un modo semplice per calcolare il tempo impiegato nell'esecuzione di piccole parti di codice Python"[37]

Analizziamo il seguente codice:

```
import timeit

# 1. Codice con ciclo tradizionale
codice_tradizionale = """
risultato = []
for i in range(10000):
    risultato.append(i)
"""

# 2. Codice con list comprehension
codice_comprehension = """
risultato = [i for i in range(10000)]
"""

# Eseguiamo ciascun codice 1.000 volte e misuriamo il tempo totale
tempo_tradizionale = timeit.timeit(codice_tradizionale,
number=1000)
tempo_comprehension = timeit.timeit(codice_comprehension,
number=1000)

print(f"Tempo ciclo tradizionale: {tempo_tradizionale:.4f}
secondi")
```

[37] https://docs.python.it/html/lib/module-timeit.html

```
print(f"Tempo list comprehension: {tempo_comprehension:.4f}
secondi")
print(f"La list comprehension è circa
{tempo_tradizionale/tempo_comprehension:.1f} volte più veloce!")
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Tempo ciclo tradizionale: 0.2119 secondi
Tempo list comprehension: 0.1301 secondi
La list comprehension è circa 1.6 volte più veloce!
```

Dall'analisi del codice e dell'output osserviamo che:

➔ con la list comprehension il codice è parecchio più veloce.

### 12.3.2 *Dictionary comprehension e set comprehension*

Lo stesso principio delle *list comprehension* si applica alla costruzione di dizionari e insiemi. Cambia solo il tipo di parentesi (le graffe: `{}`) e, nel caso del dizionario, la presenza dei due punti per separare chiave e valore. Riportiamo anche in questo caso la sintassi:

```
# dictionary comprehension
{chiave: valore for elemento in iterabile if condizione}
# set comprehension
{espressione for elemento in iterabile if condizione}
```

Vogliamo scrivere una procedura che ripulisce un elenco di prodotti da spazi e maiuscole superflue (usando un set, che elimina automaticamente i duplicati) e poi genera un listino prezzi fittizio. Ecco il codice python:

```
prodotti_grezzi = ["  Caffè  ", "tè verde", "CAFFÈ", " Cioccolato
", "Tè Verde"]

# Set comprehension: normalizzazione ed eliminazione dei duplicati
prodotti_puliti = {prodotto.strip().capitalize() for prodotto in
prodotti_grezzi}

# Dictionary comprehension: un prezzo fittizio per ogni prodotto
univoco
listino_prezzi = {prodotto: round(len(prodotto) * 1.5, 2) for
prodotto in prodotti_puliti}

print("Insieme dei prodotti normalizzati (Set):", prodotti_puliti)
print("Dizionario prezzi generato (Dict):", listino_prezzi)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Insieme dei prodotti normalizzati (Set): {'Cioccolato',
'Caffè', 'Tè verde'}
Dizionario prezzi generato (Dict): {'Cioccolato': 15.0,
'Caffè': 7.5, 'Tè verde': 12.0}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Le stringhe `"CAFFÈ"` e `"  Caffè  "` diventano, dopo `.strip()` e `.capitalize()`, lo stesso identico valore (`"Caffè"`), il *set*, per sua natura, elimina i duplicati, quindi nel risultato compare una sola volta;
- ➔ la *dictionary comprehension* successiva parte proprio dall'insieme già ripulito.

Questo è un esempio di come si possano concatenare più *comprehension* per costruire, passo dopo passo, una struttura dati più complessa.

# 12.4 Il protocollo di iterazione: iterabili e iteratori

## 12.4.1 Un'analogia per non confonderli

Abbiamo già usato il termine *iterabile* nel capitolo dedicato al ciclo `for`, asserendo che liste, stringhe, tuple, dizionari e insiemi lo sono tutti. Adesso vogliamo capire come funziona davvero, dietro le quinte, l'iterazione, distinguendo con precisione due concetti che spesso vengono confusi tra loro: l'*iterabile* e l'*iteratore*.

Immaginiamo un magazzino chiuso, pieno di scatole di merce. Il magazzino, da solo, non fa nulla: contiene la merce, ma non è in grado di consegnarla. Il magazzino è il nostro *iterabile*.

Per prelevare le scatole, una alla volta, serve un nastro trasportatore che è un dispositivo che sa esattamente dove si trova, che scatola ha già consegnato, e qual è la prossima. Il nastro non conosce il numero totale di scatole nel magazzino, e non può tornare indietro: il suo unico compito è fornire, quando richiesto, la scatola successiva, finché il magazzino non sarà vuoto. Il nastro trasportatore è il nostro *iteratore*.

Esaminiamo il seguente programma per vedere il meccanismo in azione:

```
numeri = [10, 20, 30]

# Otteniamo l'iteratore dalla lista
iteratore = iter(numeri)

# Estraiamo gli elementi uno alla volta con next()
print("Primo elemento:",next(iteratore))
print("Secondo elemento:",next(iteratore))
```

```
print("Terzo elemento:",next(iteratore))

# Se proviamo a chiamare next() ancora, otteniamo un errore
print("Quarto elemento:",next(iteratore))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Primo elemento: 10
Secondo elemento: 20
Terzo elemento: 30
Traceback (most recent call last):
  File "/home/utente/prova.py", line 12, in <module>
    print("Quarto elemento:",next(iteratore))
                             ^^^^^^^^^^^^^^^^
StopIteration
```

Dall'analisi del codice e dell'output osserviamo che:

- La funzione `iter()` trasforma la lista in un iteratore;
- La funzione `next()` estrae un elemento alla volta, ricordando la posizione corrente;
- Quando gli elementi sono finiti, viene sollevata l'eccezione `StopIteration` (che è esattamente ciò che fa il ciclo for internamente, fermandosi in modo pulito);
- Una volta esaurito, l'*iteratore* non può essere riutilizzato: se vogliamo ripartire dall'inizio dobbiamo crearne uno nuovo con `iter()`.

### 12.4.2 I metodi `__iter__()` e `__next__()`

Dal punto di vista del linguaggio Python, un oggetto è *iterabile* se implementa il metodo speciale `__iter__()`, e questo metodo deve restituire un *iteratore*.

Un oggetto è invece un *iteratore* se implementa entrambi i seguenti metodi:

- `__iter__()`: per un *iteratore*, questo metodo si limita a restituire se stesso (`return self`). E' ciò che permette di usare un *iteratore* direttamente in un ciclo `for`;
- `__next__()`: restituisce l'elemento successivo della sequenza. Quando gli elementi sono terminati, deve sollevare l'eccezione `StopIteration`.

In pratica, quando scriviamo `for elemento in collezione:`, l'interprete esegue esattamente le seguenti operazioni:

1. chiama `iter(collezione)` per ottenere un iteratore;
2. chiama ripetutamente `next()` su quell'*iteratore*;
3. quando `next()` solleva l'eccezione `StopIteration`, l'interprete la intercetta silenziosamente e termina il ciclo.

Come esempio implementiamo un iteratore che generi la *successione di Fibonacci*[38] fino a

[38] https://it.wikipedia.org/wiki/Successione_di_Fibonacci

un valore massimo. Esamina il seguente codice:

```
class SuccessioneFibonacci:
    """Iteratore personalizzato che genera la sequenza di
Fibonacci fino a un valore massimo."""

    def __init__(self, limite_massimo):
        self.limite_massimo = limite_massimo
        self.valore_precedente = 0
        self.valore_corrente = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.valore_precedente > self.limite_massimo:
            raise StopIteration

        valore_da_restituire = self.valore_precedente

        prossimo_valore = self.valore_precedente +
self.valore_corrente
        self.valore_precedente = self.valore_corrente
        self.valore_corrente = prossimo_valore

        return valore_da_restituire

fib_iteratore = SuccessioneFibonacci(100)

print("Attraversamento della sequenza di Fibonacci:")
for numero in fib_iteratore:
    print(numero, end=" ")
print()
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
0 1 1 2 3 5 8 13 21 34 55 89
```

Dall'analisi del codice e dell'output osserviamo che:

- nel metodo `__init__` inizializziamo lo *stato interno* dell'oggetto, cioè due attributi che tengono traccia del valore corrente e di quello precedente della successione;
- a ogni chiamata di `__next__()` lo stato viene aggiornato, e viene restituito il valore calcolato in precedenza. La classe "ricorda" a che punto della successione si trova, esattamente come il nastro trasportatore dell'esempio del magazzino ricordava quale fosse l'ultima scatola consegnata;
- quando il valore supera il limite fissato (`100`), viene sollevata l'eccezione

`StopIteration` e il ciclo `for` che lo utilizza si ferma senza bisogno di alcuna istruzione esplicita da parte nostra;

➔ il vantaggio principale di questo approccio è che i numeri di Fibonacci vengono calcolati e restituiti uno alla volta: non dobbiamo mai avere l'intera successione già pronta in una lista.

# 12.5 Generatori e valutazione pigra

## 12.5.1 Dalle classi alla parola chiave `yield`

L'esempio della classe `SuccessioneFibonacci` mostra bene il funzionamento del protocollo di *iterazione*, ma è anche piuttosto lungo da scrivere per un'esigenza, in fondo, semplice. Python offre un'alternativa molto potente: i `generatori`.

Un generatore è una funzione che, invece di restituire un valore con `return`, restituisce una sequenza di valori uno alla volta grazie alla parola chiave `yield`.

Il comportamento è diverso da quello di una funzione normale, è importante capirlo bene:

1. quando chiamiamo una funzione generatrice, il suo corpo non viene eseguito subito; viene invece creato e restituito un oggetto generatore, "congelato" al suo stato iniziale;
2. alla prima chiamata di `next()` sul generatore, il codice viene eseguito fino al primo `yield`;
3. arrivato a `yield`, il valore indicato viene restituito al chiamante, e l'esecuzione della funzione viene sospesa, tutte le variabili locali restano in memoria così come sono;
4. alla chiamata successiva di `next()`, l'esecuzione riprende esattamente dall'istruzione successiva a `yield`, come se la funzione non si fosse mai fermata.

Questo comportamento si chiama *valutazione pigra* (*lazy evaluation*): i valori non vengono calcolati tutti insieme in anticipo, ma solo uno alla volta, quando servono davvero.
Esaminiamo il seguente codice:

```
def genera_quadrati(limite):
    """Funzione generatrice: produce i quadrati dei numeri da 0 a
limite-1, uno alla volta."""
    for n in range(limite):
        yield n**2

# Chiamando la funzione otteniamo un oggetto generatore, non
ancora dei valori
quadrati = genera_quadrati(5)
print(quadrati)

# Estraiamo i valori uno alla volta con next()
print(next(quadrati))
print(next(quadrati))

# Estraiamo i rimanenti con un ciclo for
```

```
print("Valori restanti:")
for valore in quadrati:
    print(valore)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
<generator object genera_quadrati at 0x7c399a3798a0>
0
1
Valori restanti:
4
9
16
```

Dall'analisi del codice e dell'output osserviamo che:

- la chiamata `genera_quadrati(5)` non esegue il corpo della funzione: restituisce subito un oggetto generatore, pronto a produrre valori solo quando richiesto;
- le prime due chiamate a `next(quadrati)` fanno eseguire il corpo della funzione fino al primo `yield` incontrato, restituiscono il valore (`0`, poi `1`), e sospendono l'esecuzione mantenendo intatto lo stato interno (in questo caso il valore di n);
- il ciclo `for valore in quadrati:` che segue continua l'iterazione esattamente da dove `next()` l'aveva lasciata: infatti stampa `4, 9, 16` e non ripete `0` e `1`, perché quei valori sono già stati "consumati";
- questo conferma che un *generatore*, una volta esaurito, non torna indietro. Se provassimo a fare un altro `for valore in quadrati:` dopo la fine del ciclo precedente, non stamperebbe più nulla.

### 12.5.3 Espressioni generatrici

Oltre alle *funzioni generatrici* basate su `yield`, Python mette a disposizione una scorciatoia ancora più compatta: le *espressioni generatrici*, che hanno la stessa sintassi delle *list comprehension* ma usano le parentesi tonde `()` al posto di quelle quadre `[]`.

Mentre una *list comprehension* costruisce subito, per intero, la lista dei risultati, un'*espressione generatrice* si comporta esattamente come una *funzione generatrice*: non calcola nulla finché non viene interrogata con `next()` o percorsa con un ciclo `for`, e mantiene sempre un ingombro di memoria minimo, indipendentemente da quanti elementi dovrà produrre.

Per questo motivo, ogni volta che una list comprehension serve solo per "passare" i dati a un'altra elaborazione (ad esempio dentro `sum()`, `max()` o un altro ciclo `for`), e non per conservarli tutti insieme, conviene sostituire le parentesi quadre con quelle tonde. Così facendo si ottiene lo stesso risultato logico ma con un notevole risparmio di memoria e nessuna perdita di leggibilità (capiremo meglio quanto qui detto con il secondo esempio di codice).

Consideriamo subito questo semplice esempio di espressione generatrice:

```
# Espressione generatrice
generatore_quadrati = (x**2 for x in range(10))

print(generatore_quadrati)
for _ in range(3):
    print(next(generatore_quadrati))

for val in generatore_quadrati:
     print(val)

print(next(generatore_quadrati))
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
<generator object <genexpr> at 0x79f1bc179b10>
0
1
4
9
16
25
36
49
64
81
Traceback (most recent call last):
  File "/home/utente/prova.py", line 12, in <module>
    print(next(generatore_quadrati))
          ^^^^^^^^^^^^^^^^^^^^^^^^^
StopIteration
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ l'espressione generatrice si comporta esattamente come una funzione generatrice in quanto crea un'oggetto che appartiene alla classe `generator`
- ➔ Inoltre possiamo scorrere gli aelemnti attraverso il ciclo `for`
- ➔ Possiamo utilizzare `next()` per generare l'elemento successivo, esattamente come accadeva con l'*iteratore* di Fibonacci scritto a mano, ma con molte meno righe di codice
- ➔ E alla fine degli elementi un successivo `next()` genera l'eccezione `StopIteration`
- ➔ un *generatore*, una volta esaurito, non può essere riutilizzato. Se ci servisse di nuovo l'intera sequenza, dovremmo crearne uno nuovo.

Adesso analizziamo un programma, che confronta il consumo di memoria di una lista e di

un generatore che devono produrre gli stessi dieci milioni di valori e che fa comprendere come, per alcune elaborazioni, sia più performante utilizzare un'*espressione generatrice* che un *list comprehension*.

```
import sys

N = 10_000_000 # PEP 515 – Underscores in Numeric Literals

# Metodo 1: list comprehension (valutazione immediata, "eager")
lista_quadrati = [x**2 for x in range(N)]

# Metodo 2: espressione generatrice (valutazione pigra, "lazy")
generatore_quadrati = (x**2 for x in range(N))

spazio_lista = sys.getsizeof(lista_quadrati)
spazio_generatore = sys.getsizeof(generatore_quadrati)

print(f"Spazio occupato dalla lista in RAM: {spazio_lista} byte
(circa {spazio_lista / 1024 / 1024:.2f} MB)")
print(f"Spazio occupato dal generatore in RAM: {spazio_generatore}
byte")

# Calcoliamo la somma di tutti gli elementi quadratici
totale_1 = sum(lista_quadrati)
totale_2 = sum(generatore_quadrati)

print("Totale somma lista:", totale_1)
print("Totale somma generatore:", totale_2)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
Spazio occupato dalla lista in RAM: 89095160 byte (circa 84.97 MB)
Spazio occupato dal generatore in RAM: 200 byte
Totale somma lista: 333333283333335000000
Totale somma generatore: 333333283333335000000
```

Dall'analisi del codice e dell'output osserviamo che:

- la *lista* occupa decine di megabyte, perché tutti e dieci i milioni di quadrati sono stati calcolati e allocati in RAM contemporaneamente;
- il *generatore* occupa poche centinaia di byte, indipendentemente da quanti valori dovrà produrre, non contiene i dati, contiene solo lo stato necessario per calcolarli quando richiesto;
- La funzione della list comprehension e dell'espressione generatrice è semplicemente quella di creare i quadrati che poi serviranno per essere sommati tra loro, non ci serve memorizzare i valori stessi. Quindi si comprende come, in questo caso, al posto della *list comprehension* sia molto più performante utiliizzare

un *generatore*.

### 12.5.2 Lista o generatore? Un confronto riassuntivo

La tabella seguente riassume le differenze principali tra le due strutture, ed è utile per decidere quale usare caso per caso:

| Caratteristica | List comprehension / Lista | Generator expression / Generatore |
|---|---|---|
| Sintassi di definizione | Parentesi quadre `[]` | Parentesi tonde `()` |
| Tempo di inizializzazione | Proporzionale alla dimensione dei dati | Praticamente istantaneo |
| Consumo di memoria RAM | Cresce in modo proporzionale alla dimensione dei dati | Costante, tipicamente pochi byte |
| Sequenze infinite | Impossibili (la memoria si esaurirebbe) | Pienamente supportate |
| Riutilizzo | Sì, la lista resta in memoria e si può percorrere più volte | No, il generatore si esaurisce dopo l'uso |

**REGOLA PRATICA**: se ci serve l'intera collezione più volte, o se dobbiamo accedere ai suoi elementi in ordine sparso (ad esempio tramite indice), usiamo una *lista*. Se dobbiamo solo scorrere i dati una volta sola, magari in grande quantità, o addirittura senza un limite definito a priori, il *generatore* è la scelta corretta.

## 12.6 Non esagerare: la trappola della complessità

Gli strumenti visti in questo capitolo sono potenti, ma proprio per questo vanno maneggiati con giudizio. Un errore molto comune, specialmente in chi ha appena scoperto le *comprehension*, è quello di provare a comprimere qualsiasi logica, anche complessa, in un'unica riga di codice. Il risultato, più che *Pythonic*, diventa quella che qualcuno chiama scherzosamente una *PythonINcomprehension*: un'istruzione tecnicamente corretta, ma che nessuno, nemmeno l'autore stesso a distanza di qualche settimana, riesce più a leggere agevolmente!

Consideriamo il seguente problema: trovare tutte le terne di coordinate `(x, y, z)`, comprese tra 0 e 9, tali che `x * y > 20`, `z` sia pari e la somma `x + y + z` sia minore di 15. Confrontiamo due modi di risolvere lo stesso problema:

**Approccio sconsigliato** (una *list comprehension* con tre cicli e tre condizioni compressi in un'unica istruzione):

```
coordinate_incomprensibili = [
```

```
(x, y, z) for x in range(10) for y in range(10) for z in range(10)
if x * y > 20 if z % 2 == 0 if (x + y + z) < 15
]
```

**Approccio consigliato** (una *funzione generatrice*, con la stessa logica ma distribuita su più righe leggibili):

```
def genera_coordinate_valide(limite):
    """Genera in modo pigro le coordinate tridimensionali che
rispettano i vincoli geometrici."""
    for x in range(limite):
        for y in range(limite):
            if x * y <= 20:
                continue
            for z in range(limite):
                if z % 2 != 0:
                    continue
                if (x + y + z) < 15:
                    yield (x, y, z)

coordinate_leggibili = list(genera_coordinate_valide(10))

assert coordinate_incomprensibili == coordinate_leggibili
print(f"Generati con successo {len(coordinate_leggibili)} punti di
coordinate.")
```

Le due versioni producono esattamente lo stesso risultato (lo verifichiamo con l'`assert`), ma la seconda ci offre diversi vantaggi concreti:

- ➔ **separazione delle responsabilità**: la logica di filtro è isolata dentro la funzione, e il codice che la utilizza resta pulito;
- ➔ **manutenibilità**: se in futuro dovessimo aggiungere un nuovo vincolo, ci basta modificare o aggiungere una riga dentro la funzione, senza toccare un'espressione compressa e fragile;
- ➔ **debug più semplice**: possiamo inserire un `print()` di controllo dentro un qualsiasi punto del ciclo tradizionale; farlo dentro una *comprehension* a tre livelli è molto più scomodo.

**REGOLA PRATICA:** non superare mai il secondo livello di annidamento in una *comprehension*. Se serve un terzo `for` o una seconda condizione `if` oltre al primo filtro, è quasi sempre il segnale che conviene passare a un ciclo tradizionale o a un *generatore* ben commentato.

## 12.7 Ricapitolo sui concetti

Riassumiamo le indicazioni pratiche emerse in questo capitolo:

- ➔ le *lambda* vanno usate solo per operazioni minime e locali, tipicamente dentro

`map()`, `filter()`, oppure come chiave di ordinamento (`sorted()`);

- ➔ le *comprehension* (di *liste*, *insiemi* e *dizionari*) sono lo strumento ideale per trasformare e filtrare collezioni di dimensioni contenute, a patto che la logica resti semplice e leggibile su poche righe;
- ➔ il *protocollo di iterazione* ( `__iter__` e `__next__` ) è l'infrastruttura su cui si basa ogni ciclo `for` che scriviamo, e ci permette, quando serve, di costruire i nostri iteratori personalizzati;
- ➔ i *generatori*, grazie alla valutazione pigra, sono la soluzione da preferire quando dobbiamo elaborare grandi quantità di dati, o flussi di dati potenzialmente senza fine, mantenendo il consumo di memoria costante e ridotto al minimo.

## ### Esercizi

1. Usando `map()` e una lambda, converti una lista di lunghezze espresse in metri in una lista di lunghezze espresse in centimetri.
2. Scrivi una list comprehension che, data una lista di parole, restituisca solo quelle con più di 5 lettere, in maiuscolo.
3. Scrivi una classe iteratore che, dato un numero `n`, restituisca tutti i divisori di `n`, uno alla volta.
4. Riscrivi l'esercizio precedente come funzione generatrice con `yield`, e confronta la lunghezza e la leggibilità delle due soluzioni.
5. Scrivi un generatore infinito che produce i numeri primi, uno alla volta, e stampane i primi 20 usando un ciclo `for` con un contatore di uscita (attenzione: un generatore infinito non va mai usato dentro un `for` senza una condizione di arresto esterna).